

嵌入式软件开发/Linux驱动开发笔试面试知识点整理

申明：本资料部分内容转载自网络，请勿用于商业用途。如有侵权，请联系作者删除。

作者在准备秋招的过程中，整理了一些资料，最后顺利拿到了小米，兆易创新，大华股份等10家公司的offer。现将这部分资料分享出来，希望能对大家有帮助！

如果大家在网上看到了不错的资料，或者在笔试面试中遇到了资料中没有的知识点，大家可以关注我的公众号或者通过QQ群联系我，我替大家整理。资料如有错误或者不合适的地方，请及时联系作者。

作者还整理了部分公司（华为，小米，深信服，寒武纪等）的笔试题目，面经等，这些资料都放在了公众号中，关注后回复“秋招大礼包”可以免费领取。同时，如果需要嵌入式软硬件相关学习资料（视频，软件，电子书等）的同学，可以扫描下方二维码进群，在群文件获取。

最后希望大家都能在秋招中获得自己心仪的offer!

CSDN博客：[嵌入式与Linux那些事](#)

微信公众号：



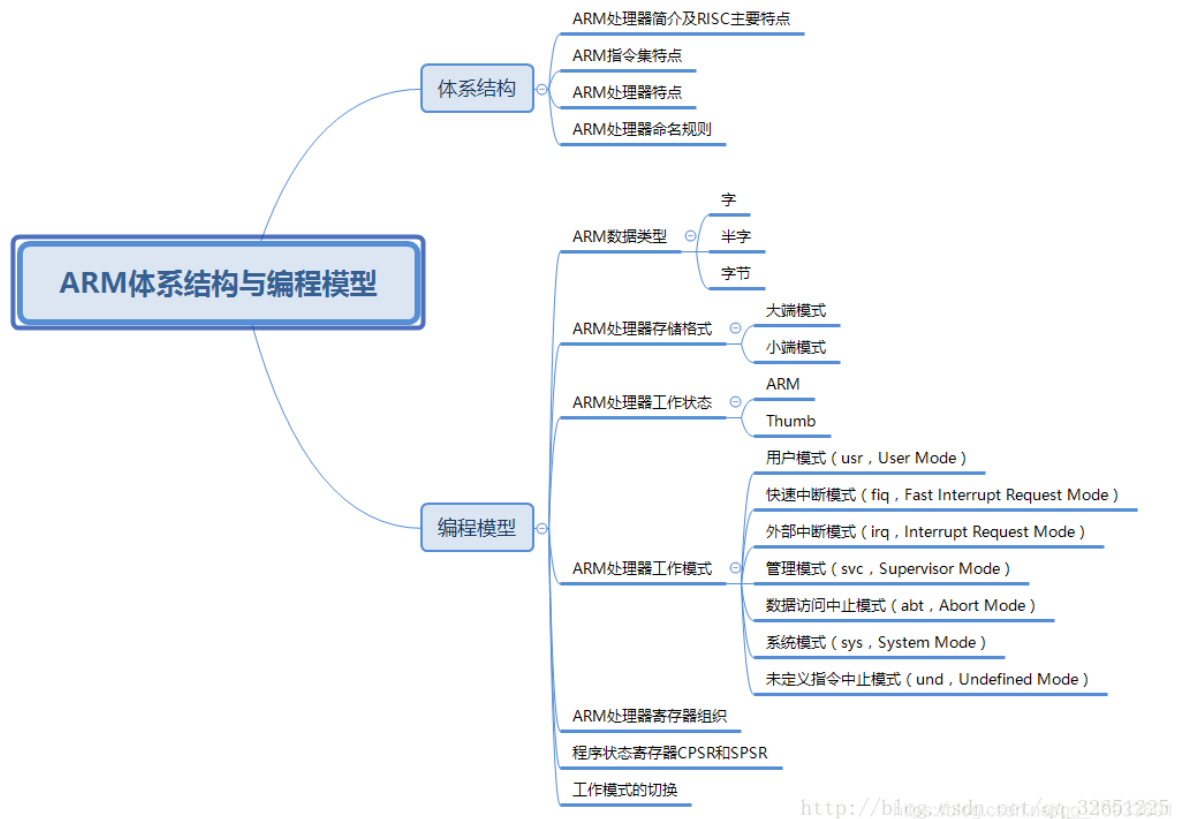
技术交流群：



ARM

ARM体系结构与编程模型总结

ARM体系结构与编程模型



ARM体系结构

一、ARM处理器简介及RISC特点

ARM处理器简介

ARM (Advanced RISC Machines) 是一个32位RISC (精简指令集) 处理器架构，ARM处理器则是ARM架构下的微处理器。ARM处理器广泛的使用在许多嵌入式系统。ARM处理器的特点有指令长度固定，执行效率高，低成本等。

RISC设计主要特点

- 1、指令集——RISC减少了指令集的种类，通常一个周期一条指令，采用固定长度的指令格式，编译器或程序员通过几条指令完成一个复杂的操作。而CISC指令集的指令长度通常不固定。
- 2、流水线——RISC采用单周期指令，且指令长度固定，便于流水线操作执行。
- 3、寄存器——RISC的处理器拥有更多的通用寄存器，寄存器操作较多。例如ARM处理器具有37个寄存器。
- 4、Load/Store结构——使用加载/存储指令批量从内存中读写数据，提高数据的传输效率。
- 5、寻址方式简化，指令长度固定，指令格式和寻址方式种类减少。

二、ARM指令集特点

ARM处理器是基于RISC的，但不是纯粹的RISC体系结构。为了使ARM处理器能够更好的满足嵌入式系统的需要，ARM指令集和单纯的RISC指令集有以下几点不同：

1、一些特定的指令周期数可变。例如多寄存器装载或存储的Load/Store指令执行周期就是不确定的，这个会根据相关的寄存器个数而定。如果是访问连续的内存地址，就可以改善性能，因为连续的内存访问比随机访问要快。根据这个特点，由于在函数的起始和结尾通常会有多个寄存器与内存进行数据交换的操作，因此相应操作的指令条数会减少，提高了代码的密度。

2、内嵌的桶形移位寄存器产生了更复杂的指令。桶形移位寄存器是一个硬件部件，在一个寄存器被一条指令使用之前，桶形移位寄存器可以处理这个寄存器中的数据。桶形移位寄存器扩展了许多指令的功能，以此改善内核的性能，提高代码密度。

3、Thumb16位指令集。ARM处理器有两种工作状态，一种是ARM状态，一种是Thumb状态。ARM状态下指令长度为32位，Thumb状态下指令长度为16位。这种特点使得ARM既能执行16位指令，又能执行32位指令，从而增强了ARM内核的功能。

4、条件执行。只有当某个特定条件满足时指令才会被执行。这个特性可以减少分支指令的数目，从而改善性能，提高代码密度。

5、增强指令。一些功能强大的数字信号处理器（DSP）指令被加入到标准的ARM指令中，以支持快速的16*16乘法操作及饱和运算。ARM的这些增强指令，使得ARM处理器不需要加上DSP即可实现。

三、ARM处理器特点

- 1、ARM指令都是32位定长的
- 2、寄存器数量丰富（37个寄存器）
- 3、普通的Load/Store指令
- 4、多寄存器的Load/Store指令
- 5、指令的条件执行
- 6、单时钟周期中的单条指令完成数据移位操作和ALU操作
- 7、通过变种和协处理器来扩展ARM处理器的功能
- 8、扩展了16位的Thumb指令来提高代码密度

四、ARM处理器命名规则

ARM {x} {y} {z} {T} {D} {M} {I} {E} {J} {F} {S}

x:系列

x=7 表示ARM7处理器；
x=9 表示ARM9处理器；
x=10 表示ARM10处理器；
x=11 表示ARM11处理器；

y: 存储单元/保护单元。

y=2 表示带Cache和MMU；
y=3 表示物理地址标记的Cache和MMU；
y=4 表示Cache和MPU；
y=6 表示写缓冲但无cache大小。

z: Cache.

z=0 表示标准Cache大小；
z=2 表示缩小的Cache；
z=6 表示包含紧密耦合SRAM (TCM)。

T：支持Thumb指令集，ARM V6及以后的内核都自动包含T。

D：支持片上Debug。

M：内嵌硬件乘法器。

I：嵌入式ICE，支持片上断点和调试。

E：DSP运算的增强指令。

J：使用Jazelle技术。

F：ARM核通过向浮点（VFP）结构支持硬件浮点。

S：综合版本，意味这处理器内核是以源代码形式提供的。这种源代码形式又可以被编译成一种易于EDA工具使用的形式。

http://https://blog.csdn.net/qq_16933601

ARM编程模型

一、ARM数据类型

- 1、字（Word）：在ARM体系结构中，字的长度为32位。
- 2、半字（Half-Word）：在ARM体系结构中，半字的长度为16位。
- 3、字节（Byte）：在ARM体系结构中，字节的长度为8位。

二、ARM处理器存储格式

ARM体系结构将存储器看作是从0地址开始的字节的线性组合。作为32位的微处理器，ARM体系结构所支持的最大寻址空间为4GB。

ARM体系结构可以用两种方法存储字数据，分别为大端模式和小端模式。

大端模式（高地高低）：字的高字节存储在低地址字节单元中，字的低字节存储在高地址字节单元中。

小端模式（高高低低）：字的高字节存储在高地址字节单元中，字的低字节存储在低地址字节单元中。

三、ARM处理器工作状态

从编程的角度来看，ARM微处理器的工作状态一般ARM和Thumb有两种，并可在两种状态之间切换。

- 1、ARM状态：此时处理器执行32位的字对齐ARM指令，绝大部分工作在此状态。
- 2、Thumb状态：此时处理器执行16位的半字对齐的Thumb指令。

四、ARM处理器工作模式

- 1、用户模式（usr，User Mode）：ARM处理器正常的程序执行状态。
- 2、快速中断模式（fiq，Fast Interrupt Request Mode）：用于高速数据传输或通道处理。当触发快速中断时进入此模式。
- 3、外部中断模式（irq，Interrupt Request Mode）：用于通用的中断处理。当触发外部中断时进入此模式。

4、管理模式（svc，Supervisor Mode）：操作系统使用的保护模式。在系统复位或执行软件中断指令SWI时进入。

5、数据访问中止模式（abt，Abort Mode）：当数据或指令预取中止时进入该模式，可用于虚拟存储及存储保护。

6、系统模式（sys，System Mode）：运行具有特权的操作系统任务。

7、未定义指令中止模式（und，Undefined Mode）：当未定义的指令执行时进入该模式，可用于支持硬件协处理器的软件仿真。

除了用户模式之外，其余六种模式都是特权模式。除了用户模式和系统模式之外，其余五种模式都是异常模式。

在特权模式下程序可以访问所有的系统资源。非特权模式和特权模式之间的区别在于有些操作只能在特权模式下才被允许，例如直接改变模式和中断使能等。而且为了保证数据安全，一般MMU会对地址空间进行划分，只有特权模式才能访问所有的地址空间。而用户模式如果需要访问硬件，必须切换到特权模式下，才允许访问硬件。

五、ARM处理器寄存器组织

ARM共有37个32位寄存器，其中31个为通用寄存器，6个为状态寄存器，包括R0-R15，R8_fiq-R14_fiq，R13_svc，R14_svc，R13_abt，R14_abt，R13_irq，R14_irq，R13_und，R14_und，CPSR，SPSR_fiq，SPSR_svc，SPSR_abt，SPSR_irq，SPSR_und。如图。

ARM State General Registers and Program Counter

System & User	FIQ	Supervisor	Abort	IRQ	Undefined
R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7
R8	R8_fiq	R8	R8	R8	R8
R9	R9_fiq	R9	R9	R9	R9
R10	R10_fiq	R10	R10	R10	R10
R11	R11_fiq	R11	R11	R11	R11
R12	R12_fiq	R12	R12	R12	R12
R13	R13_fiq	R13_svc	R13_abt	R13_irq	R13_und
R14	R14_fiq	R14_svc	R14_abt	R14_irq	R14_und
R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)

ARM State Program Status Registers

CPSR	CPSR SPSR_fiq	CPSR SPSR_svc	CPSR SPSR_abt	CPSR SPSR_irq	CPSR SPSR_und
------	------------------	------------------	------------------	------------------	------------------

通用寄存器包括R0-R15，可以分为3类：

1、未分组寄存器R0-R7

在所有运行模式下，未分组寄存器都指向同一个物理寄存器，他们未被系统用作特殊的用途。因此在中断或异常处理进行异常模式转换时，由于不同的处理器运行模式均使用相同的物理寄存器，所以可能造成寄存器中数据的破坏。

2、分组寄存器R8-R14

对于分组寄存器，他们每次所访问的物理寄存器都与当前的处理器运行模式相关。具体如上图。

R13常用作存放堆栈指针，用户也可以使用其他寄存器存放堆栈指针，但在Thumb指令集下，某些指令强制要求使用R13存放堆栈指针。

R14称为链接寄存器（LR，Link Register），当执行子程序时，R14可得到R15（PC）的备份，执行完子程序后，又将R14的值复制回PC，即使用R14保存返回地址。

3、程序计数器PC（R15）

寄存器R15用作程序计数器（PC），在ARM状态下，位[1:0]为0，位[31:2]用于保存PC；在Thumb状态下，位[0]为0，位[31:1]用于保存PC。

由于ARM体系结构采用了多级流水线技术，对于ARM指令集而言，PC总是指向当前指令的下两条指令的地址，即PC的值为当前指令的地址值加8个字节。

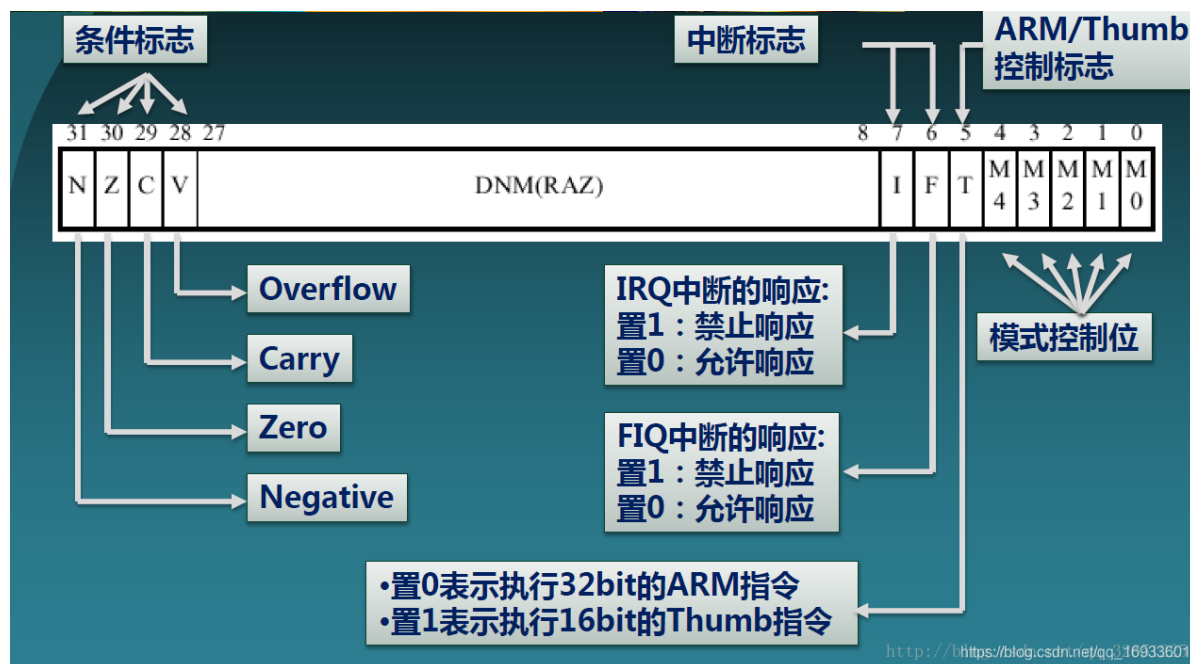
六、程序状态寄存器CPSR和SPSR

CPSR（Current Program Status Register，当前程序状态寄存器），CPSR可在任何运行模式下被访问，它包括条件标志位、中断禁止位、当前处理器模式标志位以及其他一些相关的控制和状态位。

每一种运行模式下都有一个专用的物理状态寄存器，称为SPSR（Saved Program Status Register，备份的程序状态寄存器），当异常发生时，SPSR用于保存当前CPSR的值，从异常退出时则可由SPSR来恢复CPSR。

由于用户模式和系统模式不属于异常模式，这两种状态下没有SPSR，因此在这两种状态下访问SPSR，结果是未知的。

CPSR保存数据的结构：



1、N（Negative）：当用两个补码表示的带符号数进行运算时，N=1表示结果为负，N=0表示结果为正数或零

2、Z（Zero）：Z=1表示运算结果为0，Z=0表示运算结果非零

3、C（Carry）：有4种方法可以设置C的值：

- (1) 加法指令（包括比较指令CMP）
- (2) 当运算产生进位时（无符号数溢出），C=1，否则C=0
- (3) 减法运算（包括比较指令CMP）
- (4) 当运算产生了借位（无符号数溢出），C=0，否则C=1

对于包含移位操作的非加/减运算指令，C为移出值的最后一位。对于其他的非加/减运算指令，C的值通常不变。

4、V（Overflow）：有2种方法设置V的值：

- (1) 对于加/减法运算指令，当操作数和运算结果为二进制的补码表示的带符号数时，V=1表示符号位溢出。
- (2) 对于其他的非加减法运算指令，V的值通常不变。

5、I（Interrupt Request）：I=1表示禁止响应irq，I=0表示允许响应

6、F（Fast Interrupt Request）：F=1表示禁止响应fiq，F=0表示允许响应

7、T（Thumb）：T=0表示当前状态位ARM状态，T=1表示为Thumb状态

8、M4-M0：表示当前处理器的工作模式，如图：

M[4:0]	Mode	Visible THUMB state registers	Visible ARM state registers
10000	User	R7..R0, LR, SP PC, CPSR	R14..R0, PC, CPSR
10001	FIQ	R7..R0, LR_fiq, SP_fiq PC, CPSR, SPSR_fiq	R7..R0, R14_fiq..R8_fiq, PC, CPSR, SPSR_fiq
10010	IRQ	R7..R0, LR_irq, SP_irq PC, CPSR, SPSR_irq	R12..R0, R14_irq, R13_irq, PC, CPSR, SPSR_irq
10011	Supervisor	R7..R0, LR_svc, SP_svc, PC, CPSR, SPSR_svc	R12..R0, R14_svc, R13_svc, PC, CPSR, SPSR_svc
10111	Abort	R7..R0, LR_abt, SP_abt, PC, CPSR, SPSR_abt	R12..R0, R14_abt, R13_abt, PC, CPSR, SPSR_abt
11011	Undefined	R7..R0 LR_und, SP_und, PC, CPSR, SPSR_und	R12..R0, R14_und, R13_und, PC, CPSR
11111	System	R7..R0, LR, SP PC, CPSR	R14..R0, PC, CPSR http://blog.csdn.net/ggq32651225

七、工作模式的切换

(1) 执行软中断（SWI）或复位命令（Reset）指令。如果在用户模式下执行SWI指令，CPU就进入管理（Supervisor）模式。当然，在其他模式下执行SWI，也会进入该模式，不过一般操作系统不会这么做，因为除了用户模式属于非特权模式，其他模式都属于特权模式。执行SWI指令一般是为了访问系统资源，而在特权模式下可以访问所有的系统资源。SWI指令一般用来为操作系统提供API接口。

(2) 有外部中断发生。如果发生了外部中断，CPU就会进入IRQ或FIQ模式。

(3) CPU执行过程中产生异常。最典型的异常是由于MMU保护所引起的内存访问异常，此时CPU会切换到Abort模式。如果是无效指令，则会进入Undefined模式。

(4) 有一种模式是CPU无法自动进入的，这种模式就是System模式，要进入System模式必须由程序员编写指令来实现。要进入System模式只需改变CPSR的模式位为System模式对应的模式位即可。进入System模式一般是为了利用System模式和用户模式下的寄存器相同的特点，因此一般情况下，操作系统在通过SWI进入Supervisor模式后，做一些操作后，就进入System模式。

(5) 在任何特权模式下，都可以通过修改CPSR的MODE域来进入其他模式。不过需要注意的是由于修改的CPSR是该模式下的影子CPSR，即SPSR，因此并不是实际的CPSR，所以一般的做法是修改影子CPSR，然后执行一个MOVS指令来恢复执行某个断点并切换到新模式。

什么是bootloader

Linux系统要启动就必须需要一个bootloader程序，也就说芯片上电以后先运行一段bootloader程序。这段bootloader程序会先**初始化时钟，看门狗，中断，SDRAM**，等外设，然后将Linux内核从flash (NAND, NOR FLASH, SD, MMC等) **拷贝**到SDRAM中，最后**启动Linux内核**。当然了，bootloader的实际工作要复杂的多，但是它最主要的工作就是启动Linux内核，bootloader和Linux内核的关系就跟PC上的BIOS和Windows的关系一样，bootloader就相当于BIOS。总得来说，**Bootloader就是一小段程序，它在系统上电时开始执行，初始化硬件设备、准备好软件环境，最后调用操作系统内核。**

什么是根文件系统

根文件系统首先是一种文件系统，该文件系统不仅具有普通文件系统的存储数据文件的功能，但是相对于普通的文件系统，它的特殊之处在于，它是内核启动时所挂载 (mount) 的**第一个文件系统**，内核代码的映像文件保存在根文件系统中，系统引导启动程序会在根文件系统挂载之后从中把一些**初始化脚本** (如rcS, inittab) 和**服务**加载到内存中去运行，里面包含了Linux系统能够运行**所必需的应用程序、库等**，比如可以给用户提供操作Linux的控制界面的shell程序、动态连接的程序运行时需要的glibc库等。我们要明白文件系统和内核是完全独立的两个部分。在嵌入式中移植的内核下载到开发板上，是没有办法真正的启动Linux操作系统的，会出现无法加载文件系统的错误。

根文件系统为什么这么重要

根文件系统之所以在前面加一个“根”，说明它是加载其它文件系统的“根”，那么**如果没有这个根，其它的文件系统也就没有办法进行加载的**。根文件系统包含系统启动时所必须的目录和关键性的文件，以及使其他文件系统得以挂载 (mount) 所必要的文件。例如：

- (1) init进程的应用程序必须运行在根文件系统上
- (2) 根文件系统提供了根目录“/”
- (3) linux挂载分区时所依赖的信息存放于根文件系统/etc/fstab这个文件中
- (4) shell命令程序必须运行在根文件系统上，譬如ls、cd等命令

总之：一套linux体系，只有内核本身是不能工作的，必须要rootfs (上的etc目录下的配置文件、/bin /sbin等目录下的shell命令，还有/lib目录下的库文件等…) 相配合才能工作。

Linux启动时，**第一个必须挂载的是根文件系统**。若系统不能从指定设备上挂载根文件系统，则系统会出错而退出启动。成功之后可以自动或手动挂载其他的文件系统。**因此，一个系统中可以同时存在不同的文件系统**。在Linux中将一个文件系统与一个存储设备关联起来的过程称为挂载 (mount)。使用mount命令将一个文件系统附着到当前文件系统层次结构中 (根)。在执行挂装时，要提供文件系统类型、文件系统和一个挂装点。根文件系统被挂载到根目录下“/”上后，在根目录下就有根文件系统的各个目录，文件：/bin /sbin /mnt等，再将其他分区挂接到/mnt目录上，/mnt目录下就有这个分区的各个目录和文件。

如何构建根文件系统可以参考这篇文章[构建根文件系统](#)

公众号：嵌入式与Linux那些事 CSDN： [嵌入式与Linux那些事](#) 来源网络，转载声明

Bootloader (Uboot) 启动流程分析

Bootloader的启动过程可以分为单阶段、多阶段两种。通常多阶段的Bootloader能提供更为复杂的功能以及更好的可移植性。从固态存储设备上启动的Bootloader大多都是两阶段的启动过程。第一阶段使用汇编来实现，它完成一些依赖于CPU体系结构的初始化，并调用第二阶段的代码；第二阶段则通常使用C语言来实现，这样可以实现更复杂的功能，而且代码会有更好的可读性和可移植性。

一般而言，这两个阶段完成的功能可以如下分类：

Bootloader第一阶段的功能

硬件设备初始化

首先需要设置时钟，设置MPLL（具体参见下面的FCLK HCLK PCLK 部分）。接着设置CLKDIVN地址为0x4C000014，写入0x05,表示设置分频系数为FCLK:HCLK:PCLK=1:4:8。接着，关闭看门狗，关中断，启动ICACHE，关闭DCACHE和TLB，关闭MMU（ICACHE为指令缓存，可以不关闭，指令直接操作的硬件，实际的物理地址。但是DCACHE就必须关闭，此时MMU没有使能，虚拟地址映射不成功，sdram无法访问，DCACHE无数据）。start.s具体代码如下：

```

1      /* 设置时钟 */
2      ldr r0, =0x4c000014
3      // mov r1, #0x03;
4      mov r1, #0x05;          // FCLK:HCLK:PCLK=1:4:8
5      str r1, [r0]
6
7      /* 如果HCLK非0，CPU的总线模式应该从“fast bus mode”变为“asynchronous bus
mode” */
8      mrc p15, 0, r1, c1, c0, 0      /* 读出控制寄存器 */
9      orr r1, r1, #0xc0000000        /* 设置为“asynchronous bus mode” */
10     mcr p15, 0, r1, c1, c0, 0      /* 写入控制寄存器 */
11
12     /* MPLLCON = S3C2440_MPLL_200MHZ */
13     ldr r0, =0x4c000004
14     ldr r1, =S3C2440_MPLL_400MHZ
15     str r1, [r0]
16     /* 启动ICACHE */
17     mrc p15, 0, r0, c1, c0, 0      @ read control reg
18     orr r0, r0, #(1<<12)
19     mcr p15, 0, r0, c1, c0, 0      @ write it back

```

这里具体讲下是如何设置FCLK HCLK PCLK。

FCLK又称为**内核时钟**，是提供给ARM920T 的时钟。

HCLK又称为**总线时钟**，是提供给用于存储器控制器，中断控制器，LCD 控制器，DMA 和 USB 主机模块的 AHB总线(advanced high-performance bus)的时钟。

PCLK又称为**I/O接口时钟**，是提供给用于外设如WDT，IIS，I2C，PWM 定时器，MMC/SD 接口，ADC，UART，GPIO，RTC 和SPI的 APB (advanced peripherals bus)总线的时钟。

S3C2440 FCLK值为400MHz，HCLK值为100MHz、PCLK值为50MHz。那么这些值通过什么方法计算出来呢？S3C2440上的时钟源是12MHz，如果想让CPU工作在更高频率上，就需要通过PLL（锁相环）来提高主频。S3C2440上的PLL有两种，一种是MPLL，它是用来产生FCLK、HCLK、PCLK的高频工作时钟；还有一种是UPLL，用来为USB提供工作频率。S3C2440时钟体系如下：

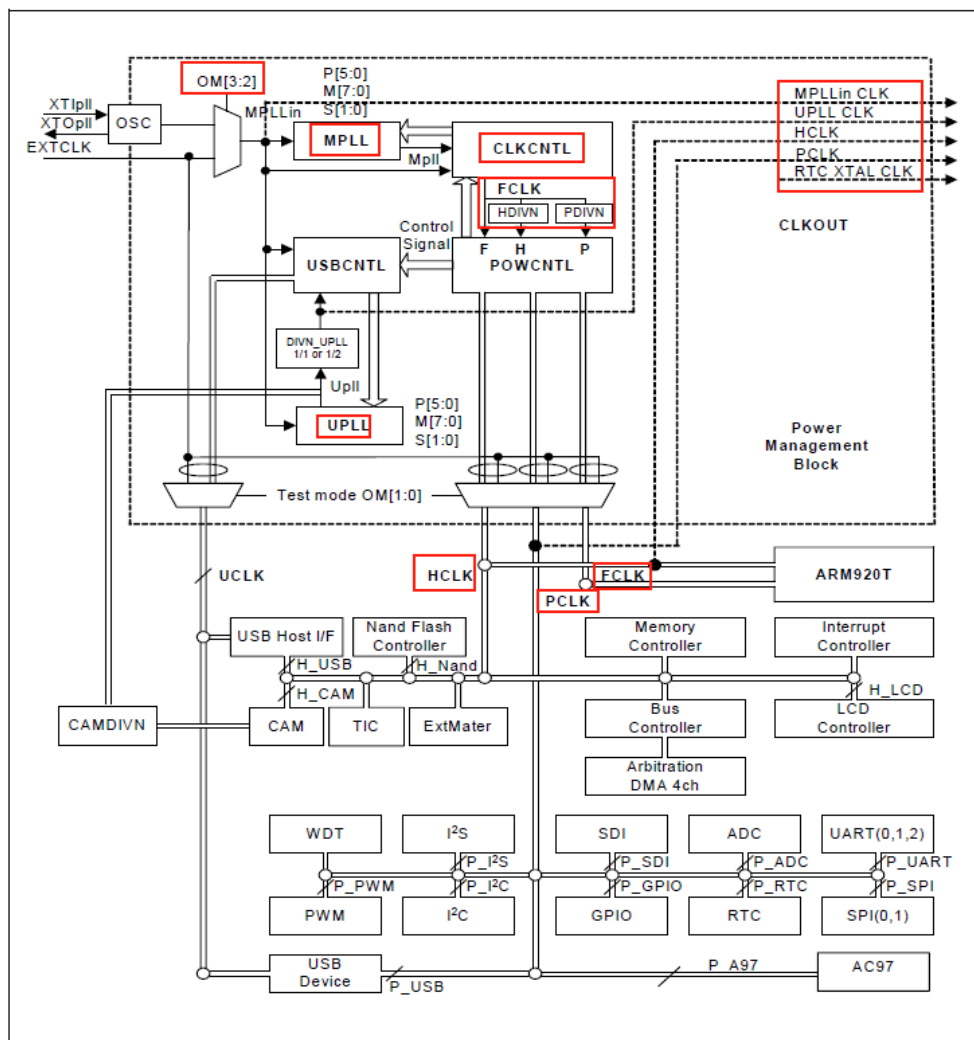
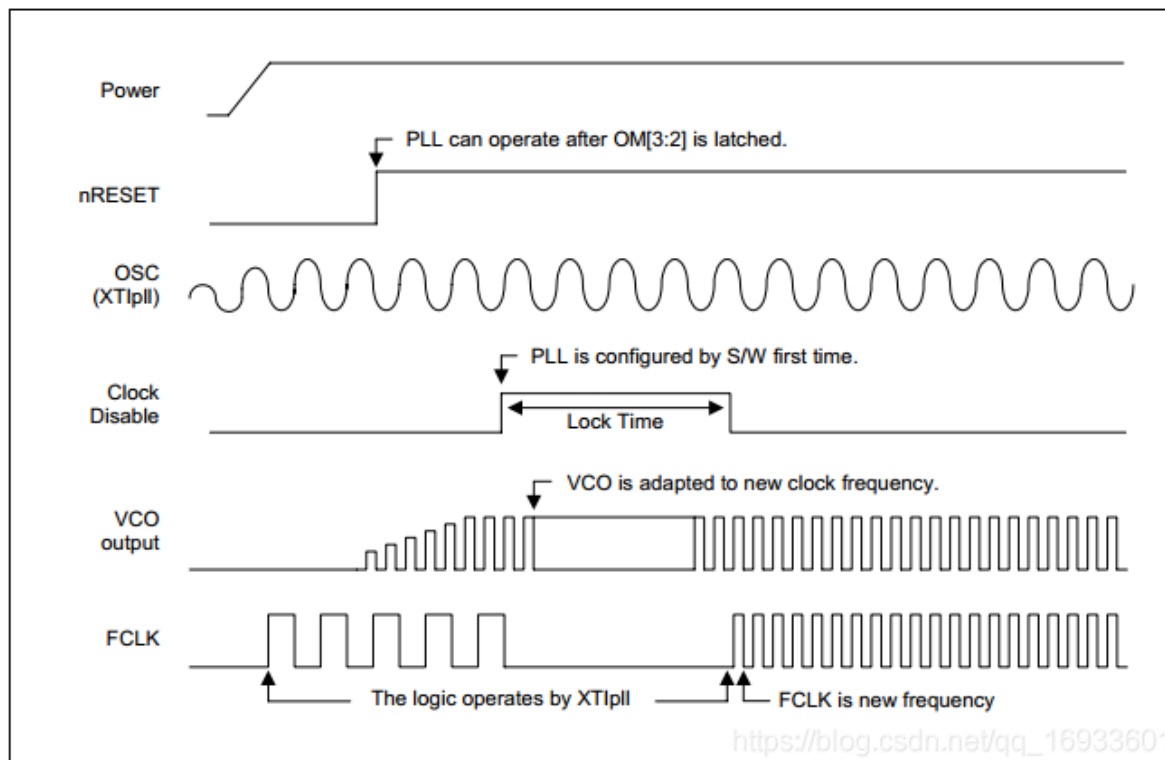


Figure 7-1. Clock Generator Block Diagram

https://blog.csdn.net/qq_16933601



https://blog.csdn.net/qq_16933601

从时序图中，我们可以看到，上电之后，如果什么都不设置，FCLK和晶振的频率相等。当设置PLL后，CPU并不是马上就使用设置好的高频时钟，而是有一段锁定时间，在这段时间里，CPU停止运行，等12MHz变成高频时钟稳定以后，整个系统再重新运行。

开启MPLL的过程：

- 1、设置LOCKTIME变频锁定时间
- 2、设置FCLK与晶振输入频率（Fin）的倍数
- 3、设置FCLK，HCLK，PCLK三者之间的比例

从手册上可以看到，LOCKTIME的默认时间是0xFFFFFFFF，控制方法如图：

（刚设置好PLL时，系统认为这是PLL还没稳定，所有这时不用PLL的时钟，而用外部晶振做时钟，将PLL锁住，过了LOCKTIME后认为PLL已经稳定了，才使用PLL给系统提供时钟）

LOCKTIME	Bit	Description	Initial State
U_LTIME	[31:16]	UPLL lock time count value for UCLK. (U_LTIME > 300uS)	0xFFFF
M_LTIME	[15:0]	MPLL lock time count value for FCLK, HCLK, and PCLK (M_LTIME > 300uS)	0xFFFF

FCLK与Fin的倍数通过MPLLCON寄存器设置，三者之间有以下关系：

$$MPLL(FCLK) = (2 * m * Fin) / (p * 2^s)$$

其中：m = MDIV + 8, p = PDIV + 2, s = SDIV

PLL配置寄存器如图：

PLLCON	Bit	Description	Initial State
MDIV	[19:12]	Main divider control	0x96 / 0x4d
PDIV	[9:4]	Pre-divider control	0x03 / 0x03
SDIV	[1:0]	Post divider control	0x0 / 0x0

当设置完MPLL之后，就会自动进入LockTime变频锁定期间，LOCKTIME之后，MPLL输出稳定时钟频率。

FCLK、HCLK、PCLK的设置比例如图：

HDIVN	PDIVN	HCLK3_HALF/ HCLK4_HALF	FCLK	HCLK	PCLK	Divide Ratio
0	0	—	FCLK	FCLK	FCLK	1 : 1 : 1 (Default)
0	1	—	FCLK	FCLK	FCLK / 2	1 : 1 : 2
1	0	—	FCLK	FCLK / 2	FCLK / 2	1 : 2 : 2
1	1	—	FCLK	FCLK / 2	FCLK / 4	1 : 2 : 4
3	0	0 / 0	FCLK	FCLK / 3	FCLK / 3	1 : 3 : 3
3	1	0 / 0	FCLK	FCLK / 3	FCLK / 6	1 : 3 : 6
3	0	1 / 0	FCLK	FCLK / 6	FCLK / 6	1 : 6 : 6
3	1	1 / 0	FCLK	FCLK / 6	FCLK / 12	1 : 6 : 12
2	0	0 / 0	FCLK	FCLK / 4	FCLK / 4	1 : 4 : 4
2	1	0 / 0	FCLK	FCLK / 4	FCLK / 8	1 : 4 : 8
2	0	0 / 1	FCLK	FCLK / 8	FCLK / 8	1 : 8 : 8
2	1	0 / 1	FCLK	FCLK / 8	FCLK / 16	1 : 8 : 16

Register	Address	R/W	Description	Reset Value
CLKDIVN	0x4C000014	R/W	Clock divider control register	0x00000000

CLKDIVN	Bit	Description	Initial State
DIVN_UPLL	[3]	UCLK select register(UCLK must be 48MHz for USB) 0: UCLK = UPLL clock 1: UCLK = UPLL clock / 2 Set to 0, when UPLL clock is set as 48MHz Set to 1. when UPLL clock is set as 96MHz.	0
HDIVN	[2:1]	00 : HCLK = FCLK/1. 01 : HCLK = FCLK/2. 10 : HCLK = FCLK/4 when CAMDIVN[9] = 0. HCLK= FCLK/8 when CAMDIVN[9] = 1. 11 : HCLK = FCLK/3 when CAMDIVN[8] = 0. HCLK = FCLK/6 when CAMDIVN[8] = 1.	00
PDIVN	[0]	0: PCLK has the clock same as the HCLK/1. 1: PCLK has the clock same as the HCLK/2.	0

如果HDIV设置为非0，CPU的总线模式要进行改变，默认情况下FCLK = HCLK，CPU工作在fast bus mode快速总线模式下，HDIV设置为非0后，FCLK与HCLK不再相等，要将CPU改为asynchronous bus mod异步总线模式，可以通过下面的嵌入汇编代码实现：

```

1  __asm__(
2      "mrc    p15, 0, r1, c1, c0, 0\n"          /* 读出控制寄存器 */
3      "orr    r1, r1, #0xc0000000\n"          /* 设置为“asynchronous bus mode”
4      "mcr    p15, 0, r1, c1, c0, 0\n"          /* 写入控制寄存器 */
5      );

```

为加载 Bootloader的第二阶段代码准备RAM空间（初始化内存空间）

lowlevel_init中设置相应BANK地址，主要用来设置SDRAM。内存是被映射在了0x30000000-0x40000000的位置，即**bank6与bank7**。那么在内存时序设置的时候，主要关心的，就是bank6与bank7。当然，bank0也是需要关注的，因为它是启动时，启动程序存放的位置。但是bank0是由OM[1:0]，即板子上的那几个小开关中的两个来控制的，所以这里程序上是不用管它的。

```

1  SMRDATA:
2      .long 0x22011110    //BWSCON
3      .long 0x00000700    //BANKCON0
4      .long 0x00000700    //BANKCON1
5      .long 0x00000700    //BANKCON2
6      .long 0x00000700    //BANKCON3
7      .long 0x00000740    //BANKCON4
8      .long 0x00000700    //BANKCON5
9      .long 0x00018005    //BANKCON6
10     .long 0x00018005    //BANKCON7
11     .long 0x008C04F4    // REFRESH
12     .long 0x000000B1    //BANKSIZE
13     .long 0x00000030    //MRSRB6
14     .long 0x00000030    //MRSRB7

```

接下来设置栈地址指向NAND，准备初始化NANDFLASH。

```

1    ldr sp, =(CONFIG_SYS_INIT_SP_ADDR)//等于0x30000f80
2    bic sp, sp, #7 /* 8-byte alignment for ABI compliance */
3
4    bl nand_init_ll

```

初始化NANDFLASH，其中包括设置时序NFCONF，(参考芯片手册和2440手册设置nandflash的启动时序)。TACLS表示的建立所用的时间，TWRPH0表示nWE写控制信号的持续时间，TWRPH1表示数据生效所用的时间，什么时候可以读数据。最后就是使能NFCONT NAND Flash控制器，初始化ECC，禁止片选。到这里，NANDFLASH的初始化就完成了。下面就可以进行重定位了。

```

1    void nand_init_ll(void)
2    {
3        #define TACLS    0
4        #define TWRPH0    1
5        #define TWRPH1    0
6        /* 设置时序 */
7        NFCONF = (TACLS<<12) | (TWRPH0<<8) | (TWRPH1<<4);
8        /* 使能NAND Flash控制器，初始化ECC，禁止片选 */
9        NFCONT = (1<<4) | (1<<1) | (1<<0);
10   }

```

复制 Bootloader的第二阶段代码到SDRAM空间中（重定位）

首先判断是NOR启动还是NAND启动，如果是NAND启动就直接拷贝数据。拷贝代码之前，要传递给拷贝函数三个参数，源，目的，长度。读取NAND的话要参考芯片手册的NAND读取数据的时序，选中NAND，发出读命令，发出地址，发出读命令，判断状态，读取数据，取消选中等。

```

1    bl copy_code_to_sdram
2    bl clear_bss //清除bss段(参考自制uboot章节)

```

```

1    void copy_code_to_sdram(unsigned char *src, unsigned char *dest, unsigned
    int len)
2    {
3        int i = 0;
4
5        /* 如果是NOR启动 */
6        if (isBootFromNorFlash())
7        {
8            while (i < len)
9            {
10               dest[i] = src[i];
11               i++;
12            }
13        }
14        else
15        {
16            //nand_init();
17            nand_read_ll((unsigned int)src, dest, len);
18        }
19    }
20
21    void clear_bss(void)
22    {
23        extern int __bss_start, __bss_end__;
24        int *p = &__bss_start;

```

```

25
26     for (; p < &__bss_end__; p++)
27         *p = 0;
28 }

```

最后要清除bss。bss段不占用空间，都是未初始化的全局变量或者已经初始化为零的变量，本来就是零，直接清零就好。不清零的话未初始化的变量可能会存在未知的数值。

设置好栈

设置栈跳转到SDRAM执行。

```

1 | ldr pc,=call_board_init_f           //绝对跳转,跳到SDRAM上执行

```

跳转到第二阶段代码的C入口点

跳转到SDRAM执行剩下的程序。

```

1 | call_board_init_f:
2 | .globl base_sp
3 | base_sp:
4 |     .long 0
5 |
6 |     ldr r0,=0x00000000
7 |     bl  board_init_f
8 |
9 |     /*unsigned int id 的值存在r0中，正好给board_init_r使用*/
10 |    ldr r1,=_TEXT_BASE
11 |    /*重新设置栈到之前的位置 指向原来addr_sp += 128;*/
12 |    ldr sp,base_sp
13 |    /*调用第二阶段代码*/
14 |    bl  board_init_r

```

Bootloader第二阶段的功能

初始化本阶段要使用到的硬件设备

为了方便开发，至少要初始化一个串口以便程序员与 Bootloader进行交互。

检测系统内存映射（ memory map ）

所谓检测内存映射，就是确定板上使用了多少内存、它们的地址空间是什么。由于嵌入式开发中 Bootloader多是针对某类板子进行编写，所以可以根据板子的情况直接设置，不需要考虑可以适用于各类情况的复杂算法。

将内核映象和根文件系统映象从 Flash上读到SDRAM空间中

Flash上的内核映象有可能是经过压缩的，在读到SDRAM之后，还需要进行解压。当然，对于有自解压功能的内核，不需要 Bootloader来解压。将根文件系统映象复制到SDRAM中，这不是必需的。这取决于是什么类型的根文件系统以及内核访问它的方法。

将内核存放在适当的位置后，直接跳到它的入口点即可调用内核。调用内核之前，下列条件要满足：

（1）CPU寄存器的设置

R0=0（规定）。

R1=机器类型ID；对于ARM结构的CPU，其机器类型ID可以参见 linux/arch/arm tools/ mach-types

R2=启动参数标记列表在RAM中起始基地址（下面会详细介绍如何传递参数）。

（2）CPU工作模式

必须禁止中断（IRQ和FIQ，uboot启动是一个完整的过程，没有必要也不能被打断）

CPU必须为SVC模式（为什么呢？主要是像异常模式、用户模式都不合适。具体深入的原因自己可以查下资料）。

（3）Cache和MMU的设置

MMU必须关闭。

指令 Cache可以打开也可以关闭。

数据 Cache必须关闭。

为内核设置启动参数

Bootloader与内核的交互是单向的，Bootloader将各类参数传给内核。由于它们不能同时行，传递办法只有一个：**Bootloader将参数放在某个约定的地方之后，再启动内核，内核启动后从这个地方获得参数。**

除了约定好参数存放的地址外，还要规定参数的结构。Linu2.4x以后的内核都期望以**标记列表**（tagged_list）的形式来传递启动参数。标记，就是一种**数据结构**；标记列表，就是挨着存放的多个标记。标记列表以标记 ATAG CORE 开始，以标记 ATAG NONE 结束。

标记的数据结构为tag，它由一个 tag_header结构和一个联合(union)组成。tag_header结构表小标记的类型及长度，比如是表示内存还是表示命令行参数等。对于不同类型的标记使用不同的联合(union)，比如表示内存时使用 tag_mem32，表示命令行时使用 tag_cmdline。

bootloader与内核约定的参数地址，设置内存的起始地址和大小，指定根文件系统在那个分区，系统启动后执行的第一个程序linuxrc，控制台ttySAC0等。

调用内核

调用内核就是uboot启动的最后一步了。到这里就uboot就完成了他的使命。

uboot启动内核详解

下面我们来展开说下uboot具体是如何调用内核的，引导内核启动的。

uboot与Linux内核之间的参数传递

我们知道，uboot启动后已经完成了基本的硬件初始化（如：内存、串口等），接下来它的主要任务就是加载Linux内核到开发板的内存，然后跳转到Linux内核所在的地址运行。

PS:只要问到uboot，面试官必问uboot和内核的参数传递，所以一定要知道！

具体是如何跳转呢？做法很简单，**直接修改PC寄存器的值为Linux内核所在的地址**，这样CPU就会从Linux内核所在的地址去取指令，从而执行内核代码。

在前面我们已经知道，在跳转到内核以前，uboot需要做好以下三件事情：

（1）CPU寄存器的设置

R0=0。

R1=机器类型ID；对于ARM结构的CPU，其机器类型ID可以参见 linux/arch/arm tools/ mach-types

R2=启动参数标记列表在RAM中起始基地址。

（2）CPU工作模式

必须禁止中断（IRQs和FIQs）

CPU必须为SVC模式

（3）Cache和MMU的设置

MMU必须关闭

指令 Cache可以打开也可以关闭

数据 Cache必须关闭

其中上面第一步CPU寄存器的设置中，就是通过R0,R1,R2三个参数给内核传递参数的。（[ATPCS规则可以参考](#)）

为什么要给内核传递参数呢？

在此之前，uboot已经完成了硬件的初始化，可以说已经“适应了”这块开发板。然而，内核并不是对于所有的开发板都能完美适配的（如果适配了，可想而知这个内核有多庞大，又或者有新技术发明了，可以完美的适配各种开发板），此时对于开发板的环境一无所知。所以，要想启动Linux内核，uboot必须要给内核传递一些必要的信息来告诉内核**当前所处的环境**。

如何给内核传递参数？

因此，uboot就把**机器ID通过R1传递给内核**，Linux内核运行的时候首先就从R1中读取机器ID来判断是否支持当前机器。这个机器ID实际上就是开发板**CPU的ID**，每个厂家生产出一款CPU的时候都会给它指定一个唯一的ID，大家可以到uboot源码的arch\arm\include\asm\mach-type.h文件中去查看。

#define MACH_TYPE IXDP2801	300
#define MACH_TYPE IQ31244	327
#define MACH_TYPE BAST	331
#define MACH_TYPE H1940	347
#define MACH_TYPE ENP2611	356
#define MACH_TYPE S3C2440	362
#define MACH_TYPE GUMSTIX	373
#define MACH_TYPE OMAP_H2	382
#define MACH_TYPE E740	384

R2存放的是块内存的基地址，这块内存中存放的是uboot给Linux内核的其他参数。这些参数有内存的**起始地址、内存大小、Linux内核启动后挂载文件系统的方式**等信息。很明显，参数有多个，不同的参数有不同的内容，为了让Linux内核能精确的解析出这些参数，双方在传递参数的时候要求参数在存放的时候需要**按照双方规定的格式存放**。

除了约定好参数存放的地址外，还要规定参数的结构。Linux2.4.x以后的内核都期望以标记列表（tagged_list）的形式来传递启动参数。标记，就是一种数据结构；标记列表，就是挨着存放的多个标记。标记列表以标记 **ATAG_CORE** 开始，以标记 **ATAG_NONE** 结束。

标记的数据结构为tag，它由一个tag_header结构和一个联合（union）组成。tag_header结构表示标记的类型及长度，比如是表示内存还是表示命令行参数等。对于不同类型的标记使用不同的联合（union），比如表示内存时使用tag_mem32，表示命令行时使用tag_cmdline。具体代码见arch\arm\include\asm\setup.h。

```

struct tag {
    struct tag_header hdr;
    union {
        struct tag_core core;
        struct tag_mem32 mem;
        struct tag_videotext videotext;
        struct tag_ramdisk ramdisk;
        struct tag_initrd initrd;
        struct tag_serialnr serialnr;
        struct tag_revision revision;
        struct tag_videolfb videolfb;
        struct tag_cmdline cmdline;
    };
    /*
     * Acorn specific
     */
    struct tag_acorn acorn;
    /*
     * DC21285 specific
     */
    struct tag_memclk memclk;
} ? end {anonu} ? u;
} ? end tag ? ;

```

ATAG_CORE 标识tag参数的开始
 ATAG_MEM 标识内存信息
 ATAG_CMDLINE 标识命令行参数
 ATAG_NONE 标识tag参数的结束

内存信息。如内存的基地址，内存的大小。

描述命令参数信息，如Linux内核，挂载文件系统方式等

从上面可以看出，struct_tag结构体由structtag_header+联合体union构成，结构体struct_tag_header用来描述每个tag的头部信息，如tag的类型，tag大小。联合体union用来描述每个传递给Linux内核的参数信息。

下面以传递内存标记、传递命令行参数为例来说明参数的传递。

(1)设置开始标记ATAG_CORE

```
1 tag->hdr.tag = ATAG_CORE;
2 tag->hdr.size = tag_size(tag_core);
3 tag->u.core.flags = params->u1.s.flags & FLAG_READONLY;
4 tag->u.core.pagesize = params->u1.s.page_size;
5 tag->u.core.rootdev = params->u1.s.rootdev;
6
7 tag = tag_next(tag);
```

涉及到的结构体定义如下

```
1 struct tag_header {
2     __u32 size;
3     __u32 tag;
4 };
5
6 /* The list must start with an ATAG_CORE node */
7 #define ATAG_CORE 0x54410001
8
9 struct tag_core {
10     __u32 flags; /* bit 0 = read-only */
11     __u32 pagesize;
12     __u32 rootdev;
13 };
```

其中tag_next，tag_size定义如下，指向当前标记的结尾

```
1 #define tag_next(t) ((struct tag *)((u32 *)(t) + (t)->hdr.size))
2 #define tag_size(type) ((sizeof(struct tag_header) + sizeof(struct type)) >> 2)
```

(2)设置内存标记

```
1 t->hdr.tag = ATAG_MEM;
2 t->hdr.size = tag_size(tag_mem32);
3 t->u.mem.start = CFG_GLOBAL_RAM_BASE;
4 t->u.mem.size = CFG_GLOBAL_RAM_SIZE;
5
6 t = tag_next(t);
```

相关结构体定义如下

```
1 #define ATAG_MEM 0x54410002
2
3 struct tag_mem32 {
4     __u32 size;
5     __u32 start; /* physical start address */
6 };
```

(3)设置命令行参数标记

命令行参数是一个字符串，一般用它来告诉内核挂载根文件系统的方式。由uboot的bootargs环境变量提供，它的内容有如下两种格式

```
1 root=nfs nfsroot=202.193.61.237:/work/nfs_root/first_fs ip=202.193.61.196
  init=/linuxrc console=ttySAC0,115200
```

```
1 root=/dev/mtdblock2 ip=202.193.61.196 init=/linuxrc console=ttySAC0,115200
```

名称	含义
root	告诉Linux内核挂载根文件系统的方式，nfs表示以NFS服务的方式挂载根文件系统，/dev/mtdblock2表示根文件系统在MTD设置的第二个分区上。
nfsroot	告诉Linux内核，以NFS方式挂载根文件系统时，根文件系统所在主机的IP地址和路径
ip	告诉Linux内核，启动后它的IP地址
init	告诉Linux内核，启动的第一个应用程序是根目录下的linuxrc程序
console	告诉Linux区内核，控制台为ttySAC0,波特率为115200

```
1 tag = tag_next(tag);
2 tag->hdr.tag = ATAG_CMDLINE;
3 tag->hdr.size = (strlen(params->commandline) + 3 +
4                 sizeof(struct tag_header)) >> 2;
5 strcpy(tag->u.cmdline.cmdline, params->commandline);
6
7 tag = tag_next(tag);
```

相关结构体定义如下

```
1 /* command line: \0 terminated string */
2 #define ATAG_CMDLINE 0x54410003
3
4 struct tag_cmdline {
5     char cmdline[1]; /* this is the minimum size */
6 };
```

(4)设置结束标记

```
1 tag->hdr.tag = ATAG_NONE;
2 tag->hdr.size = 0;
```

我们明白了运行Linux区内核的时候，uboot需要给内核的传递的参数，接下来我们就来看看如何从uboot中跳到Linux内核。

uboot跳转到Linux内核

在uboot中可以使用go和bootm来跳转到内核，这两个命令的区别如下：

(1) go命令仅仅修改PC的值到指定地址

格式：go addr

(2) bootm命令是uboot专门用来启动ulmage格式的Linux内核，它在修改pc的值到指定地址之前，会设置传递给Linux内核的参数，用法如下：

格式：bootm addr

uboot中bootm命令实现

bootm命令在uboot源码common/cmd_bootm.c中实现，它的功能如下：

- (1) 读取ulmage头部，把内核拷贝到合适的地方。
- (2) 把参数给内核准备好。
- (3) 引导内核。

当我们使用我们在uboot使用bootm命令后，bootm命令会从ulmage头中读取信息后，发现是Linux内核，就会调用do_bootm_linux()函数，函数的具体实现bootm.c中

```

1  int do_bootm_linux(int flag, int argc, char *argv[], bootm_headers_t
    *images)
2  {
3      /* No need for those on ARM */
4      if (flag & BOOTM_STATE_OS_BD_T || flag & BOOTM_STATE_OS_CMDLINE)
5          return -1;
6
7      if (flag & BOOTM_STATE_OS_PREP) {
8          boot_prep_linux(images);
9          return 0;
10     }
11
12     if (flag & BOOTM_STATE_OS_GO) {
13         boot_jump_linux(images);
14         return 0;
15     }
16
17     boot_prep_linux(images);
18     boot_jump_linux(images);
19     return 0;
20 }
```

do_bootm_linux 函数最终会跳转执行 boot_prep_linux 和 boot_jump_linux 函数，首先分析 boot_prep_linux 函数(位于 bootm.c 文件中)：

```

1  static void boot_prep_linux(bootm_headers_t *images)
2  {
3      char *commandline = getenv("bootargs");           //从环境变量中获取
    bootargs 的值
4
5      . . . . .
6      setup_board_tags(&params);
7      setup_end_tag(gd->bd);           //将 tag 参数保存在指定位置
8  } else {
9      printf("FDT and ATAGS support not compiled in - hanging\n");
10     hang();
11 }
12 do_nonsec_virt_switch();
13 }
```

从代码可以看出来，boot_prep_linux，主要功能是将 tag 参数保存到指定位置，比如 bootargs 环境变量 tag，串口 tag，接下来分析 boot_jump_linux 函数(位于 bootm.c 文件中)：

```

1 static void boot_jump_linux(bootm_headers_t *images, int flag)
2 {
3     unsigned long machid = gd->bd->bi_arch_number;           //获取机器id (在
board/samsung/jz2440/jz2440.c 中设置，为 MACH_TYPE_SMDK2410(193))
4     char *s;
5     void (*kernel_entry)(int zero, int arch, uint params);
6     unsigned long r2;
7     int fake = (flag & BOOTM_STATE_OS_FAKE_GO);
8
9     kernel_entry = (void (*)(int, int, uint))images->ep;       //获取 kernel
的入口地址，此处应为 30000000
10
11     s = getenv("machid");                                     //从环境变量里获取机器id (本例中还未在环境
变量里设置过机器 id)
12     if (s) {                                                  //判断环境变量里是否设置机器id
13         strict_strtoul(s, 16, &machid);                     //如果设置则用环境变量里的机器id
14         printf("Using machid 0x%x from environment\n", machid);
15     }
16
17     debug("## Transferring control to Linux (at address %08lx)" \
18           "... \n", (ulong) kernel_entry);
19     bootstage_mark(BOOTSTAGE_ID_RUN_OS);
20     announce_and_cleanup(fake);
21
22     if (IMAGE_ENABLE_OF_LIBFDT && images->ft_len)
23         r2 = (unsigned long)images->ft_addr;
24     else
25         r2 = gd->bd->bi_boot_params;                         //获取 tag参数地址，gd->bd-
>bi_boot_params在 setup_start_tag 函数里设置
26     if (!fake) kernel_entry(0, machid, r2); }                //进入内核
    
```

通过分析可以看出，最终进入内核的函数为：

```

1 kernel_entry(0, machid, r2)
    
```

到这里bootm就成功给内核传递了参数，并跳转到了内核。关于go命令的实现可以自己参考内核，在cmd_boot.c文件中，所不同的是，go命令实现的时候没有设置参数，只是简单的跳转执行。如果想要使用go来跳转到Linux内核，我们需要做简单的修改，有兴趣的可以自己研究下，这里就不展开讲了。

至此，uboot就启动了内核。启动内核后就是挂载根文件系统了，下篇将具体介绍是如何挂载根文件系统的。

[构建根文件系统](#)

内核镜像格式vmlinux和zImage和uImage

最后插讲下内核的不同映像格式的区别：

(1) uboot经过编译直接生成的elf格式的可执行程序是u-boot，这个程序类似于windows下的exe格式，在操作系统下是可以直接执行的。但是这种格式**不能用来烧录下载**。我们用来烧录下载的是u-boot.bin，这个东西是由u-boot使用arm-linux-objcopy工具进行加工（主要目的是去掉一些无用的）得到的。这个u-boot.bin就叫镜像（image），镜像就是用来烧录到iNand中执行的。

(2) linux内核经过编译后也会生成一个elf格式的可执行程序，叫vmlinux或vmlinuz，这个就是**原始的未经任何处理加工的原版内核elf文件**；嵌入式系统部署时烧录的一般不是这个vmlinuz/vmlinux，而是要用objcopy工具去制作成烧录镜像格式（就是u-boot.bin这种，但是内核没有.bin后缀），经过制作加工成烧录镜像的文件就叫Image（制作把78M大的精简成了7.5M，因此这个制作烧录镜像主要目的就是**缩减大小，节省磁盘**）。

(3) 原则上Image就可以直接被烧录到Flash上进行启动执行（类似于u-boot.bin），但是实际上并不是这么简单。实际上linux的作者们觉得Image还是太大了所以对Image进行了压缩，并且在image压缩后的文件的**前端附加了一部分解压压缩代码**。构成了一个压缩格式的镜像就叫zImage。（因为当年Image大小刚好比一张软盘（软盘有2种，1.2M的和1.44MB两种）大，为了节省1张软盘的钱于是乎设计了这种压缩Image成zImage的技术）。

(4) uboot为了启动linux内核，还发明了一种内核格式叫ulImage。ulImage是由zImage加工得到的，uboot中有一个工具，可以将zImage加工生成ulImage。注意：ulImage不关linux内核的事，linux内核只管生成zImage即可，然后uboot中的mkimage工具再去由zImage加工生成ulImage来给uboot启动。**这个加工过程其实就是在zImage前面加上64字节的ulImage的头信息即可。**

(5) 原则上uboot启动时应该给他ulImage格式的内核镜像，但是实际上uboot中也可以支持zImage，是否支持就看x210_sd.h中是否定义了LINUX_ZIMAGE_MAGIC这个宏。所以大家可以看出：有些uboot是支持zImage启动的，有些则不支持。但是**所有的uboot肯定都支持ulImage启动**。

(6) 如果直接在kernel底下去make ulImage会提供mkimage command not found。解决方案是去uboot/tools下cp mkimage /usr/local/bin/，复制mkimage工具到系统目录下。再去make ulImage即可。

通过上面的介绍我们了解了内核镜像的各种格式，如果通过uboot启动内核，**Linux必须为ulImage格式**。

代码段，数据段，bss段，堆，栈等

代码段：代码段通常用来存放程序**执行代码**的一块区域。这部分区域的大小在程序运行前就已经确定了，通常这块内存区域属于**只读**，有些架构也允许可写，在代码段中也有可能包含以下只读的常数变量，例如字符串常量等。程序段为程序代码在内存中映射一个程序可以在内存中有多个副本。

数据段：数据段通常用来存放程序中**已初始化的全局变量和已初始化为非0的静态变量**的一块内存区域，属于静态内存分配。直观理解就是C语言程序中的全局变量（注意：**全局变量才算是程序的数据，局部变量不算程序的数据，只能算是函数的数据**）

BSS段：bss段通常是指用来存放程序中**未初始化的全局变量和未初始化的静态变量或者初始化为0的静态变量**一块区域。bss英文Block started by symbol，bss属于静态内存分配。bss段的特点就是**被初始化为0**，bss段**本质上也是属于数据段**，bss段就是被初始化为0的数据段。注意区分：

数据段(.data)和bss段的区别和联系：二者本来没有本质区别，都是用来存放C程序中的全局变量的。区别在于把显示初始化为非零的全局变量存在.data段中，而把**显式初始化为0或者并未显式初始化（C语言规定未显式初始化的全局变量值默认为0）**的全局变量存在bss段。

rodata段存放的是只读数据，一般是程序里面的**只读变量（如const修饰的变量）和字符串常量**。单独设立".rodata"段有很多好处，不光是在语义上支持了C/C++的const关键字，而且操作系统在加载的时候可以将".rodata"段的属性映射成只读，这样对于这个段的任何修改操作都会作为非法操作处理，保证了程序的安全性。另外在某些嵌入式平台下，有些存储区域是采用只读存储器的，如ROM，这样将".rodata"段放在该存储区域中就可以保证程序访问存储器的正确性。

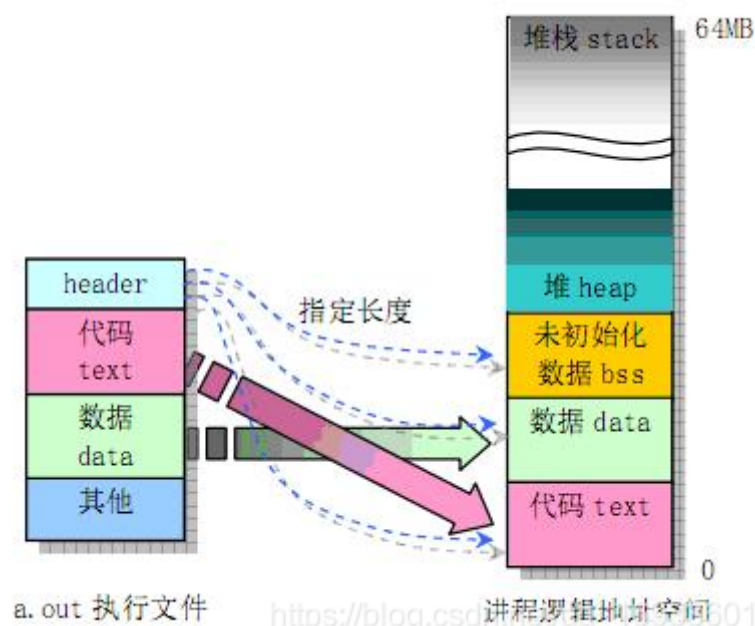
comment段存放程序的注释。

堆(heap)：堆是用来存放进程中被动态分配的内存段，它的大小**并不固定，可动态扩张或缩减**。当进程调用malloc等函数分配内存时，新分配的内存就被动态分配到堆上，当利用free等函数释放内存时，被释放的内存从堆中被剔除。

堆存放new出来的对象、栈里面所有对象都是在堆里面有指向的、假如栈里指向堆的指针被删除、堆里的对象也要释放（C++需要手动释放）、当然我们现在好面向对象程序都有'垃圾回收机制'、会定期的把堆里没用的对象清除出去。

栈（stack）：栈又称堆栈，是用户存放程序临时创建的变量，也就是我们函数{}中定义的变量，但不包括static声明的变量，static意味着在数据段中存放变量。除此之外，在函数被调用时，其参数也会被压入发起调用的进程栈中，并且待到调用结束后，函数的返回值也会被存放回栈中，由于栈的先进后出特点，所以栈特别方便用来保存、恢复调用现场。从这个意义上讲，我们可以把堆栈看成一个寄存，交换临时数据的内存区。

栈存放局部变量、临时变量、声明、返回值、指向堆对象的地址（指针）、总之存放一些小的东西、当不需要时候、栈会自动清除、比如一个加法方法里面、声明了两个int并赋值、这两个就是放在栈里面、类里面的8个基本变量常量类型、声明赋值以后都会在栈里、其他的放在堆里。注：（byte、short、int、long、char、float、double、boolean）（听说跟数据结构的栈很像、但是我学数据结构的时候其实就想到了这个栈）



```

1
2  int a = 0;    // 数据段
3
4  char *p1; // BSS段
5
6  main()
7

```

```

8  {
9
10 int b; // 栈
11
12 char s[] = "abc";// 栈
13
14 char *p2;// 栈
15
16 char *p3 = "123456"; // 123456\0在常量区，p3在栈上。
17
18 static int c =0; // BSS段
19
20 Class c1 = new Class();//new出的对象就在堆区
21
22
23 strcpy(p1, "123456"); //123456\0放在常量区，编译器可能会将它与p3所指向的"123456"优化成一个地方。
24
25 }

```

为什么堆的空间是不连续的

很多人都有这样的疑问：堆分配的内存空间到底是连续的还是不连续的，如malloc分配的内存空间是连续的吗？

问题的引出：

堆包含一个**链表来维护已用和空闲的内存块**。在堆上新分配（用 new 或者 malloc）内存是从空闲的内存块中找到一些满足要求的合适块。所以可能让人觉得只要有很多不连续的零散的小区域，只要总数达到申请的内存块，就可以分配。

但事实上是不行的，这又让人觉得是不是零散的内存块不能连接成一个大的空间，而必须要一整块连续的内存空间才能申请成功呢。

答案：

申请和释放许多小的块可能会产生如下状态：在已用块之间存在很多小的空闲块。进而申请大块内存失败，虽然空闲块的总和足够，但是**空闲的小块是零散的，不连续的**，不能满足申请的大小，这叫做“堆碎片”。

当旁边有空闲块的已用块被释放时，新的空闲块会与相连的空闲块合并成一个大的空闲块，这样就可以有效的减少“堆碎片”的产生。

堆分配的空间在**逻辑地址（虚拟地址）上是连续的**，但在**物理地址上是不连续的**（因为采用了页式内存管理,windows下有段机制、分页机制），如果逻辑地址空间上已经没有一段连续且足够大的空间，则分配内存失败。

哈佛结构和冯诺依曼结构

区别

冯诺依曼结构采用指令和数据**统一编址**，使用**同条总线**传输，CPU读取指令和数据的操作无法重叠。

哈佛结构采用指令和数据**独立编址**，使用**两条独立的总线**传输，CPU读取指令和数据的操作可以重叠。

利弊

冯诺依曼结构主要用于通用计算机领域，需要对存储器中的代码和数据频繁的进行修改，统一编址有利于**节约资源**。

哈佛结构主要用于嵌入式计算机，程序固化在硬件中，有较高的可靠性、运算速度和较大的吞吐。

程序和数据都放在内存中，且不彼此分离的结构称为冯诺依曼结构。譬如Intel的CPU均采用冯诺依曼结构。

冯诺依曼结构中程序和数据不区分的放在一起，因此安全和稳定性是个问题，好处是处理起来简单。

哈佛结构中程序（一般放在ROM、flash中）和数据（一般放在RAM中）独立分开存放，因此好处是安全和稳定性高，缺点是软件处理复杂一些（需要统一规划链接地址等）

ARM流水线技术

流水线技术通过多个功能部件并行工作来缩短程序执行时间，提高处理器核的效率和吞吐率，从而成为微处理器设计中最为重要的技术之一。**ARM7**处理器核使用了典型**三级流水线的冯·诺伊曼结构**，**ARM9**系列则采用了基于**五级流水线的哈佛结构**。通过增加流水线级数简化了流水线各级的逻辑，进一步提高了处理器的性能。

PC代表程序计数器，流水线使用三个阶段，因此指令分为三个阶段执行：1.取指（从存储器装载一条指令）；2.译码（识别将要被执行的指令）；3.执行（处理指令并将结果写回寄存器）。而R15（PC）总是指向“正在取指”的指令，而不是指向“正在执行”的指令或正在“译码”的指令。一般来说，人们习惯性约定将“**正在执行的指令作为参考点**”，称之为当前第一条指令，因此PC总是指向第三条指令。当ARM状态时，每条指令为4字节长，所以PC始终指向该指令地址加8字节的地址，即：**PC值=当前程序执行位置+8**；

ARM指令是三级流水线，取指，译指，执行，同时执行的，现在**PC指向的是正在取指的地址（下一条指令）**，那么cpu正在译指的指令地址是PC-4（假设在ARM状态下，一个指令占4个字节），**cpu正在执行的指令地址是PC-8**，也就是说PC所指向的地址和现在所执行的指令地址相差8。

当突然发生中断的时候，保存的是PC的地址（PC-8+4 = PC-4 下一条指令的地址）

这样你就知道了，如果返回的时候返回PC，那么中间就有一个指令没有执行，所以用 `SUB pc lr, lr, #4`。

Cycle		1	2	3	4	5	6	7	8	9
Operation										
ADD	F	D	E							
SUB		F	D	E						
ORR			F	D	E					
AND				F	D	E				
ORR					F	D	E			
EOR						F	D	E		

F – 取指 D – 解码 E – 执行

中断与轮询

中断向量表其实是处理器内部的概念，因为处理器除了会被外部设备中断外，其内部也可能产生异常等事件，例如在MIPS中，中断只是异常的一种。当这些事件发生时，CPU必须暂停手头上的工作，转而去处理中断或异常，**因此处理器需要知道到哪里去获得这些中断或异常的处理函数的目标地址**。中断向量表就是用来解决这个问题，其中**每一项都是一个中断或异常处理函数的入口地址**，具体来说4个字节的函数指针将指向一段汇编微码（intConnectCode）执行跳转。

外部设备的中断常常对应向量表中的某一项，这是通用框架的外部中断处理函数入口，因此在进入通用的中断处理函数之后，系统必须知道正在处理的中断是哪一个设备产生的，而这正是**由软件中断号irq定的决定**。中断向量表的内容是由操作系统在初始化阶段来填写，对于外部中断，操作系统负责实现一个通用的外部中断处理函数，然后把这个函数的入口地址放到中断向量表中的对应位置。用户注册设备驱动ISR，实际上就是挂接到中断向量表中，覆盖某一项的默认处理实现特化。

中断 本质上是轮询，中断信号触发中断请求，cpu查询到中断请求之后根据当前中断寄存器配置决定是否进行中断响应。但是一般情况下的轮询是程序设定固定周期让cpu进行状态查询，是由指令控制的，没记错的话好像某本unix书里称轮询为软中断。当有设备事件产生以后,设置CPU的中断引脚触发中断,而CPU在每个时钟周期都查询是否有中断发生,在有中断时就转到中断处理程序去执行.这是典型的轮询。

最近在看Linux内核，看到中断和异常这块，想起来了以前体系结构课上讲到的关于轮询和中断的概念。二者都是操作系统和硬件交互的方式。开始时是轮询的方法，内核（通常是驱动）需要自己去轮询硬件看硬件是否有事件，如果有则处理，没有则继续轮询，直到被操作系统挂起。这显然是很低效的。于是有了中断这种更好的方案。

但我认为**中断本质上仍然是轮询**，只不过是把轮询操作放在了硬件层，提高了轮询的效率。中断是如何实现的？通过IRQ(Interrupt ReQuest)，IRQ其实就是一个硬件的轮询系统，它连接了所有的硬件，每个硬件分配一个独立的IRQ编号。硬件有消息就会放在它自己的IRQ里。然后CPU会在每个指令周期结束时查看IRQ里是否有消息，如果有就触发中断给内核去处理。

ARM有几种工作模式

1) 用户模式(USR)：

用户模式是用户程序的工作模式，它运行在操作系统的用户态，它没有权限去操作其它硬件资源，只能执行处理自己的数据，也**不能切换到其它模式下**，要想访问硬件资源或切换到其它模式只能通过软中断或产生异常。

(2) 系统模式(SYS)：

系统模式是特权模式，不受用户模式的限制。用户模式和系统模式共用一套寄存器，操作系统在该模式下可以方便的访问用户模式的寄存器，而且操作系统的一些特权任务可以使用这个模式访问一些受控的资源。

说明：用户模式与系统模式两者使用相同的寄存器，都没有SPSR（Saved Program Statement Register，已保存程序状态寄存器），但系统模式比用户模式有更高的权限，可以访问所有系统资源。

(3) 一般中断模式(IRQ)：

一般中断模式也叫普通中断模式，用于处理一般的中断请求，通常在硬件产生中断信号之后自动进入该模式，该模式为特权模式，可以自由访问系统硬件资源。

(4) 快速中断模式(FIQ)：

快速中断模式是相对一般中断模式而言的，它是用来处理对时间要求比较紧急的中断请求，主要用于高速数据传输及通道处理中。（快中断有许多（R8-R14）自己的专用寄存器，发生中断时，使用自己的寄存器就避免了保存和恢复某些寄存器。如果异常中断处理程序中使用它自己的物理寄存器之外的其他寄存器，异常中断处理程序必须保存和恢复这些寄存器。）

（5）管理模式（SVC）：

管理模式是**CPU上电后默认模式**，因此在该模式下主要用来做**系统的初始化**，软中断处理也在该模式下。当用户模式下的用户程序请求使用硬件资源时，通过软件中断进入该模式。

说明：系统复位或开机、软中断时进入到SVC模式下。

（6）终止模式(ABT)：

中止模式用于支持虚拟内存或存储器保护，当用户程序访问**非法地址**，**没有权限读取的内存地址**时，会进入该模式，linux下编程时经常出现的segment fault通常都是在该模式下抛出返回的。

（7）未定义模式(UND)：

未定义模式用于支持硬件协处理器的软件仿真，CPU在指令的译码阶段不能识别该指令操作时，会进入未定义模式。

说明：

1、用户模式外，其它6种模式称为特权模式。所谓特权模式，即具有如下权利：

a.MRS（把状态寄存器的内容放到通用寄存器）；

b.MSR（把通用寄存器的内容放到状态寄存器中）。

由于状态寄存器中的内容不能够改变，因此要先把内容复制到通用寄存器中，然后修改通用寄存器中的内容，再把通用寄存器中的内容复制给状态寄存器中，即可完成“修改状态寄存器”的任务。

2、剩下的六种模式中除去系统模式外，统称为异常模式。

快中断和一般中断

IRQ，FIQ定义：这就是个普通中断，当我们程序定义了该中断，并且在程序运行的时候产生了IRQ中断，则此时的芯片是这样运行的-----中断处理器把利用IRQ请求线来高速ARM，ARM就知道有个IRQ中断来了，然后ARM切换到IRQ模式运行。类似的如果该中断设置为FIQ，那么当该中断产生的时候，中断处理器通过FIQ请求线告诉ARM，ARM就知道有个FIQ中断来了，然后切换到FIQ模式运行。

那么问题来了，为何要有这两个中断，或者说为何分成这两种中断？原因很简单，速度不同，FIQ也就是快速中断的速度快，那么问题又来了，**为何就FIQ速度快**？

原因：

1：ARM的FIQ模式提供了**更多的banked寄存器**，**r8到r14还有SPSR**，而IRQ模式就没有那么多，R8,R9,R10,R11,R12对应的banked的寄存器就没有，这就意味着在ARM的IRQ模式下，中断处理程序**自己要保存R8到R12这几个寄存器**，然后**退出中断处理时程序要恢复这几个寄存器**，而FIQ模式由于这几个寄存器都有banked寄存器，模式**切换时CPU自动保存这些值到banked寄存器**，退出FIQ模式时自动恢复，所以这个过程FIQ比IRQ快.不要小看这几个寄存器，ARM在编译的时候，如果你FIQ中断处理程序足够用这几个独立的寄存器来运作，它就**不会进行通用寄存器的压栈**，这样也省了一些时间。

2：FIQ比IRQ有**更高优先级**，如果FIQ和IRQ同时产生，那么FIQ先处理。

3：在symbian系统里，当CPU处于FIQ模式处理FIQ中断的过程中，预取指令异常，未定义指令异常，软件中断全被禁止，所有的中断被屏蔽。所以FIQ就会很快执行，不会被其他异常或者中断打断，所以它又比IRQ快了。而IRQ不一样，当ARM处理IRQ模式处理IRQ中断时，如果来了一个FIQ中断请求，那正在执行的IRQ中断处理程序会被抢断，ARM切换到FIQ模式去执行这个FIQ，所以FIQ比IRQ快多了。

4：另外FIQ的入口地址是0x1c,IRQ的入口地址是0x18。写过完整汇编系统的都比较明白这点的差别，18只能放一条指令，为了不与1C处的FIQ冲突，**这个地方只能跳转**，而FIQ不一样，1C以后没有任何中断向量表了，这样**可以直接在1C处放FIQ的中断处理程序**，由于跳转的范围限制，至少少了一条跳转指令。

Linux操作系统与ARM工作模式

首先，ARM开发板在刚上电或复位后都会首先进入**SVC即管理模式**，此时，程序计数器R15PC值会被赋为0x0000 0000；bootloader就是在此模式下，位于0x0000 0000的NOR FLASH或SRAM中装载的，因此、开机或重启后bootloader会被首先执行。

接着，bootloader引导Linux内核，此时、Linux内核一样运行在ARM的SVC即管理模式下；当内核启动完毕、准备进入用户态init进程时，内核将ARM的当前程序状态CPSR寄存器M[4:0]设置为10000，进而用户态程序只能运行在ARM的用户模式。

由于ARM用户模式下对资源的访问受限，因此、可以达到保护Linux操作系统内核的目的。

需要强调的是：**Linux内核态是从ARM的SVC即管理模式下启动的**，但在某些情况下、如：硬件中断、程序异常（被动）等情况下进入ARM的其他特权模式，这时仍然可以进入内核态（因为就是可以操作内核了）；同样，Linux用户态是从ARM用户模式启动的，但当进入ARM系统模式时、仍然可以操作Linux用户态程序（进入用户态，如init进程的启动过程）。

即：**Linux内核从ARM的SVC模式下启动，但内核态不仅仅指ARM的SVC模式（还包括可以访问内核空间的所有ARM模式）**；**Linux用户程序从ARM的用户模式启动，但用户态不仅仅指ARM的用户模式。**

什么是PLL（锁相环）

简单来说，输入时钟的存在是作为“参考源”。锁相环不是为了单纯产生同频同相信号，而是一般集成进某种“频率综合电路”，产生一个不同频。但锁相的信号。有点绕，打个比方：某参考晶振10Mhz，频率综合器A使用该参考源产生了900Mhz时钟，而频率综合器B产生了1Ghz时钟。虽然两路频率不同，但**由于使用的通一个参考源，他们俩仍然是同源信号**。相反，如果不同源，那么即便同频他们也不可能一致，因为世界上没有两个钟能做到完全一样，总有微弱的频差，导致相位漂移。在很多现实应用中要求同源时钟的场合，所以锁相环被广泛应用。锁相环的另外一项衍生应用是相干解调，可以自己查查相关资料。

用PLL是为了生产一系列跟输入时钟有一定关联的时钟，不同的输出供给系统不同的模块，但每个模块的时钟我们都是可以预知的。

NAND FLASH 和NOR FLASH异同

不相同点

类别	NOR	NAND
读	快 像访问SRAM一样，可以随机访问任意地址的数据；如：unsighed short *pwAddr = (unsighed short *)0x02;unisignded short wVal;wVal = *pwAddr	快,有严格的时序要求，需要通过一个函数才能读取数据，先发送读命令->发送地址->判断nandflash是否就绪->读取一页数据读命令、发送地址、判断状态、读数据都是通过操作寄存器实现的,如数据寄存器NFDATA
写	慢，写之前需要擦除，因为写只能是1->0,擦除可以使0->1	快，写之前需要擦除，因为写只能是1->0,擦除可以使0->1
擦除	非常慢（5S）	快（3ms）
XIP	代码可以直接在NOR FLASH上运行	NO
可靠性	比较高，位反转的比例小于NAND FLASH的10%	比较低，位反转比较常见，必须有校验措施
接口	与RAM接口相同，地址和数据总线分开	I/O接口
可擦除次数	10000~100000	100000~1000000
容量	小，1MB~32MB	大，16MB~512MB
主要用途	常用于保存代码和关键数据	用于保存数据
价格	高	低

注意：nandflash和norflash的0地址是不冲突的，norflash占用BANK地址，而nandflash不占用BANK地址，它的0地址是内部的。

相同点

1	写之前都要先擦除，因为写操作只能使1->0，而擦除动作是为了把所有位都变1
2	擦除单元都以块为单位

你是怎么接收、发送串口数据的

这个问题其实比较宽泛，一般经验少的会说使用查询方式，但是查询方式效率是非常低下的，所以如果你只能回答这个，100分的题你只能得个30分。如果你说用中断方式，那么请问你具体是如何处理的？如果你回答说一个字节接收完之后再接收下一个字节，那么可以得个50分。

紧接着又问你，你是怎么接收一帧数据的（这个其实不应该由面试官问，而是由你自己补充全面），如果你说采用帧头、帧尾判断的方式接收的，那么这道题还是给你50分，但是你说用空闲中断，那么70分以上，如果你说用DMA+空闲中断的方式接收的，那么90分以上（这是我认为最好的方式了，可能会有其他更好的方式也说不定）。

那么现在说说空闲中断，为什么你说了空闲中断之后，一下子从不及格到及格了？

空闲中断，顾名思义，就是串口空闲后产生的中断。我们都知道，数据一般是按照数据帧来发送的，即一个数据帧一个数据帧的发送，如果两帧发送之间能**间隔一段时间**，那么在接收端就可以产生空闲中断（关于这个空闲中断，以后可能会专门写一篇笔记介绍），有了空闲中断有什么好处？

可以接收不定长数据（这是最明显的好处）。

不需要复杂的帧格式（比如帧头、帧尾可以不要）。

一个数据帧接收错误，不会影响到下一帧数据的接收。

有了空闲中断，可谓好处多多（有的单片机没有空闲中断，那就没办法，当然也可以舍弃一个定时器资源来获得空闲中断的效果），所以当了解到这个之后，就一直使用这种方式接收了。

但是空闲中断虽好，如果你每接收一个字节都要**CPU干预**，还是效率太低，那么这时候就得配合DMA了。

怎么配合？比如说你一个数据帧的最大长度是10个字节，设置串口接收缓存区为20个字节，那么你可以设置DMA传输长度为20，这样DMA每从串口传输一个字节，传输长度就会自减，当产生空闲中断时，只要你知道开始设置的传输长度和剩余的传输长度，那么就可以得到你已经接收的数据长度，之后你再重新设置新的接收长度即可进行下一次数据帧的接收。

如此一来，接收一个数据帧只要CPU干预一次就够了，就是在接收完数据帧的时候由空闲中断通知CPU进行后续处理即可（注意不是DMA中断），极大的减少了CPU工作时间。

有的时候，数据量很大，CPU来不及处理，那么你可以通过以下方式解决：

增加消息队列（非常好的解决方式）。

增加两帧之间的发送时间（对于实时性要求很高的可能不合适）。

前面两种方式叠加。

SPI协议

SPI协议简介

SPI接口

SPI 协议是由摩托罗拉公司提出的通讯协议(Serial Peripheral Interface)，即串行外围设备接口，是一种高速全双工的通信总线。SPI(Serial Peripheral Interface--串行外设接口)总线系统是一种同步串行外设接口，它可以使MCU与各种外围设备以串行方式进行通信以交换信息。SPI总线可直接与各个厂家生产的多种标准外围器件相连，包括FLASHRAM、网络控制器、LCD显示驱动器、A/D转换器和MCU等。该接口一般使用4条线：串行时钟线（SCLK）、主机输入/从机输出数据线MISO、主机输出/从机输入数据线MOSI和低电平有效的从机选择线NSS。

应用

SPI接口的全称是"Serial Peripheral Interface"，意为串行外围接口,是Motorola首先在其MC68HCXX系列处理器上定义的。SPI接口主要应用在EEPROM、FLASH、实时时钟、AD转换器，ADC、LCD 等设备与 MCU 间，还有数字信号处理器和数字信号解码器之间，要求通讯速率较高的场合。

SPI接口是在CPU和外围低速器件之间进行同步串行数据传输，在主器件的移位脉冲下，数据按位传输，高位在前，低位在后，为全双工通信，数据传输速度总体来说比I2C总线要快，速度可达到几Mbps。

接口信号

- (1) MOSI – 主器件数据输出，从器件数据输入
- (2) MISO – 主器件数据输入，从器件数据输出
- (3) SCLK –时钟信号，由主器件产生,最大为 $f_{PCLK}/2$ ，从模式频率最大为 $f_{CPU}/2$
- (4) NSS – 从器件使能信号，由主器件控制,有的IC会标注为CS(Chip select)

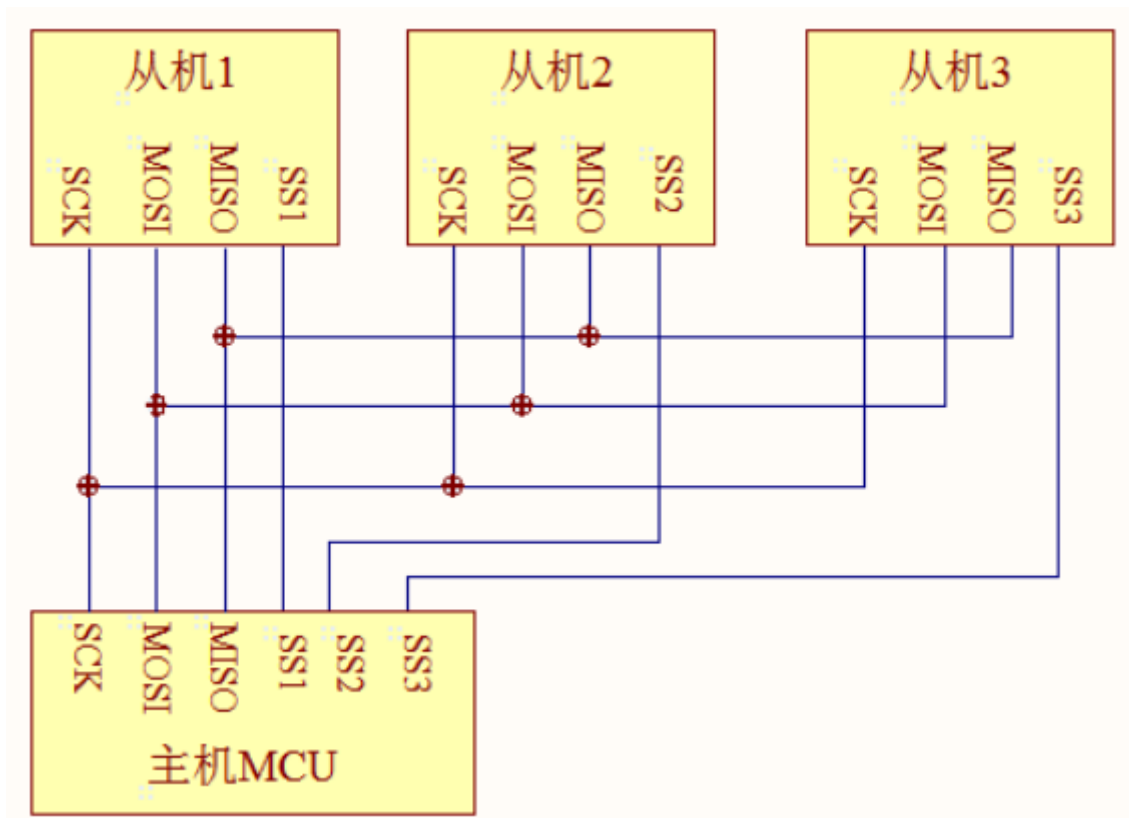
在点对点的通信中，SPI接口不需要进行寻址操作，且为全双工通信，显得简单高效。在多个从器件的系统中，每个从器件需要独立的使能信号，硬件上比I2C系统要稍微复杂一些。

SPI接口在内部硬件实际上是两个简单的移位寄存器，传输的数据为8位，在主器件产生的从器件使能信号和移位脉冲下，按位传输，高位在前，低位在后。如下图所示，在SCLK的上升沿上数据改变，同时一位数据被存入移位寄存器。

SPI通讯详解

SPI 物理层

SPI 通讯设备之间的常用连接方式见下图：



SPI 通讯使用 3 条总线及片选线，3 条总线分别为 SCK、MOSI、MISO，片选线为/S，它们的作用介绍如下：

(1) /S/ (Slave Select)：从设备选择信号线，常称为片选信号线，也称为 NSS、CS，以下用 NSS 表示。当有多个 SPI 从设备与 SPI 主机相连时，设备的其它信号线 SCK、MOSI 及 MISO 同时并联到相同的 SPI 总线上，即无论有多少个从设备，都共同只使用这 3 条总线；而每个从设备都有独立的这一条 NSS 信号线，本信号线独占主机的一个引脚，即有多少个从设备，就有多少条片选信号线。I2C 协议中通过设备地址来寻址、选中总线上的某个设备并与其进行通讯；而 SPI 协议中没有设备地址，它使用 NSS 信号线来寻址，当主机要选择从设备时，把该从设备的 NSS 信号线设置为低电平，该从设备即被选中，即片选有效，接着主机开始与被选中的从设备进行 SPI 通讯。所以 SPI 通讯以 NSS 线置低电平为开始信号，以 NSS 线被拉高作为结束信号。

(2) SCK (Serial Clock)：时钟信号线，用于通讯数据同步。它由通讯主机产生，决定了通讯的速率，不同的设备支持的最高时钟频率不一样，如 STM32 的 SPI 时钟频率最大为 $f_{pclk}/2$ ，两个设备之间通讯时，通讯速率受限于低速设备。

(3) MOSI (Master Output, Slave Input)：主设备输出/从设备输入引脚。主机的数据从这条信号线输出，从机由这条信号线读入主机发送的数据，即这条线上数据的方向为主机到从机。

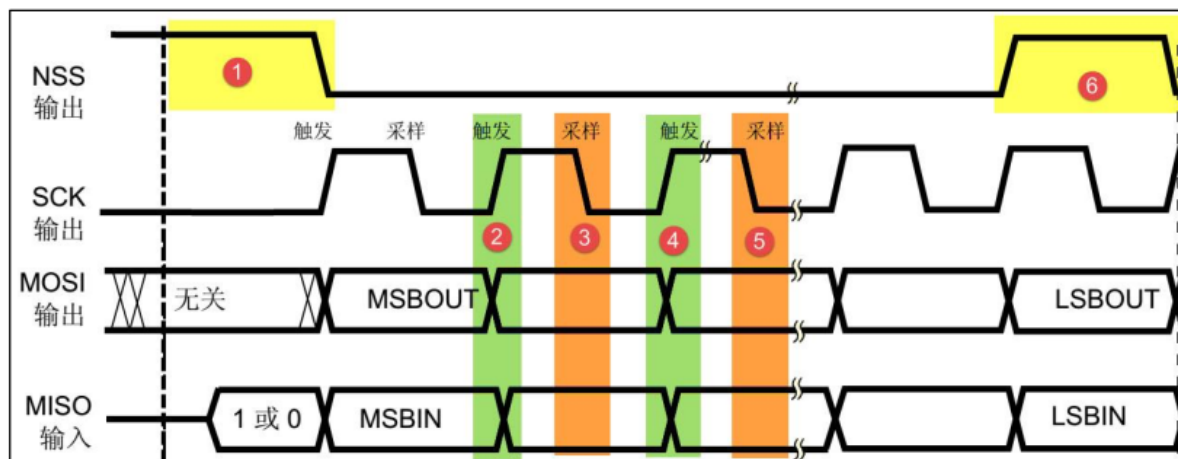
(4) MISO (Master Input, Slave Output)：主设备输入/从设备输出引脚。主机从这条信号线读入数据，从机的数据由这条信号线输出到主机，即在这条线上数据的方向为从机到主机。

协议层

与 I2C 的类似，SPI 协议定义了通讯的起始和停止信号、数据有效性、时钟同步等环节。

(1)、SPI 基本通讯过程

先看看 SPI 通讯的通讯时序，见下图：



这是一个主机的通讯时序。NSS、SCK、MOSI 信号都由主机控制产生，而 MISO 的信号由从机产生，主机通过该信号线读取从机的数据。MOSI 与 MISO 的信号只在 NSS 为低电平的时候才有效，在 SCK 的每个时钟周期 MOSI 和 MISO 传输一位数据。

(2)、通讯的起始和停止信号

在图中的标号1处，NSS 信号线由高变低，是 SPI 通讯的起始信号。NSS 是每个从机各自独占的信号线，当从机在自己的 NSS 线检测到起始信号后，就知道自己被主机选中了，开始准备与主机通讯。在图中的标号6处，NSS 信号由低变高，是 SPI 通讯的停止信号，表示本次通讯结束，从机的选中状态被取消。

(3)、数据有效性

SPI 使用 MOSI 及 MISO 信号线来传输数据，使用 SCK 信号线进行数据同步。MOSI 及 MISO 数据线在 SCK 的每个时钟周期传输一位数据，且数据输入输出是同时进行的。数据传输时，MSB 先行（高位先行）或 LSB（低位先行）先行并没有作硬性规定，但要保证两个 SPI 通讯设备之间使用同样的协定，一般都会采用上图中的 MSB 先行（高位先行）模式。

观察图中的2345标号处，MOSI及MISO的数据在SCK的上升沿期间变化输出，在SCK的下降沿时被采样。即在SCK的下降沿时刻，MOSI及MISO的数据有效，高电平时表示数据“1”，为低电平时表示数据“0”。在其它时刻，数据无效，MOSI及MISO为下一次表示数据做准备。

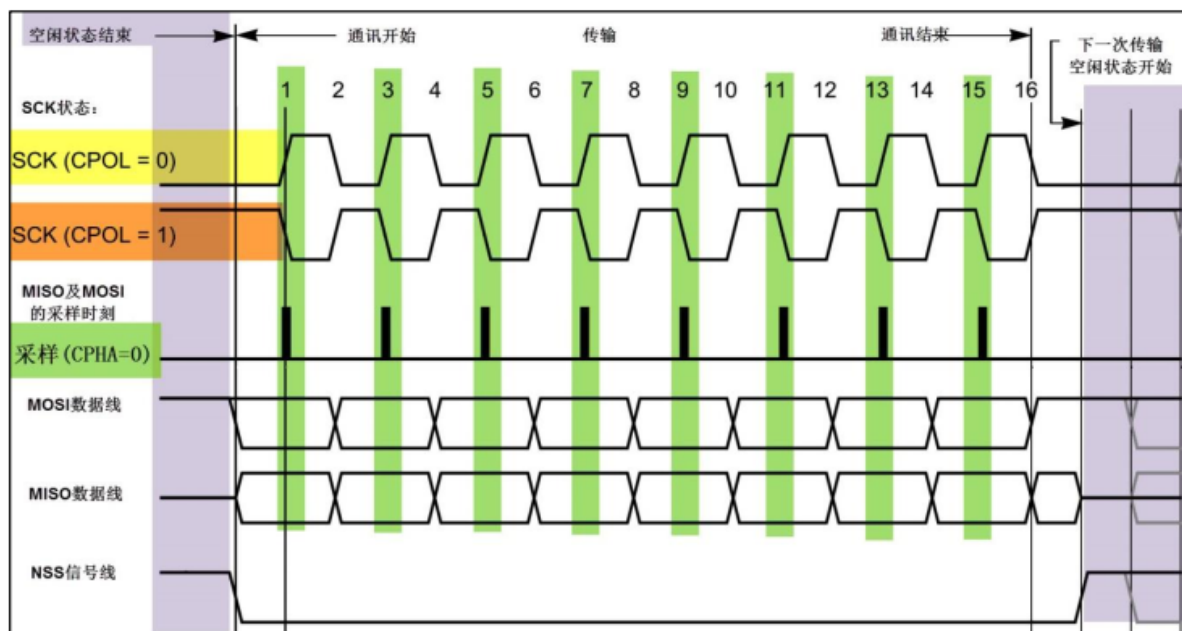
SPI每次数据传输可以8位或16位为单位，每次传输的单位数不受限制。

(4)、CPOL(时钟极性)/CPHA(时钟相位)及通讯模式

上面讲述的图中的时序只是SPI中的其中一种通讯模式，SPI一共有四种通讯模式，它们的主要区别是总线空闲时SCK的时钟状态以及数据采样时刻。为方便说明，在此引入“时钟极性CPOL”和“时钟相位CPHA”的概念。

时钟极性CPOL是指SPI通讯设备处于空闲状态时，SCK信号线的电平信号(即SPI通讯开始前、NSS线为高电平时SCK的状态)。CPOL=0时，SCK在空闲状态时为低电平，CPOL=1时，则相反。

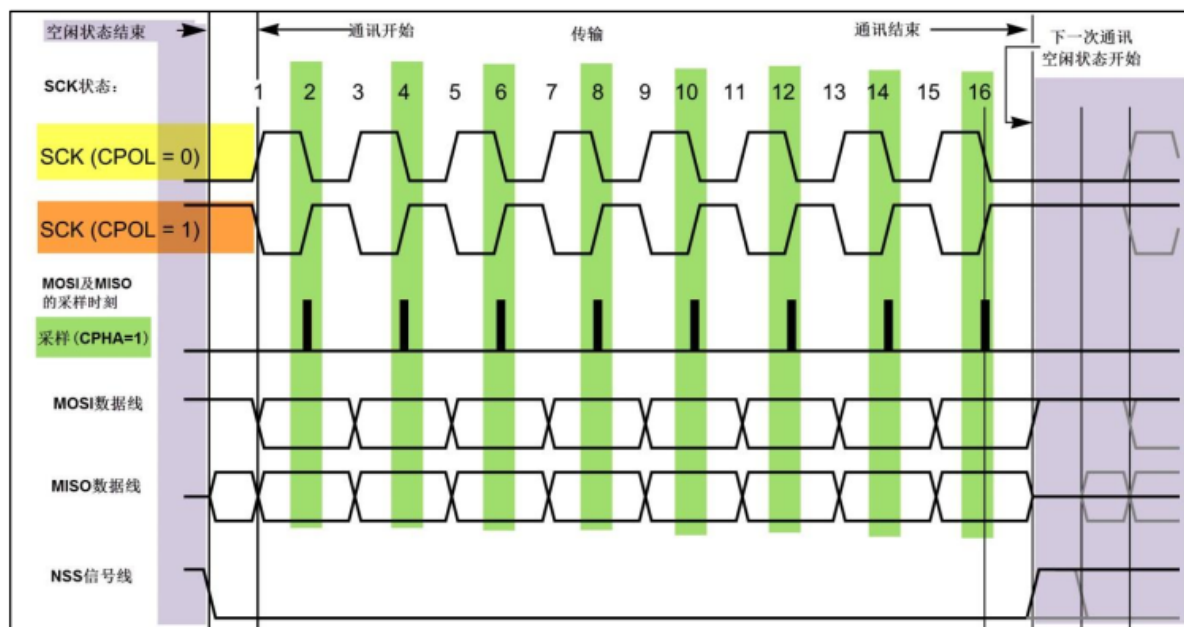
时钟相位CPHA是指数据的采样的时刻，当CPHA=0时，MOSI或MISO数据线上的信号将会在SCK时钟线的“奇数边沿”被采样。当CPHA=1时，数据线在SCK的“偶数边沿”采样。见下图：



我们来分析这个CPHA=0的时序图。首先，根据SCK在空闲状态时的电平，分为两种情况。SCK信号线在空闲状态为低电平时，CPOL=0；空闲状态为高电平时，CPOL=1。

无论CPOL=0还是=1，因为我们配置的时钟相位CPHA=0，在图中可以看到，采样时刻都是在SCK的奇数边沿。注意当CPOL=0的时候，时钟的奇数边沿是上升沿，而CPOL=1的时候，时钟的奇数边沿是下降沿。所以SPI的采样时刻不是由上升/下降沿决定的。MOSI和MISO数据线的有效信号在SCK的奇数边沿保持不变，数据信号将在SCK奇数边沿时被采样，在非采样时刻，MOSI和MISO的有效信号才发生切换。

类似地，当CPHA=1时，不受CPOL的影响，数据信号在SCK的偶数边沿被采样，见下图：



由 CPOL 及 CPHA 的不同状态，SPI 分成了四种模式，见下表，主机与从机需要工作在相同的模式下才可以正常通讯，实际中采用较多的是“模式 0”与“模式 3”

SPI 模式	CPOL	CPHA	空闲时 SCK 时钟	采样时刻
0	0	0	低电平	奇数边沿
1	0	1	低电平	偶数边沿
2	1	0	高电平	奇数边沿
3	1	1	高电平	偶数边沿

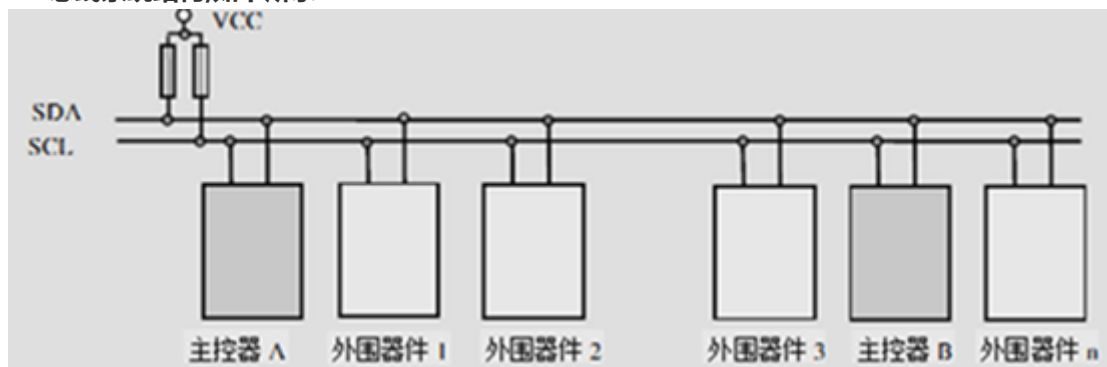
IIC协议

I2C通信介绍

它是由数据线SDA和时钟SCL构成的串行总线，可发送和接收数据,是一个多主机的**半双工通信方式**

每个挂载在总线上的器件都有个**唯一的地址**。位速在标准模式下可达 100kbit/s,在快速模式下可达 400kbit/s，在高速模式下可待3.4Mbit/s。

I2C总线系统结构,如下所示



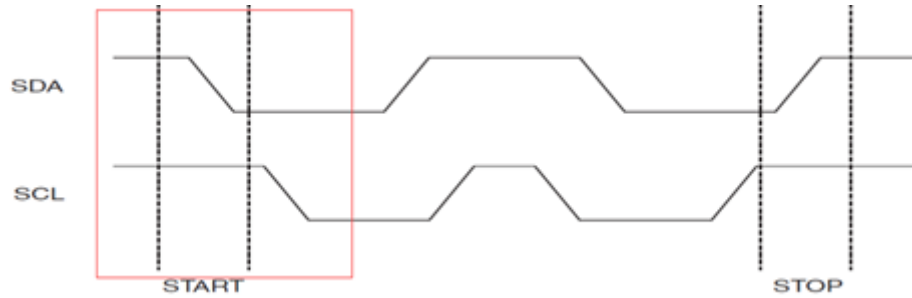
I2C时序介绍

1)空闲状态

当总线上的SDA和SCL两条信号线同时处于高电平,便是**空闲状态**,如上面的硬件图所示,当我们不传输数据时, SDA和SCL被上拉电阻拉高,即进入空闲状态

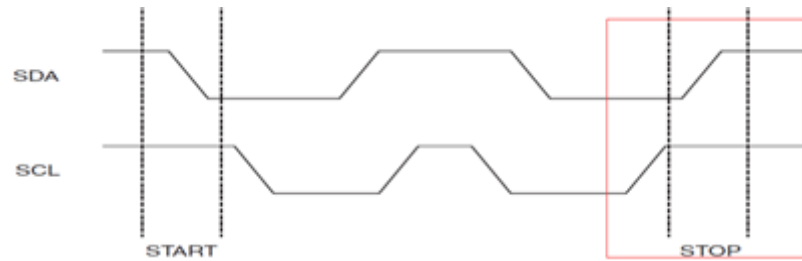
2)起始信号

当SCL为高期间，SDA由高到低的跳变；便是总线的启动信号,只能由主机发起,且在空闲状态下才能启动该信号,如下图所示:



3)停止信号

当SCL为高期间，SDA由低到高的跳变；便是总线的停止信号,表示数据已传输完成,如下图所示:



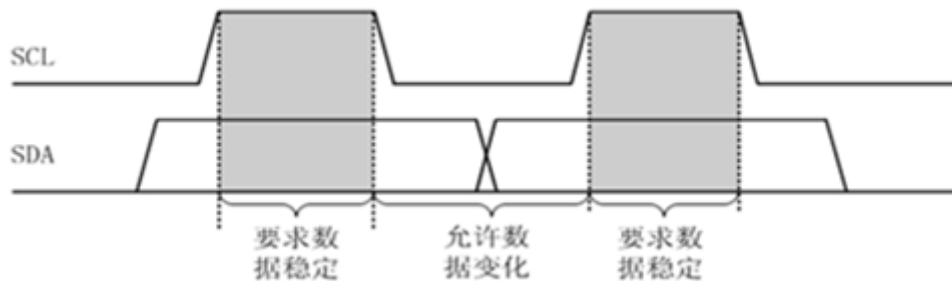
4)传输数据格式

当发了起始信号后,就开始传输数据,传输的数据格式如下图所示:

当SCL为高电平时,便会获取SDA数据值,其中SDA数据必须是稳定的(若SDA不稳定就会变成起始/停止信号)

当SCL为低电平时,便是SDA的电平变化状态

若主机在传输数据期间,需要完成其它功能(例如一个中断),可以主动拉低SCL,使I2C进入等待状态,直到处理结束再释放SCL,数据传输会继续



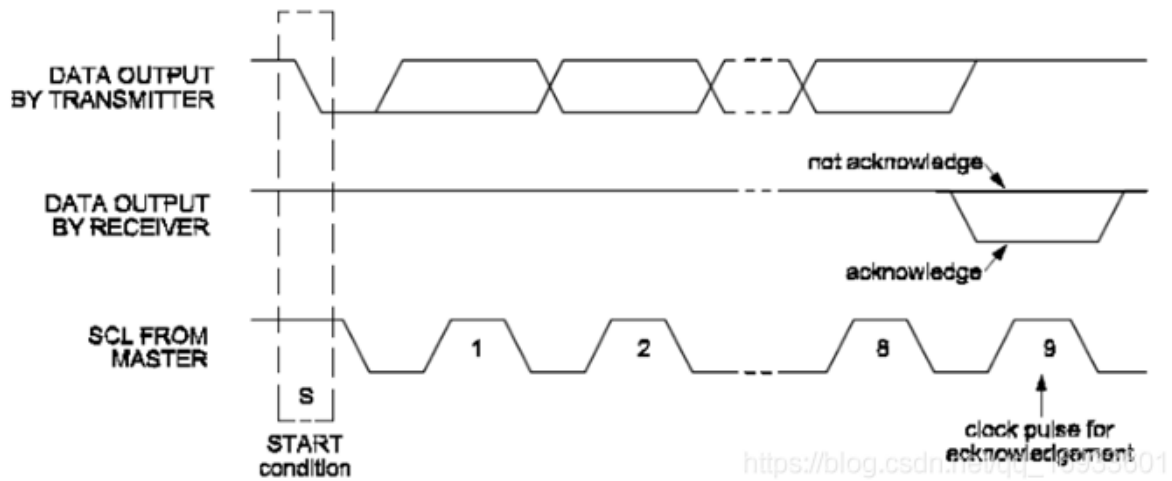
5)应答信号ACK

I2C总线上的数据都是以8位数据(字节)进行的,当发送了8个数据后,发送方会在第9个时钟脉冲期间释放SDA数据,当接收方接收该字节成功,便会输出一个ACK应答信号,当SDA为高电平,表示为非应答信号NACK,当SDA为低电平,表示为有效应答信号ACK

PS:当主机为接收方时,收到最后一个字节后,主机可以不发送ACK,直接发送停止信号来结束传输。

当从机为接收方时,没有发送ACK,则表示从机可能在忙其它事、或者不匹配地址信号和不支持多主机

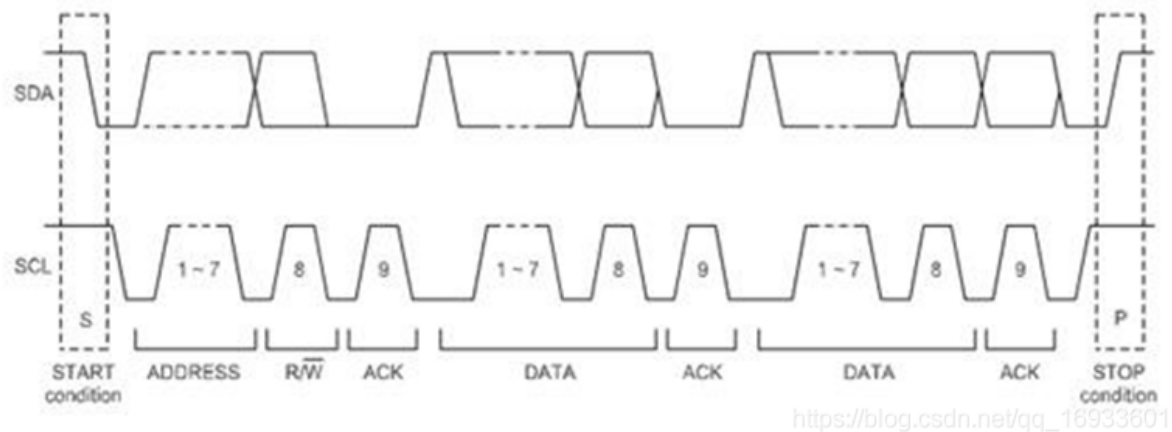
发送,主机可以发送停止信号再次发送起始信号启动新的传输



6)完整的数据传输

如下图所示,发送起始信号后,便发送一个8位的设备地址,其中第8位是对设备的读写标志,后面紧跟着的就是数据了,直到发送停止信号终止

PS:当我们第一次是读操作,然后想换成写操作时,可以再次发送一个起始信号,然后发送读的设备地址,不需要停止信号便能实现不同的地址转换



IIC传输数据的格式

1.写操作：

刚开始主芯片要发出一个**start信号**,然后发出一个 (用来确定是往哪一个芯片写数据), **方向** (读/写, 0表示写, 1表示读)。**回应**(用来确定这个设备是否存在), 然后就可以**传输数据**, 传输数据之后, 要有一个**回应信号** (确定数据是否接受完成), 然后再传输**下一个数据**。每传输一个数据, 接受方都会有一个回应信号, 数据**发送完**之后, 主芯片就会发送一个**停止信号**。

:: 白色背景：主→从

:: 灰色背景：从→主



2.读操作：

刚开始主芯片要发出一个**start信号**，然后发出一个**设备地址**(用来确定是从哪一个芯片读取数据)，**方向**(读/写，0表示写，1表示读)。**回应**(用来确定这个设备是否存在)，然后就可以**传输数据**，传输数据之后，要有一个回应信号(确定数据是否接受完成)，然后在传输**下一个数据**。每传输一个数据，接受方都会有一个**回应信号**，数据发送完之后，主芯片就会发送一个**停止信号**。

:: 白色背景：主→从

:: 灰色背景：从→主

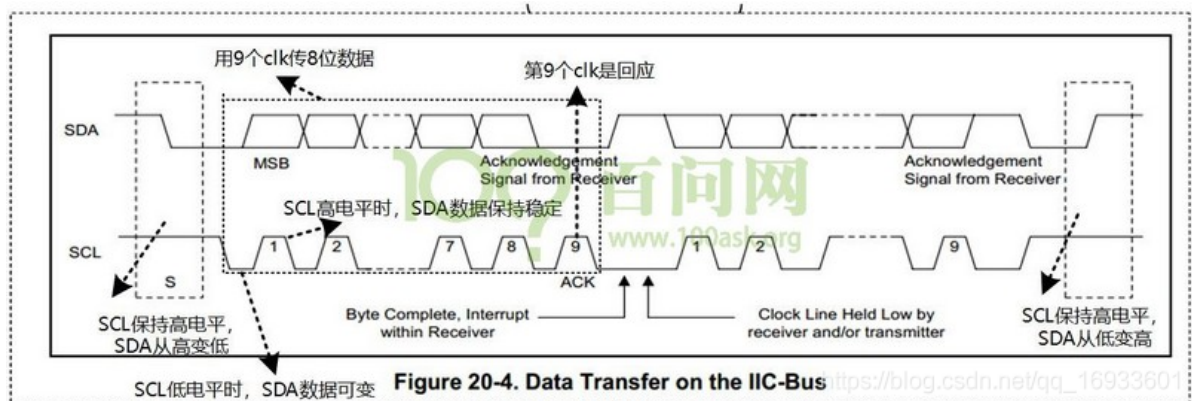


传输是以**8位为单元数据传输**的，先传输最高位(MSB)，主芯片发出start信号之后，然后发出9个时钟传输数据。

(1) 开始信号(S)：SCL为**高电平**时，SDA由**高电平向低电平跳变**，开始传送数据。

(2) 结束信号(P)：SCL为**低电平**时，SDA由**低电平向高电平跳变**，结束传送数据。

(3) 响应信号(ACK)：接收器在接收到8位数据后，在第9个时钟周期，**拉低SDA** SDA上传输的数据必须在**SCL为高电平期间保持稳定**，SDA上的数据只能在**SCL为低电平期间变化**。如图

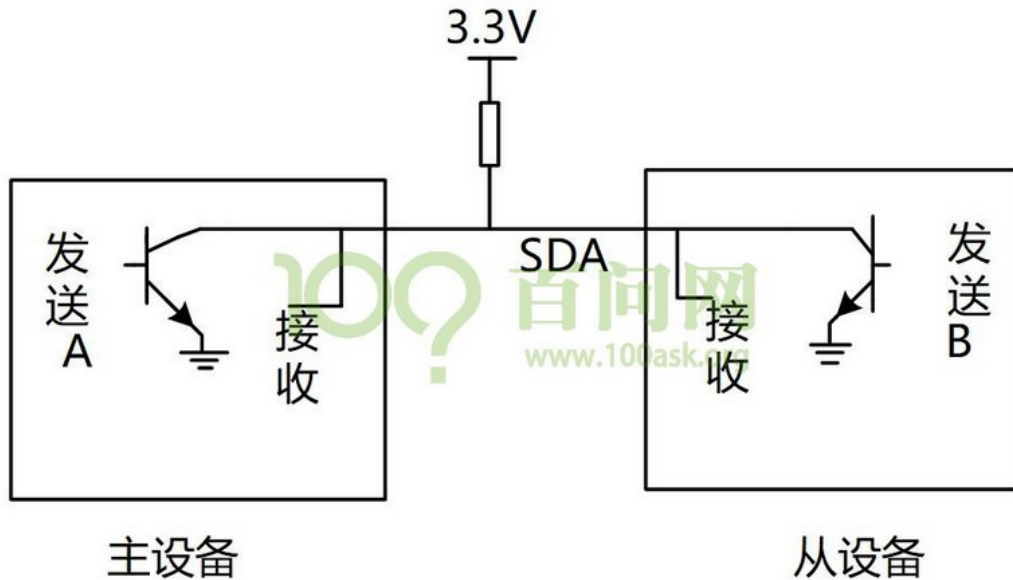


1.问题：如何在SDA上实现双向传输？

答：主芯片通过一根SDA线既可以把数据发给从设备，也可以从SDA上读取数据，连接SDA线的引脚里面**必然有两个引脚**(发送引脚/接受引脚)。

2.问题：主设备(从设备)发送数据时，从设备(主设备)的发送引脚，不影响数据的发送，怎么做到的呢？

答：里面放一个三极管，使用开极(极电集开发出去作为输出)电路，如下图



https://blog.csdn.net/qq_16933601

下面画一个真值表:

A	B	SDA
0	0	1(由上拉电阻决定)
0	1	0
1	0	0
1	1	0

https://blog.csdn.net/qq_16933601

从真值表和电路图我们可以知道，当某一个芯片不行影响SDA线时，那就不驱动这个三极管。

想输出高电平时；都不驱动(高电平就由上拉电阻决定)。

想输出低电平，就驱动三极管。

从下面的例子可以看看数据是怎么传的（实现双向传输），比如：主设备发送（8bit）给从设备

1.前8个clk

* 从设备不要影响，从设备不驱动三极管；

* 主设备决定数据；

2.第9个clk，由从设备决定数据

* 主设备不驱动三极管；

* 从设备决定数据；

从上面的例子，就可以知道，怎样在一条线上实现，双向传输的办法。这就是为什么在SDA,SCL上放上拉电阻的原因。

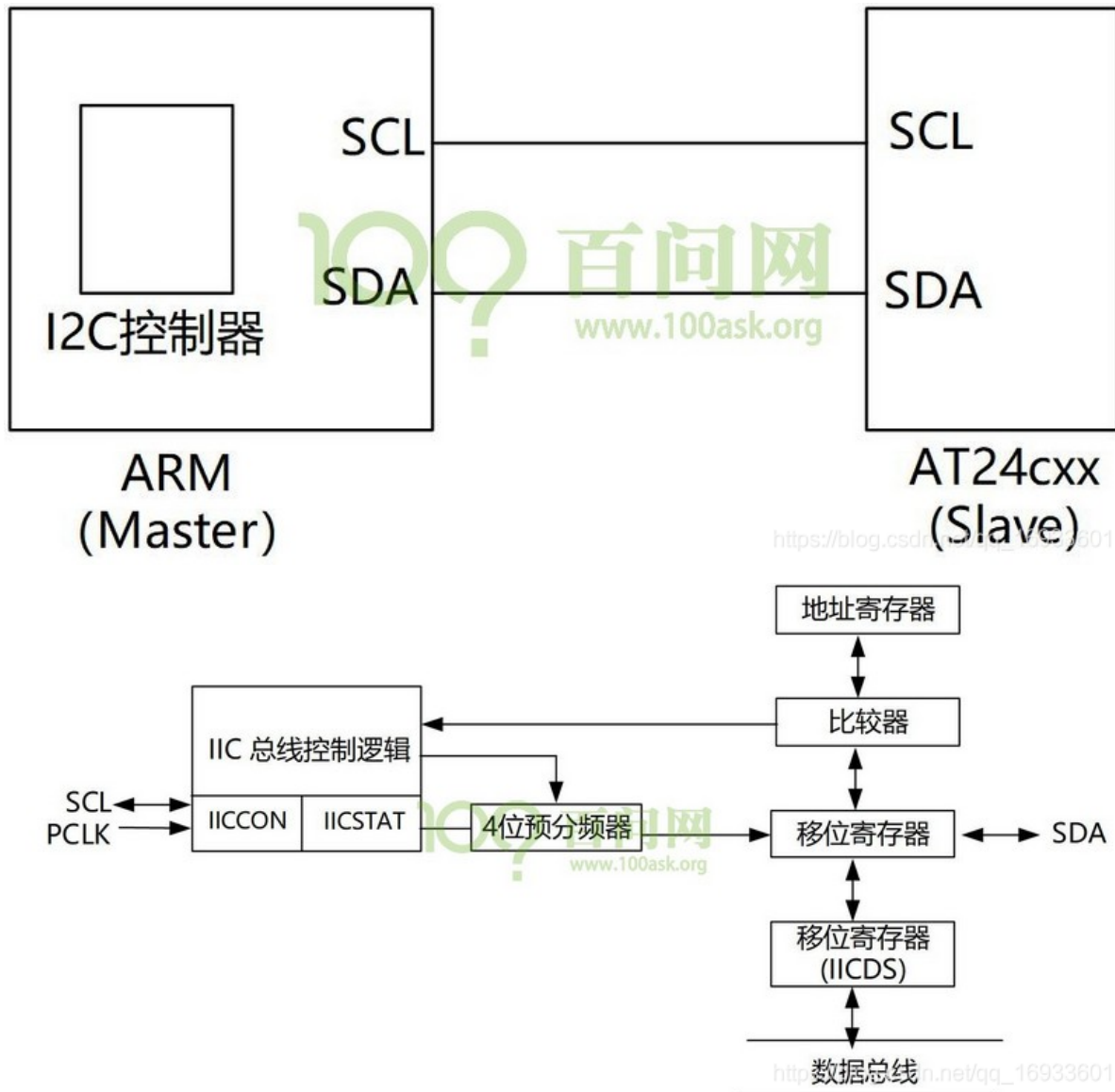
在第9个时钟之后，如果有某一方处于**繁忙状态**，它可以一直把**SCL拉低**当SCL为低电平时时候，大家都**不应该使用IIC总线**，只有当**SCL从低电平变为高电平**的时候，IIC总线才能被使用。

从前图我们也可以知道ACK信号应该是低电平。主设备不驱动三极管，如果从设备不驱动三极管，那么SDA应该是高电平，当从设备接收数据之后，发出回应信号的时候，就会驱动三极管，让SDA变为低电平。所以说：**ACK信号是低电平**。

对于IIC协议它只能规定怎么传输数据，数据什么含义它完全不能够控制，**数据的含义有从设备决定**。

在嵌入式系统里面的主控芯片一般都会有I2C控制器，要是没有可以根据I2C协议用GPIO管脚模拟，但是非常麻烦，我们要发送数据时，可以把数据放到某个寄存器，它就会自动的发出时钟，并且把数据发送给从设备，同时会等待从设备会返回回应信号。

当我们想发送一个数据的时候，要设置某个寄存器启动传输，它也一样会产生时钟，然后从设备就会把数据通过SDA传到I2C控制器里面，组装进某个寄存器里面，最终寄存器会把接收到的8位数据返回给我们的程序，从这里可以看到I2C控制器简化了I2C的操作。简短电路连接图，如图：



根据上图，我们首先设置**IICCON(来设置时钟)**，时钟源是PCLK(是50MHZ)太快了我们需要设置这个分频系数，把时钟降低，降低到我们想要的SCL,然后我们要发出start信号，我们需要设置寄存器发出start信号，之后我们需要发出数据啊，我们的程序可以把**数据写入到IICDS寄存器**，一写入就会**自动的发出时钟**，并且把这8位数据从SDA发送给从设备，数据发送之后，在第九个时钟会收到**回应信号**，可以**查询IICSTAT是否有ACK**(有ACK表示数据发送成功了)，可以继续发送数据，等发完数据之后，再来设置**IICSTAT让它发出P信号**。

在**第九个CLK**，就会产生一个**中断**，在中断处理过程中**SCL被拉为低电平**，谁都不能再使用**IIC总线**，等待中断处理完成。

怎样处理中断？

写操作：

若无**ACK**，出错，然后发出P信号结束，

:: 若有**ACK信号**，表示上一个字节成功发送出去

:: 若**仍有数据**，写入IICDS寄存器，然后**清中断**，一清中断就会**释放SCL信号**，继续发出时钟，把数据再次发送出去。

:: 若**没有数据了**，发出P信号结束。

读操作：

读到8位数时，应该回应一个**ACK信号**。

:: 还想读数据，清中断，启动传输。等它再次发生中断时，再来读取IICDS寄存器，得到数据。不想读取数据，发出P信号结束。

重点：发生中断时，我们的**IIC控制器会把SCL拉低，阻止任何设备再使用IIC总线**，清中断之后才能继续使用，这种机制就给我们中断服务程序的执行提供了时间。

读-写操作

在发送模式：

:: 1.往寄存器IICDS寄存器放入一个val值。

:: 2.发完，产生中断，并且会把 SCL拉低。

:: 3.在中断程序里，判断状态，然后往IICDS里面写入下一个数据，一旦写入下一个数据IIC继续操作，若再次发完，就会再次产生中断。

在接受模式：

:: 1.我的程序发起传输，接受数据。

:: 2.接收到数据之后，产生中断，SCL被拉低。

:: 3.中断程序里，**判断数据是否要继续接受等**，如果还有继续接受的话，再次设置，设置好之后读IICDS寄存器，一但读出来IIC。

:: 继续接受下一个数据，收到新数据之后，又会产生一个中断（就是这样循环操作）。

(1) IICCON寄存器 (Multi-masterIIC-buscontrol)

IICCON寄存器用于控制是否发出ACK信号、设置发送器的时钟、开启，i2c中断，并标识中断是否发生。它的各位含义如表：

寄存器	地址	R/W	描述	复位值
IICCON	0x54000000	R/W	IIC 总线控制寄存器	0x0x

IICCON	位	描述	初始状态
应答发生	[7]	IIC 总线应答使能位 0 = 禁止 1 = 允许 Tx 模式中，IICSDA在应答时间为空闲。 Rx 模式中，IICSDA 在应答时间为低。	0
Tx 时钟源选择	[6]	IIC 总线发送时钟预分频器的时钟源选择位 0: IICCLK = fPCLK /16 1: IICCLK = fPCLK/512	0
Tx/Rx 中断	[5]	IIC 总线 Tx/Rx 中断使能/ 禁止位 0 = 禁止 1 = 允许	0
中断挂起标志	[4]	IIC 总线 Tx/Rx 中断挂起标志。不能写 1 到此位。当此位读取到 1 时，IICSCL 限制为低并且停止 IIC。清除此位为 0 以继续操作。 0: 1) 无中断挂起（读时） 2) 清除挂起条件并且继续操作（写时） 1: 1) 中断挂起（读时） 2) N/A（写时）	0
发送时钟值	[3:0]	IIC 总线发送时钟预分频器。 IIC 总线发送时钟预分频器是由此 4 位预分频值按以下公式决定的： $Tx \text{ 时钟} = IICCLK / (IICCON[3:0] + 1)$	0

使用IICCON寄存器时，有如下注意事项。

- 1.发送模式的时钟频率由位[6]、位[3:0]联合决定,另外，IICCON[6]=0,IICCON[3:0]不能取0或10
- 2.I2C中断在以下3种情况下发生：当发出地址信息或接收到一个从机地址并且吻合时,当总线仲裁失败时，当发送/接收完一个字节的的数据（包括响应位）时。
- 3.基于SDA、SCL线上时间特性的考虑，要发送数据时，**先将数据写入IICDS寄存器，然后再清除中断。**
- 4.如果IICCON[5]=0，IICCON[14]将不能正常工作。所以，即使不使用I2C中断，也要将IICCON[5]设为1。

(2)IICSTAT寄存器(Multi-masterIIC-buscontrol/status)

IICSTAT寄存器用于选择I2C接口的工作模式，发出S信号、P信号，使能接收 / 发送功能，并标识各种状态，比如总线仲裁是否成功、作为从机时是否被寻址、是否接收到0地址、是否接收到ACK信号等。

IICSTAT寄存器的各位如表：

寄存器	地址	R/W	描述	复位值
IICSTAT	0x54000004	R/W	IIC 总线控制/状态寄存器	0x0

IICSTAT	位	描述	初始状态
模式选择	[7:6]	IIC 总线主机/从机 Tx/Rx 模式选择位 00: 从接收模式 01: 从发送模式 10: 主接收模式 11: 主发送模式	00
忙信号状态/ 起始停止条件	[5]	IIC 总线忙信号状态位 0: 读) 不忙 (读时) 写) 停止信号产生 1: 读) 忙 (读时) 写) 起始信号产生	0
串行输出	[4]	IIC 总线 Tx/Rx 中断使能/ 禁止位 0 = 禁止 1 = 允许	0
仲裁状态标志	[3]	IIC 总线仲裁过程状态标志位 0: 总线仲裁成功 1: 串行 I/O 间总线仲裁失败	0
从地址状态标志	[2]	IIC 总线从地址状态标志位 0: 发现起始/停止条件清除 1: 收到从地址与 IICADD 中地址值匹配	0
地址零状态标志	[1]	IIC 总线地址零状态标志位 0: 发现起始/停止条件清除 1: 收到从地址为 00000000b	0
最后收到位状态标志	[0]	IIC 总线最后收到位状态标志位 0: 最后收到位为 0 (已收到 ACK) 1: 最后收到位为 1 (未收到 ACK)	0

(3) IICADD寄存器(Multi-masterIIC-busaddress)

用到IICADD寄存器的位[7:11]，表示从机地址。IICADD寄存器在串行输出使能位。IICSTAT[4]为0时，才可以写入：在任何时间都可以读出。IICADD寄存器的各位如表：

寄存器	地址	R/W	描述	复位值
IICADD	0x54000008	R/W	IIC 总线地址寄存器	0x0XX

IICADD	位	描述	复位值
从地址	[7:0]	从 IIC 总线锁存的 7 位从地址。 当 IICSTAT 中串行输出使能=0 时 IICADD 为写使能。可以在任意时间读取 IICADD 的值，不用去考虑当前输出使能位 (IICSTAT) 的设置。 从地址：[7:1] 未映射：[0]	XXXXXXXX

(4) IICDS寄存器(Multi-masterIIC-busTx/Rxdatashift)

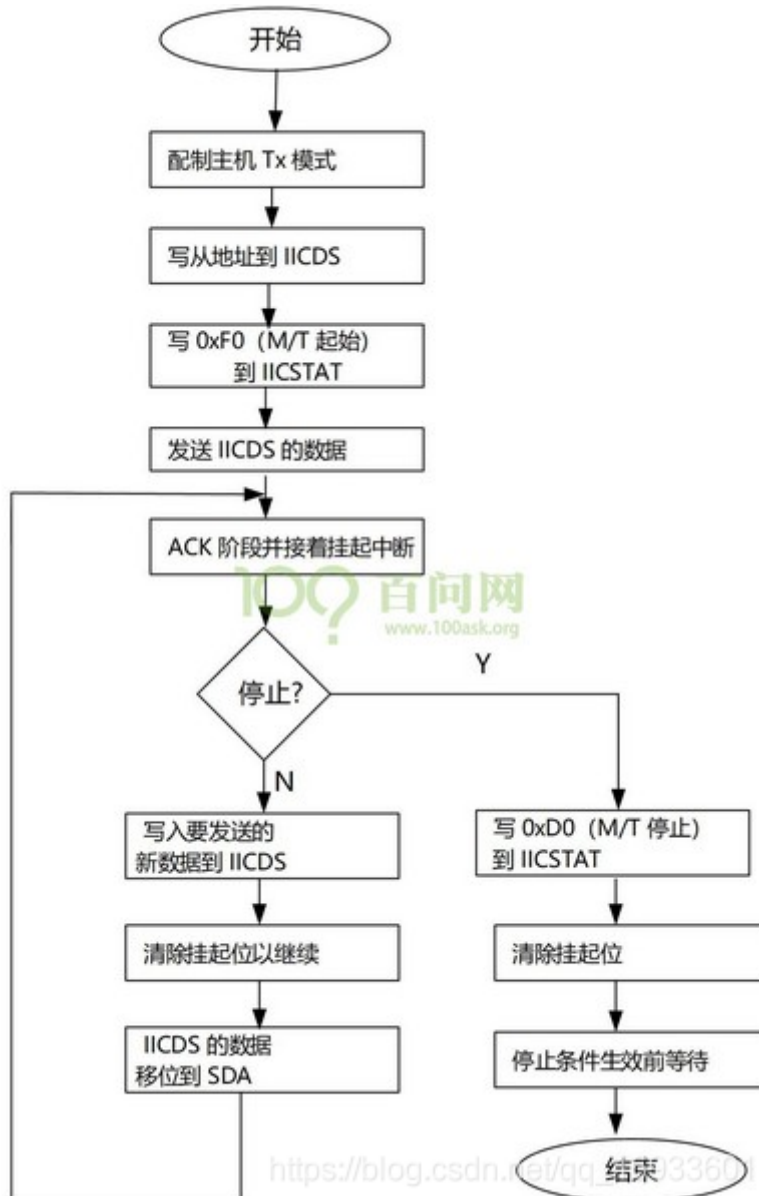
用到IICDS寄存器的位[7:0]，其中保存的是要发送或已经接收的数据。IICDS寄存器在串行输出使能位IICSTAT()1为1时，可以写入；在任何时间都可以读出。IICDS寄存器的各位如表：

寄存器	地址	R/W	描述	复位值
IICDS	0x5400000C	R/W	IIC 总线发送/接收数据移位寄存器	0x0XX

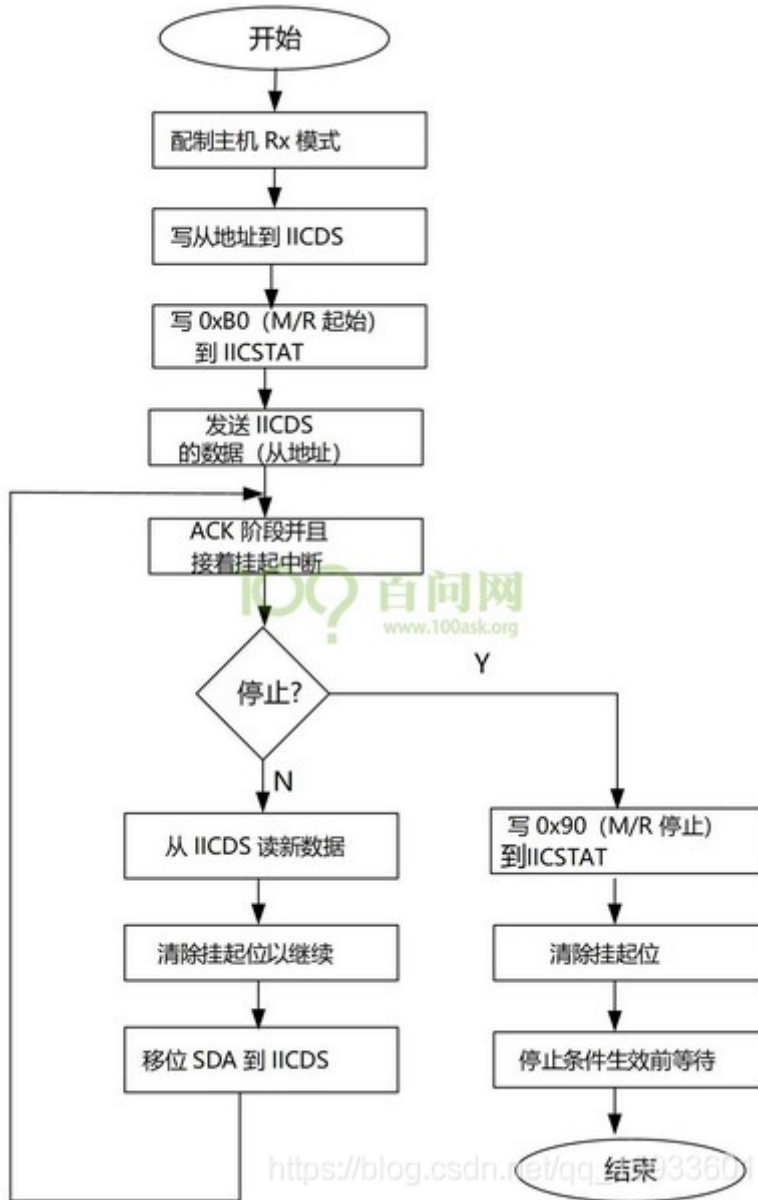
IICDS	位	描述	复位值
数据移位	[7:0]	IIC 总线 Tx/Rx 操作的 8 位数据移位寄存器。当 IICSTAT 中串行输出使能=1，IICDS 为写使能。可以在任意时间读取 IICDS 的值，不用去考虑当前输出使能位 (IICSTAT) 的设置。	XXXXXXXX

读写操作流程

主机发送器模式操作：



主机接收器模式操作:



Linux

inode和文件描述符

inode 或i节点是指对文件的索引。如一个系统，所有文件是放在磁盘或flash上，就要编个目录来说明每个文件在什么地方，有什么属性，及大小等。就像书本的目录一样，便于查找和管理。这目录是操作系统需要的，用来找文件或叫管理文件。许多操作系统都用到这个概念，如linux，某些嵌入式文件系统等。当然，对某个系统来说，有许多i节点。所以对i节点本身也是要进行管理的。

在linux中，**内核通过inode来找到每个文件**，但一个文件可以被许多用户同时打开或一个用户同时打开多次。这就有一个问题，**如何管理文件的当前位移量**，因为可能每个用户打开文件后进行的操作都不一样，这样文件位移量也不同，当然还有其他的一些问题。所以linux又搞了一个文件描述符（file descriptor）这个东西，**来分别为每一个用户服务。每个用户每次打开一个文件，就产生一个文件描述符，多次打开就产生多个文件描述符，一一对应，不管是同一个用户，还是多个用户。该文件描述符就记录了当前打开的文件的偏移量等数据。**所以一个i节点可以有0个或多个文件描述符。多个文件描述符可以对应一个i节点。

gcc -O是什么

```
gcc -O1 test.c -o test
```

使用编译优化级别1编译程序。级别为1~3，级别越大优化效果越好，但编译时间越长。

一个字符设备或者块设备都有一个主设备号和次设备号。主设备号和次设备号统称为设备号。主设备号用来表示一个特定的驱动程序。次设备号用来表示使用该驱动程序的各设备。例如一个嵌入式系统，有两个LED指示灯，LED灯需要独立的打开或者关闭。那么，可以写一个LED灯的字符设备驱动程序，可以将其主设备号注册成5号设备，次设备号分别为1和2。这里，次设备号就分别表示两个LED灯。

设备文件通常都在 /dev 目录下。如：

```
crw-rw---- 1 root uucp  4, 70 04-14 18:16 ttyS6
crw-rw---- 1 root uucp  4, 71 04-14 18:16 ttyS7
crw-rw---- 1 root tty   7,  0 08-08 18:58 vcs
crw-rw---- 1 root tty   7,  1 08-08 18:58 vcs1
crw-rw-rw- 1 root root  1,  7 08-08 18:58 full
crw-rw-rw- 1 root root  1,  3 04-14 18:16 null
```

bin文件和elf文件区别

GCC 编译出来的是ELF文件。通常gcc -o test test.c,生成的test文件就是ELF格式的，在linuxshell下输入 ./test就可以执行。

bin文件是经过压缩的可执行文件，**去掉ELF格式的东西**,是直接的内存映像的表示。**在系统没有加载操作系统的时候可以执行。**

elf (executable and link format) 文件里面包含了符号表，汇编等。

bin文件是将elf文件中的代码段，数据段，还有一些自定义的段抽取出来做成的一个内存的镜像。

在Embedded中，如果上电开始运行，没有OS系统，如果将ELF格式的文件烧写进去，包含一些ELF格式的东西，arm运行碰到这些指令，就会导致失败，如果用arm-softfloat-linux-gnu-objcopy生成纯粹的汇编 bin文件，程序就可以一步一步运行。

机器最终只认BIN，之所以有ELF格式是在有操作系统时，操作系统会根据ELF解析出代码、数据等等，最终仍是以BIN运行。由于elf文件的信息比较全，所以可以用来以单步跟踪的方式运行。关键是看loader。

Bin文件是一个没有内存修改或重定位的**纯二进制文件**，很可能它有明确的指令要加载到特定的内存地址。

ELF文件是可执行的可链接格式，它由一个符号查找表和可重定位表组成。也就是说，它可以由内核加载到任何内存地址，并且自动地将所有使用的符号调整为与该内存地址的偏移量被装入。通常，ELF文件有很多部分，比如'data', 'text', 'bss'等等。

介绍下file_operations结构体

file_operations结构体中的成员函数是**字符设备驱动程序设计的主体内容**，这些函数实际会在应用程序进行Linux的open ()、write ()、read ()、close ()等系统调用时最终被内核调用。

file_operations结构体目前已经比较庞大，具体内容如下：

```
1  /*
2  * NOTE:
3  * read, write, poll, fsync, readv, writev, unlocked_ioctl and compat_ioctl
4  * can be called without the big kernel lock held in all filesystems.
5  */
```

```

6  struct file_operations {
7      struct module *owner;
8      loff_t (*llseek) (struct file *, loff_t, int);
9      ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
10     ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
11     ssize_t (*aio_read) (struct kiocb *, const struct iovec *, unsigned
long, loff_t);
12     ssize_t (*aio_write) (struct kiocb *, const struct iovec *, unsigned
long, loff_t);
13     int (*readdir) (struct file *, void *, filldir_t);
14     unsigned int (*poll) (struct file *, struct poll_table_struct *);
15     int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned
long);
16     long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
17     long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
18     int (*mmap) (struct file *, struct vm_area_struct *);
19     int (*open) (struct inode *, struct file *);
20     int (*flush) (struct file *, fl_owner_t id);
21     int (*release) (struct inode *, struct file *);
22     int (*fsync) (struct file *, struct dentry *, int datasync);
23     int (*aio_fsync) (struct kiocb *, int datasync);
24     int (*fasync) (int, struct file *, int);
25     int (*lock) (struct file *, int, struct file_lock *);
26     ssize_t (*sendfile) (struct file *, loff_t *, size_t, read_actor_t, void
*);
27     ssize_t (*sendpage) (struct file *, struct page *, int, size_t, loff_t
*, int);
28     unsigned long (*get_unmapped_area)(struct file *, unsigned long,
unsigned long, unsigned long, unsigned long);
29     int (*check_flags)(int);
30     int (*dir_notify)(struct file *filp, unsigned long arg);
31     int (*flock) (struct file *, int, struct file_lock *);
32     ssize_t (*splice_write)(struct pipe_inode_info *, struct file *, loff_t
*, size_t, unsigned int);
33     ssize_t (*splice_read)(struct file *, loff_t *, struct pipe_inode_info
*, size_t, unsigned int);
34 };

```

llseek ()

`llseek ( )` 函数用来修改一个文件的当前读写位置，并将新位置返回，在出错时，这个函数返回一个负值。

read ()

`read ( )` 函数用来从设备中读取数据，成功时函数返回读取的字节数，出错时返回一个负值。它与用户空间应用程序中的 `ssize_t read (int fd, void*buf, size_t count)` 和 `size_t fread (void*ptr, size_t size, size_t nmemb, FILE*stream)` 对应。

write ()

`write ( )` 函数向设备发送数据，成功时该函数返回写入的字节数。如果此函数未被实现，当用户进行 `write ( )` 系统调用时，将得到-EINVAL返回值。它与用户空间应用程序中的 `ssize_t write (int fd, constvoid*buf, size_t count)` 和 `size_t fwrite (const void*ptr, size_t size, size_t nmemb, FILE*stream)` 对应。

read ()

`read ( )` 和 `write ( )` 如果返回0，则暗示 end-of-file (EOF)。

unlocked_ioctl ()

`unlocked_ioctl ( )` 提供设备相关控制命令的实现（既不是读操作，也不是写操作），当调用成功时，返回给调用程序一个非负值。它与用户空间应用程序调用的 `int fcntl (int fd, int`

`cmd, .../*arg*/` 和 `int ioctl (int d, int request, ...)` 对应。

mmap ()

`mmap ( )` 函数将设备内存映射到进程的虚拟地址空间中，如果设备驱动未实现此函数，用户进行 `mmap ( )` 系统调用时将获得-ENODEV返回值。这个函数对于帧缓冲等设备特别有意义，帧缓冲被映射到用户空间后，应用程序可以直接访问它而无须在内核和应用间进行内存复制。它与用户空间应用程序中的 `void*mmap (void*addr, size_t length, int prot, int flags, int fd, off_t offset)` 函数对应。

当用户空间调用Linux API函数 `open ( )` 打开设备文件时，设备驱动的 `open ( )` 函数最终被调用。驱动程序可以不实现这个函数，在这种情况下，设备的打开操作永远成功。与 `open ( )` 函数对应的是 `release ( )` 函数。

poll () select ()

`poll ( )` 函数一般用于询问设备是否可被非阻塞地立即读写。当询问的条件未触发时，用户空间进行 `select ( )` 和 `poll ( )` 系统调用将引起进程的阻塞。

aio_read () aio_write ()

`aio_read ( )` 和 `aio_write ( )` 函数分别对与文件描述符对应的设备进行异步读、写操作。设备实现这两个函数后，用户空间可以对该设备文件描述符执行 `sys_io_setup SYS_io_submit SYS_io_getevents`、`SYS_io_destroy` 等系统调用进行读写。

公众号：嵌入式与Linux那些事 CSDN：嵌入式与Linux那些事 来源网络，转载声明

copy_from_user open read write等常用函数总结

open

函数定义：

`int open( const char * pathname, int flags);`

`int open( const char * pathname,int flags, mode_t mode);`

参数说明：

pathname：文件的名称，可以包含（绝对和相对）路径

flags：文件打开模式

mode: 用来规定对该文件的所有者，文件的用户组及系统中其他用户的访问权限，则文件权限为：

`mode&(~umask)`

函数说明:

参数pathname 指向欲打开的文件路径字符串。下列是参数flags 所能使用的旗标:

O_RDONLY 以只读方式打开文件

O_WRONLY 以只写方式打开文件

O_RDWR 以可读写方式打开文件。上述三种旗标是互斥的，也就是不可同时使用，但可与下列的旗标利用OR(|)运算符组合。

O_CREAT 若欲打开的文件不存在则自动建立该文件。

O_EXCL 如果O_CREAT也被设置，此指令会去检查文件是否存在。文件若不存在则建立该文件，否则将导致打开文件错误。此外，若O_CREAT与O_EXCL同时设置，并且欲打开的文件为符号连接，则会打开文件失败。

O_NOCTTY 如果欲打开的文件为终端机设备时，则不会将该终端机当成进程控制终端机。

O_TRUNC 若文件存在并且以可写的方式打开时，此旗标会令文件长度清为0，而原来存于该文件的资料也会消失。

O_APPEND 当读写文件时会从文件尾开始移动，也就是所写入的数据会以附加的方式加入到文件后面。

O_NONBLOCK 以不可阻断的方式打开文件，也就是无论有无数据读取或等待，都会立即返回进程之中。

O_NDELAY 同O_NONBLOCK。

O_SYNC 以同步的方式打开文件。

O_NOFOLLOW 如果参数pathname 所指的文件为一符号连接，则会令打开文件失败。

O_DIRECTORY 如果参数pathname 所指的文件并非为一目录，则会令打开文件失败。

此为Linux2.2以后特有的旗标，以避免一些系统安全问题。参数mode 则有下列数种组合，只有在建立新文件时才会生效，此外真正建文件时的权限会受到umask值所影响，因此该文件权限应该为 (mode-umaks)。

S_IRWXU00700 权限，代表该文件所有者具有可读、可写及可执行的权限。

S_IRUSR 或S_IREAD, 00400权限，代表该文件所有者具有可读取的权限。

S_IWUSR 或S_IWRITE, 00200 权限，代表该文件所有者具有可写入的权限。

S_IXUSR 或S_IXEXEC, 00100 权限，代表该文件所有者具有可执行的权限。

S_IRWXG 00070权限，代表该文件用户组具有可读、可写及可执行的权限。

S_IRGRP 00040 权限，代表该文件用户组具有可读的权限。

S_IWGRP 00020权限，代表该文件用户组具有可写入的权限。

S_IXGRP 00010 权限，代表该文件用户组具有可执行的权限。

S_IRWXO 00007权限，代表其他用户具有可读、可写及可执行的权限。

S_IROTH 00004 权限，代表其他用户具有可读的权限

S_IWOTH 00002权限，代表其他用户具有可写入的权限。

S_IXOTH 00001 权限，代表其他用户具有可执行的权限。

返回值：

若所有欲核查的权限都通过了检查则返回0值，表示成功，只要有一个权限被禁止则返回-1。

错误代码：

EEXIST 参数pathname 所指的文件已存在，却使用了O_CREAT和O_EXCL旗标。

EACCESS 参数pathname所指的文件不符合所要求测试的权限。

EROFS 欲测试写入权限的文件存在于只读文件系统内。

EFAULT 参数pathname指针超出可存取内存空间。

EINVAL 参数mode 不正确。

ENAMETOOLONG 参数pathname太长。

ENOTDIR 参数pathname不是目录。

ENOMEM 核心内存不足。

ELOOP 参数pathname有过多符号连接问题。

EIO I/O 存取错误。

```

1  #include<unistd.h>
2  #include<sys/types.h>
3  #include<sys/stat.h>
4  #include<fcntl.h>
5  main()
6  {
7      int fd,size;
8      char s [ ]="Linux Programmer!\n",buffer[80];
9      fd=open("/tmp/temp",O_WRONLY|O_CREAT);
10     write(fd,s,sizeof(s));
11     close(fd);
12     fd=open("/tmp/temp",O_RDONLY);
13     size=read(fd,buffer,sizeof(buffer));
14     close(fd);
15     printf("%s",buffer);
16 }
```

read

函数定义：

```
ssize_t read(int fd, void * buf, size_t count);
```

函数说明：

read()会把参数fd所指的文件传送count 个字节到buf 指针所指的内存中。

返回值：

返回值为实际读取到的字节数, 如果返回0, 表示已到达文件尾或是无可读取的数据。若参数count 为0, 则read()不会有作用并返回0。

注意：

read时fd中的数据如果小于要读取的数据，就会引起阻塞。

read的用法比write较为简单，此处不叙述过多。由于笔者水平也有限，如果文中有谬误之处还恳请诸位指出，以免误导大家。

write

函数定义：

```
ssize_t write (int fd, const void * buf, size_t count);
```

函数说明：

write()会把参数buf所指的内存写入count个字节到参数放到所指的文件内。

返回值：

如果顺利write()会返回实际写入的字节数。当有错误发生时则返回-1，错误代码存入errno中。

说明：

(1) write()函数返回值一般无0，只有当如下情况发生时才会返回0：write(fp, p1+len, (strlen(p1)-len)中第三参数为0，此时write()什么也不做，只返回0。

(2) write()函数从buf写数据到fd中时，若buf中数据无法一次性读完，那么第二次读buf中数据时，其读位置指针（也就是第二个参数buf）不会自动移动，需要程序员编程控制。而不是简单的将buf首地址填入第二参数即可。如可按如下格式实现读位置移动：write(fp, p1+len, (strlen(p1)-len)。这样write第二次循环时变会从p1+len处写数据到fp, 之后的也由此类推，直至(strlen(p1)-len变为0。

以下通过一个例子具体说明write函数用法

```
1  #include <string.h>
2  #include <stdio.h>
3  #include <fcntl.h>
4  int main()
5  {
6      char *p1 = "This is a c test code";
7      volatile int len = 0;
8
9      int fp = open("/home/test.txt", O_RDWR|O_CREAT);
10     for(;;)
11     {
12         int n;
13
14         if((n=write(fp, p1+len, (strlen(p1)-len)))== 0)    //if((n=write(fp,
p1+len, 3)) == 0)
15         {                                                    //strlen(p1) = 21
16             printf("n = %d \n", n);
17             break;
18         }
19         len+=n;
```

```
20     }
21     return 0;
22 }
```

此程序中的字符串"This is a c test code"有21个字符，经笔者亲自试验，若write时每次写3个字节，虽然可以将p1中数据写到fp中，但文件test.txt中会带有很多乱码。唯一正确的做法还是将第三参数设为(strlen(p1) - len，这样当write到p1末尾时(strlen(p1) - len将会变为0，此时符合附加说明（1）中所说情况，write返回0，write结束。

（3）在write一次可以写的最大数据范围内（貌似是BUFSIZ,8192），第三参数count大小最好为buf中数据的大小，以免出现错误。（经过笔者再次试验，write一次能够写入的并不只有8192这么多，笔者尝试一次写入81920000，结果也是可以，看来其一次最大写入数据并不是8192，但内核中确实有BUFSIZ这个参数，具体指什么还有待研究）

copy_to_user

函数定义：

unsigned long copy_to_user(void *to, const void *from, unsigned long n)

参数说明：

to:目标地址（用户空间）

from:源地址（内核空间）

n:将要拷贝数据的字节数

函数说明：

从内核空间中读取数据到用户空间

返回值：

成功返回0，失败返回没有拷贝成功的数据字节数

copy_from_user

函数定义：

unsigned long copy_from_user(void *to, const void *from, unsigned long n);

参数说明：

to:目标地址（内核空间）

from:源地址（用户空间）

n:将要拷贝数据的字节数

函数说明：

从用户空间中读取数据到内核空间

返回值：

成功返回0，失败返回没有拷贝成功的数据字节数

常用Linux指令

怎么查看当前进程？怎么执行退出？怎么查看当前路径？

查看当前进程：ps

执行退出：exit

查看当前路径：pwd

Ls 命令执行什么功能？可以带哪些参数，有什么区别？

ls 执行的功能：列出指定目录中的目录，以及文件

哪些参数以及区别：a 所有文件 l 详细信息，包括大小字节数，可读可写可执行的权限等

目录创建用什么命令？创建文件用什么命令？复制文件用什么命令？

创建目录：mkdir

创建文件：典型的如 touch，vi 也可以创建文件，其实只要向一个不存在的文件输出，都会创建文件

复制文件：cp [-r] source... directory

查看文件内容有哪些命令可以使用？

vi 文件名 #编辑方式查看，可修改

cat 文件名 #显示全部文件内容

more 文件名 #分页显示文件内容

less 文件名 #与 more 相似，更好的是可以往前翻页

tail 文件名 #仅查看尾部，还可以指定行数

head 文件名 #仅查看头部,还可以指定行数

怎么向屏幕输出带空格的字符串，比如“hello world”？

向屏幕输出带空格的字符串:echo hello world

移动文件用哪个命令？改名用哪个命令？

mv

mv

删除文件用哪个命令？如果需要连目录及目录下文件一块删除呢？删除空文件夹用什么命令？

rm -r

rmdir

查找文件内容

以递归的方式查找符合条件的文件。例如，查找指定目录/etc/acpi 及其子目录（如果存在子目录的话）下所有文件中包含字符串"update"的文件，并打印出该字符串所在行的内容

```
grep -r update /etc/acpi
```

查找文件

```
find path -option [ -print ] [ -exec -ok command ] {} \;
```

find 路径 -f (一般文件)

将目前目录及其子目录下所有延伸档名是 c 的文件列出来。

```
find . -name "*.c"
```

cat命令

把 textfile1 的文档内容加上行号后输入 textfile2 这个文档里：

```
cat -n textfile1 > textfile2
```

GCC常用指令

预处理

```
1 gcc -E test.c -o test.i 或 gcc -E test.c
```

可以输出test.i文件中存放着test.c经预处理之后的代码。打开test.i文件，看一看，就明白了。后面那条指令，是直接在命令行窗口中输出预处理后的代码。

gcc的-E选项，可以让编译器在预处理后停止，并输出预处理结果。在本例中，预处理结果就是将stdio.h 文件中的内容插入到test.c中了。

编译为汇编代码(Compilation)

```
1 gcc -S test.i -o test.s
```

汇编(Assembly)

```
1 gcc -c test.s -o test.o
```

对于上一小节中生成的汇编代码文件test.s，gas汇编器负责将其编译为目标文件

连接(Linking)

```
1 gcc test.o -o test
```

gcc连接器是gas提供的，负责将程序的目标文件与所需的所有附加的目标文件连接起来，最终生成可执行文件。**附加的目标文件包括静态连接库和动态连接库。**

对于上一小节中生成的test.o，将其与C标准输入输出库进行连接，最终生成程序test

一步到位编译

```
1 gcc test.c -o test
```

多文件编译

```
1 gcc test1.c test2.c -o test
```

通常整个程序是由多个源文件组成的，相应地也就形成了多个编译单元，使用GCC能够很好地管理这些编译单元。假设有一个由test1.c和 test2.c两个源文件组成的程序，为了对它们进行编译，并最终生成可执行程序test

忽略警告

```
1 gcc -Werror test.c -o test
```

在编译程序时带上-Werror选项，那么GCC会在所有产生警告的地方停止编译

GDB调试命令

编译时生成debug有关的程序信息(供gdb使用)

```
1 gcc -g test1.c
```

开始启动调试

```
1 gdb a.out
```

显示当前的代码：l

```
(gdb) l
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <string.h>
4  int main(void)
5  {
6      int i = 0;
7      char buf[102];
8      memset(buf,0,sizeof(buf));
9      char *r = "-/|-\\";
10     while(i<=100)
11     {
12         printf("\033[33m[%-100s][%d%%][%c]\033[0m\r",buf,i,r[i%5]);
13         fflush(stdout);
14         buf[i++] = '#';
15         usleep(100000);
16     }
17     printf("\n");
18 }
19
(gdb) l
11     {
12         printf("\033[33m[%-100s][%d%%][%c]\033[0m\r",buf,i,r[i%5]);
13         fflush(stdout);
14         buf[i++] = '#';
15         usleep(100000);
16     }
17     printf("\n");
18 }
19
(gdb) l
```

显示当前程序的代码，一次只显示10行

接着上一次的代码，再显示10行

https://blog.csdn.net/qq_16933601

在某一行打断点：b 行号

```
(gdb) l
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <string.h>
4  int main(void)
5  {
6      int i = 0;
7      char buf[102];
8      memset(buf,0,sizeof(buf));
9      char *r = "-/|-\\";
10     while(i<=100)
11     {
12         printf("\033[33m[%-100s][%d%%][%c]\033[0m\r",buf,i,r[i%5]);
13         fflush(stdout);
14         buf[i++] = '#';
15         usleep(100000);
16     }
17     printf("\n");
18 }
19
(gdb) b 10
Breakpoint 1 at 0x80484d9: file test.c, line 10.
(gdb) l
```

在第10行打断点 (while(i <= 100))

https://blog.csdn.net/qq_16933601

开始运行程序：r (run)

```
(gdb) r
Starting program: /home/xj/day8.7/a.out

Breakpoint 1, main () at test.c:10
10         while(i<=100)
Missing separate debuginfos, use: debuginfo-install glibc-2.12-1.132.el6.i686
(gdb) l
```

https://blog.csdn.net/qq_37941471

逐步执行：s (step),类似于VS中的F11

```

10         while(i<=100)
(gdb) l
11         {
12             printf("\033[33m[%-100s][%d%][%c]\033[0m\r",buf,i,r[i%5]);
13             fflush(stdout);
14             buf[i++] = '#';
15             usleep(100000);
16         }
17         printf("\n");
18     }
19
(gdb) b 10
Breakpoint 1 at 0x80484d9: file test.c, line 10.
(gdb) r
Starting program: /home/xj/day8.7/a.out

Breakpoint 1, main () at test.c:10
10     while(i<=100)
Missing separate debuginfos, use: debuginfo-install glibc-2.12-1.132.el6.i686
(gdb) s
12     printf("\033[33m[%-100s][%d%][%c]\033[0m\r",buf,i,r[i%5]);
(gdb)

```

重复上条命令：不输入任何命令，直接enter

```

(gdb) l
11         {
12             printf("\033[33m[%-100s][%d%][%c]\033[0m\r",buf,i,r[i%5]);
13             fflush(stdout);
14             buf[i++] = '#';
15             usleep(100000);
16         }
17         printf("\n");
18     }
19
(gdb) b 10
Breakpoint 1 at 0x80484d9: file test.c, line 10.
(gdb) r
Starting program: /home/xj/day8.7/a.out

Breakpoint 1, main () at test.c:10
10     while(i<=100)
Missing separate debuginfos, use: debuginfo-install glibc-2.12-1.132.el6.i686
(gdb) s
12     printf("\033[33m[%-100s][%d%][%c]\033[0m\r",buf,i,r[i%5]);
(gdb)
13     fflush(stdout);
(gdb)
14     buf[i++] = '#';
(gdb)
15     usleep(100000);
(gdb)

```

查看某个变量的变化：print 变量名

```

(gdb) print i
$1 = 1
(gdb) s
10     while(i<=100)
(gdb)
12     printf("\033[33m[%-100s][%d%][%c]\033[0m\r",buf,i,r[i%5]);
(gdb)
13     fflush(stdout);
(gdb)
14     buf[i++] = '#';
(gdb)
15     usleep(100000);
(gdb) print i
$2 = 2
(gdb)

```

查看变量的类型：whatis 变量名

```

(gdb) print i
$2 = 2
(gdb) whatis i
type = int
(gdb)

```

运行至当前函数结束：finish

```

(gdb) finish
"finish" not meaningful in the outermost frame.

```

退出调试：q

```
(gdb) q
A debugging session is active.

    Inferior 1 [process 2425] will be killed.

Quit anyway? (y or n) y
[xj@localhost day8.7]$
```

内核中空间的分配函数有哪些

kmalloc

```
1 void *kmalloc(size_t size, gfp_t flags)
```

kmalloc是内核中最常用的一种内存分配方式，它通过调用kmem_cache_alloc函数来实现。kmalloc一次最多能申请的内存大小由include/linux/Kmalloc_size.h的内容来决定，在默认的2.6.18内核版本中，kmalloc一次最多能申请大小为131702B也就是**128KB字节的连续物理内存**。测试结果表明，如果试图用kmalloc函数分配大于128KB的内存，编译不能通过。

vmalloc

```
1 void *vmalloc(unsigned long size)
```

前面几种内存分配方式都是物理连续的，能保证较低的平均访问时间。但是在某些场合中，对内存区的请求不是很频繁，较高的内存访问时间也可以接受，**这是就可以分配一段线性连续，物理不连续的地址，带来的好处是一次可以分配较大块的内存**。图3-1表示的是vmalloc分配的内存使用的地址范围。vmalloc对一次能分配的内存大小没有明确限制。出于性能考虑，应谨慎使用vmalloc函数。在测试过程中，**最大能一次分配1GB的空间**。

dma_alloc_coherent

```
1 void *dma_alloc_coherent(struct device *dev, size_t size, dma_addr_t
    *dma_handle, gfp_t gfp)
```

DMA是一种硬件机制，允许外围设备和主存之间直接传输IO数据，而不需要CPU的参与，使用DMA机制能大幅提高与设备通信的吞吐量。DMA操作中，涉及到CPU高速缓存和对应的内存数据一致性的问题，必须保证两者的数据一致，在x86_64体系结构中，硬件已经很好的解决了这个问题，dma_alloc_coherent和**get_free_pages函数实现差别不大，前者实际是调用alloc_pages函数来分配内存**，因此一次分配内存的大小限制和后者一样。__get_free_pages分配的内存同样可以用于DMA操作。测试结果证明，dma_alloc_coherent函数一次能分配的最大内存也为4M。

ioremap

```
1 void * ioremap (unsigned long offset, unsigned long size)
```

ioremap是一种更直接的内存“分配”方式，使用时直接指定物理起始地址和需要分配内存的大小，然后将该段物理地址映射到内核地址空间。ioremap用到的物理地址空间都是事先确定的，和上面的几种内存分配方式并不太一样，并不是分配一段新的物理内存。ioremap多用于设备驱动，可以让CPU直接访问外部设备的IO空间。ioremap能映射的内存由原有的物理内存空间决定，所以没有进行测试。

Linux驱动开发内容

总线设备驱动模型

自内核2.6版本开始，需要关注的是总线、设备和驱动这3个实体，总线将设备和驱动绑定。在Linux内核系统中注册一个设备的时候，会寻找与之对应驱动进行匹配；相反地，系统中注册一个驱动的时候，会去寻找一个对应的设备进行匹配。匹配的工作由总线来完成。

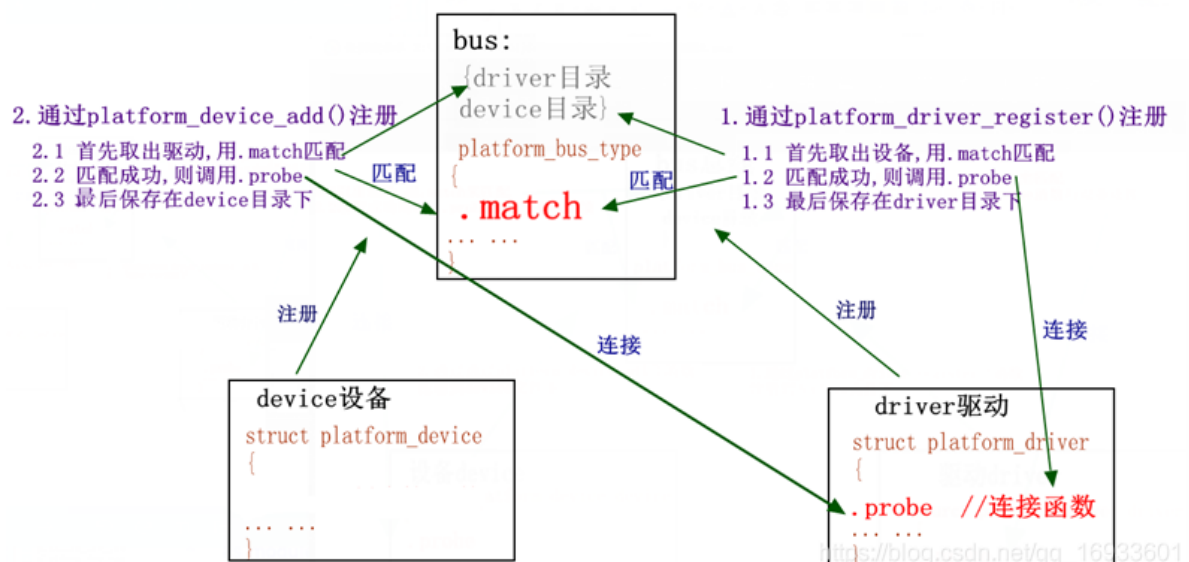
在Linux设备中有的是没有对应的物理总线的，但为了适配Linux的总线模型，内核针对这种没有物理总线的设备开发了一种虚拟总线——platform总线。

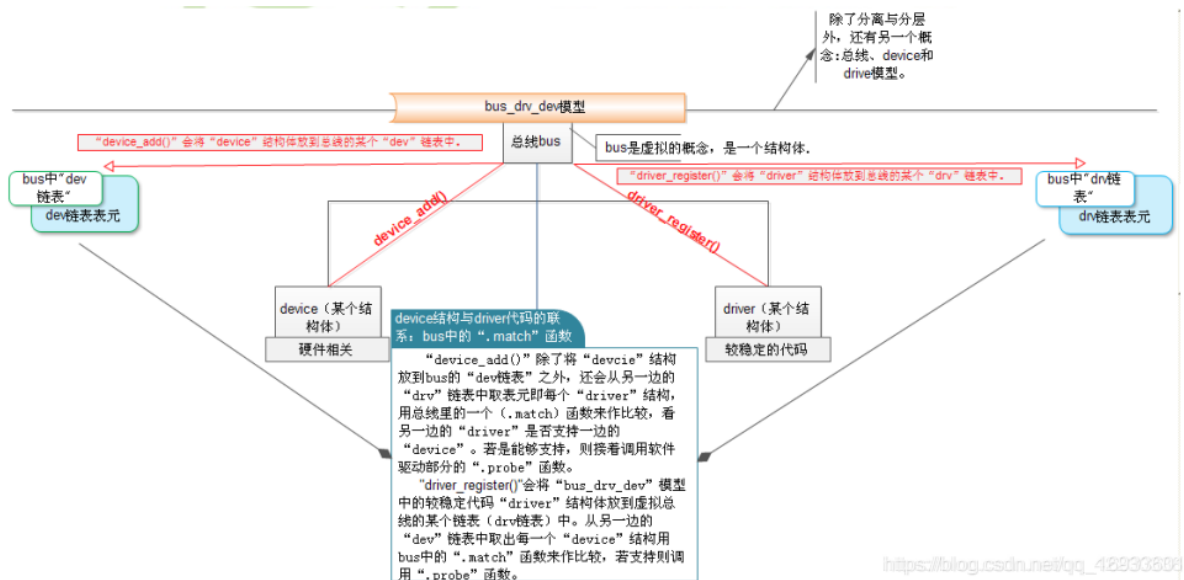
将设备和驱动独立开，驱动尽可能写的通用，当来了一个类似的设备后也可以使用这个驱动，让驱动程序可以重用。这体现了Linux驱动的软件架构设计的思想。

按照这个思路，Linux中的设备和驱动都需要挂载在一种总线上，比如i2c总线上的eeprom，eeprom作为设备，eeprom的驱动都挂载在i2c驱动上。但是在嵌入式系统中，soc系统一般都会集成独立的i2c控制器，控制器也是需要驱动的，但是再按照设备-总线-驱动模型进行设计，就会发现无法找到一个合适总线去挂载控制器设备和控制器驱动了（i2c控制器是挂载在CPU内部的总线上，而不是i2c总线），所以Linux发明了一种虚拟总线，称为platform总线，相应的设备称为platform_device（控制器设备），对应的驱动为platform_driver（控制器驱动），用platform总线来承载这些相对特殊的系统。

注意：所谓的platform_device并不是与字符设备、块设备和网络设备并列的概念，而是Linux系统提供的一种附加手段，例如，在S3C6410处理器中，把内部集成的I2C、RTC、SPI、LCD、看门狗等控制器都归纳为platform_device，而它们本身就是字符设备。我们要记住，platform驱动只是在字符设备驱动外套一层platform_driver的外壳。引入platform模型符合Linux设备模型——总线、设备、驱动，设备模型中配套的sysfs节点都可以用，方便我们的开发；当然你也可以选择不用，不过就失去了一些platform带来的便利；

设备驱动中引入platform概念，隔离BSP和驱动。在BSP中定义platform设备和设备使用的资源、设备的具体匹配信息，而在驱动中，只需要通过API去获取资源和数据，做到了板相关代码和驱动代码的分离，使得驱动具有更好的可扩展性和跨平台性。



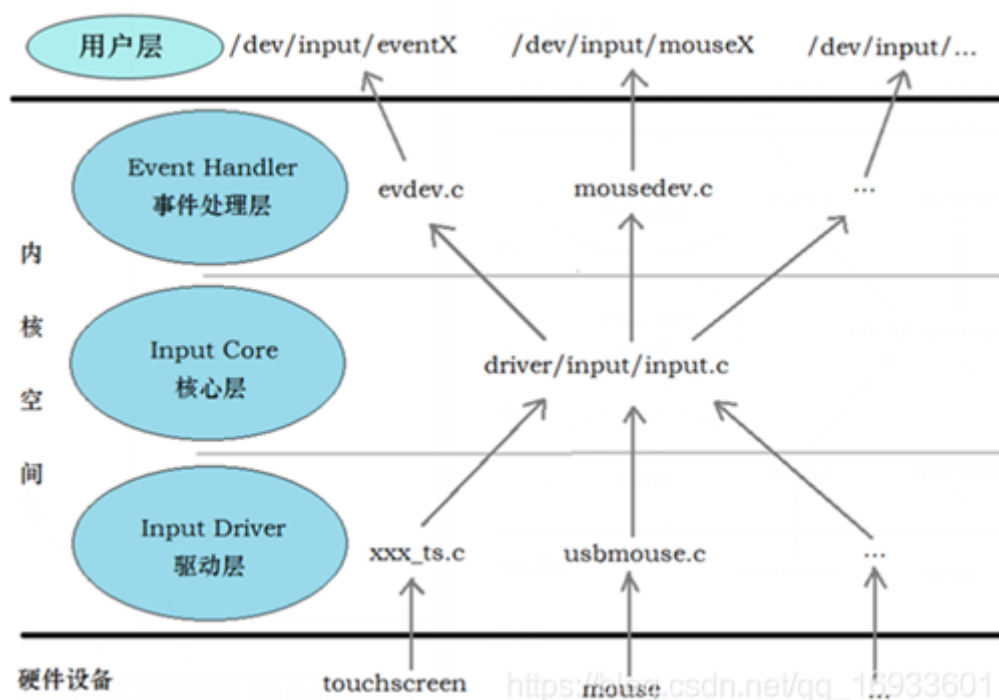


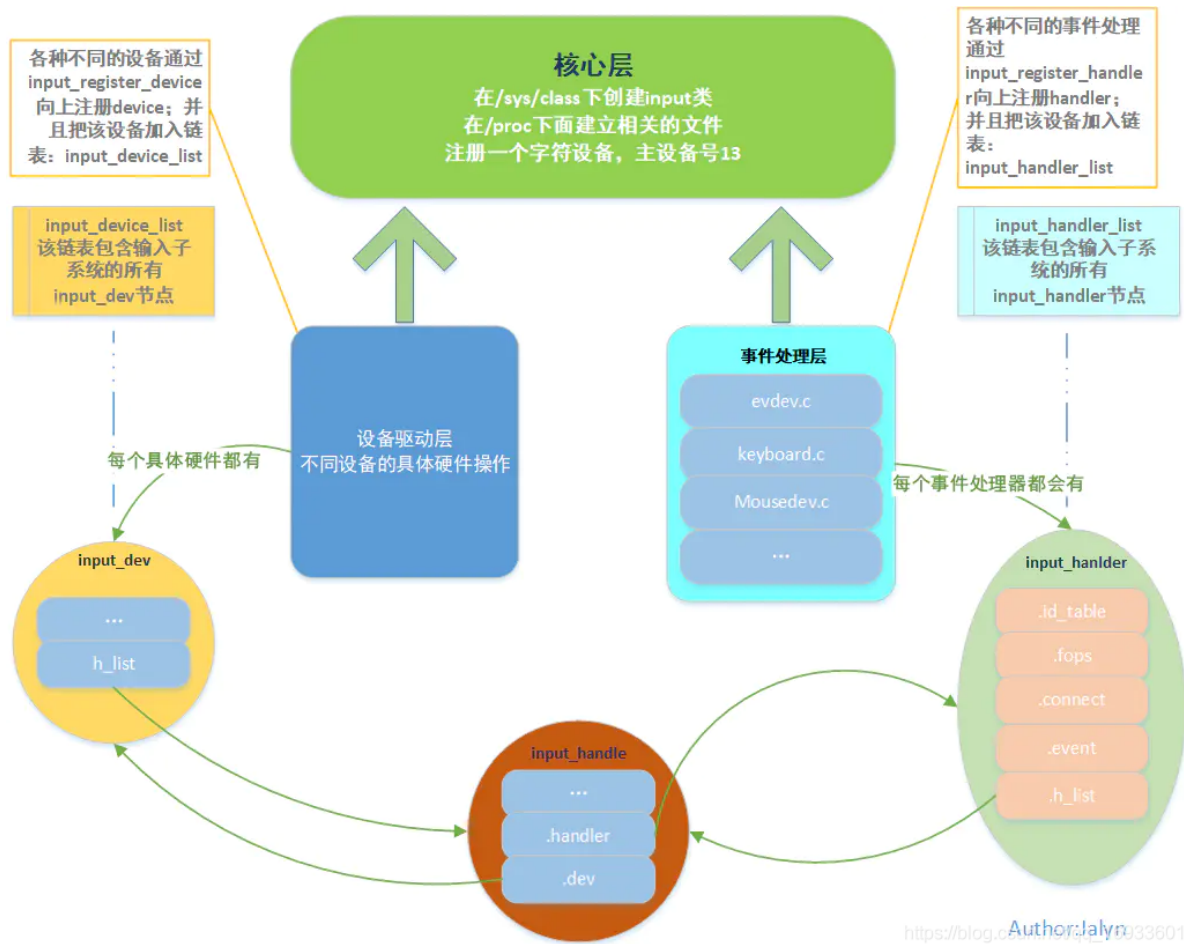
下面分析下总线设备驱动模型的匹配过程

一边的“device”结构体和另一边的“较稳定的 driver 代码”的联系：“device_add()”除了将“device”结构放到 bus 的“dev 链表”之外，还会从另一边的“drv”链表中取表元即某个“driver”结构，用总线里的一个（.match）函数来作比较，看另一边的“driver”是否支持一边的“device”。若是能够支持，则接着调用软件驱动部分的“.probe”函数。

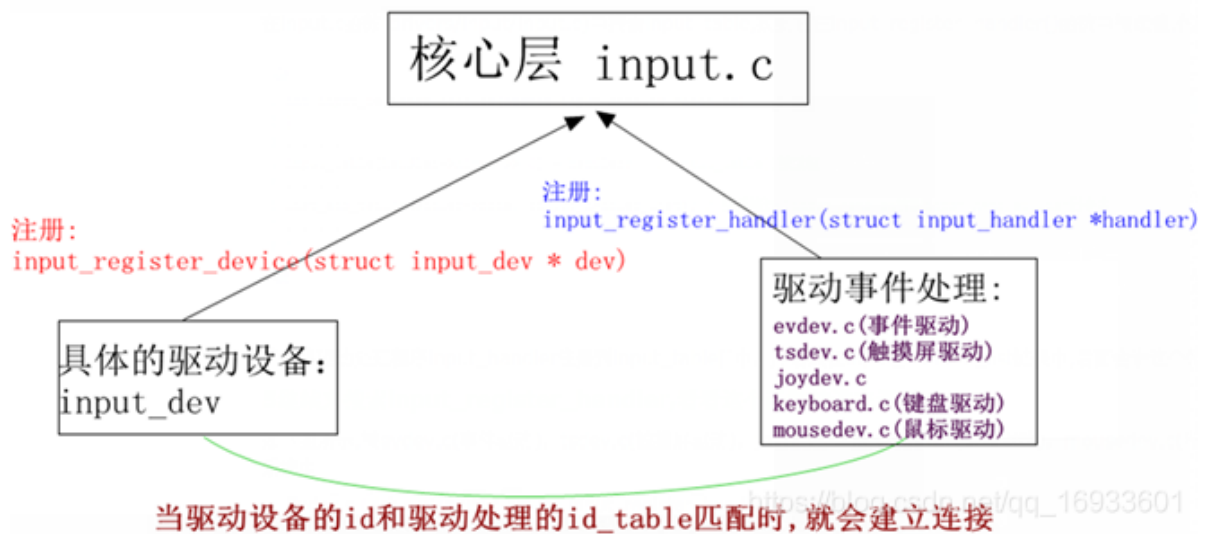
“driver_register()”会将“bus_drv_dev”模型中的较稳定代码“driver”结构体放到虚拟总线的某个链表（drv 链表）中。从另一边的“dev”链表中取出每一个“device”结构用 bus 中的“.match”函数来作比较，若支持则调用“.probe”函数。左右两个注册就建立起来的一种机制。在“.probe”函数中做的事件由自己决定，打印一句话，或注册一个字符设备，再或注册一个“input_dev”结构体等等都是由自己决定。强制的把一个驱动程序分为左右两边这种机制而已，可以把这套东西放在任何地方，这里的“driver”只是个结构体不要被这个名字迷惑，“device”也只是个结构体，里面放什么内容都是由自己决定的

输入子系统分析





https://blog.csdn.net/jalyn/article/details/16933601



每个硬件都有一个input_dev结构体, 每个软件都有一个input_handler结构体。input_dev和input_handler分别通过input_register_device(), input_register_handler()向核心层注册硬件和软件。

```
1 int input_register_device(struct input_dev *dev)    /*dev:要注册的驱动设备
2 {
3     ... ..
4     list_add_tail(&dev->node, &input_dev_list);    //(1)放入链表中
5     ... ..
6     list_for_each_entry(handler, &input_handler_list, node) //(2)
7     input_attach_handler(dev, handler);
8     ... ..
9 }
```

```
int input_register_handler(struct input_handler *handler)
{
    struct input_dev *dev;

    INIT_LIST_HEAD(&handler->h_list);

    if (handler->fops != NULL) {
        if (input_table[handler->minor >> 5])
            return -EBUSY;

        input_table[handler->minor >> 5] = handler;
    }

    list_add_tail(&handler->node, &input_handler_list);

    list_for_each_entry(dev, &input_dev_list, node)
        input_attach_handler(dev, handler);

    input_wakeup_procfs_readers();
    return 0;
} ? end input_register_handler ?
```

判断input_dev_list链表中的所有input_dev,是否支持这个新添加的input_handler处理驱动结构体

https://blog.csdn.net/qq_16933601

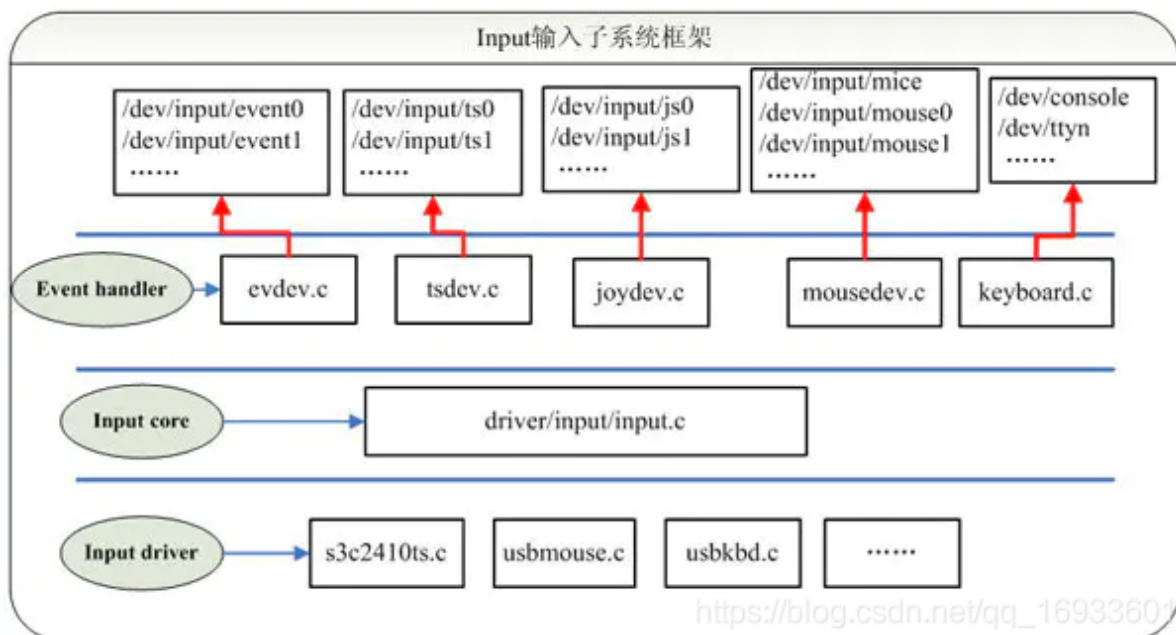
从input_dev方向分析：input设备在增加到input_dev_list链表上之后，会查找 input_handler_list 事件处理链表上的handler进行匹配，这里的匹配 方式与总线设备驱动模型的device和driver匹配过程很相似，所有的input_device都挂在input_dev_list上，所有类型的事件都挂在input_handler_list 上，进行“匹配相亲”。如果匹配上了，就调用input_handler的connect函数进行连接。设备就是在此时注册的

从input_handler方向分析：将handler挂到链表input_handler_list下，然后遍历input_dev_list链表,查找并匹配输入设备对应的事件处理层，如果匹配上了，就调用connect函数进行连接，并创建 input_handle结构。

所以,不管新添加input_dev还是input_handler,都会进入input_attach_handler()判断两者id是否有支持,若两者支持便进行连接。

驱动的分层和分离思想

分层是从外设功能来看的，尽可能高内聚低耦合，尽可能少的去修改外设硬件的一些参数就可以匹配到相应的驱动。如下图，每个外设如鼠标，键盘，操纵杆都有相应的驱动，我们最好抽象出这些外设的共性，每次增加新的外设的时候，尽可能少的修改非共性的参数就可以直接通过Input core核心层去匹配到相应的驱动程序。如下图所示。



https://blog.csdn.net/qq_16933601

分离是为了适配多个平台和设备。这个就是驱动的分隔，也就是将**主机驱动和设备驱动分隔开来**，比如I2C、SPI 等等都会采用驱动分隔的方式来简化驱动的开发。在实际的驱动开发中，一般I2C 主机控制器驱动已经由半导体厂家编写好了，而设备驱动一般也有设备器件的厂家编写好了，我们只需要提供设备信

息即可，比如I2C 设备的话提供设备连接到了哪个I2C 接口上，I2C 的速度是多少等等。相当于将设备信息从设备驱动中剥离开来，驱动使用标准方法去获取到设备信息(比如从设备树中获取到设备信息)，然后根据获取到的设备信息来初始化设备。这样就相当于驱动只需要负责驱动，设备只需要设备，想办法将两者进行匹配即可。这个就是Linux 中的总线(bus)、驱动(driver)和设备(device)模型，也就是常说的驱动分离。总线就是驱动和设备信息的月老，负责给两者牵线搭桥。如下图所示。

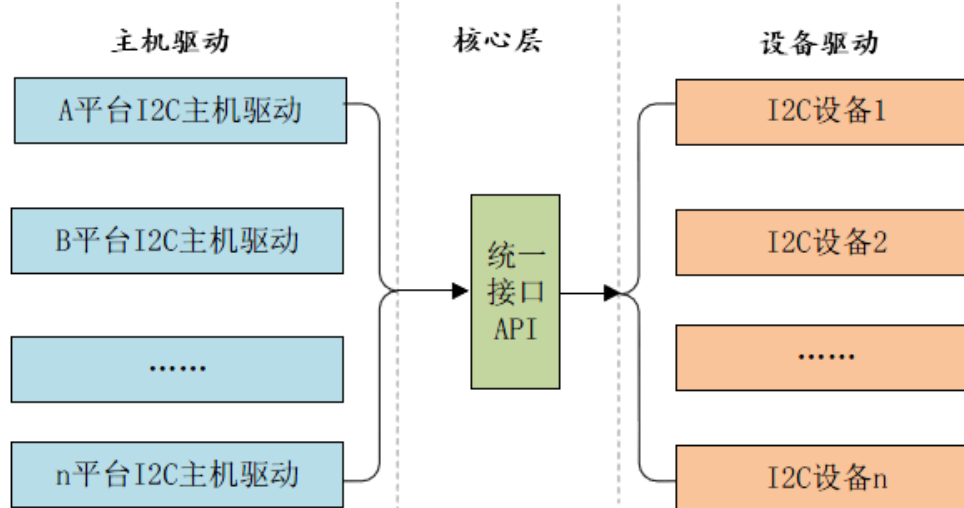


图 54.1.3 分隔后的驱动框架

https://blog.csdn.net/qq_16933601

字符设备驱动模型

以LED驱动为例，完整源码如下所示：

```

1  #include <linux/module.h>
2  #include <linux/kernel.h>
3  #include <linux/fs.h>
4  #include <linux/init.h>
5  #include <linux/delay.h>
6  #include <asm/irq.h>
7  #include <asm/arch/regs-gpio.h>
8  #include <asm/hardware.h>
9  #include <asm/uaccess.h>
10 #include <asm/io.h>
11
12
13
14 static struct class *seconddrv_class;           //创建一个class类
15 static struct class_device *seconddrv_class_devs; //创建类的设备
16
17 volatile unsigned long *GPFcon;
18 volatile unsigned long *GPFdat;
19 volatile unsigned long *GPGcon;
20 volatile unsigned long *GPGdat;
21
22 static int second_drv_open(struct inode *inode, struct file *file)
23 {
24     /*初始化按键*/
25     /* 配置 GPFCON(0x56000050)的位[0:1]、位[4:5]等于0x00(输入模式)
26        GPGCON(0x56000060)的位[6:7]、位[22:23]等于0x00*/
27     *GPFcon&=~((0x3<<0) | (0x3<<4));

```

```

28     *GPGcon&=~((0x3<<6)|(0x3<<22));
29
30     return 0;
31 }
32
33 static int second_drv_read(struct file *file, const char __user *buf, size_t
count, loff_t * ppos)
34 {
35     unsigned char key_vals[4];
36
37     /*按键0~3分别是GPF0, GPF2, GPG3, GPG11*/
38     if(count!=sizeof(key_vals))
39         return EINVAL;
40
41     key_vals[0]=(*GPFdat>>0)&0x01;
42     key_vals[1]=(*GPFdat>>2)&0x01;
43     key_vals[2]=(*GPGdat>>3)&0x01;
44     key_vals[3]=(*GPGdat>>11)&0x01;
45
46     /*上传给用户层*/
47     if(copy_to_user(buf,key_vals,sizeof(key_vals)))
48         return EFAULT;
49     return 0;
50
51 }
52
53
54
55 static struct file_operations second_drv_fops={
56     .owner = THIS_MODULE,
57     .open = second_drv_open,
58     .read = second_drv_read,};
59 volatile int second_major;           //保存主设备号
60 static int second_drv_init(void)
61 {
62     second_major=register_chrdev(0,"second_drv",&second_drv_fops); //创建驱动
63     seconddrv_class=class_create(THIS_MODULE,"second_dev"); //创建类名
64     seconddrv_class_devs=class_device_create(seconddrv_class, NULL,
MKDEV(second_major,0), NULL,"buttons");
65     /*申请虚拟地址,然后配置寄存器*/
66     /*  GPFCON(0x56000050)
67        GPGCON(0x56000060) */
68     GPFcon=ioremap(0x56000050,16);
69     GPFdat=GPFcon+1;
70
71     GPGcon=ioremap(0x56000060,16);
72     GPGdat=GPGcon+1;
73
74     return 0;
75 }
76
77 static int second_drv_exit(void)
78 {
79
80     unregister_chrdev(second_major,"second_drv"); //卸载驱动
81     class_device_unregister(seconddrv_class_devs); //卸载类设备
82     class_destroy(seconddrv_class); //卸载类
83

```

```

84  /*注销虚拟地址*/
85  iounmap(GPFcon);
86  iounmap(GPGcon);
87  return 0;
88  }
89
90
91  module_init(second_drv_init);
92  module_exit(second_drv_exit);
93  MODULE_LICENSE("GPL v2");

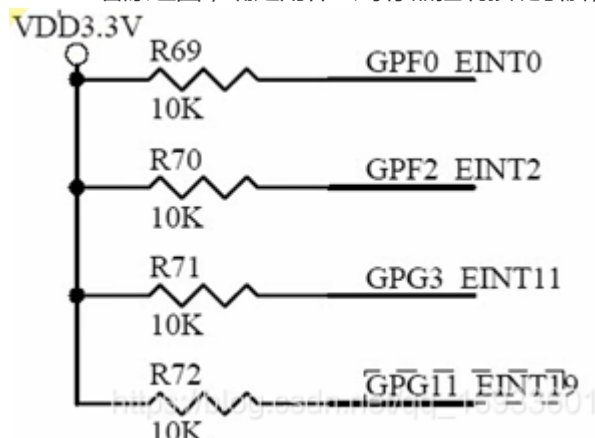
```

1.写出框架

- 1.1写file_operations结构体，second_drv_open函数，second_drv_read函数
- 1.2写入口函数,并自动创建设备节点，修饰入口函数
- 1.3写出口函数，并自动注销设备节点，修饰出口函数
- 1.4 写MODULE_LICENSE("GPL v2")声明函数许可证
- 1.5 在入口函数中，利用class_create和class_device_create自动创建设备节点。在出口函数中,利用class_destroy和class_device_unregister注销设备节点。

2.在框架中实现硬件操作

2.1看原理图，确定用什么寄存器控制按键引脚,如下图



按键0~3分别是GPF0，GPF2，GPG3，GPG11。由于是使用查询模式，并不是外部中断模式。所以配置 GPFCON(0x56000050)的位[0:1]、位[4:5]等于0x00(输入模式)。GPGCON(0x56000060)的位[6:7]、位[22:23]等于0x00。通过GPGDAT (0x56000054) 和GPGDAT(0x56000064)来查询按键状态。

2.2写代码

- init入口函数中使用ioremap()函数映射寄存器虚拟地址
- exit出口函数中使用iounmap()函数注销虚拟地址
- open函数中配置GPxCON初始化按键
- read函数中先检查读出的字符是否是4个,然后获取GPxDAT状态,用key_vals[4]数组保存4个按键值,最后使用 copy_to_user(buf, key_vals,sizeof(key_vals)) 上传给用户层

3.写测试程序Secondtext.c

用法就是./ Secondtext

使用read(fd,val,sizeof(val));函数读取内核层的数据

LCD驱动模型

写个LCD驱动入口函数,需要以下4步:

- 1) 分配一个fb_info结构体: framebuffer_alloc();
- 2) 设置fb_info
- 3) 设置硬件相关的操作
- 4) 使能LCD,并注册fb_info: register_framebuffer()

需要用到的函数:

```
1 void *dma_alloc_writecombine(struct device *dev, size_t size, dma_addr_t
   *handle, gfp_t gfp); //分配DMA缓存区给显存
2 //返回值为:申请到的DMA缓冲区的虚拟地址,若为NULL,表示分配失败,则需要使用
   dma_free_writecombine()释放内存,避免内存泄漏//参数如下:
3
4 // *dev:指针,这里填0,表示这个申请的缓冲区里没有内容
5
6 // size:分配的地址大小(字节单位)
7
8 // *handle:申请到的物理起始地址
9
10 // gfp:分配出来的内存参数,标志定义在<linux/gfp.h>,常用标志如下:
11 // GFP_ATOMIC 用来从中断处理和进程上下文之外的其他代码中分配内存. 从不睡眠.
12 // GFP_KERNEL 内核内存的正常分配. 可能睡眠.
13 // GFP_USER 用来为用户空间页来分配内存; 它可能睡眠.
```

分配一段DMA缓存区,分配出来的内存会禁止cache缓存(因为DMA传输不需要CPU)。它和 dma_alloc_coherent ()函数相似,不过 dma_alloc_coherent ()函数是分配出来的内存会禁止cache缓存以及禁止写入缓冲区。

```
1 dma_free_writecombine(dev,size,cpu_addr,handle); //释放缓存
2 //cpu_addr:虚拟地址,
3 //handle:物理地址
```

释放DMA缓冲区, dev和size参数和上面的一样

```
1 struct fb_info *framebuffer_alloc(size_t size, struct device *dev); //申
   请一个fb_info结构体,
2 //size:额外的内存,
3 // *dev:指针, 这里填0,表示这个申请的结构体里没有内容
```

```
1 int register_framebuffer(struct fb_info *fb_info);
2
3 //向内核中注册fb_info结构体,若内存不够,注册失败会返回负数
4
5 int unregister_framebuffer(struct fb_info *fb_info) ;
6
7 //注销内核中fb_info结构体
```

需要用到的结构体:

fb_info结构体如下:

```

1 struct fb_info {
2     ... ..
3     struct fb_var_screeninfo var;          //可变的参数
4     struct fb_fix_screeninfo fix;          //固定的参数
5     ... ..
6     struct fb_ops *fbops;                  //操作函数
7     ... ..
8     char __iomem *screen_base;              //显存虚拟起始地址
9     unsigned long screen_size;              //显存虚拟地址长度
10    void *pseudo_palette;
11    //假的16色调色板,里面存放了16色的数据,可以通过8bpp数据来找到调色板里面的16色颜色索引值,
    模拟出16色颜色来,节省内存,不需要的话就指向一个不用的数组即可
12    ... ..
13 };

```

其中操作函数fb_info-> fbops 结构体写法如下:

```

1 static struct fb_ops s3c_lcdfb_ops = {
2     .owner                = THIS_MODULE,
3     .fb_setcolreg          = my_lcdfb_setcolreg, //设置调色板fb_info->
    pseudo_palette,自己构造该函数
4
5     .fb_fillrect           = cfb_fillrect,        //填充矩形,用/drivers/video/
    cfbfillrect.c里的函数即可
6
7     .fb_copyarea           = cfb_copyarea,        //复制数据,
    用/drivers/video/cfbcopyarea.c里的函数即可
8
9     .fb_imageblit          = cfb_imageblit, //绘画图形,
    用/drivers/video/imageblit.c里的函数即可
10 };

```

固定的参数fb_info-> fix 结构体如下:

```

1 struct fb_fix_screeninfo {
2     char id[16];              //id名字
3     unsigned long smem_start; //framebuffer物理起始地址
4
5     __u32 smem_len;           //framebuffer长度,字节为单位
6     __u32 type;               //lcd类型,默认值0即可
7     __u32 type_aux;           //附加类型,为0
8     __u32 visual;             //画面设置,常用参数如下
9     // FB_VISUAL_MONO01        0    单色,0:白色,1:黑色
10    // FB_VISUAL_MONO10        1    单色,1:白色,0:黑色
11    // FB_VISUAL_TRUECOLOR      2    真彩(TFT:真彩)
12    // FB_VISUAL_PSEUDOCOLOR    3    伪彩
13    // FB_VISUAL_DIRECTCOLOR     4    直彩
14
15    __u16 xpanstep;             /*如果没有硬件panning就赋值为0 */
16    __u16 ypanstep;             /*如果没有硬件panning就赋值为0 */
17    __u16 ywrapstep;            /*如果没有硬件ywrap就赋值为0 */
18
19    __u32 line_length;          /*一行的字节数 ,例:(RGB565)240*320,那
    么这里就等于240*16/8 */
20    /*以下成员都可以不需要*/

```

```

20     unsigned long mmio_start;          /*内存映射IO的起始地址,用于应用层直接访问
寄存器,可以不需要*/
21     __u32 mmio_len;                    /* 内存映射IO的长度,可以不需要*/
22     __u32 accel;
23     __u16 reserved[3];
24
25 };

```

可变的参数fb_info->var 结构体如下:

```

1  struct fb_var_screeninfo{
2      __u32 xres;                        /*可见屏幕一行有多少个像素点*/
3      __u32 yres;                        /*可见屏幕一列有多少个像素点*/
4      __u32 xres_virtual;                /*虚拟屏幕一行有多少个像素点 */
5      __u32 yres_virtual;                /*虚拟屏幕一列有多少个像素点*/
6      __u32 xoffset;                     /*虚拟到可见屏幕之间的行偏移,若可见和虚拟的分辨
率一样,就直接设为0*/
7      __u32 yoffset;                     /*虚拟到可见屏幕之间的列偏移*/
8      __u32 bits_per_pixel;              /*每个像素的位数即BPP,比如:RGB565则填入16*/
9      __u32 grayscale;                   /*非0时,指的是灰度,真彩直接填0即可*/
10
11     struct fb_bitfield red;              //fb缓存的R位域, fb_bitfield结构体成员如下:
12     //__u32 offset;                      区域偏移值,比如RGB565中的R,就在第11位
13     //__u32 length;                      区域长度,比如RGB565的R,共有5位
14     //__u32 msb_right; msb_right ==0,表示数据左边最大, msb_right!=0,表示数据右边最大
15
16     struct fb_bitfield green;            /*fb缓存的G位域*/
17     struct fb_bitfield blue;             /*fb缓存的B位域*/      /*以下参数都可以不填,默认
为0*/
18     struct fb_bitfield transp;           /*透明度,不需要填0即可*/
19     __u32 nonstd;                         /* != 0表示非标准像素格式*/
20     __u32 activate;                       /*设为0即可*/
21     __u32 height;                         /*外设高度(单位mm),一般不需要填*/
22     __u32 width;                          /*外设宽度(单位mm),一般不需要填*/
23     __u32 accel_flags;                    /*过时的参数,不需要填*/
24
25     /* 除了pixclock本身外,其他的都以像素时钟为 单位*/
26     __u32 pixclock;                       /*像素时钟(皮秒)*/
27     __u32 left_margin;                    /*行切换,从同步到绘图之间的延迟*/
28     __u32 right_margin;                   /*行切换,从绘图到同步之间的延迟*/
29     __u32 upper_margin;                   /*帧切换,从同步到绘图之间的延迟*/
30     __u32 lower_margin;                   /*帧切换,从绘图到同步之间的延迟*/
31     __u32 hsync_len;                      /*水平同步的长度*/
32     __u32 vsync_len;                      /*垂直同步的长度*/
33     __u32 sync;
34     __u32 vmode;
35     __u32 rotate;
36     __u32 reserved[5];                    /*保留*/
37
38 }

```

在驱动init入口函数中:

1)分配一个fb_info结构体

2)设置fb_info

2.1)设置固定的参数fb_info-> fix。id,lcd的名字

2.2) 设置可变的参数fb_info-> var。可见屏幕一行有多少个像素点，虚拟屏幕一行有多少个像素点，每个像素的位数即BPP,比如:RGB565则填入16

2.3) 设置操作函数fb_info-> fbops

2.4) 设置fb_info 其它的成员，my_lcdfb_setcolreg调色板，cfb_copyarea复制数据

3)设置硬件相关的操作

3.1)配置LCD引脚 GPBcon = ioremap(0x56000010, 8); GPBdat = GPBcon+1;

3.2)根据LCD手册设置LCD控制器 VSYNC HSYNC 等参数

3.3)分配显存(framebuffer),把地址告诉LCD控制器和fb_info

4)开启LCD,并注册fb_info: register_framebuffer()

4.1) 直接在init函数中开启LCD(后面讲到电源管理,再来优化)

控制LCDCON5允许PWREN信号,

然后控制LCDCON1输出PWREN信号,

输出GPB0高电平来开背光,

4.2) 注册fb_info

在驱动exit出口函数中:

1)卸载内核中的fb_info

2) 控制LCDCON1关闭PWREN信号,关背光,iounmap注销地址

3)释放DMA缓存地址dma_free_writcombine()

4)释放注册的fb_info

LCD驱动完整代码如下所示：

```

1  #include <linux/module.h>
2  #include <linux/kernel.h>
3  #include <linux/errno.h>
4  #include <linux/string.h>
5  #include <linux/mm.h>
6  #include <linux/slab.h>
7  #include <linux/delay.h>
8  #include <linux/fb.h>
9  #include <linux/init.h>
10 #include <linux/dma-mapping.h>
11 #include <linux/interrupt.h>
12 #include <linux/workqueue.h>
13 #include <linux/wait.h>
14 #include <linux/platform_device.h>
15 #include <linux/clock.h>
16 #include <asm/io.h>
17 #include <asm/uaccess.h>
18 #include <asm/div64.h>
19 #include <asm/mach/map.h>
20 #include <asm/arch/regs-lcd.h>
21 #include <asm/arch/regs-gpio.h>
22 #include <asm/arch/fb.h>
23
24 /*LCD : 480*272 */
25 #define LCD_xres 480 //LCD 行分辨率
26 #define LCD_yres 272 //LCD列分辨率
27
28
29 /* GPIO prot */

```

```

30 static unsigned long *GPBcon;
31 static unsigned long *GPCcon;
32 static unsigned long *GPDcon;
33 static unsigned long *GPGcon; //GPG4:控制LCD信号
34 static unsigned long *GPBdat; //GPB0: 控制背光
35
36 /* LCD control */
37 struct lcd_reg{
38     unsigned long    lcdcon1;
39     unsigned long    lcdcon2;
40     unsigned long    lcdcon3;
41     unsigned long    lcdcon4;
42     unsigned long    lcdcon5;
43     unsigned long    lcdsaddr1;
44     unsigned long    lcdsaddr2;
45     unsigned long    lcdsaddr3 ;
46     unsigned long    redlut;
47     unsigned long    greenlut;
48     unsigned long    bluelut;
49     unsigned long    reserved[9];
50     unsigned long    dithmode;
51     unsigned long    tpa1 ;
52     unsigned long    lcdintpnd;
53     unsigned long    lcdsrcpnd;
54     unsigned long    lcdintmsk;
55     unsigned long    tconsel;
56 };
57
58 static struct lcd_reg *lcd_reg;
59
60 static struct fb_info *my_lcd; //定义一个全局变量
61 static u32 pseudo_palette[16]; //调色板数组,被fb_info->pseudo_palette调用
62
63 static inline unsigned int chan_to_field(unsigned int chan, struct
fb_bitfield *bf)
64 {
65     /*内核中的单色都是16位,默认从左到右排列,比如G颜色[0x1f],那么chan就等于0xF800*/
66     chan    &= 0xffff;
67     chan    >>= 16 - bf->length; //右移,将数据靠到位0上
68     return chan << bf->offset; //左移一定偏移值,放入16色数据中对应的位置
69 }
70
71 static int my_lcdfb_setcolreg(unsigned int regno, unsigned int red,unsigned
int green, unsigned int blue,unsigned int transp, struct fb_info *info)
    //设置调色板函数,供内核调用
72 {
73     unsigned int val;
74     if (regno >=16) //调色板数组不能大于15
75         return 1;
76
77     /* 用red,green,blue三个颜色值构造出16色数据val */
78     val = chan_to_field(red, &info->var.red);
79     val |= chan_to_field(green, &info->var.green);
80     val |= chan_to_field(blue, &info->var.blue);
81
82     ((u32 *) (info->pseudo_palette))[regno] = val; //放到调色板数组中
83     return 0;
84 }

```

```

85
86
87 static struct fb_ops my_lcdfb_ops = {
88     .owner          = THIS_MODULE,
89     .fb_setcolreg    = my_lcdfb_setcolreg, //调用my_lcdfb_setcolreg()函数,来设
置调色板fb_info-> pseudo_palette
90     .fb_fillrect     = cfb_fillrect,      //填充矩形
91     .fb_copyarea     = cfb_copyarea,      //复制数据
92     .fb_imageblit    = cfb_imageblit,     //绘画图形,
93 };
94
95 static int lcd_init(void)
96 {
97     /*1. 申请一个fb_info结构体*/
98     my_lcd= framebuffer_alloc(0,0);
99
100     /*2. 设置fb_info*/
101
102     /* 2.1设置固定的参数fb_info-> fix */
103     /*my_lcd->fix.smem_start 物理地址后面注册MDA缓存区设置*/
104     strcpy(my_lcd->fix.id, "mylcd"); //名字
105     my_lcd->fix.smem_len =LCD_xres*LCD_yres*2; //地址长
106     my_lcd->fix.type     =FB_TYPE_PACKED_PIXELS; // TFT STN
107     my_lcd->fix.visual    =FB_VISUAL_TRUECOLOR; //真彩色
108     my_lcd->fix.line_length =LCD_xres*2; //LCD 一行的字节
109
110     /* 2.2 设置可变的参数fb_info-> var */
111     my_lcd->var.xres      =LCD_xres; //可见屏x 分辨
率
112     my_lcd->var.yres      =LCD_yres; //可见屏y 分辨
率
113     my_lcd->var.xres_virtual =LCD_xres; //虚拟屏x分辨率
114     my_lcd->var.yres_virtual =LCD_yres; //虚拟屏y分辨率
115     my_lcd->var.xoffset     = 0; //虚拟到可见屏幕之间
的行偏移
116     my_lcd->var.yoffset     =0; //虚拟到可见屏幕之间
的行偏移
117
118     my_lcd->var.bits_per_pixel=16; //像素为16BPP
119     my_lcd->var.grayscale    = 0; //灰色比例
120
121     my_lcd->var.red.offset   = 11;
122     my_lcd->var.red.length   = 5;
123     my_lcd->var.green.offset = 5;
124     my_lcd->var.green.length = 6;
125     my_lcd->var.blue.offset  = 0;
126     my_lcd->var.blue.length  = 5;
127
128     /* 2.3 设置操作函数fb_info-> fbops */
129     my_lcd->fbops            = &my_lcdfb_ops;
130
131
132     /* 2.4 设置fb_info 其它的成员 */
133     /*my_lcd->screen_base 虚拟地址在后面注册MDA缓存区设置*/
134     my_lcd->pseudo_palette   =pseudo_palette; //保存调色板数组
135     my_lcd->screen_size      =LCD_xres * LCD_yres *2; //虚拟地址长
136
137     /*3 设置硬件相关的操作*/

```

```

138  /*3.1 配置LCD引脚*/
139  GPBcon = ioremap(0x56000010, 8);
140  GPBdat = GPBcon+1;
141  GPCcon = ioremap(0x56000020, 4);
142  GPDcon = ioremap(0x56000030, 4);
143  GPGcon = ioremap(0x56000060, 4);
144
145  *GPBcon      &=~(0x03<<(0*2));
146  *GPBcon      |= (0x01<<(0*2));      //PGB0背光
147  *GPBdat      &=~(0x1<<0);          //关背光
148  *GPCcon      =0xaaaaaaaa;
149  *GPDcon      =0xaaaaaaaa;
150  *GPGcon      |= (0x03<<(4*2));      //GPG4:LCD信号
151
152  /*3.2 根据LCD手册设置LCD控制器,参考之前的裸机驱动*/
153  lcd_reg=ioremap(0x4D000000, sizeof( lcd_reg) );
154  /*HCLK:100Mhz */
155  lcd_reg->lcdcon1 = (4<<8) | (0x3<<5) | (0x0C<<1) ;
156  lcd_reg->lcdcon2 = ((3)<<24) | (271<<14) | ((1)<<6) | ((0)<<0);
157  lcd_reg->lcdcon3 = ((16)<<19) | (479<<8) | ((10));
158  lcd_reg->lcdcon4 = (4);
159  lcd_reg->lcdcon5 = (1<<11) | (1<<9) | (1<<8) | (1<<0);
160
161  lcd_reg->lcdcon1 &=~(1<<0);          // 关闭PWREN信号输出
162  lcd_reg->lcdcon5 &=~(1<<3);          //禁止PWREN信号
163
164  /* 3.3 分配显存(framebuffer),把地址告诉LCD控制器和fb_info*/
165  my_lcd->screen_base=dma_alloc_writecombine(0,my_lcd->fix.smem_len,
166  &my_lcd->fix.smem_start, GFP_KERNEL);
167
168  /*lcd控制器的地址必须是物理地址*/
169  lcd_reg->lcdsaddr1 =(my_lcd->fix.smem_start>>1)&0x3FFFFFFF; //保存缓冲起
170 始地址A[30:1]
171  lcd_reg->lcdsaddr2 =((my_lcd->fix.smem_start+my_lcd-
172  >screen_size)>>1)&0x1FFFFFFF; //保存缓冲结束地址A[21:1]
173  lcd_reg->lcdsaddr3 =LCD_xres& 0x3ff;          //OFFSIZE[21:11]:保存
174  LCD上一行结尾和下一行开头的地址之间的
175  差          //PAGEWIDTH [10:0]:保
176  存LCD一行占的宽度(半字数为单位)
177
178  /*4开启LCD,并注册fb_info: register_framebuffer()*/
179  /*4.1 直接在init函数中开启LCD(后面讲到电源管理,再来优化)*/
180  lcd_reg->lcdcon1 |=1<<0;          //输出PWREN信号
181  lcd_reg->lcdcon5 |=1<<3;          //允许PWREN信号
182  *GPBdat          |= (0x1<<0);      //开背光
183
184  /*4.2 注册fb_info*/
185  register_framebuffer(my_lcd);
186  return 0;
187
188  static int lcd_exit(void)
189  {
190      /* 1卸载内核中的fb_info*/
191      unregister_framebuffer(my_lcd);
192      /*2 控制LCDCON1关闭PWREN信号,关背光,iounmap注销地址*/
193      lcd_reg->lcdcon1 &=~(1<<0);      // 关闭PWREN信号输出
194      lcd_reg->lcdcon5 &=~(1<<3);      //禁止PWREN信号
195      *GPBdat          &=~(0x1<<4);      //关背光

```

```

190     iounmap(GPBcon);
191     iounmap(GPCcon);
192     iounmap(GPDcon);
193     iounmap(GPGcon);
194
195     /*3. 释放DMA缓存地址dma_free_writcombine()*/
196     dma_free_writcombine(0,my_lcd->screen_size,my_lcd->screen_base,my_lcd-
>fix.smem_start);
197
198     /*4. 释放注册的fb_info*/
199     framebuffer_release(my_lcd);
200
201     return 0;
202 }
203
204 module_init(lcd_init);
205 module_exit(lcd_exit);
206 MODULE_LICENSE("GPL");

```

触摸屏驱动

触摸屏原理介绍

我们现在总结下使用触摸屏的流程

<1>按下触摸屏 按下触摸屏时，对于一个高效的系统，产生中断，这是触摸屏中断

<2>在触摸中断程序中 启动ADC，(获得数据，xy坐标)启动ADC就开始模数转换，不可能瞬间完成，

<3>ADC完成，产生中断

<4>ADC中断中读取x y坐标，我们来想想，在这个流程里，启动触摸屏的源头是按下触摸屏，那如果长按触摸屏，我按下之后一直不松开 滑动手指呢

那么谁来触发后续的多次ADC转换呢 不可能只启动一次吧，

为了支持 长按 滑动操作，我们需要启用定时器。

<5> 启动定时器

<6> 定时器中断发生，判断触摸屏是否仍被按下，如果按下就循环上述过程

<6.1>在触摸中断程序中 启动ADC，(获得数据，xy坐标)启动ADC就开始模数转换，不可能瞬间完成

<6.2>ADC完成，产生中断

<6.3>ADC中断中读取x y坐标，)

<7> 松开结束一个流程

这就是整个触摸屏的使用流程

触摸屏驱动框架

1 初始化ADC/TouchScreen接口ADCCON时钟接口

2 一开始触摸屏是没有被按下的，设置TS处于等待中断模式

3 设置中断。INTSUBMSK使能ADC中断和触摸屏中断,还有INTMSK设置这个寄存器使能ANT_ADC让他能够发给CPU

4 按下触摸屏，进入TS中断

4.1 进入自动采集模式(自动转换XY坐标)

4.2 启动ADC

5.转换完之后产生ADC中断

5.1 读数据

5.2 再次进入“等待中断”模式

5.3 启动定时器，处理长按或者滑动

6 定时器中断

6.1 判断是否松开，若松开结束

6.2 若按下重新执行 4.2启动ADC步骤

关于触摸屏的一些重要参数：

x y分离转换模式，

看看我们的X Y坐标原理图，可以单独转换X坐标 单独转换Y坐标。换句话说就是逐个去测量X Y坐标，他首先会启动X坐标的ADC转换，转换成功后数据会保存在ADCDAT0里，同时会产生一个中断，在这个中断服务程序里，就可以把X坐标读取出来，让后可以启动Y坐标的转换，转换成功后数据会保存在ADCDAT1，同时会产生一个中断，进入这个中断把Y坐标读取出来。测量一次会产生2个中断，一个时X坐标中断，一个是Y坐标中断

自动的或连续的X / Y坐标转换模式

也就是说不需要单独控制，不需要单独去读取X坐标Y坐标，可以设置寄存器，让它一次性的测量X坐标测量Y坐标，X坐标保存在ADCDAT0 Y坐标保存在ADCDAT1，最后产生一个中断，也就是读取X / Y坐标只需要产生一次中断

等待中断模式

所谓等待中断模式，就是等待按下或者等待松开。对于下面这幅图，我按下的时候XP从高电平变为低电平，松开时，XP从低电平变为高电平，这就是按下松开都可以检测到我们要等待按下或者松开时 需要设置rADCTSC =0xd3这个值

寄存器内容相关：

ADCDATA0 ADCDATA1 存放ADC转换后的数据

ADCTSC 手动转换坐标，自动连续转换坐标

ADCUPDN触摸屏按下或者松开检查寄存器

ADC中断源

INTERRUPT SOURCES

The interrupt controller supports 60 interrupt sources as shown in the table below.

Sources	Descriptions	Arbiter Group
INT_ADC	ADC EOC and Touch interrupt (INT_ADC_S/INT_TC)	ARB5

ADC结束中断或者触摸屏中断，看来他们合起来用一个中断
既然合并必然还会有一个寄存器来分辨到底是ADC还是触摸屏发生的中断变化

SRCPND寄存器

31位为ADC中断

Register	Address	R/W	Description	Reset Value
SRCPND	0X4A000000	R/W	Indicate the interrupt request status. 0 = The interrupt has not been requested. 1 = The interrupt source has asserted the interrupt request.	0x00000000

SOURCE PENDING (SRCPND) REGISTER (Continued)

SRCPND	Bit	Description	Initial State
INT_ADC	[31]	0 = Not requested, 1 = Requested	0

SUBSOURCE PENDING寄存器

INT_ADC_S Bit[10]表示ADC中断

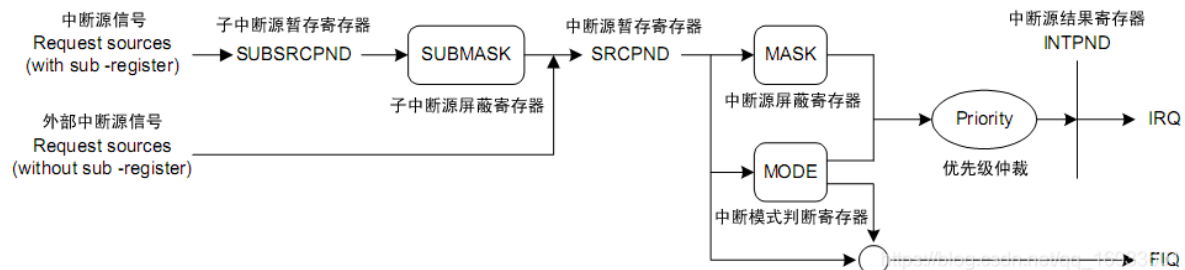
INT_TC Bit[9]表示触摸屏中断

SUB SOURCE PENDING (SUBSRCPND) REGISTER

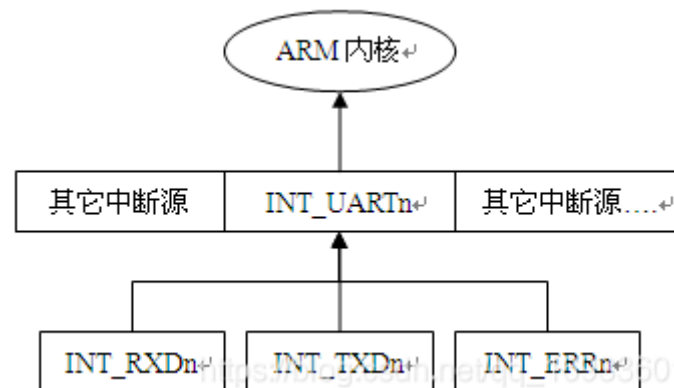
You can clear a specific bit of the SUBSRCPND register by writing a data to this register. It clears only the bit positions of the SUBSRCPND register corresponding to those set to one in the data. The bit positions corresponding to those that are set to 0 in the data remains as they are.

Register	Address	R/W	Description	Reset Value
SUBSRCPND	0X4A000018	R/W	Indicate the interrupt request status. 0 = The interrupt has not been requested. 1 = The interrupt source has asserted the interrupt request.	0x00000000

SUBSRCPND	Bit	Description	Initial State
Reserved	[31:15]	Not used	0
INT_AC97	[14]	0 = Not requested, 1 = Requested	0
INT_WDT	[13]	0 = Not requested, 1 = Requested	0
INT_CAM_P	[12]	0 = Not requested, 1 = Requested	0
INT_CAM_C	[11]	0 = Not requested, 1 = Requested	0
INT_ADC_S	[10]	0 = Not requested, 1 = Requested	0
INT_TC	[9]	0 = Not requested, 1 = Requested	0



S3C2440A将中断源分为两级：中断源和子中断源，中断源里包含单一中断源和复合中断源，复合中断源是子中断源的复合信号。如实时时钟中断，该硬件只会产生一种中断，它是单一中断源，直接将其中断信号线连接到中断源寄存器上。对于复合中断源，以UART串口为例进行说明，S3C2440A可以支持3个UART串口，每个串口对应一个复合中断源信号INT_UARTn，每个串口可以产生三种中断，也就是三个子中断：接收数据中断INT_RXDn，发送数据中断INT_TXDn，数据错误中断INT_ERRn，这三个子中断信号在中断源寄存器复合为一个中断信号，三种中断任何一个产生都会将中断信号传递给对应的中断源INT_UARTn，然后通过中断信号线传递给ARM内核。



INTERRUPT SOURCES

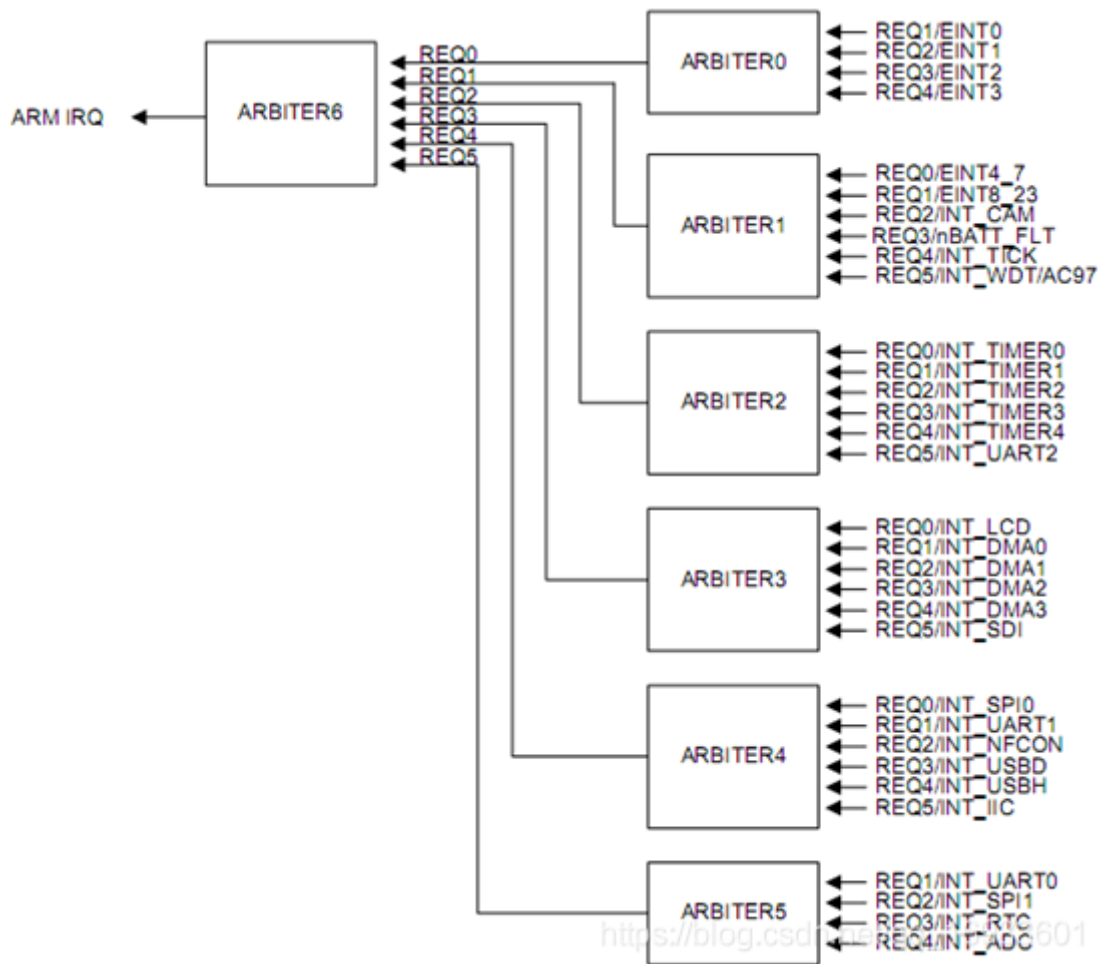
The interrupt controller supports 60 interrupt sources as shown in the table below.

Sources	Descriptions	Arbiter Group
INT_ADC	ADC EOC and Touch interrupt (INT_ADC_S/INT_TC)	ARB5
INT_RTC	RTC alarm interrupt	ARB5
INT_SPI1	SPI1 interrupt	ARB5
INT_UART0	UART0 Interrupt (ERR, RXD, and TXD)	ARB5
INT_IIC	IIC interrupt	ARB4
INT_USBH	USB Host interrupt	ARB4
INT_USBD	USB Device interrupt	ARB4
INT_NFCON	Nand Flash Control Interrupt	ARB4
INT_UART1	UART1 Interrupt (ERR, RXD, and TXD)	ARB4
INT_SPI0	SPI0 interrupt	ARB4
INT_SDI	SDI interrupt	ARB 3
INT_DMA3	DMA channel 3 interrupt	ARB3
INT_DMA2	DMA channel 2 interrupt	ARB3
INT_DMA1	DMA channel 1 interrupt	ARB3
INT_DMA0	DMA channel 0 interrupt	ARB3
INT_LCD	LCD interrupt (INT_FrSyn and INT_FiCnt)	ARB3
INT_UART2	UART2 Interrupt (ERR, RXD, and TXD)	ARB2
INT_TIMER4	Timer4 interrupt	ARB2
INT_TIMER3	Timer3 interrupt	ARB2
INT_TIMER2	Timer2 interrupt	ARB2
INT_TIMER1	Timer1 interrupt	ARB 2
INT_TIMER0	Timer0 interrupt	ARB2
INT_WDT_AC97	Watch-Dog timer interrupt(INT_WDT, INT_AC97)	ARB1
INT_TICK	RTC Time tick interrupt	ARB1
nBATT_FLT	Battery Fault interrupt	ARB1
INT_CAM	Camera Interface (INT_CAM_C, INT_CAM_P)	ARB1
EINT8_23	External interrupt 8 – 23	ARB1
EINT4_7	External interrupt 4 – 7	ARB1
EINT3	External interrupt 3	ARB0
EINT2	External interrupt 2	ARB0
EINT1	External interrupt 1	ARB0
EINT0	External interrupt 0	ARB0

https://blog.csdn.net/qq_16933601

Sub Sources	Descriptions	Source
INT_AC97	AC97 interrupt	INT_WDT_AC97
INT_WDT	Watchdog interrupt	INT_WDT_AC97
INT_CAM_P	P-port capture interrupt in camera interface	INT_CAM
INT_CAM_C	C-port capture interrupt in camera interface	INT_CAM
INT_ADC_S	ADC interrupt	INT_ADC
INT_TC	Touch screen interrupt (pen up/down)	INT_ADC
INT_ERR2	UART2 error interrupt	INT_UART2
INT_TXD2	UART2 transmit interrupt	INT_UART2
INT_RXD2	UART2 receive interrupt	INT_UART2
INT_ERR1	UART1 error interrupt	INT_UART1
INT_TXD1	UART1 transmit interrupt	INT_UART1
INT_RXD1	UART1 receive interrupt	INT_UART1
INT_ERR0	UART0 error interrupt	INT_UART0
INT_TXD0	UART0 transmit interrupt	INT_UART0
INT_RXD0	UART0 receive interrupt	INT_UART0

https://blog.csdn.net/qq_16933601



在init入口函数中:

1)分配一个input_dev结构体

2)设置input_dev的成员

2.1)设置input_dev->evbit支持按键事件,绝对位移事件(触摸屏:通过按键BTN_TOUCH获取按下/松开,通过绝对位移获取坐标)

2.2)设置input_dev-> keybit支持BTN_TOUCH触摸屏笔尖按下

2.3)设置input_dev-> absbit 支持ABS_X、ABS_Y、ABS_PRESSURE

input_set_abs_params(ts.dev, ABS_X, 0, 0x3FF, 0, 0);

input_set_abs_params(ts.dev, ABS_Y, 0, 0x3FF, 0, 0); // 0x3FF:最大值为10位ADC,

input_set_abs_params(ts.dev, ABS_PRESSURE, 0, 1, 0, 0); //压力最多就是1

3)注册input_dev 驱动设备到内核中

4)设置触摸屏相关的硬件

4.1)开启ADC时钟,使用clk_get ()和clk_enable()函数

4.2) ioremap获取寄存器地址,设置寄存器ADCCON =(1<<14)|(49<<6),分频

4.3)设置寄存器ADCDLY=0xffff,ADC启动延时时间设为最大值,使触摸按压更加稳定

4.4)开启IRQ_TC笔尖中断、开启IRQ_ADC中断获取XY坐标

4.5)初始化定时器,增加触摸滑动功能

4.6)最后设置寄存器ADCTSC=0x0d3,开启IRQ_TC中断

在出口函数中:

1)注销内核里的input_dev、

2)释放中断、删除定时器、iounmap注销地址、

3)释放input_dev、

在IRQ_TC中断函数中:

- 1)若判断笔尖为松开,设置寄存器ADCTSC =0XD3(按下中断)
- 2)若判断笔尖按下,设置为XY自动转换模式,启动一次ADC转换,ADC转换成功,会进入ADC中断

在IRQ_ADC中断函数中:

- 1)获取ADCDAT0的位[9:0],来算出XY方向坐标值
- 2)测量n次值保存在数组中,然后再次设置为XY自动转换模式,启动ADC(PS:要启动ADC转换之前必须设置一次XY为自动转换模式,不然获取的数据会不准)
- 3)采集完毕,使用快速排序将n次值排序后,以最小值为基准,如有误差非常大的数,则舍弃,如果没有则打印数组的中间值,实现中值滤波。(PS: 使用快速排序,比冒泡更快,详解:<http://www.cnblogs.com/lifexy/p/7597276.html>)
- 4)打印数据后,必须设置寄存器ADCTSC =0X1D3(松开中断IRQ_TC)(PS:在ADC采样模式下是判断不到ADCDAT0的bit15位的,因为ADCDAT0已被自动设置为X坐标的采样值)
- 5)设置定时器10ms超时时间

在定时器超时函数中:

- 1)判断ADCDAT0的bit15位,若还在按下再次启动ADC转换(实现触摸滑动功能)
- 2)若松开,设置寄存器ADCTSC =0XD3(按下中断)

触摸屏驱动代码如下所示：

```

1  #include <linux/errno.h>
2  #include <linux/kernel.h>
3  #include <linux/module.h>
4  #include <linux/slab.h>
5  #include <linux/input.h>
6  #include <linux/init.h>
7  #include <linux/serio.h>
8  #include <linux/delay.h>
9  #include <linux/platform_device.h>
10 #include <linux/clock.h>
11 #include <asm/io.h>
12 #include <asm/irq.h>
13 #include <asm/plat-s3c24xx/ts.h>
14 #include <asm/arch/regs-adc.h>
15 #include <asm/arch/regs-gpio.h>
16
17
18
19 static struct input_dev *ts_dev;
20 static struct clk *ADC_CLK; //adc时钟
21 static struct timer_list ts_timer; //定时器
22
23 struct adc_regs{
24     unsigned long  adcon;
25     unsigned long  adctsc;
26     unsigned long  adcdly;
27     unsigned long  adcdat0;
28     unsigned long  adcdat1;
29     unsigned long  adcupdn;
30 };
31
32 static volatile struct adc_regs *adc_regs;
33
34 /*启动TC 函数*/
35 static void set_pen_down(void)
36 {

```

```

37     /* 设置寄存器ADCTSC=0x0d3,开启IRQ_TC中断*/
38     adc_regs->adctsc = 0xd3;
39 }
40
41 static void set_pen_up(void)
42 {
43     /* 设置寄存器ADCTSC=0x0d3,开启IRQ_TC中断*/
44     adc_regs->adctsc = 0xd3;
45 }
46
47
48
49 /*启动ADC 转换函数*/
50 static void adc_start(void)
51 {
52     adc_regs->adctsc= (1<<3) | (1<<2);    //启动XY自动转换
53     adc_regs->adcon|= (1<<0);              //启动1次ADC转换
54 }
55
56
57
58 /*快速排序,比冒泡更快*/
59 /*快速排序详解:http://www.cnblogs.com/lifexy/p/7597276.html*/
60 static void find_frst(int *s,int lift,int right)
61 {
62     int i=lift,j=right,temp;  //(1)初始化i、j
63     if(lift>=right)
64         return ;
65     temp=s[i];                //(2)以第一个数组为比较值,保存到temp中
66     while(i<j)
67     {
68         while(j>i&& s[j]>=temp)  //(3)j--,找小值
69             j--;
70         s[i]= s[j];             //保存小值,到s[i]上
71         while(i<j&& s[i]<=temp)  //(4)i++,找大值
72             i++;
73         s[j--]=s[i];           //保存大值 到s[j]上
74     }
75
76     s[i]=temp;                 //(5)将比较值放在s[i]上
77
78     /*(6)拆分成两个数组 s[0,i-1]、s[i+1,n-1]又开始排序 */
79     find_frst(s, lift,i-1);    //左
80     find_frst(s,i+1,right);    //右
81 }
82
83
84 /*查找x y坐标偏移值是否太大*/
85 /*return:  0误差大,    1误差小          */
86 static int find_xy_offset(int x[], int y[],int n)
87 {
88     int i;
89     for(i=n;i>=1;i--)
90     {
91         if(x[i]-x[i-1]>10)    //判断是否大于误差10,
92             return 0;
93         if(y[i]-y[i-1]>10)    //判断是否大于误差10,
94             return 0;

```

```

95     }
96
97     return 1;
98 }
99
100
101     /*定时器函数,实现触摸滑动功能    */
102 void pen_updown_timer(unsigned long cnt)
103 {
104     if((adc_regs->adcdat0>>15)&0x01)    //此时笔尖已经抬起
105     {
106         set_pen_down();    //设置TC中断
107     }
108     else
109     {
110         adc_start();    //启动一次ADC转换
111     }
112 }
113
114
115     /*触摸中断IRQ_TC    */
116 static irqreturn_t tc_handler(int irq, void *dev_id)
117 {
118
119     if((adc_regs->adcdat0>>15)&0x01)    //此时笔尖已经抬起
120     {
121         set_pen_down();
122     }
123
124     else
125     {
126         adc_start();    //启动一次ADC转换
127     }
128     return IRQ_HANDLED;
129 }
130
131
132     /*ADC中断IRQ_ADC:测XY坐标    */
133 static irqreturn_t adc_handler(int irq, void *dev_id)
134 {
135     static int                adc_x[5],adc_y[5];    //保存XY
136     坐标
137     static unsigned char      xy_cnt=0;    //计数
138
139     adc_y[xy_cnt] =adc_regs->adcdat1&0x3ff;    //10位ADC
140     adc_x[xy_cnt] =adc_regs->adcdat0&0x3ff;    //10位ADC
141
142     if (adc_regs->adcdat0 & (1<<15))
143     {
144         /* 已经松开 */
145         xy_cnt = 0;
146         set_pen_down();
147     }
148     else{
149         xy_cnt++;
150         if(xy_cnt>=5)
151         {

```

```

151         xy_cnt=0;
152         find_frst(adc_x,0,4);           // 快速排序x
153         find_frst(adc_y,0,4);           // 快速排序y
154
155         if(find_xy_offset(adc_x,adc_y,4))
156         {
157             printk("x: %04d,y: %04d\n",adc_x[2],adc_y[2]); //中值滤波
158         }
159         set_pen_up();
160         mod_timer(&ts_timer ,jiffies+HZ/100);           //
        启动定时10ms
161     }
162     else //在测一次
163     {
164         adc_start(); //启动 ADC
165     }
166 }
167 return IRQ_HANDLED;
168 }
169
170
171
172
173
174 /*入口函数*/
175 static int myts_init(void)
176 {
177     /*1. 申请input_dev */
178     ts_dev=input_allocate_device();
179     /*2. 设置input_dev*/
180     set_bit(EV_ABS, ts_dev->evbit);
181     set_bit(EV_KEY, ts_dev->evbit);
182     set_bit(BTN_TOUCH, ts_dev->keybit);
183     input_set_abs_params(ts_dev, ABS_X , 0 , 0x3ff , 0 , 0); //adc
        是个10位的,所以为0X3FF
184     input_set_abs_params(ts_dev, ABS_Y , 0 , 0x3ff , 0 , 0); //adc
        是个10位的,所以为0X3FF
185     input_set_abs_params(ts_dev, ABS_PRESSURE, 0 , 1 , 0 , 0);
        //adc是个10位的,所以为0X3FF
186
187     /*3.注册input_dev 驱动设备到内核中*/
188     input_register_device(ts_dev);
189
190     /*4.设置触摸屏相关的硬件*/
191     /*4.1 开启ADC时钟 */
192     ADC_CLK =clk_get(0,"adc");
193     clk_enable(ADC_CLK);
194
195     /*4.2 设置寄存器ADCCON分频,*/
196     adc_regs=ioremap(0x58000000, sizeof(struct adc_regs));
197     adc_regs->adcccon=(1<<14) | (49<<6); //50Mhz/(49+1)=1Mhz
198
199
200     /*4.3 设置中断IRQ_TC IRQ_ADC */

```

```

201     request_irq(IRQ_TC      , tc_handler, IRQF_SAMPLE_RANDOM,
    "pen_updown", 0);
202     request_irq(IRQ_ADC     , adc_handler, IRQF_SAMPLE_RANDOM, "adc"
    , 0);
203
204
205     /*4.4设置寄存器ADCDLY=0xffff */
206     adc_regs->adcdly  =0xffff;
207
208     /*4.5 初始化定时器*/
209     init_timer(&ts_timer);
210     ts_timer.function  =pen_updown_timer;
211     add_timer(&ts_timer);
212
213     /*4.6设置寄存器ADCTSC=0x0d3,开启IRQ_TC中断*/
214     set_pen_down();
215     return 0;
216 }
217
218 /*出口函数*/
219 static void myts_exit(void)
220 {
221     /*1.注销内核里的input_dev、*/
222     input_unregister_device(ts_dev);
223     /*2.释放中断、删除定时器、iounmap注销地址、*/
224     free_irq(IRQ_TC, NULL);
225     free_irq(IRQ_ADC, NULL);
226
227     del_timer(&ts_timer);
228     iounmap(adc_regs);
229
230     /*3.释放input_dev、*/
231     input_free_device(ts_dev);
232 }
233
234 module_init(myts_init);
235 module_exit(myts_exit);
236 MODULE_LICENSE("GPL");

```

常见的调试方法及技巧总结

- 1.printk
- 2.设置内核的oops[Linux驱动调试-根据oops定位错误代码行](#) oops可以实现大容量代码的错误定位。[什么是 Linux “oops” ?](#)

3.内核的KGDB，板子连接仿真器在主机打断点调试内核

4.strace命令

在Linux系统中，strace是一种相当有效的跟踪工具，它的主要特点是可以被用来**监视系统调用**。我们不仅可以用strace调试一个新开始的程序，也可以调试一个已经在运行的程序（这意味着把 strace绑定到个已有的PID上）。[Linux应用调试-strace命令](#)

linux中内核空间及用户空间的区别？

本文以 32 位系统为例介绍内核空间(kernel space)和用户空间(user space)。

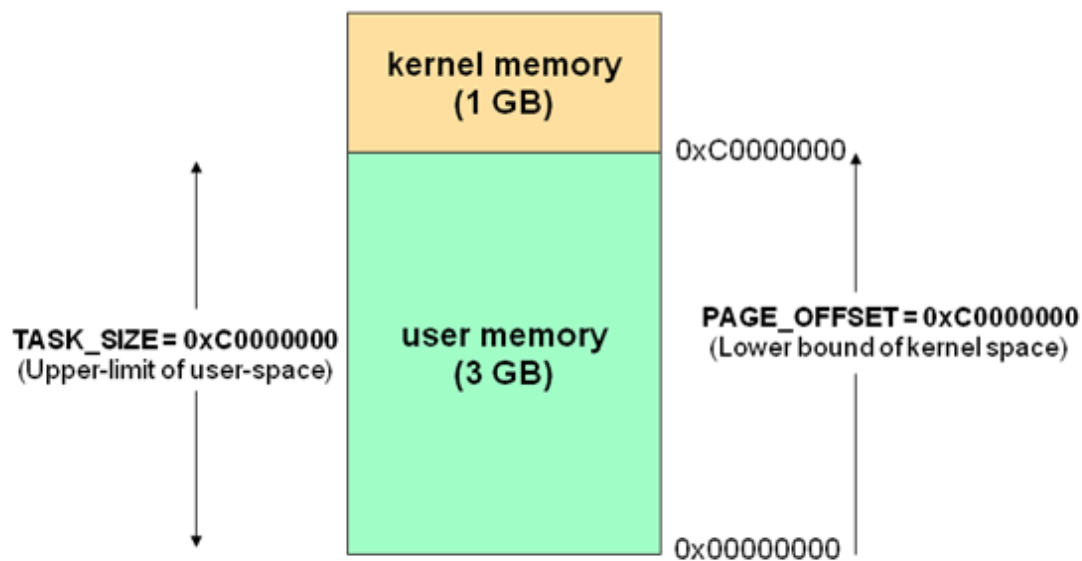
内核空间 and 用户空间

对 32 位操作系统而言，它的寻址空间（虚拟地址空间，或叫线性地址空间）为 4G（ 2^{32} 字节）。也就是说一个进程的最大地址空间为 4G。操作系统的核心是内核(kernel)，它独立于普通的应用程序，可以访问受保护的内存空间，也有访问底层硬件设备的所有权限。为了保证内核的安全，现在的操作系统一般都强制用户进程不能直接操作内核。具体的实现方式基本都是由操作系统将虚拟地址空间划分为两部分，一部分为内核空间，另一部分为用户空间。针对 Linux 操作系统而言，最高的 1G 字节(从虚拟地址 0xC0000000 到 0xFFFFFFFF)由内核使用，称为内核空间。而较低的 3G 字节(从虚拟地址 0x00000000 到 0xBFFFFFFF)由各个进程使用，称为用户空间。

对上面这段内容我们可以这样理解：

每个进程的 4G 地址空间中，最高 1G 都是一样的，即内核空间。只有剩余的 3G 才归进程自己使用。换句话说就是，最高 1G 的内核空间是被所有进程共享的！

下图描述了每个进程 4G 地址空间的分配情况：



https://blog.csdn.net/qq_16933601

为什么需要区分内核空间与用户空间

在 CPU 的所有指令中，有些指令是非常危险的，如果错用，将导致系统崩溃，比如清内存、设置时钟等。如果允许所有的程序都可以使用这些指令，那么系统崩溃的概率将大大增加。

所以，CPU 将指令分为特权指令和非特权指令，对于那些危险的指令，只允许操作系统及其相关模块使用，普通应用程序只能使用那些不会造成灾难的指令。比如 Intel 的 CPU 将特权等级分为 4 个级别：Ring0~Ring3。

其实 Linux 系统只使用了 Ring0 和 Ring3 两个运行级别(Windows 系统也是一样的)。当进程运行在 Ring3 级别时被称为运行在用户态，而运行在 Ring0 级别时被称为运行在内核态。

内核态与用户态

好了我们现在需要再解释一下什么是内核态、用户态：

当进程运行在内核空间时就处于内核态，而进程运行在用户空间时则处于用户态。

在内核态下，进程运行在内核地址空间中，此时 CPU 可以执行任何指令。运行的代码也不受任何的限制，可以自由地访问任何有效地址，也可以直接进行端口的访问。

在用户态下，进程运行在用户地址空间中，被执行的代码要受到 CPU 的诸多检查，它们只能访问映射其地址空间的页表项中规定的在用户态下可访问页面的虚拟地址，且只能对任务状态段(TSS)中 I/O 许可位图(I/O Permission Bitmap)中规定的可访问端口进行直接访问。

对于以前的 DOS 操作系统来说，是没有内核空间、用户空间以及内核态、用户态这些概念的。可以认为所有的代码都是运行在内核态的，因而用户编写的应用程序代码可以很容易的让操作系统崩溃掉。

对于 Linux 来说，通过区分内核空间和用户空间的设计，隔离了操作系统代码(操作系统的代码要比应用程序的代码健壮很多)与应用程序代码。即便是单个应用程序出现错误也不会影响到操作系统的稳定性，这样其它的程序还可以正常的运行(Linux 可是个多任务系统啊！)。所以，区分内核空间和用户空间本质上是要提高操作系统的稳定性及可用性。

如何从用户空间进入内核空间

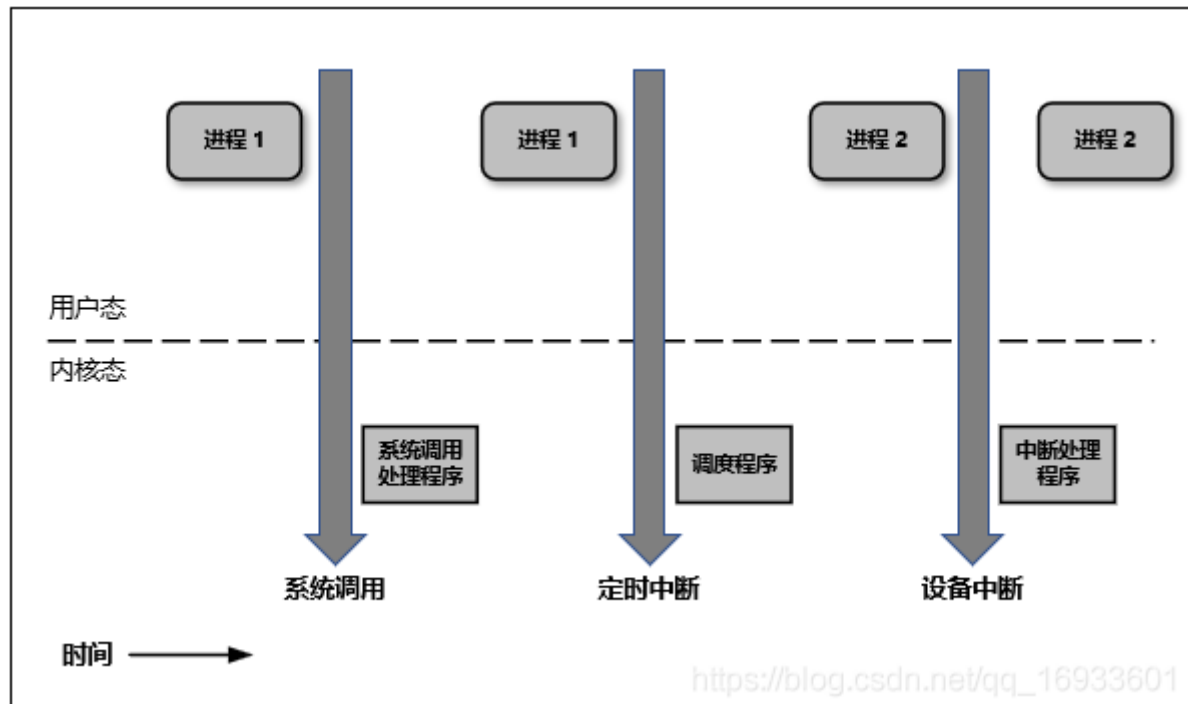
其实所有的系统资源管理都是在内核空间中完成的。比如读写磁盘文件，分配回收内存，从网络接口读写数据等等。我们的应用程序是无法直接进行这样的操作的。但是我们可以通过内核提供的接口来完成这样的任务。

比如应用程序要读取磁盘上的一个文件，它可以向内核发起一个 "系统调用" 告诉内核："我要读取磁盘上的某某文件"。其实就是通过一个特殊的指令让进程从用户态进入到内核态(到了内核空间)，在内核空间中，CPU 可以执行任何的指令，当然也包括从磁盘上读取数据。具体过程是先把数据读取到内核空间中，然后再把数据拷贝到用户空间并从内核态切换到用户态。此时应用程序已经从系统调用中返回并且拿到了想要的文件，可以开开心心的往下执行了。

简单说就是应用程序把高科技的事情(从磁盘读取文件)外包给了系统内核，系统内核做这些事情既专业又高效。

对于一个进程来讲，从用户空间进入内核空间并最终返回到用户空间，这个过程是十分复杂的。举个例子，比如我们经常接触的概念 "堆栈"，其实进程在内核态和用户态各有一个堆栈。运行在用户空间时进程使用的是用户空间中的堆栈，而运行在内核空间时，进程使用的是内核空间中的堆栈。所以说，Linux 中每个进程有两个栈，分别用于用户态和内核态。

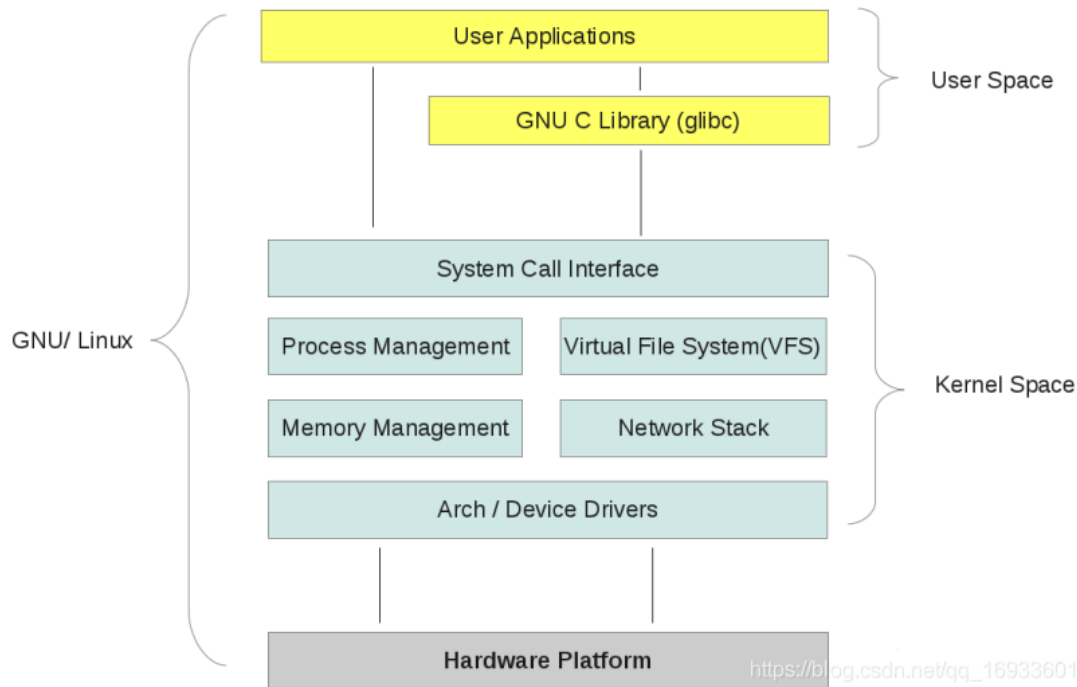
下图简明的描述了用户态与内核态之间的转换：



既然用户态的进程必须切换成内核态才能使用系统的资源，那么我们接下来就看看进程一共有多少方式可以从用户态进入到内核态。概括的说，有三种方式：**系统调用**、**软中断**和**硬件中断**。这三种方式每一种都涉及到大量的操作系统知识，所以这里不做展开。

整体结构

接下来我们从内核空间和用户空间的角度看一看整个 Linux 系统的结构。它大体可以分为三个部分，从下往上依次为：硬件 -> 内核空间 -> 用户空间。如下图所示：



在硬件之上，内核空间中的代码控制了硬件资源的使用权，用户空间中的代码只有通过内核暴露的系统调用接口(System Call Interface)才能使用到系统中的硬件资源。其实，不光是 Linux，Windows 操作系统的设计也是大同小异。

实际上我们可以将每个处理器在任何指定时间点上的活动概括为下列三者之一：

运行于用户空间，执行用户进程。

运行于内核空间，处于进程上下文，代表某个特定的进程执行。

运行于内核空间，处于中断上下文，与任何进程无关，处理某个特定的中断。

以上三点几乎包括所有的情况，比如当 CPU 空闲时，内核就运行一个空进程，处于进程上下文，但运行在内核空间。

说明：Linux 系统的中断服务程序不在进程的上下文中执行，它们在一个与所有进程都无关的、专门的中断上下文中执行。之所以存在一个专门的执行环境，就是为了保证中断服务程序能够在第一时间响应和处理中断请求，然后快速地退出。

总结

现代的操作系统大都通过内核空间和用户空间的设计来保护操作系统自身的安全性和稳定性。所以在我们阅读有关操作系统的资料时经常遇到内核空间、用户空间和内核态、用户态等概念，希望本文能够帮助您理解这些基本的概念。

用户空间与内核通信方式有哪些？

下面总结了7种方式，主要对以前不是很熟悉的方式做了编程实现，以便加深印象。

1.使用API：这是最常使用的一种方式了

A.get_user(x, ptr)：在内核中被调用，获取用户空间指定地址的数值并保存到内核变量x中。

B.put_user(x, ptr)：在内核中被调用，将内核空间的变量x的数值保存到用户空间指定地址处。

C.Copy_from_user()/copy_to_user()：主要应用于设备驱动读写函数中，通过系统调用触发。

2.使用proc文件系统：和sysfs文件系统类似，也可以作为内核空间和用户空间交互的手段。

/proc 文件系统是一种虚拟文件系统，通过他可以作为一种linux内核空间和用户空间的。与普通文件不同，这里的虚拟文件的内容都是动态创建的。

使用/proc文件系统的方式很简单。调用create_proc_entry，返回一个proc_dir_entry指针，然后去填充这个指针指向的结构就好了，我下面的这个测试用例只是填充了其中的read_proc属性。

下面是一个简单的测试用例，通过读虚拟出的文件可以得到内核空间传递过来的“proc ! test by qiankun !”字符串。

```
[root@localhost proc]# make clean
make -C /lib/modules/2.6.32-504.el6.i686/build M=/home/melody/proc clean
make[1]: Entering directory `/usr/src/kernels/2.6.32-504.el6.i686'
CLEAN /home/melody/proc/.tmp_versions
CLEAN /home/melody/proc/Module.symvers /home/melody/proc/modules.order
make[1]: Leaving directory `/usr/src/kernels/2.6.32-504.el6.i686'
[root@localhost proc]# make
make -C /lib/modules/2.6.32-504.el6.i686/build M=/home/melody/proc modules
make[1]: Entering directory `/usr/src/kernels/2.6.32-504.el6.i686'
CC [M] /home/melody/proc/proc.o
/home/melody/proc/proc.c: 在函数 'init' 中:
/home/melody/proc/proc.c: 21: 警告：从不兼容的指针类型赋值
Building modules, stage 2.
MODPOST 1 modules
CC /home/melody/proc/proc.mod.o
LD [M] /home/melody/proc/proc.ko.unsigned
NO SIGN [M] /home/melody/proc/proc.ko
make[1]: Leaving directory `/usr/src/kernels/2.6.32-504.el6.i686'
[root@localhost proc]# insmod proc.ko 模块加载
[root@localhost proc]# cat /proc/0Kproctest 读取虚拟文件
proc ! test by qiankun ! 成功读取内核空间的数据
[root@localhost proc]#
```

3.使用sysfs文件系统+kobject：其实这个以前是编程实现过得，但是那天太紧张忘记了，T_T。每个在内核中注册的kobject都对应着sysfs系统中的一个目录。可以通过读取根目录下的sys目录中的文件来获得相应的信息。除了sysfs文件系统和proc文件系统之外，一些其他的虚拟文件系统也能同样达到这个效果。

4.netlink：netlink socket提供了一组类似于BSD风格的API，用于用户态和内核态的IPC。相比于其他的用户态和内核态IPC机制，netlink有几个好处：1.使用自定义一种协议完成数据交换，不需要添加一个文件等。2.可以支持多点传送。3.支持内核先发起会话。4.异步通信，支持缓存机制。

对于用户空间，使用netlink比较简单，因为和使用socket非常的类似，下面说一下内核空间对netlink的使用，主要说一下最重要的create函数，函数原型如下：

```
1 extern struct sock *netlink_kernel_create(struct net *net,
2                                           int unit,unsigned int groups,
3                                           void (*input)(struct sk_buff *skb),
4                                           struct mutex *cb_mutex,
5                                           struct module *module);
```

第一个参数一般传入&init_net。

第二个参数指的是netlink的类型，系统定义了16个，我们如果使用的话最好自己定义。这个需和用户空间所使用的创建socket的第三个参数一致，才可以完成通信。

第四个参数指的是一个回调函数，当接受到一个消息的时候会调用这个函数。回调函数的参数为struct sk_buff类型的结构体。通过分析其结构成员可以得到传递过来的数据

第六个参数一般传入的是THIS_MODULE。指当前模块。

下面是对netlink的一个简单测试，将字符串“netlink test by qiankun”通过netlink输出到内核，内核再把字符串返回。Netlink类型使用的是22。

```
[root@localhost netlink]# insmod netlink.ko 加载netlink内核模块
[root@localhost netlink]# vim send.c
[root@localhost netlink]# gcc send.c -o send
[root@localhost netlink]# ./send 运行用户控件程序
test success ,rcv from kernel: netlink test by qiankun! 成功接收到字符串
[root@localhost netlink]#
```

5.文件：应该说这是一种比较笨拙的做法，不过确实可以这样用。当处于内核空间的时候，直接操作文件，将想要传递的信息写入文件，然后用户空间可以读取这个文件便可以得到想要的數據了。下面

是一个简单的测试程序，在内核态中，程序会向“/home/melody/str_from_kernel”文件中写入一条字符串，然后我们在用户态读取这个文件，就可以得到内核态传输过来的数据了。

```
root@localhost file# make
make -C /lib/modules/2.6.32-504.el6.i686/build M=/home/melody/file modules
make[1]: Entering directory '/usr/src/kernels/2.6.32-504.el6.i686'
CC [M] /home/melody/file/file.o
/home/melody/file/file.c:11: 警告：函数声明不是一个原型
/home/melody/file/file.c:28: 警告：函数声明不是一个原型
/home/melody/file/file.c: 在函数 '_exittest'中:
/home/melody/file/file.c:32: 警告：返回了不兼容的指针类型
Building modules, stage 2.
MODPOST 1 modules
CC /home/melody/file/file.mod.o
LD [M] /home/melody/file/file.ko.unsigned
NO SIGN [M] /home/melody/file/file.ko
make[1]: Leaving directory '/usr/src/kernels/2.6.32-504.el6.i686'
root@localhost file# ls
file.c file.ko file.ko.unsigned file.mod.c file.mod.o file.o Makefile modules.order Module.symvers str_from_kernel
root@localhost file# cat str_from_kernel
现在文件中没有数据
root@localhost file# insmod file.ko
加载内核模块，在这个内核模块中会向str_from_kernel文件中写入一条字符串
root@localhost file# cat str_from_kernel
再次读取文件，发现数据已经被内核模块成功写入
string from kernel: test by qiankun!
root@localhost file#
```

6.使用mmap系统调用：可以将内核空间的地址映射到用户空间。在以前做嵌入式的时候用到几次。一方面可以在driver中修改Struct file_operations结构中的mmap函数指针来重新实现一个文件对应的映射操作。另一方面，也可以直接打开/dev/mem文件，把物理内存中的某一页映射到进程空间中的地址上。

其实，除了重写Struct file_operations中mmap函数，我们还可以重写其他的方法如ioctl等，来达到驱动内核空间和用户空间通信的方式。

7.信号：从内核空间向进程发送信号。这个倒是经常遇到，用户程序出现重大错误，内核发送信号杀死相应进程。

linux中中断的实现机制，tasklet与workqueue的区别及底层实现区别？为什么要区分上半部和下半部？

tasklet和workqueue区别？

tasklet运行于中断上下文，不允许阻塞、休眠，而workqueue运行与进程上下文，可以休眠和阻塞。

为什么要区分上半部和下半部？

中断服务程序异步执行，可能会中断其他的重要代码，包括其他中断服务程序。因此，为了避免被中断的代码延迟太长的时间，中断服务程序需要尽快运行，而且执行的时间越短越好，所以中断程序只作必须的工作，其他工作推迟到以后处理。所以Linux把中断处理切为两个部分：上半部和下半部。上半部就是中断处理程序，它需要完成的工作越少越好，执行得越快越好，一旦接收到一个中断，它就立即开始执行。**像对时间敏感、与硬件相关、要求保证不被其他中断打断的任务往往放在中断处理程序中执行；而剩下的与中断有相关性但是可以延后的任务，如对数据的操作处理，则推迟一点由下半部完成。**下半部分延后执行且执行期间可以相应所有中断，这样可使系统处于中断屏蔽状态的时间尽可能的短，提高了系统的响应能力。实现了程序运行快同时完成的工作量多的目标。

由于软中断必须使用可重入函数，这就导致设计上的复杂度变高，作为设备驱动程序的开发来说，增加了负担。而如果某种应用并不需要在多个CPU上并行执行，那么软中断其实是没有必要的。因此诞生了弥补以上两个要求的tasklet。它具有以下特性：

- a) 一种特定类型的tasklet只能运行在一个CPU上，不能并行，只能串行执行。
- b) 多个不同类型的tasklet可以并行在多个CPU上。

c) 软中断是静态分配的，在内核编译好之后，就不能改变。但tasklet就灵活许多，可以在运行时改变（比如添加模块时）。tasklet是在两种软中断类型的基础上实现的，因此如果不需要软中断的并行特性，tasklet就是最好的选择。也就是说tasklet是软中断的一种特殊用法，即延迟情况下的串行执行。

linux中断的响应执行流程？中断的申请及何时执行(何时执行中断处理函数)？

中断的响应流程：cpu接受中断->保存中断上下文跳转到中断处理历程->执行中断上半部->执行中断下半部->恢复中断上下文。

中断的申请request_irq的正确位置：应该是在第一次打开、硬件被告知终端之前。

驱动中操作物理绝对地址为什么要先ioremap?

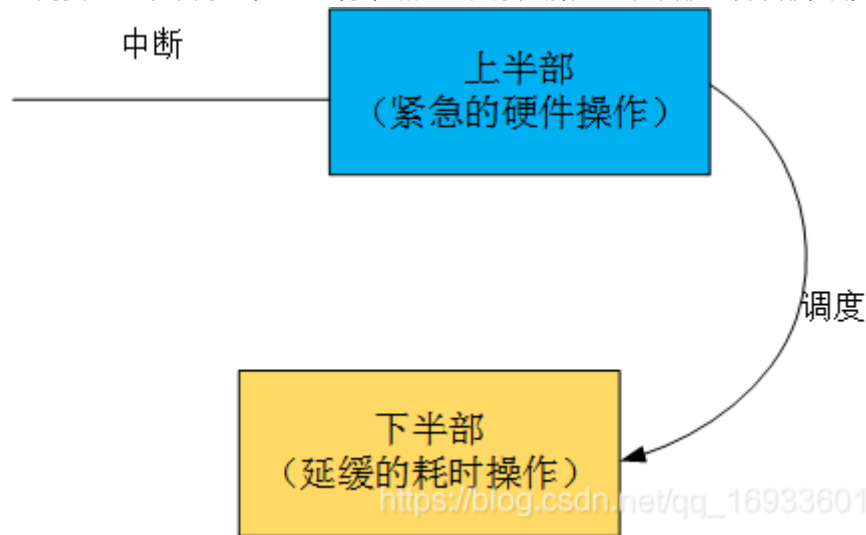
因为内核没有办法直接访问物理内存地址，必须先通过ioremap获得对应的虚拟地址。

Linux内核中断顶半部和底半部的理解

中断上半部、下半部的概念

设备的中断会打断内核进程中的正常调度和运行，系统对更高吞吐率的追求势必要求中断服务程序尽量短小精悍。但是，这个良好的愿望往往与现实并不吻合。在大多数真实的系统中，当中断到来时，要完成的工作往往并不会是短小的，它可能要进行较大的耗时处理。

下图描述了Linux内核的中断处理机制。为了在中断执行时间尽量短和中断处理需完成的工作尽量大之间找到一个平衡点，Linux将中断处理程序分解为两个半部：顶半部和底半部。



顶半部用于完成尽量少的**比较紧急的功能**，它往往只是简单地**读取寄存器中的中断状态**，并在清除中断标志后就进行“**登记中断**”的工作。“登记中断”意味着将底半部处理程序挂到该设备的底半部执行队列中去。这样，顶半部执行的速度就会很快，从而可以服务更多的中断请求。

现在，中断处理工作的重心就落在了底半部的头上，需用它来完成中断事件的绝大多数任务。底半部**几乎做了中断处理程序所有的事情**，而且可以被**新的中断打断**，这也是底半部和顶半部的最大不同，因为**顶半部往往被设计成不可中断**。底半部相对来说并不是非常紧急的，而且相对比较耗时，不在硬件中断服务程序中执行。

尽管顶半部、底半部的结合能够善系统的响应能力，但是，僵化地认为Linux设备驱动中的中断处理一定要分两个半部则是不对的。**如果中断要处理的工作本身很少，则完全可以直接在顶半部全部完成。**

其他操作系统中对中断的处理也采用了类似于Linux的方法，真正的硬件中断服务程序都斥尽量短。因此，许多操作系统都提供了中断上下文和非中断上下文相结合的机制，将中断的耗时工作保留到非中断上下文去执行。

实现中断下半部的三种方法

软中断

软中断（Softirq）也是一种传统的底半部处理机制，它的执行时机通常是**顶半部返回的时候**，tasklet是基于软中断实现的，因此也运行于软中断上下文。

在Linux内核中，用 softirq_action结构体表征一个软中断，这个结构体包含软中断处理函数指针和传递给该函数的参数。使用 open_softirq（）函数可以注册软中断对应的处理函数，而 raise_softirq（）函数可以触发一个软中断。

软中断和 tasklet运行于软**中断上下文**，仍然属于原子上下文的一种，而工作队列则运行于**进程上下文**。因此，在软中断和 tasklet处理函数中**不允许睡眠**，而在**工作队列处理函数中允许睡眠**。

local_bh_disable（）和 local_bh_enable（）是内核中用于禁止和使能软中断及 tasklet底半部机制的函数

软中断模版

```

1  asmlinkage void do_softirq(void)
2  {
3      __u32 pending;
4      unsigned long flags;
5
6      /* 判断是否在中断处理中，如果正在中断处理，就直接返回 */
7      if (in_interrupt())
8          return;
9
10     /* 保存当前寄存器的值 */
11     local_irq_save(flags);
12
13     /* 取得当前已注册软中断的位图 */
14     pending = local_softirq_pending();
15
16     /* 循环处理所有已注册的软中断 */
17     if (pending)
18         __do_softirq();
19
20     /* 恢复寄存器的值到中断处理前 */
21     local_irq_restore(flags);
22 }
```

tasklet

tasklet的使用较简单，它的执行上下文是软中断，执行时机通常是顶半部返回的时候。我们只需要定义 tasklet及其处理函数，并将两者关联则可，例如

```

1  void my_tasklet_func(unsigned long); /*定义一个处理函数*/
2  DECLARE_TASKLET(my_tasklet, my_tasklet_func, data);
3  /*定义一个tasklet结构my_tasklet，与my_tasklet_func(data)函数相关联*/
```

代码DECLARE_TASKLET（my_tasklet，my_tasklet_func，data）实现了定义名称为my_tasklet的tasklet，并将其与my_tasklet_func（）这个函数绑定，而传入这个函数的参数为data。在需要调度tasklet的时候引用一个tasklet_schedule（）函数就能使系统在适当的时候进行调度运行：

```

1  tasklet_schedule(&my_tasklet);
```

使用tasklet作为底半部处理中断的设备驱动程序模板下所示（仅包含与中断相关的部分）。

tasklet函数模版

```

1  /* 定义tasklet和底半部函数并将它们关联 */
2  void xxx_do_tasklet(unsigned long);
3  DECLARE_TASKLET(xxx_tasklet, xxx_do_tasklet, 0);
4
5  /* 中断处理底半部 */
6  void xxx_do_tasklet(unsigned long)
7
8  ...
9
10
11 /* 中断处理顶半部 */
12 irqreturn_t xxx_interrupt(int irq, void *dev_id)
13 {
14     ...
15     tasklet_schedule(&xxx_tasklet);
16     ...
17 }
18
19 /* 设备驱动模块加载函数 */
20 int __init xxx_init(void)
21 {
22     ...
23     /* 申请中断 */
24     result = request_irq(xxx_irq, xxx_interrupt,
25     0, "xxx", NULL);
26     ...
27     return IRQ_HANDLED;
28 }
29
30 /* 设备驱动模块卸载函数 */
31 void __exit xxx_exit(void)
32 {
33     ...
34     /* 释放中断 */
35     free_irq(xxx_irq, xxx_interrupt);
36     ...
37 }
```

上述程序在模块加载函数中申请中断（第24~25行），并在模块卸载函数free_irq(xxx_irq, xxx_interrupt);中释放它。对应于xxx_irq的中断处理程序被设置为xxx_interrupt（）函数，在这个函数中，tasklet_schedule（&xxx_tasklet）调度被定义的tasklet函数xxx_do_tasklet（）在适当的时候执行。

工作队列

工作队列的使用方法和tasklet非常相似，但是工作队列的执行上下文是内核线程，因此可以调度和睡眠。下面的代码用于定义一个工作队列和一个底半部执行函数

```

1  struct work_struct my_wq; /* 定义一个工作队列 */
2  void my_wq_func(struct work_struct *work); /* 定义一个处理函数 */
```

通过INIT_WORK（）可以初始化这个工作队列并将工作队列与处理函数绑定：

```
1 INIT_WORK(&my_wq, my_wq_func);
2 /* 初始化工作队列并将其与处理函数绑定 */
```

与tasklet_schedule () 对应的用于调度工作队列执行的函数为schedule_work ()，如：

```
1 schedule_work(&my_wq); /* 调度工作队列执行 */
```

工作队列函数模版

```
1 /* 定义工作队列和关联函数 */
2 struct work_struct xxx_wq;
3 void xxx_do_work(struct work_struct *work);
4
5 /* 中断处理底半部 */
6 void xxx_do_work(struct work_struct *work)
7
8 ...
9
10
11 /*中断处理顶半部*/
12 irqreturn_t xxx_interrupt(int irq, void *dev_id)
13 {
14 ...
15 schedule_work(&xxx_wq);
16 ...
17 return IRQ_HANDLED;
18 }
19
20 /* 设备驱动模块加载函数 */
21 int xxx_init(void)
22 {
23 ...
24 /* 申请中断 */
25 result = request_irq(xxx_irq, xxx_interrupt,
26 0, "xxx", NULL);
27 ...
28 /* 初始化工作队列 */
29 INIT_WORK(&xxx_wq, xxx_do_work);
30 ...
31 }
32
33 /* 设备驱动模块卸载函数 */
34 void xxx_exit(void)
35 {
36 ...
37 /* 释放中断 */
38 free_irq(xxx_irq, xxx_interrupt);
39 ...
40 }
```

工作队列早期的实现是在每个CPU核上创建一个worker内核线程，所有在这个核上调度的工作都在该worker线程中执行，其并发性显然差强人意。在Linux 2.6.36以后，转而实现“Concurrency-managedworkqueues”，简称cmwq，cmwq会自动维护工作队列的线程池以提高并发性，同时保持了API的向后兼容。

软中断和硬中断的区别

硬中断：

1. 硬中断是由**硬件产生的**，比如，像磁盘，网卡，键盘，时钟等。**每个设备或设备集都有它自己的IRQ（中断请求）**。基于IRQ，CPU可以将相应的请求分发到对应的硬件驱动上（注：硬件驱动通常是内核中的一个子程序，而不是一个独立的进程）。

2. 处理中断的驱动是需要运行在CPU上的，因此，当中断产生的时候，CPU会中断当前正在运行的任务，来处理中断。在有多核心的系统上，一个中断通常只能中断一颗CPU（也有一种特殊的情况，就是在大型主机上是有硬件通道的，它可以在没有主CPU的支持下，可以同时处理多个中断。）。

3. **硬中断可以直接中断CPU**。它会引起内核中相关的代码被触发。对于那些需要花费一些时间去处理的进程，中断代码本身也可以被其他的硬中断中断。

4. 对于时钟中断，内核调度代码会将当前正在运行的进程挂起，从而让其他的进程来运行。它的存在是为了让调度代码（或称为调度器）可以调度多任务。

软中断：

1. 软中断的处理非常像硬中断。然而，它们仅仅是由**当前正在运行的进程**所产生的。

2. 通常，软中断是一些对I/O的请求。这些请求会调用内核中可以调度I/O发生的程序。**对于某些设备，I/O请求需要被立即处理，而磁盘I/O请求通常可以排队并且可以稍后处理**。根据I/O模型的不同，进程或许会被挂起直到I/O完成，此时内核调度器就会选择另一个进程去运行。I/O可以在进程之间产生并且调度过程通常和磁盘I/O的方式是相同。

3. 软中断仅与内核相联系。而内核主要负责对需要运行的任何其他进程进行调度。一些内核允许设备驱动的一些部分存在于用户空间，并且当需要的时候内核也会调度这个进程去运行。

4. **软中断并不会直接中断CPU**。**也只有当前正在运行的代码（或进程）才会产生软中断**。这种中断是一种需要内核为正在运行的进程去做一些事情（通常为I/O）的请求。有一个特殊的软中断是Yield调用，它的作用是请求内核调度器去查看是否有一些其他的进程可以运行。

硬中断、软中断和信号的区别

硬中断是**外部设备对CPU的中断**，软中断是**中断底半部的一种处理机制**，而信号则是由**内核（或其他进程）对某个进程的中断**。在涉及系统调用的场合，人们也常说通过软中断（例如ARM为swi）陷入内核，此时软中断的概念是指由**软件指令引发的中断**，和我们这个地方说的softirq是两个完全不同的概念，一个是software，一个是soft。

需要特别说明的是，软中断以及基于软中断的tasklet如果在某段时间内大量出现的话，内核会把后续软中断放入ksoftirqd内核线程中执行。总的来说，**中断优先级高于软中断，软中断又高于任何一个线程**。软中断适度线程化，可以缓解高负载情况下系统的响应。

linux中RCU原理

答：RCU机制是Linux2.6之后提供的一种数据一致性访问的机制，从RCU（read-copy-update）的名称上看，我们就能对他的实现机制有一个大概的了解，在修改数据的时候，首先需要读取数据，然后生成一个副本，对副本进行修改，修改完成之后再老数据update成新的数据，此所谓RCU。

RCU的思路实际上很简单，如下所示：

a. 对于读操作，可以直接对共享资源进行访问，但是前提是需要CPU支持访存操作的原子化，现代CPU对这一点都做了保证。但是RCU的读操作上下文是不可抢占的（这一点在下面解释），所以读访问共享资源时可以采用read_rcu_lock()，该函数的工作是停止抢占。

b. 对于写操作，其需要将原来的老数据作一次备份（copy），然后对备份数据进行修改，修改完毕之后再用新数据更新老数据，更新老数据时采用了rcu_assign_pointer（）宏，在该函数中首先屏障一下memory，然后修改老数据。这个操作完成之后，需要进行老数据资源的回收。操作线程向系统注册回收方法，等待回收。采用数据备份的方法可以实现读者与写者之间的并发操作，但是不能解决多个写者之间的同步，所以当存在多个写者时，需要通过锁机制对其进行互斥，也就是在同一时刻只能存在一个写者。

c. 在RCU机制中存在一个垃圾回收的daemon，当共享资源被update之后，可以采用该daemon实现老数据资源的回收。回收时间点就是在update之前的所有的读者全部退出。由此可见写者在update之后是需要睡眠等待的，需要等待读者完成操作，如果在这个时刻读者被抢占或者睡眠，那么很可能会导致系统死锁。因为此时写者在等待读者，读者被抢占或者睡眠，如果正在运行的线程需要访问读者和写者已经占用的资源，那么死锁的条件就很有可能形成了。

RCU思想是比较简单的，其核心内容紧紧围绕“写时拷贝”，采用RCU机制，能够保证在读写操作共享资源时，基本不需要取锁操作，能够在一定程度上提升性能。但是该机制的应用是有条件的，对于读多写少的应用，机制的开销比较小，性能会大幅度提升，但是如果写操作较多时，开销将会增大，性能不一定会有所提升。总体来说，RCU机制是对rw_lock的一种优化。

linux中软中断的实现原理

答：中断的执行的执行过程分为上下两部分，但是下半部分执行过程中有两个缺陷：（1）在任意时刻，系统只能有一个CPU可以执行下半部代码，以防止两个或多个CPU同时来执行下半部函数而相互干扰。因此下半部代码的执行是严格“串行化”的。（2）下半部函数不允许嵌套。这两个缺点在单CPU系统中是无关紧要的，但在多CPU系统中却是非常致命的。因为下半部机制的严格串行化执行显然没有充分利用多cpu系统的多CPU特点。为此，Linux2.4内核在下半部机制的基础上进行了扩展，这就是所谓的“软中断请求”（softirq）机制。整个softirq机制的设计与实现中自始至终都贯彻了一个思想：“谁触发，谁执行”（Who marks，Who runs），也即触发软中断的那个CPU负责执行它所触发的软中断，而且每个CPU都由它自己的软中断触发与控制机制。这个设计思想也使得softirq机制充分利用了SMP系统的性能和特点。

软中断的工作过程模拟了硬中断的处理过程：

- a、当某一软中断事件发生后，首先需要设置对应的中断标记位，触发中断、设置软中断状态。
- b、然后唤醒守护线程去检测中断状态寄存器（在Linux中 软中断daemon线程函数为do_softirq（））
- c、如果通过查询发现某一软中断事务发生之后，那么通过软中断向量表调用软中断服务程序action（）。

这就是软中断的过程，与硬件中断唯一不同的地方是从中断标记到中断服务程序的映射过程。在CPU的硬件中断发生之后，CPU需要将硬件中断请求通过向量表映射成具体的服务程序，这个过程是硬件自动完成的，但是软中断不是，其需要守护线程去实现这一过程，这也就是软件模拟的中断，故称之为软中断。

内核中的并发和竞争简介

在早期的Linux内核中，并发的来源相对较少。早期内核不支持对称多处理（symmetric multi processing,SMP），因此，导致并发执行的唯一原因是对硬件中断的服务。这种情况处理起来较为简单，但并不适用于为获得更好的性能而使用更多处理器且强调快速响应事件的系统。

为了响应现代硬件和应用程序的需求，Linux内核已经发展到同时处理更多事情的时代。Linux系统是个多任务操作系统，会存在多个任务同时访问同一片内存区域的情况，这些任务可能会相互覆盖这段内存中的数据，造成内存数据混乱。针对这个问题必须要做处理，严重的话可能会导致系统崩溃。现在的Linux系统并发产生的原因很复杂，总结一下有下面几个主要原因：

1. 多线程并发访问，Linux是多任务（线程）的系统，所以多线程访问是最基本的原因。

2. 抢占式并发访问，内核代码是可抢占的，因此，我们的驱动程序代码可能在任何时候丢失对处理器的独占
3. 中断程序并发访问，设备中断是异步事件，也会导致代码的并发执行。
4. SMP（多核）核间并发访问，现在ARM架构的多核SOC很常见，多核CPU存在核间并发访问。正在运行的多个用户空间进程可能以一种令人惊讶的组合方式访问我们的代码，SMP系统甚至可在不同的处理器上同时执行我们的代码。

只要我们的程序在运转当中，就有可能发生并发和竞争。比如，当两个执行线程需要访问**相同的数据结构**（或硬件资源）时，混合的可能性就永远存在。因此在设计自己的驱动程序时，就应该避免资源的共享。**如果没有并发的访问，也就不会有竞态的产生**。因此，仔细编写的内核代码应该具有**最少的共享**。这种思想的最明显应用就是**避免使用全局变量**。如果我们将资源放在多个执行线程都会找到的地方（临界区），则必须有足够的理由。

如何防止我们的数据被并发访问呢？这个时候就要建立一种保护机制，下面介绍几种内核提供的几种并发和竞争的处理方法。

原子操作

原子操作简介

原子，在早接触到是在化学概念中。原子指化学反应不可再分的基本微粒。同样的，在内核中所说的原子操作表示这一个访问是一个步骤，必须一次性执行完，不能被打断，不能再进行拆分。

例如，在多线程访问中，我们的线程一对a进行赋值操作，`a=1`，线程二也对a进行赋值操作 `a=2`，我们理想的执行顺序是线程一先执行，线程二再执行。但是很有可能在线程一执行的时候被其他操作打断，使得线程一最后的执行结果变为 `a=2`。要解决这个问题，必须保证我们的线程一在对数据访问的过程中不能被其他的操作打断，一次性执行完成。

整型原子操作函数

函数	描述
ATOMIC_INIT(int i)	定义原子变量的时候对其初始化。
int atomic_read(atomic_t *v)	读取 v的值，并且返回
void atomic_set(atomic_t *v, int i)	向 v写入 i值。
void atomic_add(int i, atomic_t *v)	给 v加上 i值。
void atomic_sub(int i, atomic_t *v)	从 v减去 i值。
void atomic_inc(atomic_t *v)	给 v加 1，也就是自增。
void atomic_dec(atomic_t *v)	从 v减 1，也就是自减。
int atomic_dec_return(atomic_t *v)	从 v减 1，并且返回v的值。
int atomic_inc_return(atomic_t *v)	给 v加 1，并且返回 v的值。
int atomic_sub_and_test(int i, atomic_t *v)	从 v减 i，如果结果为0就返回真，否则就返回假
int atomic_dec_and_test(atomic_t *v)	从 v减 1，如果结果为0就返回真，否则就返回假
int atomic_inc_and_test(atomic_t *v)	给 v加 1，如果结果为0就返回真，否则就返回假
int atomic_add_negative(int i, atomic_t *v)	给 v加 i，如果结果为负就返回真，否则返回假

注：64位的整型原子操作只是将“atomic”前缀换成“atomic64”，将int换成long long。

位原子操作函数

函数	描述
void set_bit(int nr, void *p)	将p地址的nr位置1
void clear_bit(int nr,void *p)	将p地址的nr位清零
void change_bit(int nr, void *p)	将p地址的nr位反转
int test_bit(int nr, void *p)	获取p地址的nr位的值
int test_and_set_bit(int nr, void *p)	将p地址的nr位置1，并且返回nr位原来的值
int test_and_clear_bit(int nr, void *p)	将p地址的nr位清0，并且返回nr位原来的值
int test_and_change_bit(int nr, void *p)	将p地址的nr位翻转，并且返回nr位原来的值

原子操作例程

```

1  /* 定义原子变量，初值为1*/
2  static atomic_t xxx_available = ATOMIC_INIT(1);
3  static int xxx_open(struct inode *inode, struct file *filp)
4  {
5      ...
6      /* 通过判断原子变量的值来检查LED有没有被别的应用使用 */
7      if (!atomic_dec_and_test(&xxx_available)) {
8          /* 小于0的话就加1,使其原子变量等于0*/
9          atomic_inc(&xxx_available);
10         /* LED被使用，返回忙*/
11         return - EBUSY;
12     }
13     ...
14     /* 成功 */
15     return 0;
16     static int xxx_release(struct inode *inode, struct file *filp)
17     {
18         /* 关闭驱动文件的时候释放原子变量 */
19         atomic_inc(&xxx_available);
20         return 0;
21     }

```

自旋锁

上面我们介绍了原子变量，从它的操作函数可以看出，原子操作只能针对**整型变量或者位**。假如我们有一个结构体变量需要被线程A所访问，在线程A访问期间不能被其他线程访问，这怎么办呢？自旋锁就可以完成对结构体变量的保护。

自旋锁简介

自旋锁，顾名思义，我们可以把他理解成厕所门上的一把锁。这个厕所门只有一把钥匙，当我们进去时，把钥匙取下来，进去后反锁。那么当第二个人想进来，必须等我们出去后才可以。当第二个人在外面等待时，可能会一直等待在门口转圈。

我们的自旋锁也是这样，自旋锁只有锁定和解锁两个状态。当我们进入拿上钥匙进入厕所，这就相当于自旋锁锁定的状态，期间谁也不可以进来。当第二个人想要进来，这相当于线程B想要访问这个共享资源，但是目前不能访问，所以线程B就一直在原地等待，一直查询是否可以访问这个共享资源。当我们从厕所出来后，这个时候就“解锁”了，只有再这个时候线程B才能访问。

假如，在厕所的人待的时间太长怎么办？外面的人一直等待吗？如果换做是我们，肯定不会这样，简直浪费时间，可能我们会寻找其他方法解决问题。自旋锁也是这样的，如果线程A持有**自旋锁时间过长，显然会浪费处理器的时间，降低了系统性能**。我们知道CPU最伟大的发明就在于多线程操作，这个时候让线程B在这里傻傻的不知道还要等待多久，显然是不合理的。因此，如果**自旋锁只适合短期持有**，如果遇到需要长时间持有的情况，我们就要换一种方式了（下文的互斥体）。

自旋锁操作函数

函数	描述
DEFINE_SPINLOCK(spinlock_t lock)	定义并初始化一个自旋变量
int spin_lock_init(spinlock_t *lock)	初始化自旋锁
void spin_lock(spinlock_t *lock)	获取指定的自旋锁，也叫加锁
void spin_unlock(spinlock_t *lock)	释放指定的自旋锁。
int spin_trylock(spinlock_t *lock)	尝试获取指定的锁，如果没有获取到，返回0
int spin_is_locked(spinlock_t *lock)	检查指定的自旋锁是否被获取，如果没有被获取返回非0，否则返回0.

自旋锁是主要为了**多处理器系统**设计的。对于**单处理器且内核不支持抢占的系统**，一旦进入了自旋状态，则会永远自旋下去。因为，没有任何线程可以获取CPU来释放这个锁。因此，在单处理器且内核不支持抢占的系统中，**自旋锁会被设置为空操作**。

以上列表中的函数**适用于SMP或支持抢占的单CPU**下线程之间的并发访问，也就是用于线程与线程之间，被自旋锁保护的临界区**一定不能调用任何能够引起睡眠和阻塞（其实本质仍然是睡眠）的API函数**，否则的话可能会导致死锁现象的发生。自旋锁会**自动禁止抢占**，也就说当线程A得到锁以后会**暂时禁止内核抢占**。如果线程A在持有锁期间进入了**休眠状态**，那么线程A会**自动放弃CPU使用权**。线程B开始运行，线程B也想要获取锁，但是此时锁被A线程持有，而且**内核抢占还被禁止了**！线程B无法被调度出去，那么线程A就无法运行，锁也就无法释放**死锁**发生了！

当线程之间发生并发访问时，如果此时中断也要插一脚，中断也想访问共享资源，那该怎么办呢？首先可以肯定的是，中断里面使用自旋锁，但是在中断里面使用自旋锁的时候，在获取锁之前一定要**先禁止本地中断**（也就是**本CPU中断**，对于多核SOC来说会有多个CPU核），否则可能导致锁死现象的发生。看下下面一个例子：

```

1 //线程A
2 spin_lock(&lock);
3 .....
4 functionA();
5 .....
6 spin_unlock(&lock);
7
8 //中断发生，运行线程B
9 spin_lock(&lock);
10 .....
11 functionA();
12 .....
13 spin_unlock(&lock);

```

线程A先运行，并且获取到了lock这个锁，当线程A运行 functionA函数的时候中断发生了，**中断抢走了CPU使用权**。下边的中断服务函数也要获取lock这个锁，但是这个锁被线程A占有着，中断就会一直自旋，等待锁有效。但是在中断服务函数执行完之前，线程A是不可能执行的，线程A说“你先放手”，中断说“你先放手”，场面就这么僵持着死锁发生！

使用了自旋锁之后可以保证临界区不受别的CPU和本CPU内的抢占进程的打扰，但是得到锁的代码在执行临界区的时候，还可能受到中断和底半部的影响，为了防止这种影响，建议使用以下列表中的函数：

函数	描述
void spin_lock_irq(spinlock_t *lock)	禁止本地中断，并获取自旋锁
void spin_unlock_irq(spinlock_t *lock)	激活本地中断，并释放自旋锁
void spin_lock_irqsave(spinlock_t *lock, unsigned long flags)	保存中断状态，禁止本地中断，并获取自旋锁
void spin_unlock_irqrestore(spinlock_t *lock, unsigned long flags)	将中断状态恢复到以前的状态，并且激活本地中断，释放自旋锁

在多核编程的时候，如果进程和中断可能访问同一片临界资源，我们一般需要在进程上下文中调用 spin_lock_irqsave () spin_unlock_irqrestore () ，在中断上下文中调用 spin_lock () spin_unlock () 。这样，在CPU上，无论是进程上下文，还是中断上下文获得了自旋锁，此后，如果CPU1无论是进程上下文，还是中断上下文，想获得同一自旋锁，都必须忙等待，这避免一切核间并发的可能性。同时，由于每个核的进程上下文持有锁的时候用的是 spin_lock_irqsave () ，所以该核上的中断是不可能进入的，这避免了核内并发的可能性。

```

1 DEFINE_SPINLOCK(lock) /* 定义并初始化一个锁 */
2 /* 线程A */
3 void functionA () {
4     unsigned long flags; /* 中断状态 */
5     spin_lock_irqsave(&lock, flags) /* 获取锁 */
6     /* 临界区 */
7     spin_unlock_irqrestore(&lock, flags) /* 释放锁 */
8 }
9 /* 中断服务函数 */
10 void irq() {
11     spin_lock(&lock) /* 获取锁 */
12     /* 临界区 */
13     spin_unlock(&lock) /* 释放锁 */

```

14 } }

自旋锁例程

```

1 static int xxx_open(struct inode *inode, struct file *filp)
2 {
3     ...
4     spinlock(&xxx_lock);
5     if (xxx_count) { /* 已经打开 */
6         spin_unlock(&xxx_lock);
7         return -EBUSY;
8     }
9     xxx_count++; /* 增加使用计数 */
10    spin_unlock(&xxx_lock);
11    ...
12    return 0; /* 成功 */
13 }
14
15 static int xxx_release(struct inode *inode, struct file *filp)
16 {
17     ...
18     spinlock(&xxx_lock);
19     xxx_count--; /* 减少使用计数 */
20     spin_unlock(&xxx_lock);
21     return 0;
22 }

```

读写自旋锁

当临界区的一个文件可以被**同时读取**，但是并**不能被同时读和写**。如果一个线程在读，另一个线程在写，那么很可能会读取到错误的**不完整的数据**。读写自旋锁是可以**允许对临界区的共享资源进行并发读操作的**。但是并不允许多个线程并发读写操作。如果想要并发读写，就要用到了顺序锁。

读写自旋锁的读操作函数如下所示：

函数	描述
DEFINE_RWLOCK(rwlock_t lock)	定义并初始化读写锁
void rwlock_init(rwlock_t *lock)	初始化读写锁
void read_lock(rwlock_t *lock)	获取读锁
void read_unlock(rwlock_t *lock)	释放读锁
void read_unlock_irq(rwlock_t *lock)	打开本地中断，并且释放读锁
void read_lock_irqsave(rwlock_t *lock, unsigned long flags)	保存中断状态，禁止本地中断，并获取读锁
void read_unlock_irqrestore(rwlock_t *lock, unsigned long flags)	将中断状态恢复到以前的状态，并且激活本地中断，释放读锁
void read_lock_bh(rwlock_t *lock)	关闭下半部，并获取读锁
void read_unlock_bh(rwlock_t *lock)	打开下半部，并释放读锁

读写自旋锁的写操作函数如下所示：

函数	描述
void write_lock(rwlock_t *lock)	获取写锁
void write_unlock(rwlock_t *lock)	释放写锁
void write_lock_irq(rwlock_t *lock)	禁止本地中断，并且获取写锁。
void write_unlock_irq(rwlock_t *lock)	打开本地中断，并且释放写锁
void write_lock_irqsave(rwlock_t *lock,unsigned long flags)	保存中断状态，禁止本地中断，并获取写锁
void write_unlock_irqrestore(rwlock_t *lock,unsigned long flags)	将中断状态恢复到以前的状态，并且激活本地中断，释放写锁
void write_lock_bh(rwlock_t *lock)	关闭下半部，并获取写锁
void write_unlock_bh(rwlock_t *lock)	打开下半部，并释放写锁

读写锁例程

```

1  rwlock_t lock; /* 定义rwlock */
2  rwlock_init(&lock); /* 初始化rwlock */
3  /* 读时获取锁*/
4  read_lock(&lock);
5  ... /* 临界资源 */
6  read_unlock(&lock);
7  /* 写时获取锁*/
8  write_lock_irqsave(&lock, flags);
9  ... /* 临界资源 */
10 write_unlock_irqrestore(&lock, flags);

```

顺序锁

顺序锁是读写锁的优化版本，读写锁不允许同时读写，而使用顺序锁可以完成同时进行读和写的操作，但并不允许同时的写。虽然顺序锁可以同时进行读写操作，但并不建议这样，读取的过程并不能保证数据的完整性。

顺序锁操作函数

顺序锁的读操作函数如下所示：

函数	描述
DEFINE_SEQLOCK(seqlock_t sl)	定义并初始化顺序锁
void seqlock_ini seqlock_t *sl)	初始化顺序锁
void write_seqlock(seqlock_t *sl)	顺序锁写操作
void write_sequnlock(seqlock_t *sl)	获取写顺序锁
void write_seqlock_irq(seqlock_t *sl)	禁止本地中断，并且获取写顺序锁
void write_sequnlock_irq(seqlock_t *sl)	打开本地中断，并且释放写顺序锁
void write_seqlock_irqsave(seqlock_t *sl,unsigned long flags)	保存中断状态，禁止本地中断，并获取写顺序
void write_sequnlock_irqrestore(seqlock_t *sl,unsigned long flags)	将中断状态恢复到以前的状态，并且激活本地中断，释放写顺序锁
void write_seqlock_bh(seqlock_t *sl)	关闭下半部，并获取读写锁
void write_sequnlock_bh(seqlock_t *sl)	打开下半部，并释放读写锁

顺序锁的写操作函数如下所示：

函数	描述
DEFINE_RWLOCK(rwlock_t lock)	读单元访问共享资源的时候调用此函数，此函数会返回顺序锁的序号
unsigned read_seqretry(const seqlock_t *sl,unsigned start)	读结束以后调用此函数检查在读的过程中有没有对资源进行写操作，如果有的话就要重读

自旋锁使用注意事项

1. 因为在等待自旋锁的时候处于“自旋”状态，因此锁的持有时间不能太长，一定要短，否则的话会降低系统性能。如果临界区比较大，运行时间比较长的话要选择其他的并发处理方式，比如稍后要讲的信号量和互斥体。
2. 自旋锁保护的临界区内不能调用任何可能导致线程休眠的API函数，比如copy_from_user ()、copy_to_user ()、kmallo () 和msleep () 等函数，否则的话可能导致死锁。
3. 不能递归申请自旋锁，因为一旦通过递归的方式申请一个你正在持有的锁，那么你就必须“自旋”，等待锁被释放，而你正处于“自旋”状态，根本没法释放锁。结果就是自己把自己锁死了
4. 在编写驱动程序的时候我们必须考虑到驱动的可移植性，因此不管你用的是单核的还是多核的SOC，都将其当做多核SOC来编写驱动程序。

copy_from_user的使用是结合进程上下文的，因为他们要访问“user”的内存空间，这个“user”必须是某个特定的进程。如果在驱动中使用这两个函数，必须是在实现系统调用的函数中使用，不可在实现中断处理的函数中使用。如果在中断上下文中使用了，那代码就很可能操作了根本不相关的进程地址空间。其次由于操作的页面可能被换出，这两个函数可能会休眠，所以同样不可在中断上下文中使用。

信号量

信号量简介

信号量和自旋锁有些相似，不同的是信号量会发出一个信号告诉你还需要等多久。因此，不会出现傻傻等待的情况。比如，有100个停车位的停车场，门口电子显示屏上实时更新的停车数量就是一个信号量。这个停车的数量就是一个信号量，他告诉我们是否可以停车进去。当有车开进去，信号量加一，当有车开出来，信号量减一。

比如，厕所一次只能让一个人进去，当A在里面的时候，B想进去，如果是自旋锁，那么B就会一直在门口傻傻等待。如果是信号量，A就会给B一个信号，你先回去吧，我出来了叫你。这就是一个信号量的例子，B听到A发出的信号后，可以先回去睡觉，等待A出来。

因此，信号量显然可以提高系统的执行效率，避免了许多无用功。信号量具有以下特点：

1. 因为信号量可以使等待资源线程进入休眠状态，因此适用于那些**占用资源比较久**的场合。
2. 因此信号量不能用于中断中，因为信号量会引起休眠，**中断不能休眠**。
3. **如果共享资源的持有时间比较短，那就不适合使用信号量了，因为频繁的休眠、切换线程引起的开销要远大于信号量带来的那点优势。**

信号量操作函数

函数	描述
DEFINE_SEMAPHORE(name)	定义一个信号量，并且设置信号量的值为1
void sema_init(struct semaphore *sem, int val)	初始化信号量sem，设置信号量值为val
void down(struct semaphore *sem)	获取信号量，因为会导致休眠，因此不能在中断中使用
int down_trylock(struct semaphore *sem);	尝试获取信号量，如果能获取到信号量就获取，并且返回0.如果不能就返回非0，并且不会进入休眠
int down_interruptible(struct semaphore	获取信号量，和down类似，只是使用dow进入休眠状态的线程不能被信号打断。而使用此函数进入休眠以后是可以被信号打断的
void up(struct semaphore *sem)	释放信号量

信号量例程

```

1 struct semaphore sem; /* 定义信号量 */
2 sema_init(&sem, 1); /* 初始化信号量 表示只能由一个线程同时访问这块资源 */
3 down(&sem); /* 申请信号量 */
4 /* 临界区 */
5 up(&sem); /* 释放信号量 */

```

互斥体

互斥体简介

互斥体表示一次只有一个线程访问共享资源，**不可以递归申请互斥体**。

信号量也可以用于互斥体，当信号量用于互斥时（即避免多个进程同时在一个临界区中运行），信号量的值应初始化为1.这种信号量在任何给定时刻只能由单个进程或线程拥有。在这种使用模式下，一个信号量有时也称为一个“互斥体（mutex）”，它是互斥（mutual exclusion）的简称。Linux内核中

几乎所有的信号量均用于互斥。

互斥体操作函数

函数	描述
DEFINE_MUTEX(name)	定义并初始化一个 mutex变量
void mutex_init(mutex *lock)	初始化 mutex
void mutex_lock(struct mutex *lock)	获取 mutex，也就是给 mutex上锁。如果获取不到就进休眠
void mutex_unlock(struct mutex *lock)	释放 mutex，也就给 mutex解锁
int mutex_trylock(struct mutex *lock)	判断 mutex是否被获取，如果是的话就返回，否则返回0
int mutex_lock_interruptible(struct mutex *lock)	使用此函数获取信号量失败进入休眠以后可以被信号打断

互斥体例程

```

1  struct mutex lock; /* 定义一个互斥体 */
2  mutex_init(&lock); /* 初始化互斥体 */
3  mutex_lock(&lock); /* 上锁 */
4  /* 临界区 */
5  mutex_unlock(&lock); /* 解锁*/

```

互斥体与自旋锁

互斥体和自旋锁都是解决互斥问题的一种手段。互斥体是进程级别的，互斥体在使用的时候会发生进程间的切换，因此，使用互斥体资源开销比较大。自旋锁可以节省上下文切换的时间，如果持有锁的时间不长，使用自旋锁是比较好的选择，如果持有锁时间比较长，互斥体显然是更好的选择。

互斥体使用注意事项

1. 当锁不能被获取到时，使用互斥体的开销是**进程上下文切换时间**，使用自旋锁的开销是等待**获取自旋锁**（由临界区执行时间决定）。若临界区比较小，宜使用自旋锁，若临界区很大，应使用互斥体。
2. 互斥体所保护的临界区可包含可能引起阻塞的代码，而自旋锁则绝对要避免用来保护包含这样代码的临界区。因为**阻塞意味着要进行进程的切换**，如果进程被切换出去后，另一个进程企图获取本自旋锁，死锁就会发生。
3. 互斥体存在于进程上下文。因此，如果被保护的共享资源需要在中断或软中断情况下使用，则在互斥体和自旋锁之间只能选择自旋锁。当然，如果一定要使用互斥体，则只能通过mutex trylock（）方式进行，不能获取就立即返回以避免阻塞。

Linux内核中的阻塞和异步通知机制

阻塞/非阻塞简介

阻塞操作是指在执行设备操作时，若不能获得资源，则**挂起进程**直到满足可操作的条件后再进行操作。被挂起的进程进入**睡眠状态**，被从调度器的运行队列移走，**直到等待的条件被满足**。而非阻塞操作的进程在不能进行设备操作时，**并不挂起，它要么放弃，要么不停地查询**，直至可以进行操作为止。

阻塞/非阻塞例程

阻塞方式

```
1 int fd;
2 int data = 0;
3 fd = open("/dev/xxx_dev", O_RDWR); /* 阻塞方式打开 */
4 ret = read(fd, &data, sizeof(data)); /* 读取数据 */
```

非阻塞方式

```
1 int fd;
2 int data = 0;
3 fd = open("/dev/xxx_dev", O_RDWR | O_NONBLOCK); /* 非阻塞方式打开 */
4 ret = read(fd, &data, sizeof(data)); /* 读取数据 */
```

等待队列简介

等待队列是内核中一个重要的数据结构。阻塞方式访问设备时，如果设备不可操作，那么进程就会进入**休眠状态**。等待队列就是来完成进程休眠操作的一种数据结构。

等待队列相关函数

定义等待队列

```
1 wait_queue_head_t my_queue;
```

wait_queue_head_t是__wait_queue_head结构体的一个typedef。

初始化等待队列头

```
1 void init_waitqueue_head(wait_queue_head_t *q)
```

参数q就是要初始化的等待队列头，也可以使用宏 **DECLARE_WAIT_QUEUE_HEAD (name)**来一次性完成等待队列头的定义的初始化。

定义并初始化一个等待队列项

```
1 DECLARE_WAITQUEUE(name, tsk)
```

name就是等待队列项的名字，tsk表示这个等待队列项属于哪个任务进程，一般设置为current，在Linux内核中**current**相当于一个全局变量，表示**当前进程**。因此宏DECLARE_WAITQUEUE就是给当前正在运行的进程创建并初始化了一个等待队列项。

将队列项添加到等待队列头

```
1 void add_wait_queue(wait_queue_head_t *q, wait_queue_t *wait)
```

q：等待队列项要加入的等待队列头

wait：要加入的等待队列项

返回值：无

将队列项从等待队列头移除

```
1 void remove_wait_queue(wait_queue_head_t *q, wait_queue_t *wait)
```

q:要删除的等待队列项所处的等待队列头

wait：要删除的等待队列项。

返回值：无

等待唤醒

```
1 void wake_up(wait_queue_head_t *q)
2 void wake_up_interruptible(wait_queue_head_t *q)
```

q:就是要唤醒的等待队列头，这两个函数会将这个等待队列头中的所有进程都唤醒

wake_up函数可以唤醒处于 TASK_INTERRUPTIBLE和 TASK_UNINTERRUPTIBLE状态的进程，而 wake_up_interruptible函数只能唤醒处于 TASK_INTERRUPTIBLE状态的进程

等待事件

```
1 wait_event(wq, condition)
```

等待以wq为等待队列头的等待队列被唤醒，前提是 **condition条件必须满足（为真）**，否则一直阻塞。此函数会将进程设置为TASK_UNINTERRUPTIBLE状态

```
1 wait_event_timeout(wq, condition, timeout)
```

功能和 wait_event类似，但是此函数可以添加**超时时间**，以 **jiffies**为单位。此函数有返回值，如果返回0的话表示超时时间到，而且 condition为假。为1的话表示 condition为真，也就是条件满足了。

```
1 wait_event_interruptible(wq, condition)
```

与 wait_event函数类似，但是此函数将进程设置为 TASK_INTERRUPTIBLE，就是**可以被信号打断**。

```
1 wait_event_interruptible_timeout(wq, condition, timeout)
```

与 wait_event_timeout函数类似，此函数也将进程设置为 TASK_INTERRUPTIBLE，**可以被信号打断**。

轮询

当应用程序以非阻塞的方式访问设备时，会一遍一遍的去查询我们的设备是否可以访问，这个查询操作就叫做轮询。内核中提供了**poll**，**epoll**，**select**函数来处理轮询操作。当应用程序在上层通过 poll，epoll，select函数来查询设备时，驱动程序中的poll，epoll，select函数就要在底层实现查询，如果可以操作的话，就会从读取设备的数据或者向设备写入数据。

select

函数原型

```
1 int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,
   struct timeval *timeout)
```

nfds：要操作的文件描述符个数。

readfds、writefds和 exceptfds：这三个指针指向**描述符集合**，这三个参数指明了关心哪些描述符、需要满足哪些条件等等，这三个参数都是fd_set类型的，**fd_set类型变量的每一个位都代表了一个文件描述符**。readfds用于监视指定描述符集的**读变化**，也就是监视这些文件是否可以读取，只要这些集合里面有一个文件可以读取，那么 select就会返回一个大于0的值表示文件可以读取。如果没有文件可以读取，那么就会根据 timeout参数来判断是否超时。可以将**reads设置为NULL，表示不关心任何文件的读变化**。writefds和 reads类似，只是 writers用于监视这些文件是否可以**进行写操作**。exceptfds用于监视这些文件的异常

timeout：超时时间，当我们调用 select函数等待某些文件描述符可以设置超时时间，超时时间使用结构体 timeval表示，结构体定义如下所示：

```
1 struct timeval {
2     long tv_sec; /* 秒 */
3     long tv_usec; /* 微妙 */
4 };
```

当 timeout为NULL的时候就表示**无限期的等待返回值**。0，表示的话就表示超时发生，但是没有任何文件描述符可以进行操作；-1，发生错误；其他值，可以进行操作的文件描述符个数。

操作fd_set变量的函数

```
1 void FD_ZERO(fd_set *set)
2 void FD_SET(int fd, fd_set *set)
3 void FD_CLR(int fd, fd_set *set)
4 int FD_ISSET(int fd, fd_set *set)
```

FD_ZERO用于将 fd set变量的**所有位都清零**，FD_SET用于将 fd_set变量的某个位置**1**，也就是向 fd_set添加一个文件描述符，参数fd就是要加入的文件描述符。FD_CLR用户将 fd_set变量的**某个位清零**，也就是将一个文件描述符从 fd_set中删除，参数fd就是要删除的文件描述符。FD_ISSET用于**测试fd_set的某个位是否置1**，也就是判断某个文件**是否可以进行操作**，参数fd就是要判断的文件描述符。

```
1
2 void main(void)
3 { int ret, fd; /* 要监视的文件描述符 */
4     fd_set readfds; /* 读操作文件描述符集 */
5     struct timeval timeout; /* 超时结构体 */
6     fd = open("dev_xxx", O_RDWR | O_NONBLOCK); /* 非阻塞式访问 */
7     FD_ZERO(&readfds); /* 清除readfds */
8     FD_SET(fd, &readfds); /* 将fd添加到readfds里面 */
9     /* 构造超时时间 */
10    timeout.tv_sec = 0;
11    timeout.tv_usec = 500000; /* 500ms */
12    ret = select(fd + 1, &readfds, NULL, NULL, &timeout);
13    switch (ret) {
14        case 0: /* 超时 */
15            printf("timeout!\r\n");
16            break;
17        case -1: /* 错误 */
18            printf("error!\r\n");
19            break;
20        default: /* 可以读取数据 */
21            if(FD_ISSET(fd, &readfds)) /* 判断是否为fd文件描述符 */
22            {
23                /* 使用read函数读取数据 */
24            }
```

```
25         break;
26     }
27 }
```

poll

在单个线程中，select函数能够监视的文件描述符数量有最大的限制，一般为1024，可以修改内核将监视的文件描述符数量改大，但是这样会降低效率！这个时候就可以使用poll函数，poll函数本质上和select没有太大的差别，但是poll函数**没有最大文件描述符限制**，Linux应用程序中poll函数原型如下所示：

```
1 int poll(struct pollfd *fds, nfds_t nfd, int timeout)
```

函数参数和返回值含义如下

fds：要监视的文件描述符集合以及要监视的事件，为一个数组，数组元素都是结构体 pollfd类型的，pollfd结构体如下所示

```
1 struct pollfd
2 {
3     int fd; /* 文件描述符 文件描述符 文件描述符 */
4     short events; /* 请求的事件 请求的事件 请求的事件 */
5     short revents; /* 返回的事件 返回的事件 返回的事件 */
6 };
```

fd是要监视的文件描述符，如果fd无效的话那么events监视事件也就无效，并且revents返回0。events是要监视的事件，可监视的事件类型如下所示

```
1 POLLIN      //有数据可以读取。
2 POLLPRI     //有紧急的数据需要读取。
3 POLLOUT     //可以写数据POLLERR指定的文件描述符发生错误POLLHUP指定的文件描述符挂起
              POLLNVAL无效的请求POLLRDNORM等同于 POLLIN
```

revents：返回参数，也就是返回的事件，有Linux内核设置具体的返回事件。

nfd:poll函数要监视的文件描述符数量

timeout：超时时间，单位为ms

返回值：返回 revents域中不为0的 pollfd结构体个数，也就是发生事件或错误的文件描述符数量；0，超时；-1，发生错误，并且设置errno为错误类型

```
1 void main(void)
2 {
3     int ret;
4     int fd; /* 要监视的文件描述符 */
5     struct pollfd fds;
6     fd = open(filename, O_RDWR | O_NONBLOCK); /* 非阻塞式访问 */
7     /* 构造结构体 */
8     fds.fd = fd;
9     fds.events = POLLIN; /* 监视数据是否可以读取 */
10    ret = poll(&fds, 1, 500); /* 轮询文件是否可操作，超时500ms */
11    if (ret)
12    { /* 数据有效 */
13        /* 读取数据 */
14    } else if (ret == 0)
15    {
16        /* 超时 */
17    }
```

```
17     } else if (ret < 0)
18     {
19         /* 错误 */
20     }
21 }
```

epoll

传统的 select和poll函数都会随着所监听的fd数量的增加，出现效率低下的问题，而且poll函数每次必须遍历所有的描述符来检查就绪的描述符，这个过程很浪费时间。为此，epoll应运而生，epoll就是为处理大并发而准备的，一般常常在网络编程中使用epoll函数。应用程序需要先使用 epoll_create函数创建一个 epoll句柄，epoll_create函数原至如下。

```
1 int epoll_create(int size)
```

函数参数和返回值含义如下：

size；从 Linux2.6.8开始此参数已经没有意义了，随便填写一个大于0的值就可以

返回值：epoll句柄，如果为-1的话表示创建失败,epoll句柄创建成功以后使用,epoll_ctl函数向其中添加要监视的文件描述符以及监视的函数原型如下所示

```
1 int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event)
```

函数参数和返回值含义如下

epfd；要操作的epoll句柄，也就是使用 epoll_create函数创建的epoll句柄。

p：表示要对epfd（epoll句柄）进行的操作，可以设置为

```
1 EPOLL_CTL_ADD      //向fd添加文件参数d表示的描述符EPOLL_CTL_MOD修改参数fd的
event事件。
2 EPOLL_CTL_DEL      //从f中删除过l描述符
```

fd：要监视的文件描述

event：要监视的事件类型，为 epoll_event结构体类型指针，epoll_event结构体类型如下所

```
1 struct epoll_event
2 {
3     uint32_t events; /* epoll事件 */
4     epoll_data_t data; /* 用户数据 用户数据 */
5 };
```

结构体 epoll_event的 events成员变量表示要监视的事件，可选的事件如下所示

```
1 EPOLLIN            //有数据可以读取EPOLLOUT可以写数据
2 EPOLLPRI           //有紧急的数据需要读取EPOLLERR指定的文件描述符发生错误。
3 EPOLLHUP           //指定的文件描述符挂起POLLET设置epo为边沿触发，默认触发模式为水平触发
王
4 POLLONESHOT        //一次性的监视，当监视完成以后还需要再次监视某个fd，那么就需要将fd重新
添加到epoll里面
```

上面这些事件可以进行“或”操作，也就是说可以设置监视多个事件返回值：0，成功；-1，失败，并且设置errno的值为相应的错误码。一切都设置好以后应用程序就可以通过 epoll_wait函数来等待事件的发生，类似 select函数。epoll_wait函数原型如下所示

```
1 int epoll_wait(int epfd, struct epoll_event *events, int maxevents, int
   timeout)
```

函数参数和返回值含义如下

epfd：要等待的 epoll

events：指向 epoll_event 结构体的数组，当有事件发生的时候linux内核会填写 events，调用者可以根据 events 判断发生了哪些事件。

prevents:events 数组大小，必须大于0

timeout：超时时间，单位为ms 返回值：0，超时；-1，错误；其他值，准备就绪的文件描述符数量。

epoll更多的是用在大规模的并发服务器上，因为在这种场合下 select和poll并不适合。当设计到的文件描述符（fd比较少的时候就适合用 selcet和pl本章我们就使用 sellect和poll这两个函数

异步通知概念

阻塞与非阻塞访问、poll函数提供了较好的解决设备访问的机制，但是如果有了异步通知，整套机制则更加完整了。

异步通知的意思是：**一旦设备就绪，则主动通知应用程序**，这样应用程序根本就不需要查询设备状态，这一点非常类似于硬件上“中断”的概念，比较准确的称谓是“信号驱动的异步I/O”。**信号是在软件层次上对中断机制的一种模拟**，在原理上，**一个进程收到一个信号与处理器收到一个中断请求可以说是一样的**。信号是异步的，一个进程不必通过任何操作来等待信号的到达，事实上，进程也不知道信号到底什么时候到达。

阻塞I/O意味着一直等待设备可访问后再访问，非阻塞I/O中使用poll（）意味着查询设备是否可访问，而异步通知则意味着设备通知用户自身可访问，之后用户再进行I/O处理。由此可见，这几种I/O方式可以相互补充。

Linux信号

异步通知的核心就是信号，在 arch/xtensa/include/uapi/asm/signal.h文件中定义了Linux所支持的所有信号

```
1 #define SIGHUP          1 /* 终端挂起或控制进程终止 */
2 #define SIGINT          2 /* 终端中断(Ctrl+C组合键) */
3 #define SIGQUIT        3 /* 终端退出(Ctrl+\组合键) */
4 #define SIGILL          4 /* 非法指令 */
5 #define SIGTRAP         5 /* debug使用，有断点指令产生 */
6 #define SIGABRT         6 /* 由abort(3)发出的退出指令 */
7 #define SIGIOT          6 /* IOT指令 */
8 #define SIGBUS          7 /* 总线错误 */
9 #define SIGFPE          8 /* 浮点运算错误 */
10 #define SIGKILL         9 /* 杀死、终止进程 */
11 #define SIGUSR1        10 /* 用户自定义信号1 */
12 #define SIGSEGV        11 /* 段违例(无效的内存段) */
13 #define SIGUSR2        12 /* 用户自定义信号2 */
14 #define SIGPIPE        13 /* 向非读管道写入数据 */
15 #define SIGALRM        14 /* 闹钟 */
16 #define SIGTERM        15 /* 软件终止 */
17 #define SIGSTKFLT      16 /* 栈异常 */
18 #define SIGCHLD        17 /* 子进程结束 */
19 #define SIGCONT        18 /* 进程继续 */
20 #define SIGSTOP        19 /* 停止进程的执行，只是暂停 */
21 #define SIGTSTP        20 /* 停止进程的运行(Ctrl+Z组合键) */
22 #define SIGTTIN        21 /* 后台进程需要从终端读取数据 */
23 #define SIGTTOU        22 /* 后台进程需要向终端写数据 */
24 #define SIGURG         23 /* 有"紧急"数据 */
```

```

25 #define SIGXCPU      24 /* 超过CPU资源限制 */
26 #define SIGXFSZ      25 /* 文件大小超额 */
27 #define SIGVTALRM     26 /* 虚拟时钟信号 */
28 #define SIGPROF        27 /* 时钟信号描述 */
29 #define SIGWINCH       28 /* 窗口大小改变 */
30 #define SIGIO          29 /* 可以进行输入/输出操作 */
31 #define SIGPOLL SIGIO
32 /* #define SIGLOS 29 */
33 #define SIGPWR         30 /* 断点重启 */
34 #define SIGSYS         31 /* 非法的系统调用 */
35 #define SIGUNUSED      31 /* 未使用信号 */

```

异步通知代码

我们使用中断的时候需要设置中断处理函数，同样的，如果要在应用程序中使用信号，那么就必须设置信号所使用的**信号处理函数**，在应用程序中使用 signal函数来设置指定信号的处理函数，signal函数原型如下所示

```
1 void (*signal(int signum, void (*handler))(int))(int);
```

该函数原型较难理解，它可以分解为：

```

1 typedef void (*sighandler_t)(int);
2 sighandler_t signal(int signum, sighandler_t handler);

```

第一个参数指定信号的值，第二个参数指定针对前面信号值的处理函数，若为SIG_IGN，表示忽略该信号；若为SIG_DFL，表示采用系统默认方式处理信号；若为用户自定义的函数，则信号被捕获到后，该函数将被执行。

如果signal调用成功，它返回最后一次为信号signum绑定的处理函数的handler值，失败则返回SIG_ERR。

驱动中的信号处理

fasync_struct结构体

首先我们需要在驱动程序中定义个 fasync_struct结构体指针变量，fasync_struct结构体内容如下

```

1 struct fasync_struct
2 { spinlock_t fa_lock;
3 int magic;
4 int fa_fd;
5 struct fasync_struct *fa_next;
6 struct file *fa_file;
7 struct rcu_head fa_rcu;
8 };

```

一般将 fasync_struct结构体指针变量定义到设备结构体中，比如在xxx_dev结构体中添加一个 fasync_struct结构体指针变量，结果如下所示

```

1 struct xxx_dev
2 {
3     struct device *dev;
4     struct class *cls;
5     struct cdev cdev;
6     .....
7     struct fasync_struct *async_queue; /* 异步相关结构体 */
8 };

```

fasync函数

如果要使用异步通知，需要在设备驱动中实现file_operations操作集中的 fasync函数，此函数格式如下所示：

```

1 int (*fasync) (int fd, struct file *filp, int on)

```

fasync函数里面一般通过调用 fasync_helper函数来初始化前面定义的 fasync_struct结构体指针，fasync_helper函数原型如下

```

1 int fasync_helper(int fd, struct file * filp, int on, struct fasync_struct
  **fapp)

```

fasync_helper函数的前三个参数就是 fasync函数的那三个参数，第四个参数就是要初始化的 fasync_struct结构体指针变量。当应用程序通过结构体指针变量。当应用程序通过“fcntl(fd, F_SETFL, flags | FASYNC)”改变fasync标记的时候，驱动程序 file_operations操作集中的 fasync函数就会执行。

```

1 struct xxx_dev
2 {
3     .....
4     struct fasync_struct *async_queue; /* 异步相关结构体 */
5 };
6 static int xxx_fasync(int fd, struct file *filp, int on)
7 {
8     struct xxx_dev *dev = (xxx_dev)filp->private_data;
9     if (fasync_helper(fd, filp, on, &dev->async_queue) < 0)
10    return -EIO;
11    return 0;
12 }
13 static struct file_operations xxx_ops =
14 {
15     .....
16     .fasync = xxx_fasync,
17     .....
18 };

```

在关闭驱动文件的时候需要在file_operations操作集中的 release函数中释放 fasync_struct的释放函数同样为 fasync_helper, release函数参数参考实例如下

```

1 static int xxx_release(struct inode *inode, struct file *filp)
2 {
3     return xxx_fasync(-1, filp, 0); /* 删除异步通知 */
4 }

```

```

1 static struct file_operations xxx_ops =
2 {
3     .....
4     .release = xxx_release,
5 };

```

第3行通过调用示例代码 xxx_fasync函数来完成 fasync_struct的释放工作，但是，其最终还是通过 fasync_helper函数完成释放工作。

kill_fasync函数

当设备可以访问的时候，驱动程序需要向应用程序发出信号，相当于产生“中断” kill_fasync函数负责发送指定的信号， kill_fasync函数原型如下所示

```

1 void kill_fasync(struct fasync_struct **fp, int sig, int band)

```

函数参数和返回值含义如下：

fasync struct 要操作的文件指针

sig：要发送的信号

band：可读时设置为 POLL IN，可写时设置为 POLL OUT。

返回值：无。

应用程序对异步通知的处理

应用程序对异步通知的处理包括以下三步

1、注册信号处理函数应用程序根据驱动程序所使用的信号来设置信号的处理函数，应用程序使用 signal函数来设置信号的处理函数。前面已经详细的讲过了，这里就不细讲了。

2、将本应用程序的进程号告诉给内核使用fcntl (fd, F_SETOWN, getpid)将本应用程序的进程号告诉给内核

3、开启异步通知使用如下两行程序开启异步通知：

```

1 flags = fcntl(fd, F_GETFL); /* 获取当前的进程状态*/
2 fcntl(fd, F_SETFL, flags | FASYNC); /* 开启当前进程异步通知功能 */

```

重点就是通过 fcntl函数设置进程状态为 FASYNC，经过这一步，驱动程序中的 fasync函数就会执行。

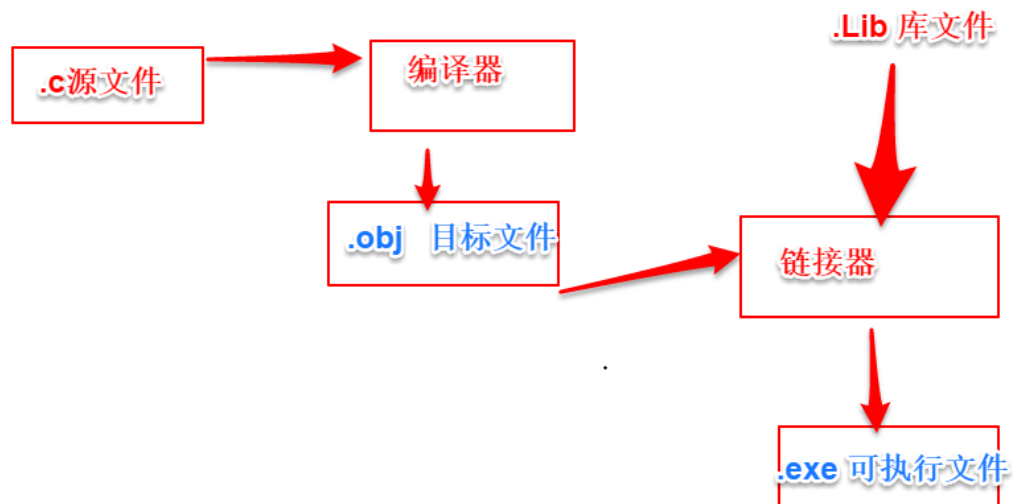
C语言

简述编译运行一段代码的过程

源程序是指未经编译的，按照一定的程序设计语言规范书写的，人类可读的文本文件，源程序就是所写好的代码。

可执行程序，即常说的.exe程序，可以执行程序，完成计算机功能。在C语言中，.c文件就是所谓的源文件。

源程序到可执行程序的过程。在这个过程中，会发生如下的变化：



https://blog.csdn.net/qq_16933601

.c文件生成.obj文件的过程，称为编译，.obj文件生成到.exe文件的过程，称为链接。

.obj文件就是一个是程序编译生成的二进制文件，当.exe文件生成以后.obj文件就会被删除。

事实上，.c文件生成.exe文件的过程总共是经历了**预处理，编译，汇编，链接，这四个过程**。

1、预处理

为了接下来能够解释的更加清楚，使用linux平台下的gcc编译器解释。先书写一个非常简单的程序来介绍：

```

1 test.c
2 #include<stdio.h>
3
4 int main()
5 {
6     printf("hello");
7
8     return 0;
9 }
    
```

直接编译得到：

```

[root@localhost learn] # vim test.c
[root@localhost learn] # gcc test.c
[root@localhost learn] # ls
a.out test.c
[root@localhost learn] # ./a.out
hello[root@localhost learn] #
[root@localhost learn] #
    
```

第一步发生的是预编译，使用-E指令会使程序只进行到预编译指令。经过预编译指令后的会生成一个.i文件。

```

[root@localhost learn] # gcc test.c -E > test.i
[root@localhost learn] # ls
a.out test.c test.i
[root@localhost learn] #
    
```

在预编译的过程中，主要处理源代码中的预处理指令，引入头文件，去除注释，处理所有的条件编译指令（#ifdef,#ifndef,#else,#elif,#endif），宏的替换，添加行号，保留所有的编译器指令。

当进行预编译以后的文件中将不再存在宏，所有的宏都被替代。当想要判断宏是否正确或者头文件包含是否正确时，也可以通过预编译来查看。

2、编译

在预处理结束后，进行的是编译。编译过程所进行的是对预处理后的文件进行语法分析，词法分析，语义分析，符号汇总，然后生成汇编代码。

```
[ root@localhost learn] #
[ root@localhost learn] # gcc test.i -S > test.s
[ root@localhost learn] #
```

3、汇编

汇编过程将汇编代码转成二进制文件，二进制文件就可以让机器来读取。每一条汇编语句都会产生一句机器语言。

```
[ root@localhost learn] #
[ root@localhost learn] # gcc test.s -O > test.o
[ root@localhost learn] #
```

在这里最终会生成一个重定位目标文件 .o 文件，类似 windows 下的 .obj 文件。这里生成的目标文件里面就是二进制文件。另外，在这里会形成符号表，给这些符号会分配虚拟地址。

4、链接

由汇编程序生成的目标文件并不能立即就被执行，其中可能还有许多没有解决的问题。例如，某个源文件中的函数可能引用了另一个源文件中定义的某个符号（如变量或者函数调用等）；在程序中可能调用了某个库文件中的函数等等。所有这些问题，都需要经链接程序的处理方能得以解决。链接程序的主要工作就是将有关的目标文件彼此相连接，也即将在一个文件中引用的符号同该符号在另外一个文件中的定义连接起来，使得所有的这些目标文件成为一个能够被操作系统装入执行的统一整体。

链接分为静态链接和动态链接：

静态链接：后缀是 .a，主要在编译的时候将库文件里面代码搬迁到可执行的文件中；

动态链接：后缀是 .so，主要在执行的时候需要转换到库文件代码执行；

两种链接的优缺点：

- （1）静态的链接产生的可执行的文件体积比较大；而动态链接的可执行文件的体积比较小；
- （2）动态的链接的编译的效率比较高；
- （3）静态链接的可执行的文件执行的效率高
- （4）静态链接的可执行的文件的“布局”比较好一点；

静态链接和动态链接有什么区别

静态链接是指把要调用的函数或者过程直接链接到可执行文件中，成为可执行文件的一部分。换句话说，函数和过程的代码就在程序的 exe 文件中，该文件包含了运行时所需的全部代码。静态链接的缺点是：当多个程序都调用相同函数时，内存中就会存在这个函数的多个复制，这样就浪费了内存资源

动态链接是相对于静态链接而言的。动态链接调用的函数代码并没有被复制到应用程序的可执行文件中，而是仅仅在其中加入了所调用函数的描述信息（往往是一些重定位信息）。仅当应用程序被装入内存开始运行时，在操作系统的管理下，才在应用程序与相应的动态链接库（dynamic link library, d）之间建立链接关系。当要执行调用，dll 文件中的函数时，根据链接产生的重定位信息，操作系统才转去执行，dll 文件中相应的函数代码。

静态链接的执行程序能够在其他同类操作系统的机器上直接运行。例如，一个 exe 文件是在 Windows 2000 系统上静态链接的，那么将该文件直接复制到另一台 Windows 2000 的机器上，是可以运行的。而动态链接的执行程序则不可以，除非把该 exe 文件所需的 dll 文件都一并复制过去，或者对方机器上也有所需的相同版本的 dll 文件，否则是不能保证正常运行的。

静态链接库和动态链接库有什么区别

静态链接库就是使用的lib文件，库中的代码最后需要链接到可执行文件中去，所以静态链接的可执行文件一般比较大一些。

动态链接库是一个包含可由多个程序同时使用的代码和数据的库，它包含函数和数据的模块的集合。程序文件（如exe文件或d文件）在运行时加载这些模块（即所需的模块映射到调用进程的地址空间）。

静态链接库和动态链接库的相同点是它们都实现了代码的共享，不同点是静态链接库lib文件中的代码被包含在调用的exe文件中，该lib文件中不能再包含其他动态链接库或者静态链接库了。而动态链接库dll文件可以被调用的exe动态地“引用”和“卸载”，该dll文件中可以包含其他动态链接库或者静态链接库。

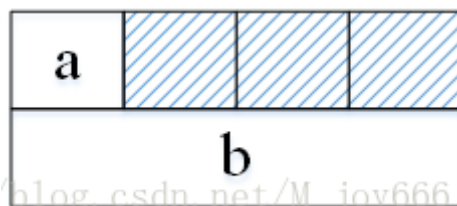
4字节对齐

什么是字节对齐？

在现代计算机中，内存空间都是按照字节(byte)划分的。从理论上讲对任何类型的变量的访问可以从任何地址开始，但实际情况是，访问特定类型的变量的时候经常在特定的内存地址访问，这就需要各种类型的数据按照一定规则在空间上排列，而不是顺序地一个接一个地排放，这种所谓的规则就是字节对齐。这么长一段话的意思是说：**字节对齐可以提升存取效率，也就是用空间换时间。**

例如：

```
1. 1 struct A
2
3 {
4
5     char a;
6
7     char reserved[3]; //使用空间换时间
8
9     int b;
10
11 };
```



注：其中的reserved成员对于程序并没有意义，只是起到了填补空间达到字节对齐的目的。当然，即使不加这个成员，编译器也会自动为我们补齐，加上它只是起到显式提醒。

为什么需要字节对齐？

因为各个硬件平台对存储空间的处理上有很大的不同，一些平台对某些特定类型的数据只能从某些特定地址开始存取。比如有些架构的CPU在访问一个没有进行对齐的变量的时候会发生错误。那么在这种架构下编程必须保证字节对齐。其他平台可能没有这种情况，但是最常见的是如果不按照适合其平台要求对数据存放进行对齐，会在存取效率上带来损失。比如有些平台每次读都是从偶地址开始，如果一个int型（假设为32位系统）如果存放在偶地址开始的地方，那么一个读周期就可以读出这32bit，而如果存放在奇地址开始的地方，就需要2个读周期，并对两次读出的结果的高低字节进行拼凑才能得到该32bit数据。显然在读取效率上下降很多。

几个基本概念

1、基本数据类型的自身对齐值

例如，char型数据的自身对齐值为1字节，short类型自身对齐值为2字节，int、float、long类型自身对齐值均为4字节，double类型自身对齐值均为8字节。（32位系统）

2、结构体或类的自身对齐值

其成员中自身对齐值最大的那个值。

3、指定对齐值

通过预编译指令 #pragma pack (value) 来指定的对齐值value。（注：取消自定义对齐值得指令为 #pragma pack () ）

4、数据成员、结构体和类的有效对齐值

其自身对齐值和指定对齐值中较小的那个值。

字节对齐的几个例子浅析

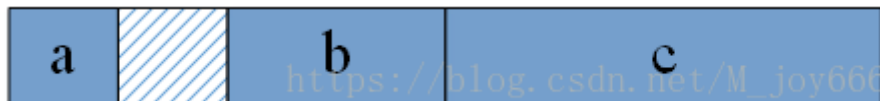
例1：设有如下两个结构体

```
1 struct A
2 {
3     char a;
4     short b;
5     int c;
6 };
```

```
1 struct B
2 {
3     short b;
4     int c;
5     char a;
6 };
```

那么上面两个结构体的大小是多少呢？

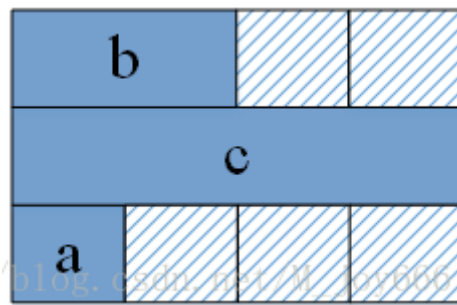
对于结构体A：a是char型数据，占用1字节内存；short型数据，占用2字节内存；int型数据，占用4字节内存。因此，结构体A的自身对齐值为4，sizeof(struct A) = 8字节。由于结构体类型数据是按顺序存储结构一个接一个向后排列的，于是其存储方式为：



对于结构体B：同理也是4字节对齐，但是sizeof(struct B) = 12字节。



想象为以下的行排列：

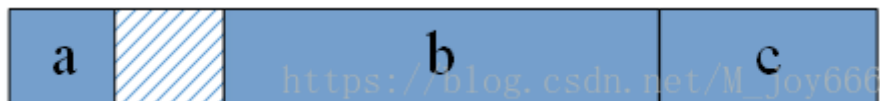


例2：为结构体指定对齐值

```

1 #pragma pack(2) //指定2字节对齐
2 struct C
3 {
4     char a;
5     int b;
6     short c;
7 };
8 #pragma pack() //取消指定对齐，恢复缺省对齐
    
```

对于结构体C：由于其自身对齐值为4字节（int b），而指定对齐值为2字节，因此该结构体的有效对齐值为较小的2字节，那么sizeof(struct C) = 8 字节。



```

1 #pragma pack(1) //指定1字节对齐
2 struct D
3 {
4     char a;
5     int b;
6     short c;
7 };
8 #pragma pack() //取消指定对齐，恢复缺省对齐
    
```

对于结构体D：同理可知，由于其自身对齐值为4字节（int b），而指定对齐值为1字节，因此该结构体的有效对齐值为较小的1字节，那么sizeof(struct D) = 7 字节。



总结

由以上分析可知，字节对齐会造成空间上的浪费。事实上，除了结构体之外，整个程序在给每个变量进行内存分配时都会遵循对齐机制，也都会产生内存空间的浪费。但我们要知道，这种浪费是值得的，因为它换来的是效率的提高。

#define和const的区别，const比#define的优点

1.const定义常量是有**数据类型**的，而#define宏定义常量却没有，一是const有类型；二是const可以有不同的作用域。

2.const常量有数据类型，而宏常量没有数据类型。编译器可以对const进行类型**安全检查**，而对后者**只进行字符替换**，没有类型安全检查，并且在字符替换中可能会产生意料不到的错误。

3.有些集成化的调试工具可以对const常量进行调试，但是不能对宏常量进行调试。

数组和指针的区别

C++/C程序中，指针和数组在不少地方都是可以相互替换使用的，这就让人产生一种错觉，以为两者是等价的。

数组或是在**静态存储区**被创建（全局数组），或是在栈上被创建。数组名**对应着**，注意不是指向，**一块内存，其地址与容量在生命期内保持不变**（当然，使用了realloc()的不算），只有数组的内容可以改变。

指针可以**随时指向**任意类型的内存块，它的特征是“可变”，所以我们常用指针来操作**动态内存**。指针远比数组灵活，当然也就更危险。

malloc、free和new、delete的区别与联系

相同点：

都可用于申请动态内存和释放内存

不同点：

（1）操作对象有所不同。

malloc和free是C++/C语言的标准库函数，new/delete是C++的运算符。对于非内部数据类的对象而言，光用malloc/free无法满足动态对象的要求。对象在创建的同时要自动执行构造函数，对象消亡之前要自动执行析构函数。由于malloc/free是库函数而不是运算符，不在编译器控制权限之内，不能够把执行构造函数和析构函数的任务强加malloc/free。

（2）用法上也有所不同。

函数malloc的原型如下：

```
1 void * malloc(size_t size);
```

用malloc申请一块长度为length的整数类型的内存，程序如下：

```
1 int *p = (int *) malloc(sizeof(int) * length);
```

我们应当把注意力集中在两个要素上：“类型转换”和“sizeof”。

1、malloc返回值的类型是void*，所以在调用malloc时要显式地进行类型转换，将void*转换成所需要的指针类型。

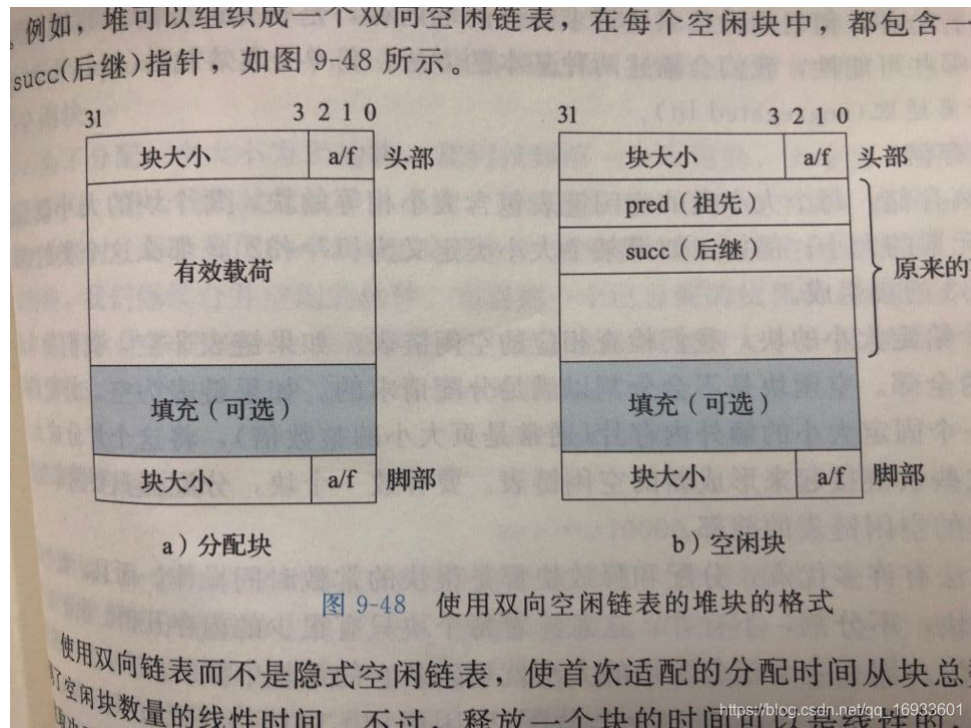
2、malloc函数本身并不识别要申请的内存是什么类型，它只关心内存的**总字节数**。

函数free的原型如下：

```
1 void free( void * memblock );
```

为什么free函数不象malloc函数那样复杂呢？这是因为**指针p的类型以及它所指的内存的容量事先都是知道的**，语句free(p)能正确地释放内存。如果p是NULL指针，那么free对p无论操作多少次都不会出问题。如果p不是NULL指针，那么free对p连续操作两次就会导致程序运行错误。

free函数怎么知道释放内存大小



内存块大概是这样的。

每个内存块是有header和footer。这个块头部标记了一个块的大小和是否已经使用。而malloc的时候是整块取用的（如果有分割合并的话这个头部信息也会有相应的更改）而free的时候只需要找到这个块头部的地址然后把这个块有效位改为未使用就可以了（大概的原理就是这样子吧）。

Assert

assert宏的原型定义在<assert.h>中，其作用是如果它的条件返回错误，则终止程序执行，原型定义：

```
1 #include <assert.h>
2 void assert( int expression );
```

assert的作用是现计算表达式 expression，如果其值为假（即为0），那么它先向stderr打印一条出错信息，然后通过调用 abort 来终止程序运行。请看下面的程序清单badptr.c：

```
1 #include <stdio.h>
2 #include <assert.h>
3 #include <stdlib.h>
4 int main( void )
5 {
6     FILE *fp;
7
8     fp = fopen( "test.txt", "w" );//以可写的方式打开一个文件，如果不存在就创建一个同名文件
9     assert( fp );                //所以这里不会出错
10    fclose( fp );
11
12    fp = fopen( "noexitfile.txt", "r" );//以只读的方式打开一个文件，如果不存在就打开文件失败
13    assert( fp );                //所以这里出错
14    fclose( fp );                //程序永远都执行不到这里来
15    return 0;
16 }
```

```

1 [root@localhost error_process]# gcc badptr.c
2 [root@localhost error_process]# ./a.out
3 a.out: badptr.c:14: main: Assertion `fp' failed.

```

已放弃使用assert()的缺点是，频繁的调用会极大的影响程序的性能，增加额外的开销。在调试结束后，可以通过在包含#include <assert.h>的语句之前插入 #define NDEBUG 来禁用assert调用，示例代码如下：

```

1 #include <stdio.h>
2 #define NDEBUG
3 #include <assert.h>

```

用法总结与注意事项：

1) 在函数开始处检验传入参数的合法性如：

```

1 int resetBufferSize(int nNewSize)
2 {
3     //功能:改变缓冲区大小,
4     //参数:nNewSize 缓冲区新长度
5     //返回值:缓冲区当前长度
6     //说明:保持原信息内容不变      nNewSize<=0表示清除缓冲区
7     assert(nNewSize >= 0);
8     assert(nNewSize <= MAX_BUFFER_SIZE);
9     ...
10 }

```

2) 每个assert只检验一个条件，因为同时检验多个条件时，如果断言失败，无法直观的判断是哪个条件失败，如：

不好：

```

1 assert(noffset>=0 && noffset+nSize<=m_nInfomationSize);

```

好：

```

1 assert(noffset >= 0);
2 assert(noffset+nSize <= m_nInfomationSize);

```

3) 不能使用改变环境的语句，因为assert只在DEBUG个生效，如果这么做，会使用程序在真正运行时遇到问题，如：

错误：

```

1 assert(i++ < 100);

```

这是因为如果出错，比如在执行之前i=100，那么这条语句就不会执行，那么i++这条命令就没有执行。

正确：

```
1  assert(i < 100);
2  i++;
```

4) assert和后面的语句应空一行，以形成逻辑和视觉上的一致感。

5) 有的地方，assert不能代替条件过滤。

以下是使用断言的几个原则：

(1) 使用断言捕捉不应该发生的非法情况。不要混淆非法情况与错误情况之间的区别，后者是必然存在的并且是一定要作出处理的。

(2) 使用断言对函数的参数进行确认。

(3) 在编写函数时，要进行反复的考查，并且自问：“我打算做哪些假定？”一旦确定了的假定，就要使用断言对假定进行检查。

(4) 一般教科书都鼓励程序员们进行防错性的程序设计，但要记住这种编程风格会隐瞒错误。当进行防错性编程时，如果“不可能发生”的事情的确发生了，则要使用断言进行报警。

ASSERT ()是一个调试程序时经常使用的宏，在程序运行时它计算括号内的表达式，如果表达式为FALSE (0), 程序将报告错误，并终止执行。如果表达式不为0，则继续执行后面的语句。这个宏通常原来判断程序中是否出现了明显非法的数据，如果出现了终止程序以免导致严重后果，同时也便于查找错误。

ASSERT只有在Debug版本中才有效，如果编译为Release版本则被忽略。

比较好的在程序中使用assert的地方：

(1)空指针检查

例如，针对一个函数的参数进行空指针检查。你可以这样使用：assert (pointer != NULL); 产生的错误会像这样：Assertion 'pointer != ((void *)0)' failed。这样，当出现空指针时，你的程序就会退出，并很好的给出错误信息。

(2)检查函数参数的值

例如，如果一个函数只能在它的一个参数foo为正值的时候被调用，你可以在函数开始时这样写:assert (foo > 0); 这将帮助你检测函数的错误使用，这也给源代码阅读者很清晰的印象，那就是在这里对函数的参数值有限制。

void 函数的返回值问题

void 函数可以写return;表示程序结束，但是没有返回值。

在软件开发过程中有一种很常见的编程方法，就是即使将参数传给无返回值的void函数也能实现对原始参数值的修改，这句话到底说的什么意思，且看示例。

一个例子：普通带参数的函数如何做返回值？(附代码)

这是一个带参数的普通函数，作用是统计字符串中的有效字符数（'\0'不算有效字符），返回值即为字符串中的字符总数，类型为int，具体代码入下：

```
#include <iostream>
using namespace std;

//普通带参数函数的返回值，以int为例
int returnFunc(char* pchar,int letterNum=0)
{
    while ((*pchar) != '\0')
    {
        pchar++;
        letterNum++;
    }
    return letterNum;
    //这里留一个问题：为什么我用传入的参数作为返回值，
    //而不在函数内声明一个int变量进行累加，最后返回呢？
}

int main()
{
    char charArr[] = "abcdefg";
    int letterSum = 0;
    letterSum = returnFunc(charArr);
    cout << "所求的字符数组中字符总数为：" << letterSum << endl;

    return 0;
}
```

https://blog.csdn.net/qq_40160695/article/details/101311630 百家号@C语言程序猿

代码中的问题是非常关键的，感兴趣的读者可以尝试在函数中声明一个变量做返回值，看看会出什么错误！

另一个例子：无返回值的带参void函数如何做返回值？（附代码）

void函数是没有返回值的，那么对于传入的参数在计算处理之后，怎么把结果返回呢？这里给出两种最常用的方式：指针和引用！这也是软件开发中实际使用的方式。

仍然以计算字符串中有效字符总数为例，代码入下：

```
#include <iostream>
using namespace std;

//void带参数函数返回处理结果:引用方式
void returnFunc1(char* pchar, int &letterNum)
{
    while ((*pchar) != '\0')
    {
        pchar++;
        letterNum++;
    }
}

//void带参数函数返回处理结果:指针方式
void returnFunc2(char* pchar, int* pletterNum)
{
    while ((*pchar) != '\0')
    {
        pchar++;
        (*pletterNum)++;
    }
}

int main()
{
    char charArr[] = "abcdefg";
    int letterSum1 = 0;
    returnFunc1(charArr, letterSum1);
    cout << "引用方式一所求的字符串中字符总数为: " << letterSum1 << endl;
    int letterSum2 = 0;
    int* pletterSum2 = &letterSum2;
    returnFunc2(charArr, pletterSum2);
    cout << "指针方式一所求的字符串中字符总数为: " << letterSum2 << endl;
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
引用方式一所求的字符串中字符总数为: 7
指针方式一所求的字符串中字符总数为: 7

C:\Users\jiangjingjing\source\repos\ConsoleApplication1\Debug\ConsoleApplication
1.exe <进程 8900>已退出, 返回代码为: 0。
若要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时
自动关闭控制台”。
按任意键关闭此窗口...
```

运行结果

其实主要采用的方式就是在主程序中直接声明一个变量，然后把这个变量的引用或者指针作为参数直接传递给void函数，当void函数在做处理时，直接将结果写到引用参数或者指针指向的主函数变量，这样就间接实现了“返回值”。

总结

在需要调用子函数做处理时，void函数利用引用或者指针来“返回”处理结果是程序员经常用到的方式。主要原因是：一可以让代码更简洁，二是能减少内存空间的占用。

希望大家两种方式都能掌握，因为很多情况下，尤其是涉及对象返回时，第一种方式的优越性就体现出来了。当然，第二种在void函数中间接“返回”处理结果的方式也要掌握，它是入门程序员和程序员之间的区别之一。

C语言函数嵌套定义与嵌套调用

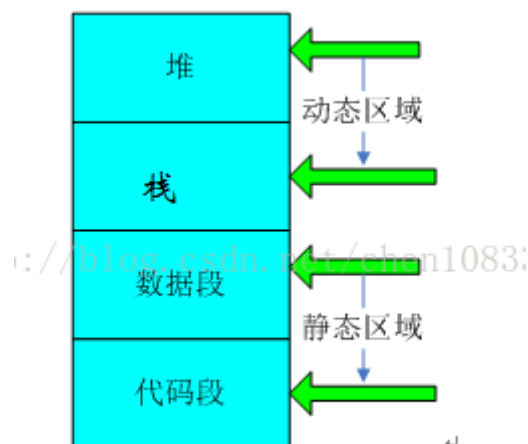
C语言中函数的定义都是相互平行、相互独立的，也就是说在函数定义时，**函数体内不能包含另一个函数的定义，即函数不能嵌套定义，但可以嵌套调用**。嵌套调用就是某个函数调用另外一个函数（即函数嵌套允许在一个函数中调用另外一个函数）。

同一个数组中的所有数组元素的数据类型必须相同吗

是的。因为数组的数据类型是根据你所定义的数据类型来规定的。

比如：C语言的数组大小和元素类型必须在编译期指定，一经指定不可改变。

静态存储区和动态存储区



动态存储方式

所谓动态存储方式是指在程序运行期间根据需要进行动态的分配存储空间的方式。

动态存储变量是在程序执行过程中，使用它时才分配存储单元，使用完毕立即释放。

典型的例子是函数的形式参数，在函数定义时并不给形参分配存储单元，只是在函数被调用时，才予以分配，调用函数完毕立即释放。如果一个函数被多次调用，则反复地分配、释放形参变量的存储单元。

动态存储区：

1. 栈和堆

栈：会存放函数的返回地址、参数和局部变量。

堆：我们通过 new 算符和 malloc 函数分配得到的空间。

堆和栈在内存中分配位置，跟硬件架构和操作系统都有关系。

x86中栈都是由高地址向低地址分配，堆是由低地址向高地址分配，不过在 Windows 和 Linux 中堆和栈的位置相反，另外存放静态数据、代码的区域位置也有一些不同。栈是属于线程的，每一个线程会有一个自己的栈。

2.局部自动变量

存放在栈区，在栈区里面其实又可以分成好几个区域，他们叫做栈帧，一个栈帧就是一个函数，需要调用该函数的时候就如入栈，函数return的时候就会弹出栈，所以他们的生命周期是从函数的开始直到函数结束。

而栈帧里面又存放着什么呢，栈帧存放着以下几种东西：参数变量的地址，局部变量的地址，return的地址（还有栈指针和基指针，想知道这个是什么百度吧）

3.自动变量(未加static声明的局部变量)；

4.函数调用时的现场保护和返回地址等；

5.动态变量

通常是由malloc new等分配的空间，生命周期是从分配的那一刻直到free结束。

静态存储方式

所谓静态存储方式是指在程序编译期间分配固定的存储空间的方式。该存储方式通常是在变量定义时就分定存储单元并一直保持不变，直至整个程序结束。全局变量，静态变量等就属于此类存储方式。

静态存储区：

一定会存在的而且会永恒存在、不会消失，这样的数据包括常量、常量（const 变量）、静态变量、全局变量等。静态、常量、全局变量就是存放在静态存储区，他们在程序编译完成后就已经分配好了，生命周期持续至程序结束。

delete和delete[]区别

delete 释放new分配的单个对象指针指向的内存。

delete[] 释放new分配的对象数组指针指向的内存。

数组指针和指针数组

数组指针就是指向数组的指针，它表示的是一个指针，这个指针指向的是一个数组，它的重点是指针。例如，`int (*pa)[8]` 声明了一个指针，该指针指向了一个有8个int型元素的数组。下面给出一个数组指针的示例。

```
1  #include<stdio. h>
2  #include<stdlib. h>
3  void main()
4  {
5      int b[12]={1,2,3,4,5,6,7,8,9,10,11,12};
6      int (*p)[4];
7      p = b;
8      printf ("%d\n",  **(++p);
9  }
```

程序的输出结果为 5。

上例中，p是一个数组指针，它指向一个**包含有4个int类型数组**的指针，刚开始p被初始化为指向数组b的首地址，++p相当于把p所指向的地址**向后移动4个int所占用的空间**，此时p指向数组{5,6,7,8}，语句 `* (++p)`；表示的是这个数组中**第一个元素**的地址(可以理解p为指向二维数组的指针，{ 1,2,3,4 }，{ 5,6,7,8 }，{ 9,10,11,12 }。p指向的就是 { 1,2,3,4 } 的地址，`*p` 就是指向元素，{ 1,2,3,4 }，`**p` 指向的就是1)，语句`** ( ++p )`会输出这个数组的第一个元素5。

指针数组表示的是一个数组，而数组中的元素是指针。下面给出另外一个指针数组的示例

```
1  #include <stdio.h>
2  int main()
3  {
4      int i;
5      int *p[4];
6      int a[4]={1,2,3,4};
7      p[0] = &a[0];
8      p[1] = &a[1];
9      p[2] = &a[2];
10     p[3] = &a[3];
11     for(i=0;i<4;i++)
12         printf ("%d",*p[i]);
13     printf("\n");
14     return 0;
15 }
16
```

程序的输出结果为1234。

函数指针和指针函数

函数指针

如果在程序中定义了一个函数，那么在编译时系统就会为这个函数代码分配一段存储空间，这段存储空间的首地址称为这个函数的地址。而且函数名表示的就是这个地址。既然是地址我们就可以定义一个指针变量来存放，这个指针变量就叫作函数指针变量，简称函数指针。

```
1  int(*p)(int, int);
```

这个语句就定义了一个指向函数的指针变量 p。首先它是一个指针变量，所以要有一个“*”，即 `(*p)`；其次前面的 int 表示这个指针变量可以指向返回值类型为 int 型的函数；后面括号中的两个 int 表示这个指针变量可以指向有两个参数且都是 int 型的函数。所以合起来这个语句的意思就是：定义了一个指针变量 p，该指针变量可以指向返回值类型为 int 型，且有两个整型参数的函数。p 的类型为 `int(*) (int, int)`。

我们看到，函数指针的定义就是将“函数声明”中的“函数名”改成“（指针变量名）”。*但是这里需要注意的是：“（指针变量名）”两端的括号不能省略*，括号改变了运算符的优先级。如果省略了括号，就不是定义函数指针而是一个函数声明了，即声明了一个返回值类型为指针型的函数。

最后需要注意的是，指向函数的指针变量没有 ++ 和 -- 运算。

```
1  # include <stdio.h>
2  int Max(int, int);  //函数声明
3  int main(void)
4  {
5      int(*p)(int, int);  //定义一个函数指针
6      int a, b, c;
7      p = Max;  //把函数Max赋给指针变量p，使p指向Max函数
8      printf("please enter a and b:");
```

```

9     scanf("%d%d", &a, &b);
10    c = (*p)(a, b); //通过函数指针调用Max函数
11    printf("a = %d\nb = %d\nmax = %d\n", a, b, c);
12    return 0;
13 }
14 int Max(int x, int y) //定义Max函数
15 {
16     int z;
17     if (x > y)
18     {
19         z = x;
20     }
21     else
22     {
23         z = y;
24     }
25     return z;
26 }
27

```

指针函数

首先它是一个函数，只不过这个函数的返回值是一个地址值。函数返回值必须用同类型的指针变量来接受，也就是说，指针函数一定有“函数返回值”，而且，在主调函数中，函数返回值必须赋给同类型的指针变量。

类型名 *函数名(函数参数列表);

其中，后缀运算符括号“()”表示这是一个函数，其前缀运算符星号“*”表示此函数为指针型函数，其函数值为指针，即它带回来的值的类型为指针，当调用这个函数后，将得到一个“指向返回值为...的指针（地址）”，“类型名”表示函数返回的指针指向的类型。

“(函数参数列表)”中的括号“()”为函数调用运算符，在调用语句中，即使函数不带参数，其参数表的一对括号也不能省略。其示例如下：

```

1 | int *pfun(int, int);

```

由于“*”的优先级低于“()”的优先级，因而pfun首先和后面的“()”结合，也就意味着，pfun是一个函数。即：

```

1 | int *(pfun(int, int));

```

接着再和前面的“*”结合，说明这个函数的返回值是一个指针。由于前面还有一个int，也就是说，pfun是一个返回值为整型指针的函数。

```

1 | #include <stdio.h>
2 | float *find(float(*pionter)[4],int n);//函数声明
3 | int main(void)
4 | {
5 |     static float score[][4]={{60,70,80,90},{56,89,34,45},{34,23,56,45}};
6 |     float *p;
7 |     int i,m;
8 |     printf("Enter the number to be found:");
9 |     scanf("%d",&m);
10 |    printf("the score of NO.%d are:\n",m);
11 |    p=find(score,m-1);
12 |    for(i=0;i<4;i++)
13 |        printf("%5.2f\t",*(p+i));

```

```

14
15     return 0;
16 }
17
18 float *find(float(*pointer)[4],int n)/*定义指针函数*/
19 {
20     float *pt;
21     pt=*(pointer+n);
22     return(pt);
23 }

```

共有三个学生的成绩，函数find()被定义为指针函数，其形参pointer是指针指向包含4个元素的一维数组的指针变量。pointer+n指向score的第n+1行。*(pointer+n)指向第一行的第0个元素。pt是一个指针变量，它指向浮点型变量。main()函数中调用find()函数，将score数组的首地址传给pointer。

定义常量谁更好？# define还是 const

尺有所短，寸有所长，define与const都能定义常量，效果虽然一样，但是各有侧重。

define既可以替代常数值，又可以替代表达式，甚至是代码段，但是容易出错，而const的引入可以增强程序的可读性，它使程序的维护与调试变得更加方便。具体而言，它们的差异主要表现在以下3个方面。

1) define只是用来进行**单纯的文本替换**，define常量的**生命周期止于编译期**，**不分配内存空间**，它存在于程序的**代码段**，在实际程序中，它只是一个常数；而const常量存在于程序的**数据段**，并在**堆栈中分配了空间**，const常量在程序中确实存在，并且可以被调用、传递

公众号：嵌入式与Linux那些事。CSDN：嵌入式与Linux那些事。来源网络，转载声明

2) const常量有数据类型，而define常量没有数据类型。编译器可以对const常量进行类型安全检查，如类型、语句结构等，而define不行。

3) 很多IDE**支持调试**const定义的常量，而不支持define定义的常量由于const修饰的变量可以排除程序之间的不安全性因素，保护程序中的常量不被修改，而且对数据类型也会进行相应的检查，极大地提高了程序的**健壮性**，所以一般**更加倾向于用const来定义常量类型**。

* (a[1]+1)、* (&a[1][1])、(* (a+1))[1] 表示什么意思

因为a[1]是第2行的地址，a[1]+1偏移一个单位（得到第2行第2列的地址），然后解引用取值，得到a[1][1]；

如果数组定义为int a[2][3]={1, 2, 3, 4, 5, 6}

那么*(a[1]+1)等价于a[1][1]

先分析下括号里面的内容

a[1]表示第二行的首地址，zhi后面那个+1是第二个维度+1，即列+1，

最后在括号外的解引用得到元素的值

还有另外一种写法是

((a+1)+1)和a[1][1]也等价（注意这里最里面a+1一定要打括号，因为*号的优先级比+号高）

[]优先级高，a[1][1]取地址再取值。

第三个a+1相当于&a[1]，所以*(a+1)=a[1]，因此*(a+1)[1]=a[1][1]

不使用流程控制语句，如何打印出1~1000的整数

方法一：宏定义多层嵌套（10 * 10 * 10），printf多次输出。

```
1 #include <stdio. h>
2 #define B P,P,P,P,P,P,P,P,P,P
3 #define P L,L,L,L,L,L,L,L,L,L
4 #define L I,I,I,I,I,I,I,I,I,I,N
5 #define I printf ("%3d", i++)
6 #define N printf ("n")
7 int main()
8 {
9     int i = 1;
10    B;
11    return 0;
12 }
13
```

简便写法,同样使用多层嵌套

```
1 #include<stdio. h>
2 #define A (x) x;x;x;x;x;x;x;x;x;
3 int main ()
4 {
5     int n=1;
6     A(A(A(printf("%d", n++));
7     return 0;
8 }
```

a是数组，（int*）（&a+1）表示什么意思

对于数组而言，一个数组名代表的含义是数组中第一个元素的位置，即地址，通过数组名可以访问该数组。程序示例如下：

```
1 #include <stdio.h>
2 int main (
3 {
4     int a[5]={1,2,3,4,5};
5     int *ptr =(int*)&a+1;
6     printf("%d\n%d\n", *(a+1), *(ptr-1));
7     printf("sizeof(&a)=%d\n", sizeof(a));
8     printf("sizeof(&a)=%d\n", sizeof(&a));
9     return 0;
10 }
```

程序的输出结果为（在32位机器上的输出结果）：

```
2
5
sizeof(b) = 20;
sizeof ( &b ) =4
```

一般而言，对指针进行加1操作，得到的将是下一个元素的地址。因此，只要确定了**指针所指向的类型**，就可以很容易地求出指针+1的结果。也就是说，一个类型为T的指针移动，是以 **sizeof (T) 为移动单位**的。

对于数组int a[5]；a表示数组首元素的地址，而&a表示数组的首地址。

a有两层含义：一是a表示数组**首元素的地址**（类型为int*）；二是**a表示一个大小为5的int数组（重要）**。数组名本身在计算中会自动转化成第一个元素的地址，但**sizeof测试的时候不做计算，不转化**。由此可以看出，在计算*(a+1)的时候，可以把a看成int类型的指针，a+1表示的是数组第二个元素的指针，由此*(a+1)的值为2。但是，sizeof(a)是把a作为一个**数组类型**来测试大小，结果是数组占用的**字节数**，由此 sizeof(a)=20

&a是一个指向int(*)[5]的指针。由于&a是一个指针，那么在32位机器上，sizeof(&a)=4，但是**&a+1的值取决于&a指向的类型**，由于&a指向int(*)[5]，所以&a+1=&a+sizeof(int(*)[5])=&a+20。&a是数组的首地址，由此&a+1表示a向后移动了**5个int**从而指向a[5]，所以ptr指向的是a[5]，由于ptr是int * 类型，那么ptr-1就指向a[4]，所以 *(ptr-1)=a[4]=5。

由此可知 (int*) &(a+1) 指向了数组a最后一个元素之后的下一个内存单元开始的4个字节 (int *) 内存。

以下程序的输出

```
1 int main()
2 {
3     //int brand[5][5]={{'a','b','c','d','e'},{'f','g','h','g','k'}};
4     char brand[5][5]={{'a','b','c','d','e'},{'f','g','h','g','k'}};
5     int *p=(int*)brand[1];
6     printf("%c\r\n",*(++p));
7 }
```

输出k。++p = p+sizeof(int *)，原数组为char型，每个元素占据1字节，p指向的是f，sizeof(int *) = 4字节，所以最后p指向的是k。

当数组换成int型时，程序的输出为g，int型数组每个元素占据4字节。

数组下标可以为负数吗

可以，因为下标只是给出了一个与**当前地址的偏移量**而已，只要根据这个偏移量能定位得到目标地址即可。下面给出一个下标为负数的示例：

数组下标取负值的情况：

```
1 #include <stdio.h>
2 int main()
3 {
4     int i;
5     int a[5]={0,1,2,3,4};
6     int *p=&a[4]
7     for (i=-4; i<=0; i++)
8         printf("%d %x\n", p[i], &p[i]);
9     return 0.
10 }
11
```

程序的输出结果为

```

1 0 b3ecf480
2 1 b3ecf484
3 2 b3ecf488
4 3 b3ecf48c
5 4 b3ecf490

```

从上例可以发现，在C语言中，数组的下标并非不可以为负数，当数组下标为负数时，编译可以通过，而且也可以得到正确的结果，只是它表示的意思却是从当前地址**向前寻址**。

C语言宏中"#"和"##"的用法

在查看linux内核源码的过程中，遇到了许多宏，这里面有许多都涉及到"#"和"##"，因此，在网上搜索了一些资料，整理如下：

一、一般用法

我们使用#把宏参数变为一个字符串,用##把两个宏参数贴合在一起。

用法:

```

1 #include<stdio>
2 #include<climits>
3 using namespace std;
4
5 #define STR(s)      #s
6 #define CONS(a,b)  int(a##e##b)
7
8 int main()
9 {
10     printf(STR(vck));           // 输出字符串"vck"
11     printf("%d\n", CONS(2,3)); // 2e3 输出:2000
12     return 0;
13 }

```

二、当宏参数是另一个宏的时候

需要注意的是凡宏定义里有用'#'或'##'的地方宏参数是不会再展开。

1, 非'#'和'##'的情况

```

1 #define TOW      (2)
2 #define MUL(a,b) (a*b)
3 printf("%d*%d=%d\n", TOW, TOW, MUL(TOW,TOW));

```

这行的宏会被展开为：

```

1 printf("%d*%d=%d\n", (2), (2), ((2)*(2)));

```

MUL里的参数TOW会被展开为(2)。

2, 当有'#'或'##'的时候

```

1 #define A      (2)
2 #define STR(s)  #s
3 #define CONS(a,b) int(a##e##b)
4
5 printf("int max: %s\n", STR(INT_MAX)); // INT_MAX #include<climits>

```

这行会被展开为：

```
1 printf("int max: %s\n", "INT_MAX");
2
3 printf("%s\n", CONS(A, A));           // compile error
```

这一行则是：

```
1 printf("%s\n", int(AeA));
```

INT_MAX和A都不会再被展开, 然而解决这个问题方法很简单. 加多一层中间转换宏.

加这层宏的用意是把所有宏的参数在这层里全部展开, 那么在转换宏里的那一个宏(_STR)就能得到正确的宏参数.

```
1 #define A          (2)
2 #define _STR(s)     #s
3 #define STR(s)      _STR(s)           // 转换宏
4 #define _CONS(a,b)  int(a##e##b)
5 #define CONS(a,b)   _CONS(a,b)       // 转换宏
6
7 printf("int max: %s\n", STR(INT_MAX)); // INT_MAX,int型的最大值, 为一个变量
```

输出为: int max: 0x7fffffff

```
1 STR(INT_MAX) --> _STR(0x7fffffff) 然后再转换成字符串;
2
3 printf("%d\n", CONS(A, A));
```

输出为：200

```
1 CONS(A, A) --> _CONS((2), (2)) --> int((2)e(2))
```

三、'#'和'##'的一些应用特例

1、合并匿名变量名

```
1 #define __ANONYMOUS1(type, var, line) type var##line
2 #define __ANONYMOUS0(type, line) __ANONYMOUS1(type, _anonymous, line)
3 #define ANONYMOUS(type) __ANONYMOUS0(type, __LINE__)
```

例：ANONYMOUS(static int); 即: static int _anonymous70; 70表示该行行号；

第一层：ANONYMOUS(static int); --> **ANONYMOUS0(static int, LINE);**

第二层：--> _ANONYMOUS1(static int, _anonymous, 70);

第三层：--> static int _anonymous70;

即每次只能解开当前层的宏，所以**LINE**在第二层才能被解开；

2、填充结构

```

1  #define FILL(a)  {a, #a}
2
3  enum IDD{OPEN, CLOSE};
4  typedef struct MSG{
5      IDD id;
6      const char * msg;
7  }MSG;
8
9  MSG _msg[] = {FILL(OPEN), FILL(CLOSE)};

```

相当于：

```

1  MSG _msg[] = {{OPEN, "OPEN"},
2               {CLOSE, "CLOSE"}};

```

3、记录文件名

```

1  #define _GET_FILE_NAME(f)  #f
2  #define GET_FILE_NAME(f)  _GET_FILE_NAME(f)
3  static char FILE_NAME[] = GET_FILE_NAME(__FILE__);

```

4、得到一个数值类型所对应的字符串缓冲大小

```

1  #define _TYPE_BUF_SIZE(type)  sizeof #type
2  #define TYPE_BUF_SIZE(type)  _TYPE_BUF_SIZE(type)
3  char buf[TYPE_BUF_SIZE(INT_MAX)];
4      --> char buf[_TYPE_BUF_SIZE(0x7fffffff)];
5      --> char buf[sizeof "0x7fffffff"];

```

这里相当于：

```

1  char buf[11];

```

“#把宏参数变为一个字符串,用##把两个宏参数贴合在一起”

```

1  #include<stdio.h>
2  #include<string.h>
3  #define STRCPY(a,b) strcpy(a##_p,#b)    //把第一个参数后边加上字符_p，把第二个参数
      变成字符串
4  int main()
5  {
6  char var1_p[20];
7  char var2_p[30];
8  strcpy(var1_p,"aaaa");
9  strcpy(var2_p,"bbbb");
10      STRCPY(var1,var2);                //等于strcpy(var1_p,"var2");
11  STRCPY(var2,var1);                    //等于strcpy(var2_p,"var1");
12  printf("%s\n",var1_p);
13  printf("%s\n",var2_p);
14  return 0;
15  }

```

typedef和 define有什么区别

typedef与 define都是替一个对象取一个别名，以此来增强程序的可读性，但是它们在使用和作用上也存在着以下4个方面的不同。

1) 原理不同

#define是C语言中定义的语法，它是预处理指令，在预处理时进行简单而机械的字符串替换，**不做正确性检查**，不管含义是否正确照样代入，只有在编译已被展开的源程序时，才会发现可能的错误并报错。

例如，`#define PI 3.1415926`，当程序执行 `area=Pr*r` 语句时，PI会被替换为3.1415926。于是该语句被替换为 `area=3.1415926*r*r`。如果把# define语句中的数字9写成了g，预处理也照样代入，而不去检查其是否合理、合法。

typedef是关键字，它在编译时处理，所以 typedef具有类型检查的功能。它在自己的作用域内给一个已经存在的类型一个别名，但是不能在一个函数定义里面使用标识符 typedef。例如，`typedef int INTEGER`，这以后就可用 INTEGER来代替int作整型变量的类型说明了，例如：`INTEGER a,b;`

用 typedef定义数组、指针、结构等类型将带来很大的方便，不仅使程序书写简单而且使意义更为明确，因而增强了可读性。例如：`typedef int a[10];`

表示a是整型数组类型，数组长度为10。然后就可用a说明变量，例如:语句 `a s1,s2`；完全等效于语句 `int s1[10],s2[10]`。同理，`typedef void (*p) (void)`表示p是一种指向void型的指针类型。

2) 功能不同

typedef用来定义类型的别名，这些类型不仅包含内部类型（int、char等），还包括自定义类型（如 struct），可以起到使类型易于记忆的功能。

例如：`typedef int (*PF)(const char *, const char*)`

定义一个指向函数的指针的数据类型PF，其中函数返回值为int，参数为 const char*。
typedef还有另外一个重要的用途，那就是定义机器无关的类型。例如，可以定义一个叫REAL的浮点类型，在目标机器上它可以获得最高的精度：`typedef long double REAL`，在不支持 long double的机器上，该 typedef看起来会是下面这样：`typedef double real`，在 double都不支持的机器上，该 typedef看起来会是这样：`typedef float REAL`。

#define不只是可以为类型取别名，还可以定义常量、变量、编译开关等。

3) 作用域不同

#define没有作用域的限制，只要是之前预定义过的宏，在以后的程序中都可以使用，而 typedef有自己的作用域。

程序示例如下：

```
1 void fun()  
2 {  
3     #define A int  
4 }  
5 void gun()  
6 {  
7     //这里也可以使用A，因为宏替换没有作用域，但如果上面用的是 typedef，那这里就不能用  
8     //A，不过，一般不在函数内使用 typedef  
9 }  
10
```

4) 对指针的操作不同

两者修饰指针类型时，作用不同。

```
1 #define INTPTR1 int*
2 typedef int* INTPTR2;
3 INTPTR1 p1, p2;
4 INTPTR2 p3, p4;
```

INTPTR1 p1, p2和INTPTR2 p3, p4的效果截然不同。INTPTR1 p1, p2进行字符串替换后变成 `int*p1, p2`，要表达的意义是声明一个指针变量p1和一个整型变量p2。而 INTPTR2 p3, p4，由于 INTPTR2是具有含义的，告诉我们是一个指向整型数据的指针，那么p3和p4都为指针变量，这句相当于 `int*p1, *p2`。从这里可以看出，进行宏替换是不含任何意义的替换，仅仅为字符串替换；而用 typedef 为一种数据类型起的别名是带有一定含义的。

程序示例如下

```
1 #define INTPTR1 int*
2 typedef int* INTPTR2
3 int a=1;
4 int b=2;
5 int c=3;
6 const INTPTR1 p1=&a;
7 const INTPTR2 p2=&b;
8 INTPTR2 const p3=&c;
```

上述代码中，const INTPTR1 p1表示p1是一个常量指针，即不可以通过p1去修改p1指向的内容，但是p1可以指向其他内容。而对于 const INTPTR2 p2，由于 INTPTR2表示的是个指针类型，因此用 const去限定，表示封锁了这个指针类型，因此p2是一个指针常量，不可使p2再指向其他内容，但可以通过p2修改其当前指向的内容。INTPTR2 const p3同样声明的是一个指针常量。

数组名和指针的区别与联系

数据保存方面

指针保存的是地址（保存目标数据地址，自身地址由编译器分配），内存访问偏移量为4个字节，无论其中保存的是何种数据均已地址类型进行解析。

数组保存的数据。数组名表示的是第一个元素的地址，内存偏移量是保存数据类型的内存偏移量；只有对数组名取地址（&数组名）时数组名才表示整个数组，**内存偏移量是整个数组的大小（sizeof(数组名)）**。

数据访问方面

指针对数据的访问方式是间接访问，需要用到解引用符号（*数组名）。

数组对数据的访问则是直接访问，可通过下标访问或数组名+元素偏移量的方式

使用环境

指针多用于动态数据结构（如链表，等等）和动态内存开辟。

数组多用于存储固定个数且类型统一的数据结构（如线性表等等）和隐式分配。

strlen("\0") = ? sizeof("\0")=? p64

strlen("\0") =0，sizeof("\0")=2

strlen用来计算字符串的长度（在C/C++中，字符串是以“\0”作为结束符的），它从内存的某个位置（可以是字符串开头，中间某个位置，甚至是某个不确定的内存区域）开始扫描直到碰到第一个字符串结束符\0为止，然后返回计数器值sizeof是C语言的关键字，它以**字节的形式**给出了其操作数的**存储大小**，操作数可以是一个表达式或括在括号内的类型名，操作数的存储大小由操作数的类型决定。

具体而言，strlen与sizeof的差别表现在以下5个方面。

1) sizeof是运算符（是不是被弄糊涂了？事实上，sizeof既是关键字，也是运算符，但不是函数），而strlen是函数。sizeof后如果是类型，则必须加括弧，如果是变量名，则可以不加括弧。

2) sizeof运算符的结果类型是size_t，它在头文件中typedef为unsigned int类型。该类型保证能够容纳实现所建立的最大对象的字节大小

3) sizeof可以用类型作为参数，strlen只能用char*作参数，而且必须是以“0”结尾的。sizeof还可以以函数作为参数，如int g（），则sizeof（g（））的值等于sizeof（int）的值，在32位计算机下，该值为4。

4) 大部分编译程序的sizeof都是在**编译**的时候计算的，所以可以通过sizeof（x）来定义数组维数。而strlen则是在**运行期**计算的，用来计算字符串的实际长度，不是类型占内存的大小。例如，char str[20] = "0123456789"，字符数组str是**编译期**大小已经固定的数组，在32位机器下，为sizeof（char）*20=20，而其strlen大小则是在**运行期**确定的，所以其值为字符串的实际长度10。

5) 当数组作为参数传给函数时，传递的是指针，而不是数组，即传递的是数组的首地址。

不使用 sizeof，如何求int占用的字节数

```
1  #include <stdio.h>
2  #define MySizeof(Value)  (char *)&value+1)-(char*)&value
3  int main()
4  {
5      int i ;
6      double f;
7      double *q;
8      printf("%d\r\n",MySizeof(i));
9      printf("%d\r\n",MySizeof(f));
10     printf("%d\r\n",MySizeof(a));
11     printf("%d\r\n",MySizeof(q));
12     return 0;
13 }
```

输出为：4 8 32 4

上例中，（char）&Value返回Value的地址的第一个字节，（char）（&Value+1）返回value的地址的下一个地址的第一个字节，所以它们之差为它所占的字节数。

不能使用任何变量，如何实现计算字符串长度函数

非递归 但是不满足题意

```
1  #include <stdio.h>
2  int MyStrlen(const char *s)
3  {
4      int len = 0;
5      if(s == NULL)
6          return 0;
7      for(;*s++!='\0';)
8          len++;
9      return len;
10 }
11
12 int main()
13 {
14     printf("%d\r\n",MyStrlen("feizhu"));
```

```
15     return 0;
16 }
17
```

递归实现

```
1  #include <stdio.h>
2  int MyStrlen(const char *s)
3  {
4      if(*s!='\0')
5          return 1+strlen(++s);
6      else
7          return 0;
8  }
9  //改进版本
10 int MyStrlen(const char *s)
11 {
12     return *s=="\0"?0:(1+strlen(++s));
13 }
14 int main()
15 {
16     printf("%d\n\r",MyStrlen("feizhu"));
17     return 0;
18 }
```

什么是逗号表达式

关于二维数组赋值的一个陷阱

```
1  int a[3][2] = {(0,1),(2,3),(4,5)};
2  int *p;
3  p = a[0];
4  printf("%d\n",p[0]);
```

程序的输出结果为 1

```
1  int a[3][2] = {{0,1},{2,3},{4,5}};
2  int *p;
3  p = a[0];
4  printf("%d\n",p[0]);
```

程序的输出结果为 0

上述两段代码很类似，为什么输出却有这么大的不同？两者的区别在于二维数组的初始化问题，第一种情况由于是逗号表达式，所以数组的元素等价于{1,3,5,0,0}，与第二种情况不一样。

C语言提供一种特殊的运算符—逗号运算符，优先级最低，它将两式连接起来。例如，
(3+4,2+8) 称为逗号表达式，其求解过程是先求表达式1的运算结果，然后求解表达式2的结果，而整个表达式的值为表达式2的值，如 (3+4,2+8) 的值是10。(a=3*5, a*4) 的值为60。

在使用逗号表达式时，首先需要弄清楚其表示形式，其表示形式如下：

表达式1，表达式2，表达式3.....表达式n

使用逗号表达式，需要注意以下3个方面的事项

1) 逗号表达式的运算过程为从左往右逐个计算表达式的值。

- 2) 逗号表达式作为一个整体，它的值为最后一个表达式的值。
- 3) 逗号运算符的优先级在所有运算符中最低。

例如，语句*i*=1,2;，虽然是逗号表达式，但是赋值符的优先级更高，所以*i*的值为1，接着执行常量2的运算，运算结果会被丢弃。在编译器中，虽然这种语句的写法是合法的，但是不提倡。再如，语句（*a*=3*5, *a**4）和语句*b*=（*a*=3*5, *a**4）的意义不一样，第一个语句中*a*的值为15，第二个语句中*a*的值为15，但*b*的值为60

引申：int i = (j=4,k=8,l=16,m=32)，则i的值是多少

输出结果为32.当一个语句是由多个被逗号运算符隔开的表达式组成时，此语句的值为最后一个表达式的值。

首先看一个例子，int i = (j=4)，它等价于*i*=*j*=4语句，即int *j*=4与int *i*=*j*。而int *i* = (*j*=4, *k*=8, *l*=16, *m*=32) 等价于int *j*=4, *k*=8, *l*=16, *m*=32, *i*=*m*=32，所以输出为32。例如，int *a*=3, 则执行 *a*+=*a*=*a*=*a***a* 运算后，*a*的值变为0，因为这个连续运算语句可以转换为以下3个语句：首先计算 *a*+=*a***a*，把*a*的值3代入，*a*+=3+3*3，则*a*的值变为12，然后计算*a*=*a*，此时*a*的值变为0，最后执行*a*+=*a*操作，*a*的值最终变为0。

C语言中 struct与 union的区别是什么

P86

struct（结构体）与 union（联合体）是C语言中两种不同的数据结构，两者都是常见的复合结构，其区别主要表现在以下两个方面。

1) 结构体与联合体虽然都是由多个不同的数据类型成员组成的，但不同之处在于联合体中所有成员**共用一块地址空间**，即联合体只存放了一个被选中的成员，而结构体中所有成员占用空间是累加的，其所有成员都存在，不同成员会存放在不同的地址。在计算一个结构型变量的总长度时，其内存空间大小等于所有成员长度之和（需要考虑字节对齐），而在联合体中，所有成员不能同时占用内存空间，它们不能同时存在，所以**一个联合型变量的长度等于其最长的成员的长度**。

2) 对于联合体的不同成员赋值，**将会对它的其他成员重写**，原来成员的值就不存在了，而对结构体的不同成员赋值是互不影响的。

```
1 typedef union {double i; int k[5]; char c;}DATE;
2 typedef struct data( int cat; DATE cow;double dog;) too;
3 DATE max;
4 printf ("%d", sizeof(too)+sizeof(max));
```

执行结果是多少？

假设为32位机器，int型占4个字节，double型占8个字节，char型占1个字节，而DATE是一个联合型变量，联合型变量共用空间，union里面最大的变量类型是int[5]，**所以占用20个字节，它的大小是20**，而由于union中double占了8个字节，因此union是要8个字节对齐，**所占内存空间为8的倍数**。为了实现8个字节对齐，**所占空间为24**。而data是一个结构体变量，每个变量分开占用空间，依次为sizeof (int) + sizeof (DATE) + sizeof (double) =4+24+8=36按照8字节对齐，占用空间为40，所以结果为40+24=64。

全局变量和静态变量的区别

1) 全局变量的作用域为程序块，而局部变量的作用域为当前函数。

2) 内存存储方式不同，全局变量（静态全局变量，静态局部变量）分配在全局数据区（静态存储空间），后者分配在栈区。

3) 生命周期不同。全局变量随主程序创建而创建，随主程序销毁而销毁，局部变量在局部函数内部，甚至局部循环体等内部存在，退出就不存在了。

4) 使用方式不同。通过声明为全局变量，程序的各个部分都可以用到，而局部变量只能在局部使用。

理解非常复杂的声明

- 1) 一个整型数。
- 2) 一个指向整型数的指针。
- 3) 一个指向指针的指针，它指向的指针是指向一个整型数
- 4) 一个有10个整型数的数组。
- 5) 一个有10个指针的数组，数组中指针指向一个整型数。
- 6) 一个指向有10个整型数数组的指针。
- 7) 一个指向函数的指针，该函数有一个整型参数并返回一个整型数。
- 8) 一个有10个指针的数组，该指针指向一个函数，该函数有一个整型参数并返回一个整型数。

对于上述问题，可以有如下解答

```
1 1) int a:
2 2) int *a:
3 3) int **a
4 4) int a[10]
5 5) int *a[10];
6 6) int(*a)[10];
7 7) int(*a)(int);
8 8) int(*a[10])(int)
```

```
1 void*(*fp1)(int)[10];
2 float(*fp2)(int,int,int)(int);
3 int(*fp3())[10]();
```

分别表示什么意思？

答案：对于这种复杂的表达式，为了能够理解它表达的意思，可以对表达式进行分解，分解成比较简单的表达式，当明确了各个简单表达式的意思后，把各个表达式结合起来就可以得出整个表达式的意思。下面通过这种方法对这3个表达式进行详细介绍

1) 为了便于理解，把 `void*(*fp1)(int)[10];` 分解为如下两个表达式

```
1 p=(*fp1)(int)
2 void*(*p)[10]
```

第一个表达式可以与上面讲解的7) 对应，fp1是一个函数的指针，这个函数的参数为int，函数的返回值为p;第二个表达式与上面讲解的6) 对应，由此可以看出p是一个指向有10个元素的数组的指针，这个数组中元素的类型为void (指向void指针)。综上所述，fp1是一个函数的指针，它指向的函数有一个int参数，这个函数的返回值是一个指针，这个指针指向一个有10个元素的数组，而且数组中的每个元素都是void。

2) 为了便于理解，把 `float(*fp2)(int,int,int)(int);` 分解为如下两个表达式

```
1 p=(*fp2)(int,int,int)
2 float(*p)(int);
```

由第一个表达式可以看出：fp2是一个指针，这个指针指向一个函数，这个函数有3个int类型的参数，这个函数的**返回值为p**；从第二个表达式可以看出p也是一个指针，这个指针指向一个函数，并且这个函数的参数为int，返回值为float。综上所述，fp2是一个指向函数的指针，函数有3个int类型的参数，函数的**返回值是一个函数的指针**（这个函数的参数为int返回值为float）。

3) 对于声明 `int>(*fp3)()[10]();`，通过同样的分析方式可以得出：fp3是一个指向函数的指针，这个函数的参数为空，函数的返回值也是一个指针（这个指针指向一个有10个元素的数组，数组中的每个元素是一个指针，这个指针指向一个函数，这个函数的参数为空，函数的返回值是int型）。

变量定义和变量声明

在一个程序中，变量有且仅有一个定义如，`int my_array[100]`，而声明（declaration）是指向程序表明变量的类型和名字。定义也是声明，当定义变量时声明了它的类型和名字。可以通过使用 `extern` 关键字声明变量名而不定义它，它所说明的并非自身，而是描述其他地方创建的对象，可以多次出现，如 `extern int my_array[]`。

“定义也是声明”，这说明定义包括声明，对于int a来说，它既是定义，又是声明，对于`extern int a`来说，它是声明，不是定义。一般为了叙述方便，把**申请存储空间的声明称为定义**，而把**不申请存储空间的声明称为声明**。

一个十六进制的数占用多少字节

1个字节是8位，二进制8位：xxxxxxx 范围从00000000 - 11111111，表示0到255。一位16进制数（用二进制表示是xxxx）最多只表示到15（即对应16进制的F），要表示到255,就还需要第二位。所以1个字节 = 2个16进制字符，一个16进制位 = 0.5个字节 = 4位2进制（xxxx）。

嵌入式编程中，什么是大端？什么是小端

ARM CPU大小端：

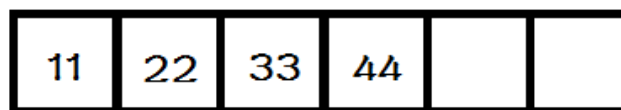
大端模式：低位字节存在高地址上，高位字节存在低地址上

小端模式：高位字节存在高地址上，低位字节存在低地址上

11 22 33 44

尾端 44

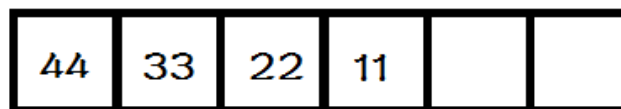
高尾端（大端）



0 1 2 3

低地址 -----> 高地址

低尾端（小端）



0 1 2 3

低地址 -----> 高地址 https://www.csdn.net/qq_16933601



STM32属于小端模式,简单的说,比如u32 temp=0X12345678;

假设temp地址在0X2000 0010.

那么在内存里面,存放就变成了:

1	地址				HEX		
2	0x2000	0010		78	56	43	12

因为是16进制的，一个数为0.5字节，所以 12 代表一个字节 34 代表一个字节
采用小端模式的CPU对操作数的存放方式是从低字节到高字节，而大端模式对操作数的存放方式是从高字节到低字节。例如，16位宽的数0x1234在小端模式CPU内存中的存放方式（假设从地址0x4000开始存放）见表1，而在大端模式CPU内存中的存放方式见表2。

表1 0x1234在小端CPU内存中的存放方式

内存地址	存放内容
0x4000	0x34
0x4001	0x12

表2 0x1234在大端CPU内存中的存放方式

内存地址	存放内容
0x4000	0x12
0x4001	0x34

32位宽的数0x12345678在小端模式CPU内存中的存放方式（假设从地址0x4000开始存放）见表3，而在大端模式CPU内存中的存放方式见表4。

表3 0x12345678在小端CPU内存中的存放方式

内存地址	存放内容
0x4000	0x78
0x4001	0x56
0x4002	0x34
0x4003	0x12

表4 0x12345678在大端CPU内存中的存放方式

内存地址	存放内容
0x4000	0x12
0x4001	0x34
0x4002	0x56
0x4003	0x78

以下程序为例：

```
1 #include <stdio.h>
2 struct mybitfields
3 {
```

```

4     unsigned short a:4;
5     unsigned short b:5;
6     unsigned short c:7;
7 }test;
8 int main()
9 {
10    int i;
11    test.a = 2;
12    test.b = 3;
13    test.c = 0;
14    i =*((short*)&test);
15    printf("%d\n",i);
16    return 0;
17 }

```

程序的输出结果为 50

上例中，sizeof (test) =2，上例的声明方式是把一个 short（也就是一块16位内存）分成3部分，各部分的大小分别是4位、5位、7位，赋值语句 i*((short*)&test) 就是把上面的16位内存转换成 short 类型进行解释。

变量a的二进制表示为0000000000000010，取其低四位是0010.变量b的二进制表示为0000000000000011，取其低五位是00011。变量c的二进制表示为0000000000000000，取其低七位是0000000。

80x86机是小端（修改分区表时要注意）模式，单片机一般为大端模式。小端一般是低位字节在高位字节的前面，也就是低位在内存地址低的一端，可以这样记（小端→低位→在前→与正常逻辑顺序相反），所以合成后得到0000000000110010，即十进制的50，下面给出另外一个例子

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4  int main()
5  {
6      unsigned int uiVal_1 = 0x12345678;
7      unsigned int uiVal_2 = 0;
8      unsigned char aucVal[4] = {0x12,0x34,0x56,0x78};
9      unsigned short usVal_1 = 0;
10     unsigned short usVal_2 = 0;
11     memcpy(&uiVal_2,aucVal,sizeof(uiVal_2));
12     usVal_1 = (unsigned short)uiVal_1;//在这里截断，都取得的是低位
13     usVal_2 = (unsigned short)uiVal_2;//在这里截断
14     printf("usVal_1:%x\n",usVal_1);//在这里又转化回来
15     printf("usVal_2:%x\n",usVal_2);//在这里又转化回来
16     return 0;
17 }
18 }

```

小端模式是低地址存放低字节，高地址存放高字节，结构如下所示

```

1  78//低地址
2  56
3  34
4  12//高地址

```

在内存里面测试机是小端，地址由小到大。

```

1 val1:78563412
2 rival2:12345678

```

结果如下：

```

1 5678
2 3412

```

引申 如何判断计算机处理器是大端，还是小端

```

1 #include <stdio.h>
2 int checkCPU()
3 {
4     {
5         union w
6         {
7             int a;
8             char b;
9         }c;
10        c.a =1;
11        return(c.b == 1);
12    }
13 }
14 int main()
15 {
16     if(checkCPU())
17         printf("小端\n");
18     else
19         printf("大端\n");
20     return 0;
21 }

```

编者的处理器为ntel处理器，因为 Intel处理器一般都是小端模式，所以此时程序的输出结果为：小端

上述代码中，如果处理器是大端，则返回0；如果处理器是小端，则返回1.联合体 union的存放顺序是所有成员都从低地址开始存放，如果能够通过改代码知道CPU对内存是采用小端模式读写，还是采用大端模式读写，一定会令面试官刮目相看。

还可以通过指针地址来判断，由于在32位计算机系统中，short占两个字节，char占1个字节，所以可以采用如下做法实现该判断。

```

1 #include <stdio.h>
2 int checkCPU()
3 {
4     unsigned short usData = 0x1122;
5     unsigned char*pucData = (unsigned char*)&usData;
6     return (*pucData == 0x22);
7 }
8 int main()
9 {
10    if(checkCPU())
11        printf("小端\n");
12    else
13        printf("大端\n");

```

```
14     return 0;
15 }
```

程序输出的结果为：小端

如何进行大小端的转换

```
1  int swapInt32(int intValue){
2
3      int temp = 0;
4
5      temp = ((intValue & 0x000000FF) <<24) |
6
7              ((intValue & 0x0000FF00) <<8) |
8
9              ((intValue & 0x00FF0000) >>8) |
10
11             ((intValue & 0xFF000000) >>24);
12     return temp;
13 }
14 /*short型: */
15 unsigned short swapShort16(unsigned short shortValue){
16
17
18
19     return ((shortValue & 0x00FF ) <<8) | ((shortValue & 0xFF00)>>8);
20
21 }
22 /*float型:*/
23 float swapFloat32(float floatValue){
24
25     typedef union SWAP_UNION{
26
27         float unionFloat;
28
29         int    unionInt;
30
31     }SWAP_UNION;
32
33     SWAP_UNION swapUnion;
34     swapUnion.unionFloat = floatValue;
35     swapUnion.unionInt = swapInt32( swapUnion.unionInt);
36
37     return    swapUnion.unionFloat;
38 }
39 /*double型换一种写法，用一下指针，不然移位移死了.....*/
40 void swapDouble64(unsigned char *pIn, unsigned char *pOut){
41
42     for( int i=0;i<8;i++)
43
44     pOut[7-i] = pIn[i];
45
46 }
47
48 int main()
49 {
50     int x = 0x12345678;
```

```

51     int y = swapInt32(x);
52     printf("%x\r\n",y);
53     return 0;
54 }

```

不用除法操作符如何实现两个正整数的除法

在回答本问题前，先介绍一些有关位运算的知识。

1) 常用的等式： $-n = \sim(n-1) = \sim n$

2) 获取整数n的二进制中最后一个1： $n \& (-n)$ 或者 $n \& \sim(n-1)$ 。例如， $n=010100$ ，则 $-n=101100$ ， $n \& (-n) = 000100$ 。

3) 去掉整数n的二进制中最后一个1： $n \& (n-1)$ ，如 $n=010100$ ， $n-1=010011$ ， $n \& (n-1) = 010000$ 。

一般求解两个正整数的除法问题时，首先考虑到的就是采用除法运算，但是除了除法运算外，还有其他方法可以执行除法运算。

方法一

可以根据除法运算的原理进行减法操作，对除数循环减被除数，减一次结果加直到刚好减为0或余数小于被除数为止。程序示例如下：

```

1  #include <stdio.h>
2  int div(int a,int b)
3  {
4      int result = 0;
5      if (b==0)
6      {
7          printf("除数不能为0\n");
8          return result;
9      }
10     while (a>=b)
11     {
12         result++;
13         a=a-b;
14     }
15     return result;
16 }
17
18 int main()
19 {
20     printf("%d\n",div(10,3));
21     return 0;
22 }

```

程序的输出结果为3

当a 很大，b很小时，这个算法的效率会非常低。

方法二，翻倍比较法。

以100/3为例，方法一提出的方法分别比较97,94,91，...，4,1，最后余数为1，退出 while循环，整个算法需要比较34次。如果每次采用将比较数翻倍的比较方法，则算法效率能够得到极大优化。这种算法的主要思路为：首先用3与100比较，显然3小于100，然后翻倍6，仍然小于100，以此类推用12,24,48，96,192与100进行比较，由于192>100，因此退回到96.这里共比较了7次，每比较一次 $k++$ ， $res+=1 \ll k$ 。

100-96=4>除数3，继续使用上面的算法，用4先跟3比较，然后6,6>4，这次比较2次。退回到3,4-3=1<除数3.算法结束，总共比较了7+2+1=10次，显然比方法一效率更高。程序如下

```

1  #include <stdio.h>
2  int MyDiv(int a,int b)
3  {
4      int k = 0;
5      int c = b;
6      int res = 0;
7      if(b== 0)
8          printf("除数不能为0\n");
9      if(a<b)
10         return 0;
11     for(;a>=c;c<<1,k++)
12         if(a-c<b)
13             return 1<<k;
14     return MyDiv(a-(c>>1),b)+(1<<(k-1));
15 }
16 int main()
17 {
18     printf("%d\n",MyDiv(100,3));
19     return 0;
20 }

```

输出结果为33

方法三

采用移位操作实现，位操作的效率一般都比较高效。程序示例如下：

```

1  #include <stdio.h>
2  int div(const int x,const int y)
3  {
4      int left_num = x;
5      int result = 0;
6      while(left_num>=y)
7      {
8          int multi = 1;
9          while(y*multi<=(left_num>>1))
10             {
11                 multi = multi << 1;
12             }
13             result+=multi;
14             left_num -=y*multi;
15         }
16     return result;
17 }
18 int main()
19 {
20     printf("%d\n".div(10,3));
21     return 0;
22 }

```

程序的输出结果3

引申：如何只用逻辑运算实现加法运算

实现两个正整数相加，一般直接使用加号运算符即可。考虑到题目中的要求，与上例中的方法二类似，也可以通过移位操作符进行正整数的加法运算。例如，5与7求和，转换为二进制求和为101与111求和，其二进制结果为1100.对于二进制的加法而言，1+1=0,1+0=1,0+1=1,0+0=0，通过对比位运算中的异或方法，不难发现，此方法与位运算中的异或类似。那么，第一个数值就是：

$tempLum=num1\wedge num2$ ；0+0的进位是0,1+0的进位是0，只有1+1的进位有效，该思路与位运算的&运算相似，即只有 $1\&1=1$ ，所以可以先 $num1\&num2$ ，由于进位是进到高一位的，与<<运算很相似，同时 $num1$ 和 $num2$ 相互&之后，如果结果为0，那么就不存在进位，运算完成，所以可以用递归的思想实现。程序示例如下

```

1  #include <stdio.h>
2
3  int add(int num1,int num2)
4  {
5      if(0 == num2)
6          return num1;
7      int sumTemp = num1^num2;
8      int carry = (num1 & num2)<<1;
9      return add(sumTemp,carry);
10 }
11 int main()
12 {
13     printf("%d\n",add(100,200));
14     return 0;
15 }
```

程序输出的结果为300

使用非递归方法实现的代码如下：

```

1  #include <stdio.h>
2  int add(int num1,int num2)
3  {
4      int sum = 0;
5      int num3 = 0;
6      int num4 = 0;
7      while((num1 & num2)>0)
8      {
9          num3 = num1^num2;
10         num4 = num1&num2;
11         num1 = num3;
12         num2 = num4 << 1;
13     }
14     sum = num1^num2;
15     return sum;
16 }
17 int main()
18 {
19     printf("%d\n",add(100,200));
20     return 0;
21 }
```

程序的结果为300

引申2：引申2：如何只用逻辑运算实现乘法运算

先看一个实例： $1011*1010$ ，因为二进制运算的特殊性，所以可以将该乘法运算表达式拆分为两个运算， $1011*0010$ 与 $1011*1000$ 的和，而对于二进制的运算，左移1位，等价于乘以0010，左移3位，等价于乘以1000，所以，两者的乘积为10110与1011000的和，即为1101110
因而乘法可以通过一系列移位和加法完成。最后一个1可通过 $b\&\sim(b-1)$ 求得，可通过 $b\&(b-1)$ 去掉，为了高效地得到左移的位数，可提前计算一个map，示例代码如下：

```

1  #include <iostream>
2  #include <map>
3  using namespace std;
4
5  int multiply(int a,int b)
6  {
7      bool neg = (b<0);
8      if(b<0)
9          b = -b;
10     int sum = 0;
11     map<int,int>bit_map;
12     for(int i = 0;i<32;i++)
13         bit_map.insert(pair<int,int>(1<<i,i));
14     while(b>0)
15     {
16         int last_bit = bit_map[b&~(b-1)];
17         sum+=(a<<last_bit);
18         b&=b-1;
19     }
20     if(neg)
21         sum =-sum;
22     return sum;
23 }
24 int main()
25 {
26     printf("%d\n",multiply(3,5));
27     return 0;
28 }

```

程序的输出结果为 15

不使用if/?: switch及其他判断语句如何找出两个int型变量中的最大值和最小值

寻找两个变量中的较大值，一般可以用判断型语句进行比较，如（a>b），按照题目中的要求，不能采用判断语句，所以可以利用绝对值的方法或是移位的方法。

方法一：

```
1  int max=((a+b)+abs(a-b))/2
```

如果a>b，则max=a；如果a<b，则max=b

```
1  int min=((a+b)-abs(a-b))/2
```

如果a>b，则min=b；如果a<b，则min=a。

上例中，abs是C语言中的用于求绝对值的函数。C语言中还有一个类似的函数，函数名为fabs

方法二

对变量的差值进行移位操作，通过判断差值的符号就可以确定两个值的大小

```

1  #include <stdio h>
2  int main()
3  {
4      int a=1,b=4;
5      int c=a-b;
6      int MAX=(unsigned)c>>(sizeof(int)*8-1);
7      if (!MAX)
8          printf("%d\n",a);
9      else
10         printf("%d\n", b);
11     return 0;
12 }

```

程序的输出结果4

方法三

通过移位操作判断。程序示例如下：

```

1  void compare(int a ,int b)
2  {
3      static char *op[]{"=", "<", ">"};
4      int i =(unsigned(a-b)>>31) +(unsigned(b-a)>>31)*2;
5      printf ("%d %s %d\n",a,op[i],b);
6  }

```

上例中，

- 1) 如果a>b，那么(unsigned (a-b) >>31)=0,(unsigned(b-a)>>31)×2=2。
- 2) 如果a=b，那么(unsigned (a-b) >>31)=0，(unsigned(b-a)>>31)×2=0。
- 3) 如果a<b，那么(unsigned (a-b) >>31)=1，(unsigned(b-a)>>31)×2=0。

方法四

通过加减运算与移位运算结合的方式实现。

```

1  void compare3(int a ,int b)
2  {
3      int min = a+(((b-a)>>31)&(b-a));
4      int max = a-(((a-b)>>31)&(a-b));
5      count<<min<<endl;
6      count<<max<<endl;
7  }

```

注意：正数的补码和原码相同。负数的补码是正数的补码按位取反，末位加1

方法五

```

1  int main()
2  {
3      int a =1,b = 4;
4      int c = a-b;
5      //涉及到有符号数和无符号数的转换
6      //int c = -1
7      //int c= 1000 0000 0000 0001
8      //(unsigned int ) c= 1111 1111 1111 1111
9
10     //int c = 3

```

```

11 //int c= 0000 0000 0000 0011
12 //(unsigned int ) c= 0111 1111 1111 1101
13 int MAX = (unsigned int )c >> (sizeof(int)*8-1);
14 printf("%d\n\r",MAX);
15 if(!MAX)
16     printf("%d\n\r",a);
17 else
18     printf("%d\n\r",b);
19 return 0;
20 }

```

一些结构声明中的冒号和数字是什么意思

C语言的结构体可以实现位段（也称位域），它的定义形式是在一个定义的结构体成员后加上冒号，然后是该成员所占的位数。位段的结构体成员必须是int或者 unsigned int类型，不能是其他类型。位段在内存中的存储方式是由具体的编译器决定的。

首先，定义位段的长度不能大于存储单元的长度。存储单元是指该位段的类型大小。其次，一个位段如果不能放在一个存储单元里，那么它会把这个存储单元中剩余的空间闲置，而从下一个存储单元开始存储下一个位段，即一个位段不能跨越两个存储单元，位段在一个存储单元中的存储是紧凑的。再次，位段名缺省时称作无名位段，无名位段的存储空间通常不用，而位段长度为0位表示下一个位段存储在一个新的存储单元中，位段长度为0的时候位段名必须缺省（不能定义位段名）。最后，一个结构体中既可以定义位段成员，也可以同时定义一般的结构体成员。这个时候，一般成员不和位段存储在同一个存储单元中。

程序示例分析如下：

```

1  #include <stdio.h>
2  typedef struct
3  {
4      int a:2;
5      int b:2;
6      int c:1;
7  }test;
8  int main()
9  {
10     test t;
11     t.a = 1;
12     t.b = 3;
13     t.c = 1;
14     printf("%d\n%d\n%d\n", t.a,t.b,t.c);
15     return 0;
16 }

```

程序的输出结果为

```

1  1
2  -1
3  -1

```

由于a占两位，而a被赋值为1，二进制就是01，因此，%d输出的时候输出1；b也占两位，赋值为3，二进制也就是11，由于使用了%d输出，表示的是将这个b作为有符号int型来输出，这样的话二进制的11将会有一位被认为是符号位，并且两位的b也会被扩展为int类型，也就是4字节，即32位。其实，a也做了这种扩展，只是扩展符号位的时候，由于数字在计算机中存储都是补码形式，因此扩展符号位的

时候正数用0填充高位，负数用1填充高位。因此，对于a来说，输出的时候被扩展为00000000 00000000 00000001也就是1，而b则扩展为11111111 11111111 11111111 11111111，也就是-1.同理，t.c的输出也是-1

如何求解整型数的二进制表示中1的个数

求解整型数的二进制表示中1的个数有以下两种方法：

方法一

判断每个数的二进制表示中每一位是否为1，如果为1，就在 count上加1，而循环的次数是常数，即n的位数。程序示例如下：

```
1  #include <stdio.h>
2  int func(unsigned int n)
3  {
4      int count = 0;
5      while(n)
6      {
7          count +=n&0x1u;
8          n >>=1;
9      }
10     return count;
11 }
12 int main()
13 {
14     printf("%d\n",func(9999));
15     return 0;
16 }
```

程序的输出结果为8

需要注意的是，上例中，0x1u表示的是十六进制的无符号数1.但该方法有一个缺陷，就是在1比较稀疏的时候效率比较低。下面给出另外一种比较高效的方法

方法二

程序代码如下

```
1  #include <stdio.h>
2  int func(int x)
3  {
4      int countx = 0;
5      while(x)
6      {
7          countx++;
8          x = x&(x-1);
9      }
10     return countx;
11 }
12 int main()
13 {
14     printf("%d\n",func(9999));
15     return 0;
16 }
```

程序输出的结果为8

在上例中，函数func ()的功能是将x转化为二进制数，然后计算该二进制数中含有的1的个数。首先以9为例来分析，9的二进制表示为1001,8的二进制表示为1000，两者执行&操作之后结果为1000，此时1000再与0111 (7的二进制位) 执行&操作之后结果为0。

为了理解这个算法的核心，需要理解以下两个操作：

- 1) 当一个数被减1时，它最右边的那个值为1的bit将变为0，同时其右边的所有的bit都会变成1。
- 2) 每次执行 $x \& (x-1)$ 的作用是把 x 对应的二进制数中的最后一位1去掉。因此，循环执行这个操作直到 x 等于0的时候，循环的次数就是 x 对应的二进制数中1的个数。

求二进制中0的个数

```

1  int CountZeroBit(int num)
2  {
3      int count = 0;
4      /*for (int i = 31; i > 0; i++) //一般的实现方式
5      {
6          if (num % 2 == 0)
7          {
8              count++;
9          }
10         else
11         {
12             continue;
13         }
14     }*/
15     while (num + 1)
16     {
17         count++;
18         num |= (num + 1); //算法转换
19     }
20     return count;
21 }
22
23 int main()
24 {
25     int value = 25;
26     int ret = CountZeroBit(value);
27     printf("%d的二进制位中0的个数为%d\n", value, ret);
28     system("pause");
29     return 0;
30 }
```

不能用 sizeof () 函数，如何判断操作系统是16位，还是32位

如果没有强调不许使用 sizeof，一般可以使用 sizeof 计算字节长度来判断操作系统的位数，如果在32位机器上，sizeof (int) =4，如果在16位机器上，sizeof (int) =2.除此之外，还有以下两种方法。

方法一

一般而言，机器位数不同，其表示的数字的最大值也不同，根据这一特性，可以判断操作系统的位数。在16位机器上，int可以表示-32768到+32767之间的数，也就是说，它能表示最大的数为32767，如果给这个数加1，它将会溢出变为-32768.但是，在32位机器上，int能表示的最大值为2147483647，因此 $32767+1=32768$ ，不会溢出。可以根据这个特性来判断系统是16位，还是32位，实现代码如下：

例如，运行如下代码：

```

1  #include <stdio.h>
2  int main()
3  {
4      int i = 65536;
5      i++;
6      if(i>0)
7          printf("32位\n");
8      else
9          printf("16位\n");
10     return 0;
11 }

```

方法二

在16位机器上，unsigned int能表示的最大值为65535，而在32位机器上，int可表示的最大值为4294967295。对类型为unsigned int的0取反，不同位数下的0值取反，其结果不一样。例如，在32位机器上，执行按位取反运算，结果为11111111111111111111111111111111 (4294967295)。而在16位机器上，取反的结果值为65535，可以根据这个特性来判断系统是16位，还是32位，运行如下代码：

```

1  #include <stdio.h>
2  int main()
3  {
4      unsigned int a = ~0;
5      if(a>65535)
6          printf("32位\n");
7      else
8          printf("16位\n");
9      return 0;
10 }

```

程序的输出结果为：32位

Makefile包含几部分；

Makefile是Make读入的惟一配置文件，因此本节的内容实际就是讲述Makefile的编写规则。在一个Makefile中通常包含如下内容：

- 需要由make工具创建的**目标体**（target），通常是目标文件或可执行文件；
- 要创建的目标体所**依赖**的文件（dependency_file）；
- 创建每个目标体时需要运行的**命令**（command）。

它的格式为：

```

1  target:
2  dependency_files
3  command

```

++a和a++那个运算效率高，为什么？

```

1  int temp = a;
2  a=a+1;
3  return temp;

```

++a的具体运算过程为

```
1 a=a+1;
2 return a;
```

后置自增运算符需要把原来变量的值复制到一个**临时的存储空间**，等运算结束后才会返回这个临时变量的值。所以前置自增运算符效率比后置自增要高

左值和右值

左值是指可以出现在等号左边的变量或表达式，它最重要的特点就是**可写**（可寻址）。也就是说，它的值**可以被修改**，如果一个**变量或表达式的值不能被修改**，那么它就不能作为左值。

右值是指**只可以出现在等号右边的变量或表达式**。它最重要的特点是**可读**。一般的使用场景都是把一个右值赋值给一个左值。

通常，左值可以作为右值，但是右值不一定是左值。

(a++) +=a是否合法

a++的具体运算过程为

```
1 int temp = a;
2 a=a+1;
3 return temp;
```

a++返回的值是编译器临时分配的**临时变量temp**，而temp并不是程序中定义的可寻址的变量，因此**不能作为左值**

++a的具体运算过程为

```
1 a=a+1;
2 return a;
```

显然，++a首先把a的值加1，然后返回a的值，由于a本身是在程序中定义的可寻址的变量，因此，++a可以作为左值

指针进行强制类型转换后与地址进行加法运算，结果是什么

假设在32位机器上，在对齐为4的情况下,sizeof(long)的结果为4字节，sizeof(char*)的结果为4字节，sizeof(short int)的结果与 sizeof(short)的结果都为2字节， sizeof(char)的结果为1字节，sizeof(int)的结果为4字节，由于32位机器上是4字节对齐，以如下结构体为例：

```
1 struct BBB
2
3 {
4
5     long num;
6
7     char *name;
8
9     short int data;
10
11     char ha;
12
13     short ba[5];
14
15 }*p;
```

当 `p=0x100000`; 则 `p+0x200=?` `(ulong)p+0x200=?` `(char*)p+0x200=?`

其实,在32位机器下, `sizeof(struct BBB)=sizeof(*p)=4+4+2+2+1+3/*补齐*/+2*5+2/*补齐*/=24字节`, 而 `p=0x100000`, 那么 `p+0x200=0x1000000+0x200*24` 指针加法, 加出来的是指针所指类型的字节长度的整倍数, 就是p偏移`sizeof(p)*0x200`。

`(ulong)p+0x200=0x100000010+0x200` 经过ulong后, 已经不再是指针加法, 而变成一个数值加法了。

`(char*)p+0x200=0x10000000+0x200*sizeof(char)` 结果类型是char*。

include< filename. h>和# nclude" filename. h"有什么区别

对于 `include< filename. h>`, 编译器先从标准库路径开始搜索 `filename.h`, 使得系统文件调用较快。而对于 `# include" filename.h"`, 编译器先从用户的工作路径开始搜索 `filename.h`, 然后去寻找系统路径, 使得自定义文件较快。

引申：头文件的作用有哪些

头文件的作用主要表现为以下两个方面：

1) 通过头文件来调用库功能。出于对源代码保密的考虑, 源代码不便 (或不准) 向用户公布, 只要向用户提供头文件和二进制的库即可。用户只需要按照头文件中的接口声明来调用库功能, 而不必关心接口是怎么实现的。编译器会从库中提取相应的代码。

2) 头文件能加强类型安全检查。当某个接口被实现或被使用时, 其方式与头文件中的声明不一致, 编译器就会指出错误, 大大减轻程序员调试、改错的负担。

如何使用 define声明个常数，用以表明1年中有多少秒（忽略闰年问题）

```
1 #define SECOND_PER_YEAR (60*60*24*365)UL
```

下面三个有什么区别？

```
1 char * const p;
2 char const * p
3 const char *p
```

```
1 char * const p; //常量指针，p的值不可以修改
2 char const * p; //指向常量的指针，指向的常量值不可以改
3 const char *p; //和char const *p
4 char str1[] = "abc";
5 char str2[] = "abc";
6 const char str3[] = "abc";
7 const char str4[] = "abc";
8 const char *str5 = "abc";
9 const char *str6 = "abc";
10 char *str7 = "abc";
11 char *str8 = "abc";
12 cout << ( str1 == str2 ) << endl;
13 cout << ( str3 == str4 ) << endl;
14 cout << ( str5 == str6 ) << endl;
15 cout << ( str7 == str8 ) << endl;
```

结果是：0011

解答：str1,str2,str3,str4是数组变量，它们有各自的内存空间；
而str5,str6,str7,str8是指针，它们指向相同的常量区域，即指针相同。

Heap与stack的差别。

Heap是堆，stack是栈。

申请方式

栈的空间由操作系统自动分配/释放，堆上的空间手动分配/释放。

申请大小的限制

栈空间有限。在Windows下,栈是向**低地址**扩展的数据结构，是一块**连续的内存的区域**。这句话的意思是栈顶的地址和栈的最大容量是系统预先规定好的，在WINDOWS下，栈的大小是2M（也有的说是1M，总之是一个编译时就确定的常数），如果申请的空间超过栈的剩余空间时，将提示overflow。因此，**能从栈获得的空间较小**

堆是很大的自由存储区。堆是向**高地址**扩展的数据结构，是不连续的内存区域。这是由于系统是用**链表来存储的空闲内存地址的**，自然是不连续的，而链表的遍历方向是由低地址向高地址。堆的大小**受限于计算机系统中有效的虚拟内存**。由此可见，堆获得的空间比较灵活，也比较大。

申请效率：

栈由系统自动分配，速度较快。但程序员是无法控制的。

堆是由new分配的内存，一般速度比较慢，而且容易产生内存碎片,不过用起来最方便。

程序运行比较

C中的malloc函数分配的内存空间即在堆上,C++中对应的是new
操作系统中的堆和栈都是指内存空间，不同的是**堆为按需申请、动态分配**，例如 C 中的 malloc 函数和 C++ 中的 new 操作（当然 C++ 的 new 不仅仅是申请内存这么简单）。内存中的空闲空间**并不是连续的，而是不同程序占用了不同的一块一块的内存**，即使是同一个程序也可能占用了不同地方的多块内存。操作系统中则会对这些空间进行统一的管理，在应用程序提出申请时，就会从堆中按照一定算法找出一块可用内存，标记占用空间等信息之后返回其起始地址给程序。在程序结束之前，操作系统不会删除已经申请的内存，而是要靠程序主动提出释放的请求（free、delete），如果使用后忘记释放，就会造成所谓的内存泄漏问题。因此堆基本上可以理解为当前可以使用的空闲内存，但是其申请和释放都要程序员自己写代码管理。

程序在编译期对变量和函数分配内存都在栈上进行,且程序运行过程中函数调用时参数的传递也在栈上进行。而操作系统的**栈则是程序运行时自动拥有的一小块内存，大小在编译期时由编译器参数决定**，用于局部变量的存放或者函数调用栈的保存。在 C 中如果声明一个局部变量（例如 int a），它存放的地方就在栈中，而当这个局部变量离开其作用域之后，所占用的内存则会被自动释放，因此在 C 中局部变量也叫自动变量。**栈的另一个作用则是保存函数调用栈**，这时和数据结构的栈就有关系了。在函数调用过程中，常常会多层甚至递归调用。每一个函数调用都有各自的局部变量值和返回值，每一次函数调用其实是先将当前函数的状态压栈，然后在栈顶开辟新空间用于保存新的函数状态，接下来才是函数执行。当函数执行完毕之后，栈先进后出的特性使得后调用的函数先返回，这样可以保证**返回值的有序传递**，也保证函数现场可以按顺序恢复。操作系统的**栈在内存中高地址向低地址增长，也即低地址为栈顶，高地址为栈底**。这就导致了栈的空间有限制，一旦局部变量申请过多（例如开个超大数组），或者函数调用太深（例如递归太多次），那么就会导致栈溢出（Stack Overflow），操作系统这时候就会直接把你的程序杀掉。

请简述栈在C语言中的作用

栈在多线程编程中的作用，栈在操作系统中的作用。

C语言中栈用来存储临时变量，临时变量包括**函数参数和函数内部定义的临时变量**。函数调用中和函数调用相关的**函数返回地址**，函数中的临时变量，寄存器等均保存在栈中，函数调用返回后从栈中恢复寄存器和临时变量等函数运行场景。

多线程编程的基础是栈，**栈是多线程编程的基石**，每一个线程都最少有一个自己专属的栈，用来存储本线程运行时各个函数的临时变量和维系函数调用和函数返回时的函数调用关系和函数运行场景。操作系统最基本的功能是支持多线程编程，支持中断和异常处理，每个线程都有专属的栈，中断和异常处理也具有专属的栈，栈是操作系统多线程管理的基石。

要对绝对地址0x100000赋值，我们可以用

```
1 | (unsigned int*)0x100000 = 1234;
```

那么要是想让程序跳转到绝对地址是0x100000去执行，应该怎么做？

```
1 | *((void (*)( ))0x100000) ( );
```

首先要将0x100000强制转换成函数指针,即:

```
1 | (void (*)( ))0x100000
```

然后再调用它:

```
1 | *((void (*)( ))0x100000) ( );
```

用typedef可以看得更直观些:

```
1 | typedef void(*)() voidFuncPtr;
2 | *((voidFuncPtr)0x100000) ( );
```

已知一个数组table，用一个宏定义，求出数据的元素个数

```
1 | #define NTBL
2 | #define NTBL (sizeof(table)/sizeof(table[0]))
```

输出多少？并分析过程

```
1 | unsigned short A = 10;
2 | printf("~A = %u\n", ~A);
3 | char c=128;
4 | printf("c=%d\n", c);
```

第一题，~A = 0xfffff5, int值为 -11，但输出的是uint。所以输出4294967285。

第二题，发生溢出，因为有符号字符型其范围为-128~127，char为1字节，8位二进制表示。127用二进制表示为：0111 1111，128表示为1000 0000，这里**发生溢出**，因为第一位为1，为符号位，**表示负数，即-128**。

A=10, short占据2字节，十六位二进制数，无符号型，转换为二进制为0000 0000 0000 0000 0000 0000 1010。所以~A的二进制位1111 1111 1111 1111 1111 1111 0101，即0xFFFFF5。因为输出的是无符号整型，无符号整型的范围为0~4294967295，而0xFFFF FFF5转换为无符号十进制整型为4294967285。

(%u是转义说明，代表输出格式为十进制无符号整数。例如:%u (默认) 输出十进制无符号int。%lu，输出十进制无符号long。%llu，输出十进制无符号long long)

如果转换为符号整型的话则为-11，（1000 0000 0000 0000 0000 0000 0000 1011）。求0xFFFFFFFF补码。

这两道题都是在考察二进制向int或uint转换时的最高位处理。

有符号数和无符号数

首先明确一点，负数存的是补码，正数属存的是原码。看有符号数的最高位是否为1，如果不为1（即为0），则无符号数就直接等于有符号数；

无符号数转有符号数：

看无符号数的最高位是否为1，如果不为1（即为0），则有符号数就直接等于无符号数；如果无符号数的**最高位为1**，则将无符号数取补码，得到的数就是有符号数。

有符号数转无符号数：

看有符号数的最高位是否为1，如果不为1（即为0），则无符号数就直接等于有符号数；如果有符号数的**最高位为1**，则将无符号数取补码，得到的数就是无符号数。

心算求补（大神求补算法）：

从最低位开始至找到的第一个1均不变，符号位不变，这之间的各位“求反”（0变1；1变0）。

原码：1010 1001 补码：1101 0111。

有符号数转十进制

求补码。去掉符号位，剩下的数按位取反，再加1，转换为十进制。记得加上符号位。或者用上面提到的心算求补。如10000，补码为11111，十进制为-16。

交换两个变量的值，不使用第三个变量。即a=3,b=5,交换之后a=5,b=3;

有两种解法，一种用算术算法，一种用^（异或）

```
1 a = a + b;
2 b = a - b;
3 a = a - b;
```

or

```
1 a = a^b; // 只能对int,char...
2 b = a^b;
3 a = a^b;
4 or
5 a ^= b ^= a;
```

求一个正整数的开方，要求不能使用库函数sqrt，结果精度在0.01即可。（深圳联发科）

直接使用二分法即可解答问题，但是题目中所说的是正整数的开方，我觉得完全可以扩展到浮点型的开方，浮点型开方需要注意一点问题，就是大于1和小于1的情况不一样，比如0.9的平方是0.81，而我们是从小于0.81取二分的，显然肯定找不到0.9这个数，所以要稍作处理。

```
1 /*
2    分治法求平方根
3 */
4 #include <stdio.h>
5 #include <stdlib.h>
```

```

6  #define EPSION 0.000001
7  double MySqrt(double des)
8  {
9      double low = EPSION;
10     double high = EPSION;
11     double mid = 0.0;
12     double sqr = 0.0;
13     high = des > 1.0 ? des : des + 1.0;
14     do
15     {
16         mid = (low + high) / 2.0;
17         sqr = mid * mid;
18         if(sqr > des)
19         {
20             high = mid;
21         }
22         else
23         {
24             low = mid;
25         }
26     }while((sqr - des) < -EPSION || (sqr - des) > EPSION);
27     return mid;
28 }
29 int main(void)
30 {
31     double num = 0.0;
32     do
33     {
34         printf("The number :");
35         scanf("%lf", &num);
36         if(num < 0)
37         {
38             printf("input error! Please input again!\n");
39         }
40     }while(num < 0);
41     printf("Its sqrt is %.4lf\n", MySqrt(num));
42     return 0;
43 }

```

用宏定义写出swap (x , y)

```

1  #define swap(x, y)\
2  x = x + y;\
3  y = x - y;\
4  x = x - y;

```

以下代码中的两个sizeof用法有问题吗？

```

1 void UpperCase( char str[] ) // 将 str 中的小写字母转换成大写字母
2 {
3     for( size_t i=0; i<sizeof(str)/sizeof(str[0]); ++i )
4         if( 'a'<=str[i] && str[i]<='z' )
5             str[i] -= ('a'-'A' );
6 }
7
8 char str[] = "aBCDe";
9 cout << "str字符长度为: " << sizeof(str)/sizeof(str[0]) << endl;
10 UpperCase( str );
11 cout << str << endl;

```

答：函数内的sizeof有问题。根据语法，**sizeof如用于数组，只能测出静态数组的大小，无法检测动态分配的或外部数组大小**。函数外的str是一个静态定义的数组，因此其大小为6，**函数内的str实际只是一个指向字符串的指针**，没有任何额外的与数组相关的信息，因此sizeof作用于上只将其当指针看，一个指针为4个字节，因此返回4。

短路求值

```

1 #include <stdio.h>
2 int main()
3 {
4     int i = 6;
5     int j = 1;
6     if(i>0 || (j++)>0);
7     printf("%D\r\n",j);
8     return 0;
9 }

```

输出结果为1。

输出为什么不是2，而是1呢？其实，这里就涉及一个短路计算的问题。由于i语句是个条件判断语句，里面是有两个简单语句进行或运算组合的复合语句，因为或运算中，只要参与或运算的两个表达式的值都为真，则整个运算结果为真，而由于变量i的值为6，已经大于0了，而该语句已经为true，则不需要执行后续的j++操作来判断真假，所以后续的j++操作不需要执行，j的值仍然为1。

因为短路计算的问题，对于&&操作，由于在两个表达式的返回值中，如果有一个为假则整个表达式的值都为假，如果前一个语句的返回值为 false，则无论后一个语句的返回值是真是假，整个条件判断都为假，不用执行后一个语句，而a>b的返回值为 false，程序不执行表达式n=c>d，所以，n的值保持为初值2。

下面代码有什么问题

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void getmemory(char *p)
5 {
6     p=(char *) malloc(100);
7     strcpy(p,"hello world");
8 }
9 int main( )
10 {
11     char *str=NULL;
12

```

```

13     getmemory(str);
14     printf("%s/n",str);
15     free(str);
16
17     return 0;
18 }

```

程序崩溃，getmemory中的malloc **不能返回动态内存**，free () 对str操作很危险。

关键字volatile有什么含意？并举出三个不同的例子？

1) 并行设备的硬件寄存器。**存储器映射的硬件寄存器通常加volatile**。例如状态寄存器。以为设备寄存器会在你的程序不知道或者不介入的时候发生改变，那是因为设备寄存器可以被外设硬件修改。相反，变量中的不会变。设备寄存器的内容是易失的，或者在不注意的时候被修改。**当声明指向设备寄存器的指针时一定要用volatile它会告诉编译器不要对存储在这个地址的数据进行假设**，编译器在优化这个变量时应该把它看作编译时未知的。

```

1 #define IOPMOD (*(volatile unsigned *)(SYSCFG + 0x5000))

```

2) 一个中断服务程序中修改的供其他程序检测的变量。

volatile提醒编译器它后面所定义的变量随时都有可能改变，因此编译后的程序每次需要存储或读取这个变量的时候，都会直接从变量地址中读取数据。如果没有volatile关键字，则编译器可能优化读取和存储，可能暂时使用寄存器中的值，如果这个变量由别的程序更新了的话，将出现不一致的现象。下面举例说明。在DSP开发中，经常需要等待某个事件的触发，所以经常会写出这样的程序：

```

short flag;
void test()
{
    do1();
    while(flag==0);
    do2();
}

```

这段程序等待内存变量flag的值变为1(怀疑此处是0,有点疑问,)之后才运行do2()。变量flag的值由别的程序更改，这个程序可能是某个硬件中断服务程序。例如：如果某个按钮按下的话，就会对DSP产生中断，在按键中断程序中修改flag为1，这样上面的程序就能够得以继续运行。但是，编译器并不知道flag的值会被别的程序修改，因此它在进行优化的时候，可能会把flag的值先读入某个寄存器，然后等待那个寄存器变为1。如果不幸进行了这样的优化，那么while循环就变成了死循环，因为寄存器的内容不可能被中断服务程序修改。为了让程序每次都读取真正flag变量的值，就需要定义为如下形式：

```
volatile short flag;
```

需要注意的是，没有volatile也可能能正常运行，但是可能修改了编译器的优化级别之后就又不能正常运行了。因此经常会出现debug版本正常，但是release版本却不能正常的问题。所以为了安全起见，只要是等待别的程序修改某个变量的话，就加上volatile关键字。

3) 多线程应用中被几个任务共享的变量。单地说就是防止编译器对代码进行优化.比如如下程序：

```

1 XBYTE[2]=0x55;
2 XBYTE[2]=0x56;
3 XBYTE[2]=0x57;
4 XBYTE[2]=0x58;

```

对外部硬件而言，上述四条语句分别表示不同的操作，会产生四种不同的动作，但是编译器却会对上述四条语句进行优化，**认为只有XBYTE[2]=0x58（即忽略前三条语句，只产生一条机器代码）**。如果键入volatile，编译器会逐一的进行编译并产生相应的机器代码（产生四条代码）。

int (*s[10])(int) 表示的是啥啊

int (*s[10])(int) 函数指针数组，每个指针指向一个int func(int param)的函数。

嵌入式系统总是要用户对变量或寄存器进行位操作。给定一个整型变量a，写两段代码，第一个设置a的bit 3，第二个清除a 的bit 3。在以上两个操作中，要保持其它位不变

对这个问题有三种基本的反应

1). 不知道如何下手。该被面者从没做过任何嵌入式系统的工作。

2). 用bit fields。Bit fields是被扔到C语言死角的东西，它保证你的代码在不同编译器之间是不可移植的，同时也保证了你的代码是不可重用的。我最近不幸看到Infineon为其较复杂的通信芯片写的驱动程序，它用到了bit fields因此完全对我无用，因为我的编译器用其它的方式来实现bit fields的。从道德讲：永远不要让一个非嵌入式的家伙粘实际硬件的边。

3). 用 #defines 和 bit masks 操作。这是一个有极高可移植性的方法，是应该被用到的方法。最佳的解决方案如下：

```

1  #define BIT3 (0x1<<3)
2  static int a;
3  void set_bit3(void)
4  {
5      a |= BIT3;
6  }
7  void clear_bit3(void)
8  {
9      a &= ~BIT3;
10 }
11
```

一些人喜欢为设置和清除值而定义一个掩码同时定义一些说明常数，这也是可以接受的。我希望看到几个要点：**说明常数、|=和&=~操作。**

访问固定的内存位置（Accessing fixed memory locations）C/C++ Development

分析下面的程序（释放malloc的空间后，为什么要把指针指向NULL看这个）

```

1  void GetMemory(char **p,int num)
2  {
3      *p=(char *)malloc(num);
4  }
5  int main()
6  {
7      char *str=NULL;
8
9      GetMemory(&str,100);
10
11     strcpy(str,"hello");
12     free(str);
```

```

13
14     if(str!=NULL)
15     {
16         strcpy(str,"world");
17     }
18
19     printf("\n str is %s",str);
20     getchar();
21 }

```

问输出结果是什么？

输出str is world。

free **只是释放的str指向的内存空间,它本身的值还是存在的**。free(str)之后，str不是null,而是指向一块随机内存，也叫**野指针**。所以free之后，有一个好的习惯就是将str=NULL。此时str指向空间的内存已被回收,如果输出语句之前还存在分配空间的操作的话,这段存储空间是可能被重新分配给其他变量的。

尽管这段程序确实是存在大大的问题，但是通常会打印出world来。这是因为，**进程中的内存管理一般不是由操作系统完成的，而是由库函数自己完成的**。

当你malloc一块内存的时候，管理库向操作系统申请一块空间（可能会比你申请的大一些），然后在这块空间中记录一些管理信息（一般是在你申请的内存前面一点），并将可用内存的地址返回。但是**释放内存的时候，管理库通常都不会将内存还给操作系统（指向随机），因此你是可以继续访问这块地址的**。

在子函数中malloc分配内存和free释放内存的方法

子函数malloc分配内存

为了增强程序可读性，有时会在子函数中malloc分配内存。测试了如下三种方法，容易想到的是第一种。事实证明这种也是错误的！

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4
5  typedef struct _dataStc
6  {
7      int buf[100];
8      int size;
9  }dataStc;
10
11 void func1(dataStc* p)
12 {
13     p = (dataStc*)malloc(sizeof(dataStc));
14     p->size = 10;
15     return;
16 }
17
18 dataStc* func2(void)
19 {
20     dataStc* p;
21     p = (dataStc*)malloc(sizeof(dataStc));
22     p->size = 10;
23     return p;
24 }
25

```

```

26 void func3(dataStc** p)
27 {
28     *p = (dataStc*)malloc(sizeof(dataStc));
29     (*p)->size = 10;
30     return;
31 }
32
33 int main(int argc, const char * argv[]) {
34
35     dataStc *p1 = NULL;
36     dataStc *p2 = NULL;
37     dataStc *p3 = NULL;
38
39     func1(p1);
40     p2 = func2();
41     func3(&p3);
42
43     if (p1 == NULL)
44     {
45         printf("p1=NULL\n");
46     }
47     else
48     {
49         printf("p1->size:%d\n", p1->size);
50     }
51     if (p2 == NULL)
52     {
53         printf("p2=NULL\n");
54     }
55     else
56     {
57         printf("p2->size:%d\n", p2->size);
58     }
59     if (p3 == NULL)
60     {
61         printf("p3=NULL\n");
62     }
63     else
64     {
65         printf("p3->size:%d\n", p3->size);
66     }
67     return 0;
68 }
69

```

执行结果如下所示：

```

p1=NULL
p2->size:10
p3->size:10

```

结论：只有后两种方法能正确分配内存。即直接返回内存指针，或将二级指针作为参数传入子函数。这是因为，在子函数中，内存指针只是被当做一个变量来处理的（虽然这个变量被定义为dataStc*），对其赋值后再返回，它的值当然没改变。

另外，由于成员选择运算符“->”的优先级比取值运算符“*”高，所以在func3中，p需要加括号。

子函数free释放内存

在子函数中释放内存，很容易想到的是写个下面这样的函数释放内存：

```
1 void func_free(dataStc* p)
2 {
3     free(p);
4     return;
5 }
```

上面的函数能释放内存，但也有个缺陷。因为C在free一段内存后，该内存的确是被释放了（可以被其他程序使用），但指针p仍然指向该段内存（而不是我们期待的NULL）。这时p就成为野指针了（不同于空指针），仍然可以访问或修改p指向的内存。如果其他程序不慎又通过p访问了内存，程序或许不报错，但可能带来不可预知的后果。

因此，保险做法是，在free后，将p赋为NULL。由于这里又将输入变量p修改了，因此仍然需要传递二级指针。正确的代码为：

```
1 void func_free(dataStc** p)
2 {
3     free(*p);
4     *p = NULL;
5     return;
6 }
```

这样，在其他程序中访问p时，就可先判断其是否为NULL了。

请问以下代码有什么问题：

```
1 char* s="AAA";
2 printf("%s",s);
3 s[0]='B';
4 printf("%s",s);
```

"AAA"是字符串常量。s是指针，指向这个**字符串常量**，所以声明s的时候就有问题。const char* s="AAA";

然后又因为是常量，所以对s[0]的赋值操作是不合法的。

声明为如下字符串数组变量是可以的 char str3[] = "abc";

分析程序

```
1 #include<iostream.h>
2 #include <string.h>
3 #include <malloc.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <memory.h>
7
8 typedef struct AA
9 {
10     int b1:5;
11     int b2:2;
12 }AA;
13
14
```

```

15 void main()
16 {
17     AA aa;
18     char cc[100];
19     strcpy(cc, "0123456789abcdefghijklmnopqrstuvwxyz");
20     memcpy(&aa, cc, sizeof(AA));
21     cout << aa.b1 << endl;
22     cout << aa.b2 << endl;
23 }

```

答案是 -16和 1。

首先sizeof(AA)的大小为4字节（4字节对齐），b1和b2分别占5bit和2bit（1字节8bit）。

经过strcpy和memcpy后,aa的4个字节所存放的值是:0,1,2,3的**ASCII码**，即00110000,00110001,00110010,00110011。所以，最后一步：显示的是这4个字节的前5位，和之后的2位分别为：10000,和01。因为int是有正负之分.所以：答案是-16和1。

分析程序

求函数返回值，输入x=9999;

```

1  int func ( x )
2  {
3      int countx = 0;
4
5      while ( x )
6      {
7          countx ++;
8          x = x&(x-1);
9      }
10
11     return countx;
12 }

```

结果呢？

知道了这是统计9999（ $10\ 0111\ 0000\ 1111\ 2^{13} + 2^{10} + 2^9 + 2^8 + 2^3 + 2^2 + 2^1 + 2^0$ ）的二进制数值中有多少个1的函数，且有 $9999 = 9 \times 1024 + 512 + 256 + 15$ 。

9×1024中含有1的个数为2；

512中含有1的个数为1；

256中含有1的个数为1；

15中含有1的个数为4；

故共有1的个数为8，结果为8。

$1000 - 1 = 0111$ ，正好是原数取反。这就是原理。

用这种方法来求1的个数是很效率很高的。

不必去一个一个地移位。循环次数最少。

程序的输出

```

1  int main()
2  {
3      int a = 0, b = 0, c = 0;
4      //printf("%d %d %d\r\n", (a-=a-5), b, c); ---5
5      c=(a-=a-5), (a=b, b+3);
6      //c=((a-=a-5), (a=b, b+3)); ----0 0 3
7      printf("%d %d %d\r\n", a, b, c);
8  }

```

不要被 `c=(a-=a-5), (a=b, b+3);` 误解了，这个式子整体是个逗号表达式，没错，但是没有变量来接收他的返回值啊。所以C的值最终就是5。但是如果加了括号，就不一样了 `c=((a-=a-5), (a=b, b+3));`，看清楚题目。

程序的输出

```

1  #define sum(num1,num2) num1+num2
2  int main()
3  {
4      printf("%d", sum(sum(2,5), sum(4,10)*sum(2,3))); //展开2+5+4+10*2+3
5  }

```

答案：34

程序的输出

```

1      int sum = 0;
2      for(int j = 1; j<4; j++)
3      {
4          switch (j)
5          {
6              case (1):
7                  continue; //结束本次循环，进行下一次循环。continue后面的都不执行，因此
sum+=j; 不会被执行
8              case (2):
9                  sum+=j;
10                 break; //break跳出的是switch 跳出后会执行下一句sum+=j;
11            }
12        sum+=j;
13    }

```

答案为：7

程序输出多少（重要）

```

1  main()
2  {
3      int a[5]={1,2,3,4,5};
4      int *ptr=(int *)(&a+1);
5
6      printf("%d,%d", *(a+1), *(ptr-1));
7  }

```

输出：2,5。

`*(a+1)` 就是`a[1]`，`*(ptr-1)` 就是`a[4]`，执行结果是2，5。`&a+1`不是首地址+1，系统会认为加一个**a数组的偏移**，是偏移了一个数组的大小（本例是5个int）

```
1 | int *ptr=(int *)(&a+1);
```

则ptr实际是`&a[5]`，也就是`a+5`。

原因如下：

`&a`是**数组指针**，其类型为`int (*)[5]`；而指针加1要根据指针类型加上一定的值，不同类型的指针+1之后增加的大小不同。`a`是长度为5的int数组指针，所以要加`5 * sizeof(int)`所以ptr实际是`a[5]`。

但是prt与`(&a+1)`类型是不一样的(这点很重要)。所以 `prr-1`只会减去 `sizeof(int*)`。a,&a的地址是一样的，但意思不一样，a是数组首地址，也就是`a[0]`的地址，&a是对象（数组）首地址，a+1是数组下一元素的地址，即`a[1]`，&a+1是下一个对象的地址，即`a[5]`。

设有以下说明和定义：

```
1 | typedef union {long i; int k[5]; char c;} DATE;
2 | struct data { int cat; DATE cow; double dog;} too;
3 | DATE max;
```

则语句 `printf("%d",sizeof(struct date)+sizeof(max));`的执行结果是：52

答：DATE是一个union, 变量公用空间. 里面最大的变量类型是int[5], 占用20个字节. 所以它的大小是20。

data是一个struct, 每个变量分开占用空间. 依次为int4 + DATE20 + double8 = 32.。所以结果是 20 + 32 = 52。

当然...在某些16位编辑器下, int可能是2字节,那么结果是 int2 + DATE10 + double8 = 20。

语句for(; 1 ;)有什么问题？它是什么意思？

答：和while(1)相同。

全局变量可不可以定义在可被多个.C文件包含的头文件中？为什么？

答：可以，在不同的C文件中以static形式来声明同名全局变量。

可以在不同的C文件中声明同名的全局变量，前提是其中只能有一个C文件中对此变量赋初值，此时连接不会出错。

如何引用一个已经定义过的全局变量？

可以用引用头文件的方式，也可以用extern关键字，如果用引用头文件方式来引用某个在头文件中声明的全局变理，假定你将那个变写错了，那么在编译期间会报错，如果你用extern方式引用时，假定你犯了同样的错误，那么在编译期间不会报错，而在连接期间报错。

static全局变量与普通的全局变量有什么区别？static函数与普通函数有什么区别？（重要）

答：全局变量(外部变量)的说明之前再冠以static 就构成了静态的全局变量。全局变量本身就是静态存储方式，静态全局变量当然也是静态存储方式。这两者在存储方式上并无不同。

这两者的区别虽在于**非静态全局变量的作用域是整个源程序**，当一个源程序由多个源文件组成时，**非静态的全局变量在各个源文件中都是有效的**。而**静态全局变量则限制了其作用域，即只在定义该变量的源文件内有效**，在同一源程序的其它源文件中不能使用它。

由于静态全局变量的作用域局限于一个源文件内，只能为该源文件内的函数公用，因此可以避免在其它源文件中引起错误。

从以上分析可以看出，**把局部变量改变为静态变量后是改变了它的存储方式即改变了它的生存期。把全局变量改变为静态变量后是改变了它的作用域，限制了它的使用范围。**

static函数与普通函数作用域不同。仅在本文件。只在当前源文件中使用的函数应该说明为内部函数(static)，内部函数应该在当前源文件中说明和定义。对于可在当前源文件以外使用的函数，应该在一个头文件中说明，要使用这些函数的源文件要包含这个头文件。

关键字static的作用是什么？

在C语言中，关键字static有三个明显的作用：

- 1) 在函数体，**只会被初始化一次**，一个被声明为静态的变量在这一函数被调用过程中维持其值不变。
- 2) 在模块内（但在**函数体外**），一个被声明为**静态的变量**可以被模块内所用函数访问，但不能被模块外其它函数访问。它是一个**本地的全局变量**（只能被当前文件使用）。
- 3) 在模块内，一个被声明为**静态的函数**只可被这一模块内的其它函数调用。那就是，这个函数被限制在声明它的模块的本地范围内使用（**只能被当前文件使用**）。

引申1：在C语言中，为什么 static变量只初始化一次？

对于所有的对象（不仅仅是静态对象），**初始化都只有一次**，而由于静态变量具有“记忆”功能，初始化后，一直都没有被销毁，**都会保存在内存区域中**，所以不会再次初始化。存放在静态区的变量的生命周期一般比较长，它与整个程序“同生死、共存亡”，所以它只需初始化一次。而auto变量，即自动变量，由于它**存放在栈区**，一旦函数调用结束，就会**立刻被销毁**。

引申2：在头文件中定义静态变量是否可行，为什么？

不可行，如果在头文件中定义静态变量，会造成资源浪费的问题，同时也可能引起程序错误。因为如果在使用了该头文件的每个C语言文件中定义静态变量，按照编译的步骤，**在每个头文件中都会单独存在一个静态变量**，从而会引起**空间浪费或者程序错误**所以，不推荐在头文件中定义任何变量，当然也包括静态变量。

常量指针，指向常量的指针，指向常量的常量指针

常量指针

```
1 | int * const p
```

先看const再看*，是p是一个常量类型的指针，**不能修改这个指针的指向**，但是这个指针所指向的地址上存储的**值可以修改**。

指向常量的指针

```
1 | const int *p
```

先看*再看const，定义一个指针指向一个常量，不能通过指针来修改这个指针**指向的值**

指向常量的常量指针

```
1 | const int *const p
```

对于“指向常量的常量指针”，就必须同时满足上述1和2中的内容，**既不可以修改指针的值，也不可以修改指针指向的值。**

const作用

常类型也称为 const类型，是指使用类型修饰符 const修饰的变量或对象。const是C和C++中常见的关键字，在C语言中，它主要有如下作用

(1) 定义变量（局部变量或全局变量）为常量，例如：

```
1 | const int N=100; //定义一个常量N
2 | N=50; //错误，常量的值不能被修改
3 | const int n; //错误，常量在定义的时候必须初始化
```

(2) 修饰函数的参数，表示在函数体内不能修改这个参数的值。

(3) 修饰函数的返回值。

a.如果给用 const修饰**返回值的类型为指针**，那么函数返回值（即指针）的内容是**不能被修改的**，而且这个返回值只能赋给被 const修饰的指针。例如：

```
1 | const char GetString() //定义一个函数
2 | char *str= GetString() //错误，因为str没有被 const修饰
3 | const char *str=GetString() //正确
```

b.如果用 const修饰**普通的返回值**，如返回int变量，由于这个返回值是一个临时变量，在函数调用结束后这个临时变量的生命周期也就结束了，因此把这些**返回值修饰为 const是没有意义的**。

(4)节省空间，避免不必要的内存分配。例如

```
1 | #define PI 3.14159 //该宏用来定义常量
2 | const double Pi=3.14159 //此时并未将P放入只读存储器中
3 | double i=Pi //此时为Pi分配内存，以后不再分配
4 | double I=PI //编译期间进行宏替换，分配内存
5 | double j=Pi //没有内存分配再次进行宏替换，又一次分配内存
```

从汇编的角度看，const定义常量只是给出了对应的内存地址，而不是像# define一样给出的是立即数，所以，const定义的常量在程序运行过程中只有一个复制品，而# define定义的常量在内存中有若干个复制品。

(5) 提高了程序的效率。编译器通常不为普通 const常量分配存储空间，而是将它们保存在符号表中，这使得它成为一个编译期间的常量，没有了存储与读内存的操作，它的效率也很高。

引申：什么情况下使用const关键字

(1) 修饰一般常量。一般常量是指简单类型的常量。这种常量在定义时，修饰符const可以用在类型说明符前，也可以用在类型说明符后。例如：

```
1 | int const x=2; const int x=2
```

(2) 修饰常数组。定义或说明一个常数组可以采用如下格式：

```
1 | int const a[8]={1,2,3,4,5,6,7,8}
2 | const int a[8]={1,2,3,4,5,6,7,8}
```

(3) 修饰常对象。常对象是指对象常量，定义格式如下：

```
1 class A:
2     const A a:
3     A const a:
```

定义常对象时，同样要进行初始化，并且该对象不能再被更新。修饰符 `const` 可以放在类名后面，也可以放在类名前面。

(4) 修饰常指针

```
1 const int*p; //指向常量的指针，p值可以变，p指向的数值内容不可变
2 int const*p; //常量指针 p指向的内存不可以变，但是p指向的数值可以变
3 int*const p; //同2
4 const int* const p; //指向常量的常量指针。即p指向的内存和数值都不可变
```

(5) 修饰常引用。被 `const` 修饰的引用变量为常引用，一旦被初始化，就不能再指向其他对象了。

(6) 修饰函数的常参数。 `const` 修饰符也可以修饰函数的传递参数，格式如下：

```
1 void Fun (const int Var)
```

告诉编译器 `Var` 在函数体中不能被改变，从而防止了使用者一些无意的或错误的修改。

(7) 修饰函数的返回值。 `const` 修饰符也可以修饰函数的返回值，表明该返回值不可被改变，格式如下：

```
1 const int FunI ();
2 const MyClass Fun2 ();
```

(8) 在另一连接文件中引用 `const` 常量。使用方式有

```
1 extern const int l;
2 extern const int j=10;
```

局部变量能否和全局变量重名？

能，局部会屏蔽全局。

局部变量可以与全局变量同名，在函数内引用这个变量时，会用到同名的局部变量，而不会用到全局变量。

对于有些编译器而言，在同一个函数内可以定义多个同名的局部变量，比如在两个循环体内都定义一个同名的局部变量，而那个局部变量的作用域就在那个循环体内。

改错题

```
1 #include <stdio.h>
2 int main(void)
3 {
4     int **p;
5     int arr[100];
6     p = &arr;
7
8     return 0;
9 }
```

搞错了,是指针类型不同,

```
1 int **p; //二级指针
2 &arr; //得到的是指向第一维为100的数组的指针
```

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int **p, *q;
6     int arr[100];
7     q = arr;
8     p = &q;
9
10    return 0;
11 }
```

下面这个程序执行后会有什么错误或者效果:

```
1 #define MAX 255
2 int main()
3 {
4     unsigned char A[MAX], i; //i被定义为unsigned char
5     for (i=0; i<=MAX; i++)
6         A[i]=i;
7 }
```

解答：死循环加数组越界访问（C/C++不进行数组越界检查）

MAX=255

数组A的下标范围为0..MAX-1,这是其一。其二.当i循环到255时,循环内执行:A[255]=255;这句本身没有问题。但是返回for (i=0;i<=MAX;i++)语句时,由于unsigned char的取值范围在(0-255),i++以后i又为0了,无限循环下去。

请找出下面代码中的所以错误

说明：以下代码是把一个字符串倒序，如“abcd”倒序后变为“dcba”

```
1 #include "string.h"
2
3 main()
4 {
5     char*src="hello,world";
6     char* dest=NULL;
7     int len=strlen(src);
```

```

8     dest=(char*)malloc(len);
9     char* d=dest;
10    char* s=src[len];
11    while(len--!=0)
12        d++=s--;
13    printf("%s",dest);
14    return 0;
15 }

```

方法1：

```

1  int main()
2  {
3      char* src = "hello,world";
4
5      int len = strlen(src);
6      char* dest = (char*)malloc(len+1);//要为\0分配一个空间
7      char* d = dest;
8      char* s = &src[len-1];//指向最后一个字符
9      while( len-- != 0 )
10         *d++=*s--;
11         *d = 0;//尾部要加\0
12     printf("%s\n",dest);
13
14     free(dest);// 使用完，应当释放空间，以免造成内存泄露
15     return 0;
16 }

```

方法2：

```

1  #include <stdio.h>
2  #include <string.h>
3
4  main()
5  {
6      char str[]="hello,world";
7      int len=strlen(str);
8      char t;
9      for(int i=0; i<len/2; i++)
10     {
11         t=str[i];
12         str[i]=str[len-i-1]; str[len-i-1]=t;
13     }
14     printf("%s",str);
15     return 0;
16 }

```

用两个栈实现一个队列的功能？要求给出算法和思路！

设2个栈为A,B，一开始均为空.

入队：

将新元素push入栈A;

出队：

(1)判断栈B是否为空；

(2)如果不为空，则将栈A中所有元素依次pop出并push到栈B；

(3)将栈B的栈顶元素pop出；

这样实现的队列入队和出队的平摊复杂度都还是 $O(1)$ ，比上面的几种方法要好。

写一个"标准"宏MIN，这个宏输入两个参数并返回较小的一个

```
1 #define MIN(A,B) ((A) <= (B) ? (A) : (B))
```

面试中需要能达到手写水平的代码

链表

实现链表的逆置

```
1 struct ListNode* reverseList(struct ListNode* head){
2
3     //判断节点长度
4     if(head == NULL || head->next == NULL)
5         return head;
6     else
7     {
8         struct ListNode* former = NULL;
9         struct ListNode* mid = head;
10        struct ListNode* temp = NULL;
11        while(mid != NULL)
12        {
13            //保存下一个节点
14            temp = mid->next;
15            //让该节点指向上一个节点
16            mid->next = former;
17            //上一个节点走到当前节点
18            former = mid;
19            //当前节点走到下一个节点
20            mid = temp;
21        }
22        return former;
23    }
24 }
```

判断单链表中是否存在环

```
1 bool IsExitsLoop(slist *head)
2 {
3     slist *slow = head, *fast = head;
4     while ( fast && fast->next )
5     {
6         slow = slow->next;
7         fast = fast->next->next;
8         if ( slow == fast ) break;
9     }
10    return !(fast == NULL || fast->next == NULL);
11 }
```

单链表相交，如何求交点

给定两个（单向）链表，判定它们是否相交并返回交点。请注意相交的定义基于节点的引用，而不是基于节点的值。换句话说，如果一个链表的第k个节点与另一个链表的第j个节点是同一节点（引用完全相同），则这两个链表相交。

示例 1：

输入：intersectVal = 8, listA = [4,1,8,4,5], listB = [5,0,1,8,4,5], skipA = 2, skipB = 3

输出：Reference of the node with value = 8

输入解释：相交节点的值为 8（注意，如果两个列表相交则不能为 0）。从各自的表头开始算起，链表 A 为 [4,1,8,4,5]，链表 B 为 [5,0,1,8,4,5]。在 A 中，相交节点前有 2 个节点；在 B 中，相交节点前有 3 个节点。

示例 2：

输入：intersectVal = 2, listA = [0,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1

输出：Reference of the node with value = 2

输入解释：相交节点的值为 2

（注意，如果两个列表相交则不能为 0）。从各自的表头开始算起，链表 A 为 [0,9,1,2,4]，链表 B 为 [3,2,4]。在 A 中，相交节点前有 3 个节点；在 B 中，相交节点前有 1 个节点。

示例 3：

输入：intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2

输出：null

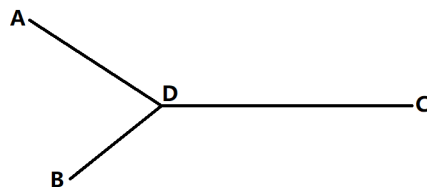
输入解释：从各自的表头开始算起，链表 A 为 [2,6,4]，链表 B 为 [1,5]。由于这两个链表不相交，所以 intersectVal 必须为 0，而 skipA 和 skipB 可以是任意值。解释：这两个链表不相交，因此返回 null。

注意：

如果两个链表没有交点，返回 null。在返回结果后，两个链表仍须保持原有的结构。可假定整个链表结构中没有循环。程序尽量满足 $O(n)$

时间复杂度，且仅用 $O(1)$ 内存。

根据题意,两个链表相交的点是指: 两个指针指向的内容相同,则说明该结点记在A链表上又在B链表上,进而说明A和B是相交的。而对于相交的情况,两条链表一定是这种结构:



https://blog.csdn.net/qg_10933601

为什么呢?因为如果链表A和链表B相交于D的话,那么说明D结点即在A上又在B上,而D之后的元素自然也都在A和B上了,因为他们是通过next指针相连的。

如果有相交的结点D的话,每条链的头结点先走完自己的链表长度,然后回头走另外的一条链表,那么两结点一定为相交于D点,因为这时每个头结点走的距离是一样的,都是 $AD + BD + DC$,而他们每次又都是前进1,所以**距离相同,速度又相同**,固然一定会在相同的时间走到相同的结点上,即D点。

```

1  struct ListNode *getIntersectionNode(struct ListNode *headA, struct ListNode
   *headB) {
2      struct ListNode *p=headA;
3      struct ListNode *q=headB;
4      while(p!=q)
5      {
6          if(p==NULL)
7              p=headB;
8          else
9              p=p->next;
10         if(q==NULL)
11             q=headA;
12         else
13             q=q->next;
14     }
15     return q;
16 }

```

求有环链表第一个入环节点

给定一个链表，返回链表开始入环的第一个节点。 如果链表无环，则返回 null。

为了表示给定链表中的环，我们使用整数 pos 来表示链表尾连接到链表中的位置（索引从 0 开始）。如果 pos 是 -1，则在该链表中没有环。

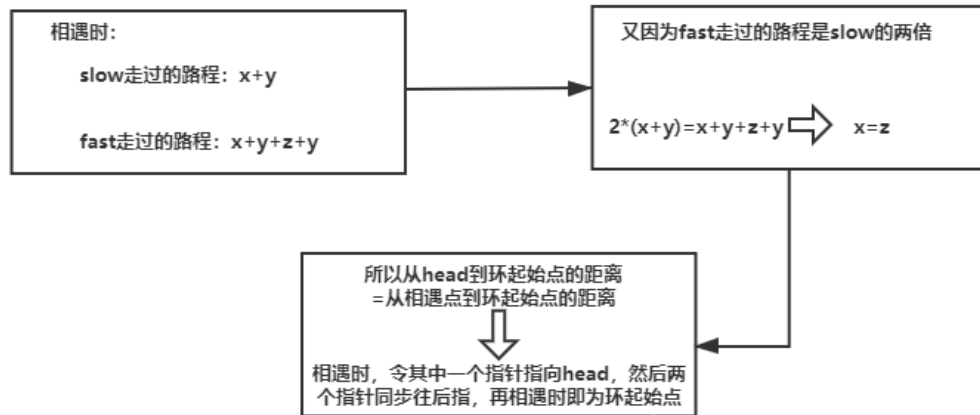
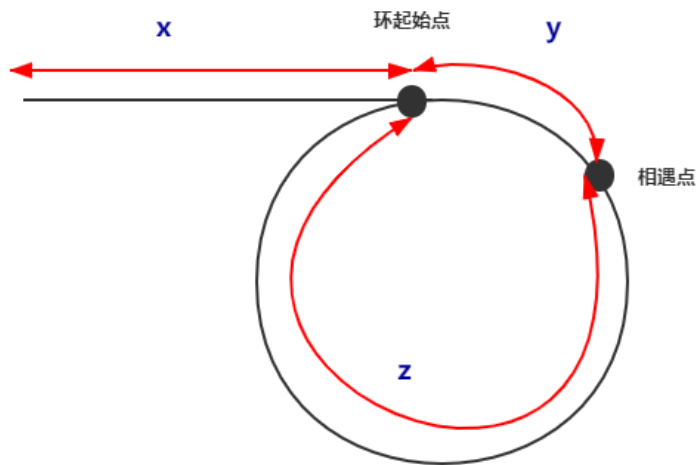
说明：不允许修改给定的链表。

示例 1：

输入：head = [3,2,0,-4], pos = 1

输出：tail connects to node index 1

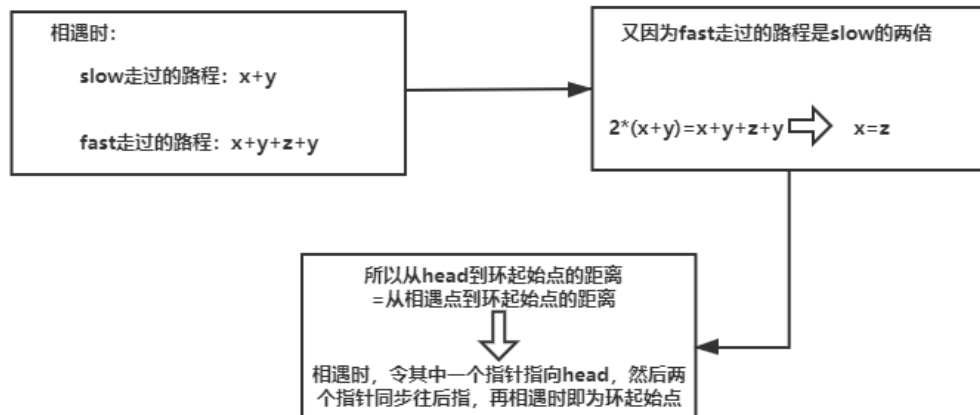
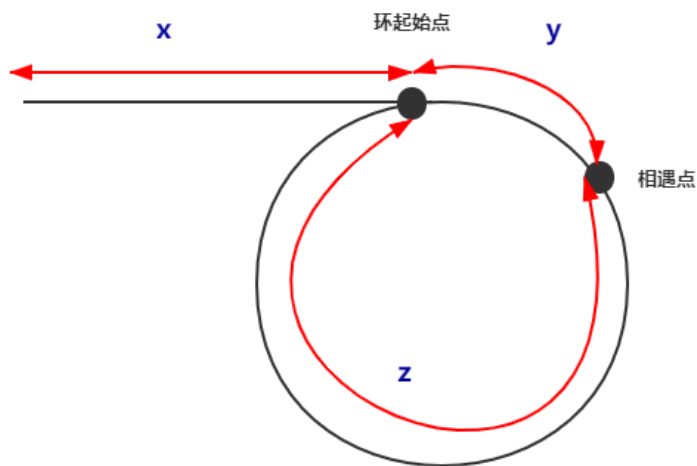
解释：链表中有一个环，其尾部连接到第二个节点。



示例 2：输入：head = [1,2], pos = 0

输出：tail connects to node index 0

解释：链表中有一个环，其尾部连接到第一个节点。

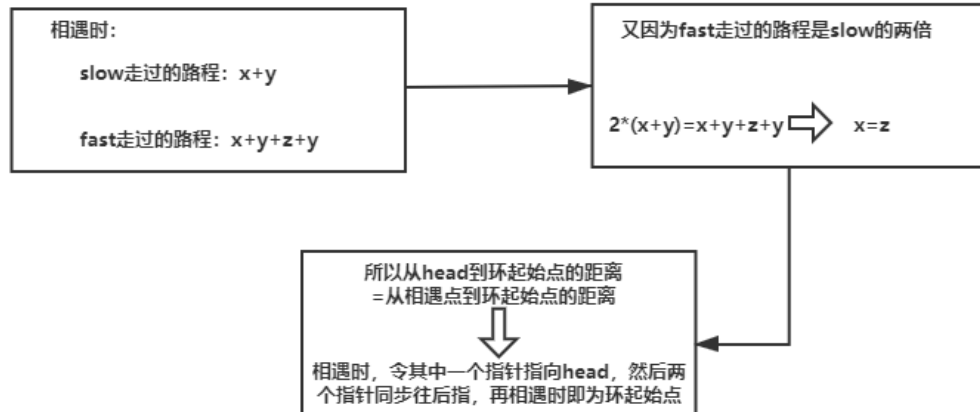
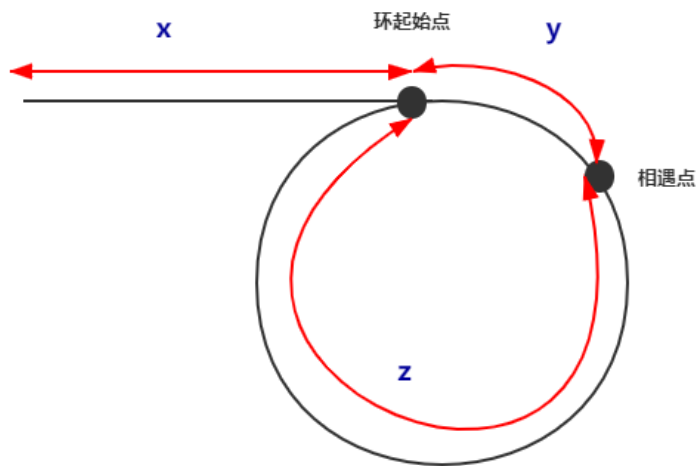


示例 3：

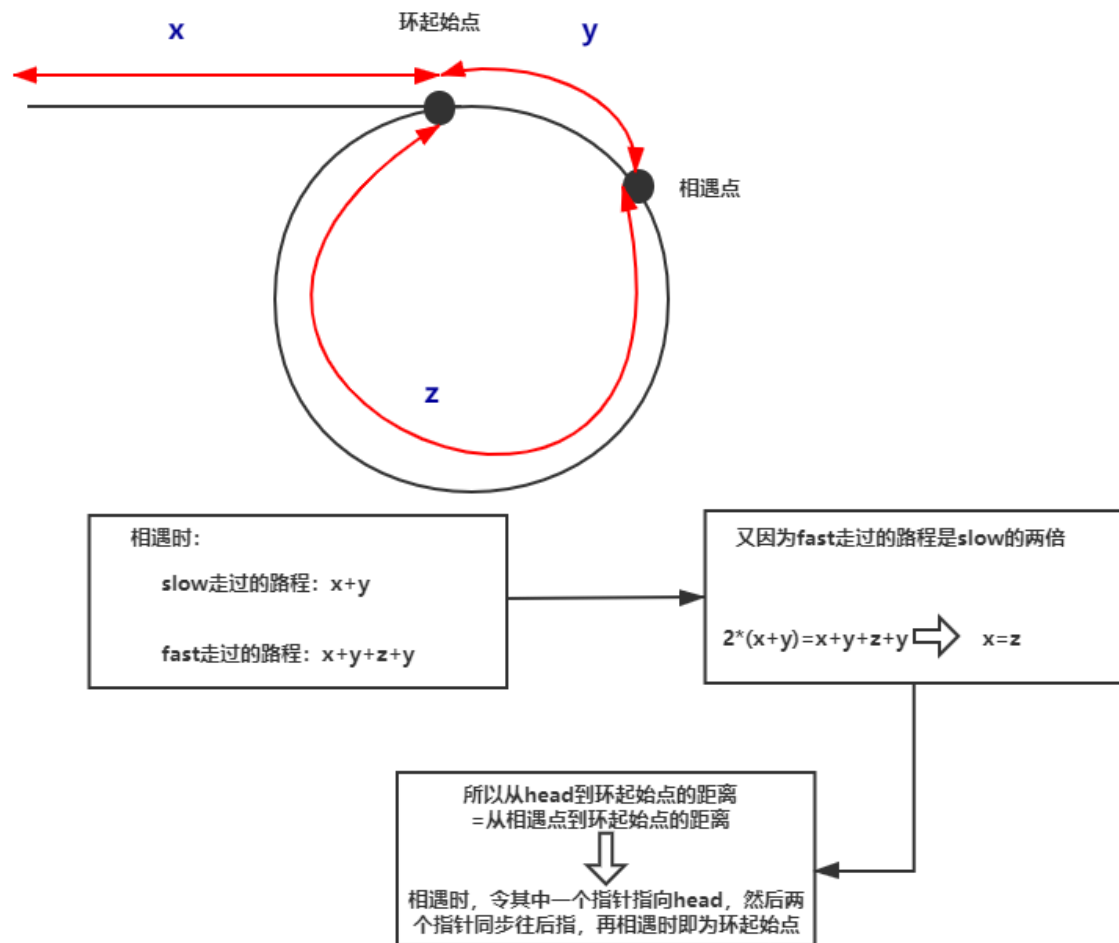
输入：head = [1], pos = -1

输出：no cycle

解释：链表中没有环。



相遇的节点并不是入环的节点，头指针距环入口点前的距离 和 相遇点距离入口的距离 相等，精简易懂！



```

1  struct ListNode *detectCycle(struct ListNode *head) {
2      struct ListNode *slow = head;
3      struct ListNode *fast = head;
4      while (fast != NULL && fast->next != NULL) {
5          slow = slow->next;
6          fast = fast->next->next;
7          if (slow == fast) {
8              fast = head;
9              while (slow != fast) {
10                 slow = slow->next;
11                 fast = fast->next;
12             }
13             return slow;
14         }
15     }
16     return NULL;
17 }
    
```

写出链表的删除一个节点的程序

给定单向链表的头指针和一个要删除的节点的值，定义一个函数删除该节点。

返回删除后的链表的头节点。

示例 1:

输入: head = [4,5,1,9], val = 5

输出: [4,1,9]

解释: 给定你链表中值为 5 的第二个节点，那么在调用了你的函数之后，该链表应变为 4 -> 1 -> 9.

示例 2:

输入: head = [4,5,1,9], val = 1

输出: [4,5,9]

解释: 给定你链表中值为 1 的第三个节点，那么在调用了你的函数之后，该链表应变为 4 -> 5 -> 9.

说明：

题目保证链表中节点的值互不相同 若使用 C 或 C++ 语言，你不需要 free 或 delete 被删除的节点

```

1  struct ListNode* deleteNode(struct ListNode* head, int val){
2      if(head==NULL)
3          return NULL;
4      struct ListNode* pre = head;
5      struct ListNode* p = head;
6      while(p)
7      {
8          if(p->val == val)
9          {
10             //删除节点为尾节点
11             if (p->next == NULL)
12             {
13                 pre->next =NULL;
14                 return head;
15             }
16             //删除节点为头节点
17             else if (p == head)
18             {
19                 return head->next;
20             }
21             //删除节点为中间节点
22             else
23             {
24                 pre->next = p->next;
25                 return head;
26             }
27         }
28         //记录下前一个节点
29         pre = p;
30         p = p->next;
31     }
32     return head;
33 }
34
35 }
```

用普通算法实现两个有序链表的合并

```

1  struct ListNode* mergeTwoLists(struct ListNode* l1, struct ListNode* l2){
2      //加个链表头
3      struct ListNode head;
4      struct ListNode *pre = &head;
```

```

5
6     while (l1 && l2) {
7         if (l1->val <= l2->val) {
8             pre->next = l1;
9             l1 = l1->next;
10        } else {
11            pre->next = l2;
12            l2 = l2->next;
13        }
14        pre = pre->next;
15    }
16    //将剩下元素直接接在最后面
17    pre->next = (NULL == l1 ? l2 : l1);
18    return head->next;
19 }

```

```

1 node *mergelinklist(node *head1,node *head2)
2 {
3     node *p1,*p2,*temp=NULL,*cur=NULL,*head=NULL;
4     p1=head1;
5     p2=head2;
6     while (p1!=NULL&&p2!=NULL) {
7
8         if (p1->value<p2->value) {
9             temp=p1;
10            p1=p1->next;
11        }
12        else
13        {
14            temp=p2;
15            p2=p2->next;
16        }
17        if (NULL==head) {
18            cur=temp;
19            head=cur;
20        }
21        else
22        {
23            cur->next=temp;
24            cur=temp;
25        }
26    }
27    temp=(p1?p1:p2);
28    while (temp!=NULL) {
29        cur->next=temp;
30        cur=cur->next;
31        temp=temp->next;
32    }
33    cur->next=NULL;
34    return head;
35 }
36 }

```

用递归算法实现两个有序列表的合并

```

1  if (!l1)
2      return l2;
3      if (!l2)
4          return l1;
5      if (l1->val < l2->val){
6          l1->next = mergeTwoLists(l1->next, l2);
7          return l1;
8      }
9      else{
10         l2->next = mergeTwoLists(l1, l2->next);
11         return l2;
12     }
13

```

```

1  node *mergeRecursion(node *head1,node *head2)
2  {
3      if(NULL==head1)
4      {
5          return head2;
6      }
7      if(NULL==head2)
8      {
9          return head1;
10     }
11     node *head=NULL;
12     if (head1->value<=head2->value) {
13         head=head1;
14         head->next=mergeRecursion(head1->next,head2);
15     }
16     else
17     {
18         head=head2;
19         head->next=mergeRecursion(head1,head2->next);
20     }
21     return head;
22 }

```

二叉树

先序遍历

```

1  /*
2   * @Description: 二叉树的先序遍历（非递归）
3   * @Version: V1.0
4   * @Autor: Carlos
5   * @Date: 2020-05-17 16:35:27
6   * @LastEditors: Carlos
7   * @LastEditTime: 2020-05-18 14:51:01
8   */
9  #include <stdio.h>
10 #include <string.h>
11 #include <stdlib.h>
12 #define TElemType int
13 int top=-1;//top变量时刻表示栈顶元素所在位置

```

```

14 //构造结点的结构体
15 typedef struct BiTNode{
16     TElemType data;//数据域
17     struct BiTNode *lchild,*rchild;//左右孩子指针
18 }BiTNode,*BiTree;
19 /**
20  * @Description: 初始化树
21  * @Param: BiTree *T 结构体指针
22  * @Return: 无
23  * @Author: Carlos
24  */
25 void CreateBiTree(BiTree *T){
26     *T=(BiTree)malloc(sizeof(BiTNode));
27     (*T)->data=1;
28     (*T)->lchild=(BiTree)malloc(sizeof(BiTNode));
29     (*T)->rchild=(BiTree)malloc(sizeof(BiTNode));
30     (*T)->lchild->data=2;
31     (*T)->lchild->lchild=(BiTree)malloc(sizeof(BiTNode));
32     (*T)->lchild->rchild=(BiTree)malloc(sizeof(BiTNode));
33     (*T)->lchild->rchild->data=5;
34     (*T)->lchild->rchild->lchild=NULL;
35     (*T)->lchild->rchild->rchild=NULL;
36     (*T)->rchild->data=3;
37     (*T)->rchild->lchild=(BiTree)malloc(sizeof(BiTNode));
38     (*T)->rchild->lchild->data=6;
39     (*T)->rchild->lchild->lchild=NULL;
40     (*T)->rchild->lchild->rchild=NULL;
41     (*T)->rchild->rchild=(BiTree)malloc(sizeof(BiTNode));
42     (*T)->rchild->rchild->data=7;
43     (*T)->rchild->rchild->lchild=NULL;
44     (*T)->rchild->rchild->rchild=NULL;
45     (*T)->lchild->lchild->data=4;
46     (*T)->lchild->lchild->lchild=NULL;
47     (*T)->lchild->lchild->rchild=NULL;
48 }
49 /**
50  * @Description: 前序遍历使用的进栈函数
51  * @Param: BiTree* a 指向树的指针 BiTree elem 进栈的元素
52  * @Return: 无
53  * @Author: Carlos
54  */
55 void push(BiTree* a,BiTree elem){
56     //指针进栈
57     a[++top]=elem;
58 }
59 /**
60  * @Description: 前序遍历使用的弹栈函数
61  * @Param: 无
62  * @Return: 无
63  * @Author: Carlos
64  */
65 void pop( ){
66     if (top== -1) {
67         return;
68     }
69     top--;
70 }
71 /**

```

```

72  * @Description: 显示函数
73  * @Param: 无
74  * @Return: 无
75  * @Author: Carlos
76  */
77 void displayElem(BiTree elem){
78     printf("%d ",elem->data);
79 }
80 /**
81  * @Description: 拿到栈顶元素
82  * @Param: 无
83  * @Return: 栈顶元素的地址
84  * @Author: Carlos
85  */
86 BiTree getTop(BiTree*a){
87     return a[top];
88 }
89 /**
90  * @Description: 先序遍历非递归算法。遇到右孩子就进栈，优先打印左孩子。遍历完做孩子，
    再出栈右孩子
91  * @Param: 无
92  * @Return: 栈顶元素的地址
93  * @Author: Carlos
94  */
95 void PreOrderTraverse(BiTree Tree){
96     //定义一个顺序栈
97     BiTree a[20];
98     int cnt1=1;
99     //临时指针
100    BiTNode * p;
101    //根结点进栈
102    push(a, Tree);
103    while (top!= -1) {
104        //取栈顶元素
105        p=getTop(a);
106        //弹栈
107        pop();
108        while (p) {
109            //调用结点的操作函数    遍历之前调用，打印子树根节点
110            displayElem(p);
111            //如果二叉树的深度为 K，那么此二叉树最多有 2K-1 个结点。 利用总结点个数
            可以求二叉树深度
112            printf("cnt1:%d:\r\n",cnt1++);
113            //如果该结点有右孩子，右孩子进栈
114            if (p->rchild) {
115                push(a,p->rchild);
116            }
117            //一直指向根结点最后一个左孩子
118            p=p->lchild;
119        }
120    }
121 }
122 }
123 }
124 /**
125  * @Description: 先序遍历。子树回退到子树的根节点，然后打印根节点
126  * @Param: BiTree T 遍历的起点元素
127  * @Return: 无

```

```

128  * @Author: Carlos
129  */
130  // void PreOrderTraverse(BiTree T){
131  //     if (T) {
132  //         displayElem(T); //调用操作结点数据的函数方法
133  //         PreOrderTraverse(T->lchild); //访问该结点的左孩子
134  //         PreOrderTraverse(T->rchild); //访问该结点的右孩子
135  //     }
136  //     //如果结点为空，返回上一层
137  //     return;
138  // }
139  int main(){
140      BiTree Tree;
141      CreateBiTree(&Tree);
142      printf("先序遍历: \n");
143      PreOrderTraverse(Tree);
144  }

```

中序遍历

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4  #define DBG_PRINTF(fmt, args...) \
5  do\
6  {\
7      printf("<<File:%s Line:%d Function:%s>> ", __FILE__, __LINE__,
__FUNCTION__); \
8      printf(fmt, ##args); \
9  }while(0)
10 #define TElemType int
11 int top=-1; //top变量时刻表示栈顶元素所在位置
12 //构造结点的结构体
13 typedef struct BiTNode{
14     //数据域
15     TElemType data;
16     //左右孩子指针
17     struct BiTNode *lchild, *rchild;
18 }BiTNode;
19 /**
20  * @Description: 初始化树
21  * @Param: BiTNode **T 结构体指针的指针
22  * @Return: 无
23  * @Author: Carlos
24  */
25 //定义一个顺序栈
26 BiTNode *a[20];
27 void CreateBiTNode(BiTNode **T){
28     *T=(BiTNode*)malloc(sizeof(BiTNode));
29     (*T)->data=1;
30     (*T)->lchild=(BiTNode*)malloc(sizeof(BiTNode));
31     (*T)->rchild=(BiTNode*)malloc(sizeof(BiTNode));
32     (*T)->lchild->data=2;
33     (*T)->lchild->lchild=(BiTNode*)malloc(sizeof(BiTNode));
34     (*T)->lchild->rchild=(BiTNode*)malloc(sizeof(BiTNode));
35     (*T)->lchild->rchild->data=5;
36     (*T)->lchild->rchild->lchild=NULL;

```

```

37     (*T)->lchild->rchild->rchild=NULL;
38     (*T)->rchild->data=3;
39     (*T)->rchild->lchild=(BitNode*)malloc(sizeof(BitNode));
40     (*T)->rchild->lchild->data=6;
41     (*T)->rchild->lchild->lchild=NULL;
42     (*T)->rchild->lchild->rchild=NULL;
43     (*T)->rchild->rchild=(BitNode*)malloc(sizeof(BitNode));
44     (*T)->rchild->rchild->data=7;
45     (*T)->rchild->rchild->lchild=NULL;
46     (*T)->rchild->rchild->rchild=NULL;
47     (*T)->lchild->lchild->data=4;
48     (*T)->lchild->lchild->lchild=NULL;
49     (*T)->lchild->lchild->rchild=NULL;
50 }
51 /**
52  * @Description: 中序遍历使用的进栈函数
53  * @Param: BitNode** a 指向树的指针数组 BitNode *elem 进栈的元素
54  * @Return: 无
55  * @Author: Carlos
56  */
57 void Push(BitNode** a, BitNode *elem){
58     //指针进栈
59     a[++top]=elem;
60 }
61 /**
62  * @Description: 前序遍历使用的弹栈函数
63  * @Param: 无
64  * @Return: 无
65  * @Author: Carlos
66  */
67 void Pop( ){
68     if (top==--1) {
69         return;
70     }
71     top--;
72 }
73 /**
74  * @Description: 显示函数
75  * @Param: BitNode *elem 指向树的指针
76  * @Return: 无
77  * @Author: Carlos
78  */
79 void PrintBiT(BitNode *elem){
80     printf("%d ",elem->data);
81 }
82 /**
83  * @Description: 拿到栈顶元素
84  * @Param: BitNode**a 指针数组
85  * @Return: 栈顶元素的地址
86  * @Author: Carlos
87  */
88 BitNode *GetTop(BitNode**a){
89     return a[top];
90 }
91 /**
92  * @Description: 中序遍历非递归算法，先左，然后回退，然后右。从根结点开始，遍历左孩子
    同时压栈，当遍历结束，说明当前遍历的结点没有左孩子，
93  * 从栈中取出来调用操作函数，然后访问该结点的右孩子，继续以上重复性的操作

```

```

94  * @Return: 栈顶元素的地址
95  * @Author: Carlos
96  */
97  void InorderTraverse1(BitNode *Tree){
98
99      //临时指针
100     BitNode *p;
101     //根结点进栈
102     Push(a, Tree);
103     //top!=-1说明栈内不为空，程序继续运行
104     while (top!=-1) {
105         //一直取栈顶元素，且不能为NULL
106         while ((p=GetTop(a)) &&p){
107             //将该结点的左孩子进栈，如果没有左孩子，NULL进栈
108             Push(a, p->lchild);
109         }
110         //跳出循环，栈顶元素肯定为NULL，将NULL弹栈。 打印的第一个元素没有右孩子，所以
        也会Pop掉，再取栈顶元素就是第一个元素的父节点
111         Pop();
112         if (top!=-1) {
113             //取栈顶元素
114             p=GetTop(a);
115             //栈顶元素弹栈
116             Pop();
117             //遍历完所有左孩子之后，打印栈顶的元素。
118             PrintBiT(p);
119             //将p指向的结点的右孩子进栈
120             Push(a, p->rchild);
121         }
122     }
123 }
124 /**
125  * @Description: 中序遍历非递归算法。中序遍历过程中，只需将每个结点的左子树压栈即可，
        右子树不需要压栈。
126  * 当结点的左子树遍历完成后，只需要以栈顶结点的右孩子为根结点，继续循环遍历即可
127  * @Param: 无
128  * @Return: 栈顶元素的地址
129  * @Author: Carlos
130  */
131 void InorderTraverse2(BitNode *Tree){
132     //定义一个顺序栈
133     //BitNode *a[20];
134     //临时指针
135     BitNode *p = Tree;
136     //当p为NULL或者栈为空时，表明树遍历完成
137     while (p || top!=-1) {
138         //如果p不为NULL，将其压栈并遍历其左子树
139         if (p) {
140             Push(a, p);
141             p=p->lchild;
142         }
143         //如果p==NULL，表明左子树遍历完成，需要遍历上一层结点的右子树 弹出时顺便访问
        右子树
144         else{
145             p=GetTop(a);
146             Pop();
147             PrintBiT(p);
148             p=p->rchild;

```

```

149     }
150 }
151 }
152 /**
153  * @Description: 中序遍历
154  * @Param: BitNode *T 结构体指针
155  * @Return: 无
156  * @Author: Carlos
157  */
158 void INOrderTraverse(BitNode *T){
159     if (T) {
160         INOrderTraverse(T->lchild); //遍历左孩子
161         PrintBit(T); //调用操作结点数据的函数方法
162         INOrderTraverse(T->rchild); //遍历右孩子
163     }
164     //如果结点为空，返回上一层
165     return;
166 }
167 int main(){
168     BitNode *Tree;
169     CreateBitNode(&Tree);
170     printf("中序遍历: \r\n");
171     InOrderTraverse2(Tree);
172     DBG_PRINTF("123456\r\n");
173     return 0;
174 }

```

后序遍历 (重点)

```

1  /**
2   * @Description: 二叉树的后序遍历（非递归）
3   * @Version: V1.0
4   * @Autor: Carlos
5   * @Date: 2020-05-18 16:23:57
6   * @LastEditors: Carlos
7   * @LastEditTime: 2020-05-18 16:24:29
8   */
9  #include <stdio.h>
10 #include <string.h>
11 #include <stdlib.h>
12 #define TElemType int
13 //top变量时刻表示栈顶元素所在位置
14 int top=-1;
15 //构造结点的结构体
16 typedef struct BitNode{
17     //数据域
18     TElemType data;
19     //左右孩子指针
20     struct BitNode *lchild,*rchild;
21 }BitNode;
22 /**
23  * @Description: 初始化树
24  * @Param: BitNode* *T 结构体指针数组
25  * @Return: 无
26  * @Author: Carlos
27  */
28 void CreateBiTree(BitNode **T){

```

```

29     *T=(BitNode*)malloc(sizeof(BitNode));
30     (*T)->data=1;
31     (*T)->lchild=(BitNode*)malloc(sizeof(BitNode));
32     (*T)->rchild=(BitNode*)malloc(sizeof(BitNode));
33     (*T)->lchild->data=2;
34     (*T)->lchild->lchild=(BitNode*)malloc(sizeof(BitNode));
35     (*T)->lchild->rchild=(BitNode*)malloc(sizeof(BitNode));
36     (*T)->lchild->rchild->data=5;
37     (*T)->lchild->rchild->lchild=NULL;
38     (*T)->lchild->rchild->rchild=NULL;
39     (*T)->rchild->data=3;
40     (*T)->rchild->lchild=(BitNode*)malloc(sizeof(BitNode));
41     (*T)->rchild->lchild->data=6;
42     (*T)->rchild->lchild->lchild=NULL;
43     (*T)->rchild->lchild->rchild=NULL;
44     (*T)->rchild->rchild=(BitNode*)malloc(sizeof(BitNode));
45     (*T)->rchild->rchild->data=7;
46     (*T)->rchild->rchild->lchild=NULL;
47     (*T)->rchild->rchild->rchild=NULL;
48     (*T)->lchild->lchild->data=4;
49     (*T)->lchild->lchild->lchild=NULL;
50     (*T)->lchild->lchild->rchild=NULL;
51 }
52 /**
53  * @Description: 后序遍历使用的弹栈函数
54  * @Param: 无
55  * @Return: 无
56  * @Author: Carlos
57  */
58 void Pop( ){
59     if (top== -1) {
60         return ;
61     }
62     top--;
63 }
64 /**
65  * @Description: 显示函数
66  * @Param: 无
67  * @Return: 无
68  * @Author: Carlos
69  */
70 void PrintBiT(BitNode* elem){
71     printf("%d ",elem->data);
72 }
73
74 //增加左右子树的访问标志
75 typedef struct SNode{
76     BitNode* p;
77     int tag;
78 }SNode;
79 //定义一个顺序栈
80 SNode a[20];
81 /**
82  * @Description: 后序遍历使用的进栈函数
83  * @Param: SNode *a 指向树和标志位的结构体的指针 BitNode* sdata 进栈的元素
84  * @Return: 无
85  * @Author: Carlos
86  */

```

```

87 void Push(SNode *a,SNode sdata){
88     a[++top]=sdata;
89 }
90 /**
91  * @Description: 后序遍历非递归算法。后序遍历是在遍历完当前结点的左右孩子之后，才调用
    操作函数，所以需要在操作结点进栈时，为每个结点配备一个标志位。
92  * 当遍历该结点的左孩子时，设置当前结点的标志位为 0，进栈；当要遍历该结点的右孩子时，设
    置当前结点的标志位为 1，进栈。这样，当遍历完成，该结点弹栈时，
93  * 查看该结点的标志位的值：如果是 0，表示该结点的右孩子还没有遍历；反之如果是 1，说明该
    结点的左右孩子都遍历完成，可以调用操作函数。
94  * @Param: 结构体指针数组
95  * @Return: 无
96  * @Author: Carlos
97  */
98 void PostOrderTraverse(BiTNode* Tree){
99
100     //临时指针
101     BiTNode* p = Tree;
102     int tag;
103     SNode sdata;
104     while (p||top!=-1) {
105         //左孩子进栈
106         while (p) {
107             //为该结点入栈做准备
108             sdata.p=p;
109             //由于遍历是左孩子，设置标志位为0
110             sdata.tag=0;
111             //压栈
112             Push(a, sdata);
113             //以该结点为根结点，遍历左孩子
114             p=p->lchild;
115         }
116         //取栈顶元素 取左孩子的父节点
117         sdata=a[top];
118         //栈顶元素弹栈
119         Pop();
120         p=sdata.p;
121         tag=sdata.tag;
122         //右孩子进栈
123         //如果tag==0，说明该结点还没有遍历它的右孩子
124         if (tag==0) {
125             //遍历该节点的右孩子
126             sdata.p=p;
127             sdata.tag=1;
128             //更改该结点的标志位，重新压栈
129             Push(a, sdata);
130             //以该结点的右孩子为根结点，重复循环
131             p=p->rchild;
132         }
133         //如果取出来的栈顶元素的tag==1，说明此结点左右子树都遍历完了，可以调用操作函
    数了
134         else{
135             PrintBiT(p);
136             p=NULL;
137         }
138     }
139 }
140 // /**

```

```

141 // * @Description: 先序遍历
142 // * @Param: BitNode* T 指针数组，存放各个节点的指针
143 // * @Return: 无
144 // * @Author: Carlos
145 // */
146 // void PreOrderTraverse(BitNode* T){
147 //     if (T) {
148 //         PreOrderTraverse(T->lchild); //访问该结点的左孩子
149 //         PreOrderTraverse(T->rchild); //访问该结点的右孩子
150 //         PrintBit(T); //调用操作结点数据的函数方法
151 //     }
152 //     //如果结点为空，返回上一层
153 //     return;
154 // }
155 int main(){
156     BitNode* Tree;
157     CreateBiTree(&Tree);
158     printf("后序遍历: \n");
159     PostOrderTraverse(Tree);
160 }

```

排序/查找算法及其改进

快速排序

```

1  /**
2   * @Description: 核心思想就是在数组中找个key点，按照key点大小对数组进行前后分割 复杂度最好  $O(n \log n)$  数组有序，最坏:  $O(n^2)$  不稳定排序
3   * 改进: 1.key值随机选取 2.key值选取数组元素的中位数，中位数可以在线性时间取得（算法导论）
4   * @Param: Mytype arr[] 数组地址 int n 数组大小
5   * @Return: 无
6   * @Author: Carlos
7   */
8  void QucikSort(Mytype arr[], int low, int high)
9  {
10     int first = low;
11     int last = high;
12     Mytype key = arr[first];
13     if(low >= high)
14         return;
15     while(first < last)
16     {
17         //从右向左找一个比key小的元素，和first交换。
18         while(first < last && arr[last] >= key)
19         {
20             last--;
21         }
22         //跳出的时候说明 找到了小于key的数，和first交换
23         arr[first] = arr[last];
24         while(first < last && arr[first] <= key)
25         {
26             first++;
27         }
28         //跳出的时候说明 找到了大于key的数，和last交换
29         arr[last] = arr[first];
30     }

```

```

31 //当first = last 跳出，把key放到合适的位置
32 arr[first] = key;
33 //继续排序剩下的
34 QucikSort(arr, low, first-1);
35 QucikSort(arr, first+1, high);
36 }

```

冒泡排序

```

1  /**
2   * @Description: 冒泡排序 复杂度O(n2) 稳定排序
3   * 改进: 1.利用布尔变量isSorted作为标记。如果在本轮排序中，元素有交换，则说明数列无序；
   如果没有元素交换，说明数列已然有序，直接跳出大循环
4   * 2.有序区的长度和排序的轮数是相等的。比如第一轮排序过后的有序区长度是1，第二轮排序过后的
   有序区长度是2 .....
5   实际上，数列真正的有序区可能会大于这个长度，比如例子中仅仅第二轮，后面5个元素实际都已经属于
   有序区。因此后面的许多次元素比较是没有意义的。
6   如何避免这种情况呢？我们可以在每一轮排序的最后，记录下最后一次元素交换的位置，那个位置也就是
   无序数列的边界，再往后就是有序区了。
7   * https://www.cnblogs.com/misttermoney/p/9550590.html
8   * @Param: Mytype arr[] 数组地址 int n 数组大小
9   * @Return: 无
10  * @Author: Carlos
11  */
12 void BubleSortImprove(Mytype arr[],int n)
13 {
14     int i,j = 0;
15     int lastExchangeIndex = 0;
16     int sortBorder = n - 1;
17     for(i = 0; i < n-1;i++)
18     {
19         bool isSorted = true;
20         for(j = 0;j < sortBorder;j++)
21         {
22             if(arr[j] < arr[j+1])
23             {
24                 Mytype temp = arr[j];
25                 arr[j] = arr[j+1];
26                 arr[j+1] = temp;
27                 isSorted = false;
28                 lastExchangeIndex = j;
29             }
30         }
31         sortBorder = lastExchangeIndex;
32         if(isSorted)
33             break;
34     }
35 }

```

堆排序

```

1 void heapify(Mytype tree[], int n, int i)
2 {
3     if (i >= n)
4     {
5         return;
6     }

```

```

7      //当前父节点的两个子节点
8      int c1 = 2 * i + 1;
9      int c2 = 2 * i + 2;
10     //假设根节点为最大值
11     int max = i;
12     //找出子树中三个节点中的最大的节点
13     // if (c1 < n )
14     // {
15     //     max = (tree[c1] > tree[max]) ? c1:c2;
16     // }
17     if (c1 < n && tree[c1] > tree[max])
18     {
19         max = c1;
20     }
21     if (c2 < n && tree[c2] > tree[max])
22     {
23         max = c2;
24     }
25     //将最大节点和当前节点交换位置
26     if (max != i)
27     {
28         swap(tree, max, i);
29         //继续进行下一个节点的 构建堆
30         heapify(tree, n, max);
31     }
32 }
33
34 int build_heap (Mytype tree[],int n)
35 {
36     int last_node = n-1;
37     //最后一个父节点
38     int parent = (last_node-1)/2;
39     int i;
40     //从最后一个父节点开始，由下到上遍历每一个父节点依次调整构建堆
41     for(i = parent;i>=0;i--)
42     {
43         heapify(tree,n,i);
44     }
45 }
46 /**
47  * @Description: 堆排序时间复杂度 O(nlogn) 思想：构建大根堆/小根堆，将堆顶元素与最后
    一个叶子节点交换，切断叶子节点 （B站正月点灯笼）
48  * @Param: Mytype tree[] 数组 int n 数组中元素的个数
49  * @Return: 无
50  * @Author: Carlos
51  */
52 int HeapSort (Mytype tree[],int n)
53 {
54     //构建堆 第一个节点为堆中最大的数
55     build_heap(tree,n);
56     int i;
57     //每次都换出来一个最大值
58     for(i = n-1;i>=0;i--)
59     {
60         //交换最后一个节点和第一个节点
61         swap(tree,i,0);
62         //每次都截断最后一个元素，我们可以默认最后一个元素已经被截断了，i即为剩下的元
        素，每次传入i的值即可继续构建0--i范围内的元素

```

```

63     heapify(tree,i,0);
64 }
65 }

```

插入排序

```

1  //插入排序
2  //直接插入排序函数 升序
3  void InsertSort(Mytype a[], int n)
4  {
5      //注意从第二个元素开始
6      for(int i= 1; i<n; i++)
7      {
8          //如果a[i-1]本来就是小于a[i]的，就不需要做任何操作。当a[i-1]是较大的元素，需
          //要从i向前找，找到合适的位置，插在a[i]前面
9          if(a[i] < a[i-1])
10             { //若第 i 个元素大于 i-1 元素则直接插入；反之，需要找到适当的插入位置后在插
                  //入。
11                 int j= i-1;
12                 int x = a[i];
13                 //在0到j-1中找到 a[i-1]<a[i]的位置
14                 //从第j个元素开始，即i的前一个。找一个比i位置元素小的位置
15                 while(j>-1 && x < a[j])
16                 { //采用顺序查找方式找到插入的位置，在查找的同时，将数组中的元素进行后移操
                      //作，给插入元素腾出空间
17                     a[j+1] = a[j];
18                     j--;
19                 }
20                 //跳出循环时，找到了x > a[j] 的恰当位置
21                 a[j+1] = x;          //插入到正确位置
22             }
23         }
24     }

```

选择排序

```

1  /**
2   * @Description: 从i的位置开始，寻找最大值
3   * @Param: int arr[] 数组名 int i 查找最大值开始的位置，int n 数组长度
4   * @Return: 最大值的位置
5   * @Author: Carlos
6   */
7  int FindMaxPos1(Mytype arr[],int i,int n)
8  {
9      //假设第i 的位置为最大值
10     int max=i;
11     //从i+1开始比较
12     while (i+1<n) {
13         if (arr[max]<arr[i+1])
14         {
15             max=i+1;
16         }
17         i++;
18     }
19     return max;
20 }
21 /**

```

```

22  * @Description: 从i的位置开始，寻找最小值
23  * @Param: int arr[] 数组名 int i 查找最小值开始的位置，int n 数组长度
24  * @Return: 最小值的位置
25  * @Author: Carlos
26  */
27 int FindMinPos1(Mytype arr[],int i,int n)
28 {
29     //假设第i 的位置为最小值
30     int min=i;
31     //从i+1开始比较
32     while (i+1<n) {
33         if (arr[min]>arr[i+1])
34         {
35             min=i+1;
36         }
37         i++;
38     }
39     return min;
40 }
41 /**
42  * @Description: 选择排序，对于具有 n 个记录的无序表遍历 n-1 次
43  * 首先在未排序序列中找到最小（大）元素，存放到排序序列的起始位置。
44  * 再从剩余未排序元素中继续寻找最小（大）元素，然后放到已排序序列的末尾。
45  * 重复第二步，直到所有元素均排序完毕。
46  * O(n²)
47  * @Param: arr[] 数组 n数组大小
48  * @Return: 无
49  * @Author: Carlos
50  */
51 void SelectSort1(Mytype arr[],int n)
52 {
53     //从第二个数开始
54     for(int i = 0;i<n;i++)
55     {
56         //每次从i开始，在i到n中找一个最小值出来
57         int pos = FindMinPos1(arr,i,n);
58         if(pos!=i)
59             swap(arr,pos,i);
60     }
61 }

```

二分查找法

```

1  int binSearch(int key,int length, int array[])
2  {
3      int mid=0;
4      int start = 0;
5      int end = length - 1;
6      while (start <= end) {
7          mid = (end - start) / 2 + start;
8          if (key == array[mid]) {
9              return mid;
10         }
11         if (key < array[mid]) {
12             end = mid - 1;
13         } else if (key > array[mid]) {
14             start = mid + 1;

```

```

15         } else {
16             return mid;
17         }
18     }
19     return -1;
20 }

```

实现字符串的库函数

strcpy

```

1 void mycpy(char *s1, char *s2)
2 {
3     while(*s1++ = *s2++);
4 }

```

strlen

```

1 int mylen(char *s1)
2 {
3     char *p = s1;
4     while(*p++);
5     return p - s - 1;
6 }

```

strcat

```

1 void mycat(char *s1, char *s2)
2 {
3     while(*s1++);
4     s1--;
5     while(*s1++ = *s2++);
6 }

```

实现atoi函数

```

1 int myatoi (char* str)
2 {
3     if(str == NULL)
4     {
5         printf("Invalid Input");
6         return -1;
7     }
8     while(*str == ' ')//去掉字符串开始的空格
9     {
10         str++;
11     }
12     int nsign = (*str=='-')?-1:1;//确定符号位
13     if(*str == '+' || *str == '-')
14     {
15         str++;
16     }
17     int nResult = 0;
18     while(*str>='0'&&*str<='9')
19     {

```

```

20     nResult = nResult*10+(*str-'0');
21     str++;
22 }
23 while(*str == ' ')
24 {
25     str++;
26 }
27 if(*str!='\0')
28 {
29     printf("Invalid Input");
30     return -1;
31 }
32 return nResult*nSign;
33 }
34
35 int main()
36 {
37     printf("%d\r\n",myatoi("123465"));
38     return 0;
39 }

```

实现itoa函数

```

1
2 char *myitoa(int num,char *str,int radix)
3 {
4     char *ptr = str;
5     int temp;
6     int sign = num,i = 0,j = 0;
7     if(sign<0)
8     {
9         num =-num;
10    }
11    while(num)
12    {
13        //先存储低位
14        *ptr++ = num%radix+'0';
15        num/=radix;
16        //小于进制数直接放进去
17        if(num<radix)
18        {
19            *ptr++=num+'0';
20            break;
21        }
22    }
23    //最高位存储符号位
24    if(sign<0)
25    {
26        *ptr++='-';
27    }
28    *ptr = '\0';
29    //存储的是反过来的 我们需要调换位置
30    j = ptr-str-1;
31    for(i=0;i<j;i++)
32    {
33        int temp = str[i];
34        str[i] = str[j];

```

```

35     str[j--]=temp;
36 }
37 return str;
38 }
39 int main()
40 {
41     char str[6];
42     myitoa(-12345,str,10);
43     printf("%s\n",str);
44     itoa(12345,str,10);
45     printf("%s\n",str);
46     return 0;
47 }

```

如何实现memmove函数

```

1  #include <iostream>
2  #include <cstdio>
3  #include <cstring>
4
5  void * my_memmove( void * const dest, const char * const src, size_t n )
6  {
7      // check parameters
8      if( 0 == n )
9      {
10         return NULL;
11     }
12     if( NULL == dest || NULL == src )
13     {
14         return NULL;
15     }
16
17     char * psrc = (char *)src;
18     char * pdest = (char *)dest;
19     if( pdest <= psrc || pdest > psrc + n )
20     {
21         std::cout << "forward overlapping" << std::endl;
22         // copy forward direction
23         for( size_t i = 0; i < n; i++ )
24         {
25             *pdest = *psrc;
26             pdest++;
27             psrc++;
28         }
29     }
30     else
31     {
32         std::cout << "backward overlapping" << std::endl;
33         // copy backward direction
34         pdest = pdest + n;
35         psrc = psrc + n;
36         for( size_t i = 0; i < n; i++ )
37         {
38             *pdest = *psrc;
39             pdest--;
40             psrc--;
41         }

```

```

42     }
43     return dest;
44 }
45
46 int main( int argc, char ** argv )
47 {
48     char *src = new char[100];
49     sprintf( src, "%s", "hello world!" );
50     char * dest = new char[100];
51     memset( dest, 0, 100*sizeof(char) );
52     std::cout << src << std::endl;
53     char * result = (char*)my_memmove( dest, src, strlen(src) );
54     std::cout << result << std::endl;
55     delete src;
56     delete dest;
57     return 0;
58 }

```

LeetCode题目频率(刷题可参考)

[【leetcode】高频题目整理 所有题目汇总篇\(High Frequency Problems, All Problems \)](#)

操作系统

创建进程有哪几种方式

创建进程的多种方式但凡是硬件，都需要有操作系统去管理，只要有操作系统，就有进程的概念，就需要有创建进程的方式，一些操作系统只为个应用程序设计，比如扫地机器人，一旦启动，所有的进程都已经存在。

而对于通用系统（跑很多应用程序），需要有系统运行过程中创建或撤销进程的能力，主要分为4中形式创建新的进程：

- 1.系统初始化**（查看进程 linux中用ps命令， windows中用任务管理器，前台进程负责与用户交互，后台运行的进程与用户无关，运行在后台并且只在需要时才唤醒的进程，称为守护进程，如电子邮件、web页面、新闻、打印）
- 2.一个进程在运行过程中开启了子进程**（如 nginx开启多进程，os. fork, subprocess Popen等）
- 3.用户的交互式请求，而创建一个新进程**（如用户用鼠标双击任意款软件图片：q微信暴风影音等）
- 4.一个批处理作业的初始化**（只在大型机的批处理系统中应用）无论哪一种，新进程的创建都是由一个已经存在的进程执行了一个用于创建进程的系统调用而创建的。

进程和线程有什么区别

进程是具有一定独立功能的程序关于某个数据集合上的一次运行活动，它是**系统进行资源分配和调度的一个独立单位**。例如，用户运行自己的程序，系统就创建一个进程，并为它**分配资源**，包括各种表格、内存空间、磁盘空间、IO设备等，然后该进程被放入到进程的就绪队列，进程调度程序选中它，为它分配CPU及其他相关资源，该进程就被运行起来。

线程是进程的一个实体，是**CPU调度和分配的基本单位**，线程自己基本上不拥有系统资源，只拥有一些在运行中必不可少的资源（如程序计数器、一组寄存器和栈），但是，它可以与同属一个进程的其他线程**共享进程所拥有的全部资源**。

在没有实现线程的操作系统中，进程既是资源分配的基本单位，又是调度的基本单位，它是系统中并发执行的单元。而在实现了线程的操作系统中，进程是资源分配的基本单位而线程是调度的基本单位，是系统中并发执行的单元。

引入线程主要有以下4个方面的优点：

- 1) 易于调度。
- 2) 提高并发性。通过线程可以方便有效地实现并发
- 3) 开销小。创建线程比创建进程要快，所需要的开销也更少。
- 4) 有利于发挥多处理器的功能。通过创建多线程，每个线程都在一个处理器上运行，从而实现应用程序的并行，使每个处理器都得到充分运行。

区别：

进程是并发执行的程序在执行过程中分配和管理资源的基本单位。

线程是进程的一个执行单元，是比进程还要小的独立运行的基本单位。一个程序至少有一个进程，一个进程至少有一个线程。两者的区别主要有以下几个方面：

1. 进程是**资源分配**的最小单位。
2. 线程是**程序执行**的最小单位，也是**处理器调度**的基本单位，但进程不是，两者均可并发执行。
3. 进程有自己的**独立地址空间**，每启动一个进程，系统就会为它分配地址空间，建立数据表来维护代码段、堆栈段和数据段，这种操作非常昂贵。而线程是**共享进程中的数据**，使用相同的地址空间，因此，CPU切换一个线程的花费远比进程小很多，同时创建一个线程的开销也比进程小很多。
4. 线程之间的通信更方便，同一进程下的线程共享全局变量、静态变量等数据，而进程之间的通信需要以通信的方式（IPC）进行。不过如何处理好同步与互斥是编写多线程程序的难点。但是多进程程序更健壮，**多线程程序只要有一个线程死掉，整个进程也跟着死掉了，而一个进程死掉不会对另外一个进程造成影响，因为进程有自己独立的地址空间。**
5. 进程切换时，消耗的资源大，效率低。所以涉及到频繁的切换时，使用线程要好于进程。同样如果要求同时进行并且又要共享某些变量的并发操作，只能用线程不能用进程。
6. 执行过程：**每个独立的进程有一个程序运行的入口、顺序执行序列和程序入口。但是线程不能独立执行，必须依存在应用程序中，由应用程序提供多个线程执行控制。**

优缺点：

线程执行开销小，但是不利于资源的管理和保护。线程适合在SMP机器（双CPU系统）上运行。

进程执行开销大，但是能够很好的进行资源管理和保护，可以跨机器迁移。

何时使用多进程，何时使用多线程？

对资源的管理和保护要求高，不限制开销和效率时，使用多进程。

要求效率高，频繁切换时，资源的保护管理要求不是很高时，使用多线程。

线程间通信都有哪些方式

进程间通信：管道，FIFO，信号，信号量，消息队列，共享内存（最快），套接字。

线程间通信：同一进程内的多线程之间通信因为共享相同的地址空间，所以更多的是处理**线程间的同步问题**。不同线程间的线程之间通信则类似进程间通信的方式。

线程间同步：互斥锁，条件变量，读写锁，信号量，临界区等。

进程间通信方式

概述

进程通信(Interprocess Communication，IPC)是一个进程与另一个进程间共享消息的一种通信方式。消息(message)是发送进程形成的一个消息块，将消息内容传送给接收进程。IPC机制是消息从一个进程的地址空间拷贝到另一个进程的地址空间。

进程通信的目的

- 数据传输
一个进程需要将其数据发送给另一进程，发送的数据量在一个字节到几M字节之间。
- 共享数据
多个进程操作共享数据
- 事件通知
一个进程需要向另一个或一组进程发送消息，通知它（它们）发生了某种事件（如进程终止时要通知父进程）。
- 资源共享
多个进程之间共享同样的资源。为了作到这一点，需要内核提供锁和同步机制。
- 进程控制
有些进程希望完全控制另一个进程的执行（如Debug进程），此时控制进程希望能够拦截另一个进程的所有陷入和异常，并能够及时知道它的状态改变。

linux使用的进程间通信方式

- 管道（pipe），流管道(s_pipe)和有名管道（FIFO）
- 信号(signal)
- 消息队列
- 共享内存
- 信号量
- 套接字（socket）

管道(pipe)

管道这种通讯方式有两种限制，一是**半双工的通信，数据只能单向流动**，二是只能在具有**亲缘关系的进程间使用**。进程的亲缘关系通常是指**父子进程关系**。

流管道s_pipe: 去除了第一种限制,可以**双向传输**（全双工）。

管道可用于具有亲缘关系进程间的通信。**命名管道**:name_pipe克服了管道没有名字的限制，因此，除具有管道所具有的功能外，它还允许**无亲缘关系**进程间的通信；

信号量(semaphore)

信号量是一个**计数器**，可以用来控制多个进程对共享资源的访问。它常作为一种锁机制，**防止某进程正在访问共享资源时，其他进程也访问该资源**。因此，主要作为进程间以及同一进程内不同线程之间的同步手段。

消息队列(message queue)

消息队列是由消息组成的链表，存放在内核中并由消息队列标识符标识。

消息队列是消息的链接表，包括Posix消息队列system V消息队列。有足够权限的进程可以向队列中添加消息，被赋予读权限的进程则可以读走队列中的消息。消息队列克服了信号承载信息量少，管道只能承载无格式字节流以及缓冲区大小受限等缺点。

信号(signal)

信号是一种比较复杂的通信方式，用于**通知接收进程某个事件已经发生**。主要作为进程间以及同一进程不同线程之间的同步手段。

共享内存(shared memory)

共享内存就是映射一段能被其他进程所访问的内存，**这段共享内存由一个进程创建，但多个进程都可以访问。共享内存是最快的IPC方式**，它是针对其他进程间通信方式运行效率低而专门设计的。它往往与其他通信机制，如信号量，配合使用，来实现进程间的同步和通信。

套接字(socket)

套解字也是一种进程间通信机制，与其他通信机制不同的是，它可用于**不同机器间的进程通信**。

各通信方式的比较和优缺点：

管道：速度慢，容量有限，只有父子进程能通讯。

FIFO：任何进程间都能通讯，但速度慢。

消息队列：容量受到系统限制，且要注意第一次读的时候，要考虑上一次没有读完数据的问题，消息队列可以不再局限于父子进程，而允许任意进程通过共享消息队列来实现进程间通信，并由系统调用函数来实现消息发送和接收之间的同步，从而使得用户在使用消息缓冲进行通信时不再需要考虑同步问题，使用方便，但是信息的复制需要额外消耗CPU的时间，**不适宜于信息量大或操作频繁的场所**。此种方法不太常用。

信号量：不能用来传递复杂消息，**只能用来同步**。

共享内存：利用内存缓冲区直接交换信息，无须复制，快捷、信息量大是其优点。共享内存块提供了在任意数量的进程之间进行高效双向通信的机制。每个使用者都可以读取写入数据，但是所有程序之间必须达成并遵守一定的协议，以防止诸如在读取信息之前覆写内存空间等竞争状态的出现。

进程间通信方式的选择

PIPE和FIFO(有名管道)用来实现进程间相互发送非常短小的、频率很高的消息，这两种方式通常适用于两个进程间的通信。

共享内存用来实现进程间共享的、非常庞大的、读写操作频率很高的数据；这种方法适用于多进程间的通信。

其他考虑用socket。主要应用在分布式开发中

线程间同步方式

1、临界区 (CCriticalSection)

当多个线程访问一个独占性共享资源时，可以使用临界区对象。拥有临界区的线程可以访问被保护起来的资源或代码段，其他线程若想访问，则被挂起，直到拥有临界区的线程放弃临界区为止。具体应用方式：

- 1、定义临界区对象CriticalSection g_CriticalSection;
- 2、在访问共享资源（代码或变量）之前，先获得临界区对象，g_CriticalSection.Lock（）；
- 3、访问共享资源后，则放弃临界区对象，g_CriticalSection.Unlock（）；

2、事件 (CEvent)

事件机制，则允许一个线程在处理完一个任务后，主动唤醒另外一个线程执行任务。比如在某些网络应用程序中，一个线程如A负责侦听通信端口，另外一个线程B负责更新用户数据，利用事件机制，则线程A可以通知线程B何时更新用户数据。每个Cevent对象可以有两种状态：有信号状态和无信号状态。Cevent类对象有两种类型：人工事件和自动事件。

自动事件对象，在被至少一个线程释放后自动返回到无信号状态；

人工事件对象，获得信号后，释放可利用线程，但直到调用成员函数ReSet()才将其设置为无信号状态。在创建Cevent对象时，默认创建的是自动事件。

1、

```
1 CEvent(BOOL bInitiallyOwn=FALSE,
2         BOOL bManualReset=FALSE,
3         LPCTSTR lpszName=NULL,
4         LPSECURITY_ATTRIBUTES lpsaAttribute=NULL);
```

bInitiallyOwn:指定事件对象初始化状态，TRUE为有信号，FALSE为无信号；

bManualReset：指定要创建的事件是属于人工事件还是自动事件。TRUE为人工事件，FALSE为自动事件；

后两个参数一般设为NULL，在此不作过多说明。

2、BOOL CEvent：：SetEvent();

将Cevent类对象的状态设置为有信号状态。如果事件是人工事件，则Cevent类对象保持为有信号状态，直到调用成员函数ResetEvent()将其重新设为无信号状态时为止。如果为自动事件，则在SetEvent（）后将事件设置为有信号状态，由系统自动重置为无信号状态。

3、BOOL CEvent：：ResetEvent();

将事件的状态设置为无信号状态，并保持该状态直至SetEvent（）被调用为止。由于自动事件是由系统自动重置，故自动事件不需要调用该函数。

一般通过调用WaitForSingleObject（）函数来监视事件状态。

3、互斥量（CMutex）

互斥对象和临界区对象非常相似，只是其允许在进程间使用，而临界区只限制与同一进程的各个线程之间使用，但是更节省资源，更有效率。

4、信号量（CSemaphore）

当需要一个计数器来限制可以使用某共享资源的线程数目时，可以使用“信号量”对象。

CSemaphore类对象保存了对当前访问某一个指定资源的线程的计数值，该计数值是当前还可以使用该资源的线程数目。如果这个计数达到了零，则所有对这个CSemaphore类对象所控制的资源的访问尝试都被放入到一个队列中等待，直到超时或计数值不为零为止。

CSemaphore 类的构造函数原型及参数说明如下：

```
1 CSemaphore(
2     LONG lInitialCount = 1,
3     LONG lMaxCount = 1,
4     LPCTSTR pstrName = NULL,
5     LPSECURITY_ATTRIBUTES lpsaAttributes = NULL
6 );
```

lInitialCount:信号量对象的初始计数值，即可访问线程数目的初始值；

lMaxCount：信号量对象计数值的最大值，该参数决定了同一时刻可访问由信号量保护的资源的线程最大数目；

后两个参数在同一进程中使用一般为NULL，不作过多讨论；

一般是将当前可用资源计数设置为最大资源计数，每增加一个线程对共享资源的访问，当前可用资源计数就减1，只要当前可用资源计数大于0，就可以发出信号量信号。如果为0，则放入一个队列中等待。线程在处理完共享资源后，应在离开的同时通过ReleaseSemaphore（）函数将当前可用资源数加1。

```
1 BOOL ReleaseSemaphore( HANDLE hSemaphore,          // hSemaphore:信号量句柄
2                        LONG lReleaseCount,          // lReleaseCount: 信号量计数值
3                        LPLONG lpPreviousCount        // 参数一般为NULL);
```

线程是否具有相同的堆栈？

真正的程序执行都是线程来完成的，程序启动的时候操作系统就帮你创建了一个主线程。

每个线程有自己的堆栈。

线程同步方法

现在流行的进程线程同步互斥的控制机制，其实是由最原始、最基本的4种方法（临界区、互斥量、信号量和事件）实现的。

1) **临界区**：通过对多线程的串行化来访问公共资源或一段代码，速度快，适合控制数据访问。在任意时刻只允许一个线程访问共享资源，如果有多个线程试图访问共享资源，那么当有一个线程进入后，其他试图访问共享资源的线程将会被挂起，并一直等到进入临界区的线程离开，临界在被释放后，其他线程才可以抢占。

2) **互斥量**：为协调对一个共享资源的单独访问而设计，只有拥有互斥量的线程，才有权限去访问系统的公共资源，因为互斥量只有一个，所以能够保证资源不会同时被多个线程访问。互斥不仅能实现同一应用程序的公共资源安全共享，还能实现不同应用程序的公共资源安全共享。

3) **信号量**：为控制一个具有有限数量的用户资源而设计。它允许多个线程在同一个时刻去访问同一个资源，但一般需要限制同一时刻访问此资源的最大线程数目

4) **事件**：用来通知线程有一些事件已发生，从而启动后继任务的开始。

线程同步的方式包括：互斥锁、读写锁、条件变量、信号量和令牌。

互斥锁和读写锁：提供对临界资源的保护，当多线程试图访问临界资源时，都必须通过获取锁的方式来访问临界资源。（临界资源：是被多线程共享的资源）当读写线程获取锁的频率差别不大时，一般采用互斥锁，如果读线程访问临界资源的频率大于写线程，这个时候采用读写锁较为合适，读写锁允许多个读线程同时访问临界资源，读写线程必须互斥访问临界资源。读写锁的实现采用了互斥锁，所以在读写次数差不多的情况下采用读写锁性能没有直接采用互斥锁来的高。

条件变量：提供线程之间的一种通知机制，当某一条件满足时，线程A可以通知阻塞在条件变量上的线程B，B所期望的条件已经满足，可以解除在条件变量上的阻塞操作，继续做其他事情。

信号量：提供对临界资源的安全分配。如果存在多份临界资源，在多个线程争抢临界资源的情况下，向线程提供安全分配临界资源的方法。如果临界资源的数量为1，将退化为锁。

令牌：一种高级的线程同步的方法。它既提供锁的安全访问临界资源的功能，又利用了条件变量使得线程争夺临界资源时是有序的。

内核线程和用户线程的区别

根据操作系统内核是否对线程可感知，可以把线程分为内核线程和用户线程。

内核线程的建立和销毁都是由操作系统负责、通过系统调用完成的，操作系统在调度时，参考各进程内的线程运行情况做出调度决定。如果一个进程中没有就绪态的线程，那么这个进程也不会被调度占用CPU。

和内核线程对应的是用户线程。**用户线程指不需要内核支持而在用户程序中实现的线程，其不依赖于操作系统核心**，用户进程利用线程库提供创建、同步、调度和管理线程的函数来控制用户线程。用户线程多见于一些历史悠久的操作系统，如UNIX操作系统，不需要用户态/核心态切换，速度快，操作系统内核不知道多线程的存在，因此一个线程阻塞将使得整个进程（包括它的所有线程）阻塞。由于这里的处理器时间片分配是以进程为基本单位的，所以每个线程执行的时间相对减少。为了在操作系统中加入线程支持，采用了在用户空间增加运行库来实现线程，这些运行库被称为“线程包”，用户线程是不能被操作系统所感知的。

引入用户线程有以下4个方面的优势

- 1) 可以在不支持线程的操作系统中实现。
- 2) 创建和销毁线程、线程切换等线程管理的代价比内核线程少得多
- 3) 允许每个进程定制自己的调度算法，线程管理比较灵活。
- 4) 线程能够利用的表空间和堆栈空间比内核级线程多。

用户线程的缺点主要有以下两点：

- 1) 同一进程中只能同时有一个线程在运行，如果有一个线程使用了系统调用而阻塞，那么整个进程都会被挂起
- 2) 页面失效也会导致整个进程被挂起内核线程的优缺点刚好跟用户线程相反。实际上，操作系统可以使用混合的方式来实现线程。

读写锁

读写锁实际是一种特殊的自旋锁，它把对共享资源的访问者划分成读者和写者，读者只对共享资源进行读访问，写者则需要对共享资源进行写操作。这种锁相对于自旋锁而言，能提高并发性，因为在多处理器系统中，它允许同时有多个读者来访问共享资源，最大可能的读者数为实际的逻辑CPU数。写者是排他性的，一个读写锁同时只能有一个写者或多个读者（与CPU数相关），但不能同时既有读者又有写者。在读写锁保持期间也是抢占失效的。

如果读写锁当前没有读者，也没有写者，那么写者可以立刻获得读写锁，否则它必须自旋在那里，直到没有任何写者或读者。如果读写锁没有写者，那么读者可以立即获得该读写锁，否则读者必须自旋在那里，直到写者释放该读写锁。

产生死锁的原因是什么？

多个并发进程因争夺系统资源而产生相互等待的现象。即：一组进程中的每个进程都在等待某个事件发生，而只有这组进程中的其他进程才能触发该事件，这就称这组进程发生了死锁。

产生死锁的本质原因为：

- 1)、系统资源有限。
- 2)、进程推进顺序不合理。

死锁的4个必要条件

1、**互斥**：某种资源一次只允许一个进程访问，即该资源一旦分配给某个进程，其他进程就不能再访问，直到该进程访问结束。

2、**占有且等待**：一个进程本身占有资源（一种或多种），同时还有资源未得到满足，正在等待其他进程释放该资源。

3、**不可抢占**：别人已经占有了某项资源，你不能因为自己也需要该资源，就去把别人的资源抢过来。

4、**循环等待**：存在一个进程链，使得每个进程都占有下一个进程所需的至少一种资源。

当以上四个条件均满足，必然会造成死锁，发生死锁的进程无法进行下去，它们所持有的资源也无法释放。这样会导致CPU的吞吐量下降。所以死锁情况是会浪费系统资源和影响计算机的使用性能的。那么，解决死锁问题就是相当有必要的了。

死锁的处理方式有哪些？

死锁的处理方式主要从预防死锁、避免死锁、检测与解除死锁这四个方面来进行处理。

预防死锁：

- 1、资源一次性分配：（破坏请求和保持条件）
- 2、可剥夺资源：即当某进程新的资源未满足时，释放已占有的资源（破坏不可剥夺条件）
- 3、资源有序分配法：系统给每类资源赋予一个编号，每一个进程按编号递增的顺序请求资源，释放

则相反（破坏环路等待条件）

避免死锁:

预防死锁的几种策略，会严重地损害系统性能。因此在避免死锁时，要施加较弱的限制，从而获得较满意的系统性能。由于在避免死锁的策略中，允许进程动态地申请资源。因而，系统在进行资源分配之前预先计算资源分配的安全性。若此次分配不会导致系统进入不安全状态，则将资源分配给进程；否则，进程等待。其中最具有代表性的避免死锁算法是银行家算法。

检测死锁：

首先为每个进程和每个资源指定一个唯一的号码；然后建立资源分配表和进程等待表。

解除死锁：

当发现有进程死锁后，便应立即把它从死锁状态中解脱出来，常采用的方法有：

- 1、剥夺资源：从其它进程剥夺足够数量的资源给死锁进程，以解除死锁状态；
- 2、撤消进程：可以直接撤消死锁进程或撤消代价最小的进程，直至有足够的资源可用，死锁状态消除为止；所谓代价是指优先级、运行代价、进程的重要性和价值等。

如何避免死锁

在有些情况下死锁是可以避免的。三种用于避免死锁的技术：

加锁顺序（线程按照一定的顺序加锁）

加锁时限（线程尝试获取锁的时候加上一定的时限，超过时限则放弃对该锁的请求，并释放自己占有的锁）

死锁检测

加锁顺序

当多个线程需要相同的一些锁，但是按照不同的顺序加锁，死锁就很容易发生。

如果能确保所有的线程都是按照相同的顺序获得锁，那么死锁就不会发生。看下面这个例子：

```

1 Thread 1:
2   lock A
3   lock B
4
5 Thread 2:
6   wait for A
7   lock C (when A locked)
8
9 Thread 3:
10  wait for A
11  wait for B
12  wait for C
    
```

如果一个线程（比如线程3）需要一些锁，那么它必须按照确定的顺序获取锁。它只有获得了从顺序上排在前面的锁之后，才能获取后面的锁。

例如，线程2和线程3只有在获取了锁A之后才能尝试获取锁C(译者注：获取锁A是获取锁C的必要条件)。因为线程1已经拥有了锁A，所以线程2和3需要一直等到锁A被释放。然后在它们尝试对B或C加锁之前，必须成功地对A加了锁。

按照顺序加锁是一种有效的死锁预防机制。但是，这种方式需要你事先知道所有可能会用到的锁(译者注：并对这些锁做适当的排序)，但总有些时候是无法预知的。

加锁时限

另外一个可以避免死锁的方法是在尝试获取锁的时候加一个超时时间，这也就意味着在尝试获取锁的过程中若超过了这个时限该线程则放弃对该锁请求。若一个线程没有在给定的时限内成功获得所有需要的锁，则会进行回退并释放所有已经获得的锁，然后等待一段随机的时间再重试。这段随机的等待时间让其它线程有机会尝试获取相同的这些锁，并且让该应用在没有获得锁的时候可以继续运行(译者注：加锁超时后可以先继续运行干点其它事情，再回头来重复之前加锁的逻辑)。

以下是一个例子，展示了两个线程以不同的顺序尝试获取相同的两个锁，在发生超时后回退并重试的场景：

```
1 Thread 1 locks A
2 Thread 2 locks B
3
4 Thread 1 attempts to lock B but is blocked
5 Thread 2 attempts to lock A but is blocked
6
7 Thread 1's lock attempt on B times out
8 Thread 1 backs up and releases A as well
9 Thread 1 waits randomly (e.g. 257 millis) before retrying.
10
11 Thread 2's lock attempt on A times out
12 Thread 2 backs up and releases B as well
13 Thread 2 waits randomly (e.g. 43 millis) before retrying.
```

在上面的例子中，线程2比线程1早200毫秒进行重试加锁，因此它可以先成功地获取到两个锁。这时，线程1尝试获取锁A并且处于等待状态。当线程2结束时，线程1也可以顺利的获得这两个锁（除非线程2或者其它线程在线程1成功获得两个锁之前又获得其中的一些锁）。

需要注意的是，由于存在锁的超时，所以我们不能认为这种场景就一定是出现了死锁。也可能是因为获得了锁的线程（导致其它线程超时）需要很长的时间去完成它的任务。

此外，如果有非常多的线程同一时间去竞争同一批资源，就算有超时和回退机制，还是可能会导致这些线程重复地尝试但却始终得不到锁。如果只有两个线程，并且重试的超时时间设定为0到500毫秒之间，这种现象可能不会发生，但是如果是10个或20个线程情况就不同了。因为这些线程等待相等的重试时间的概率就高的多（或者非常接近以至于会出现问题）。

(译者注：超时和重试机制是为了避免在同一时间出现的竞争，但是当线程很多时，其中两个或多个线程的超时时间一样或者接近的可能性就会很大，因此就算出现竞争而导致超时后，由于超时时间一样，它们又会同时开始重试，导致新一轮的竞争，带来了新的问题。)

这种机制存在一个问题，在Java中不能对synchronized同步块设置超时时间。你需要创建一个自定义锁，或使用Java5中java.util.concurrent包下的工具。写一个自定义锁类不复杂，但超出了本文的内容。后续的Java并发系列会涵盖自定义锁的内容。

死锁检测

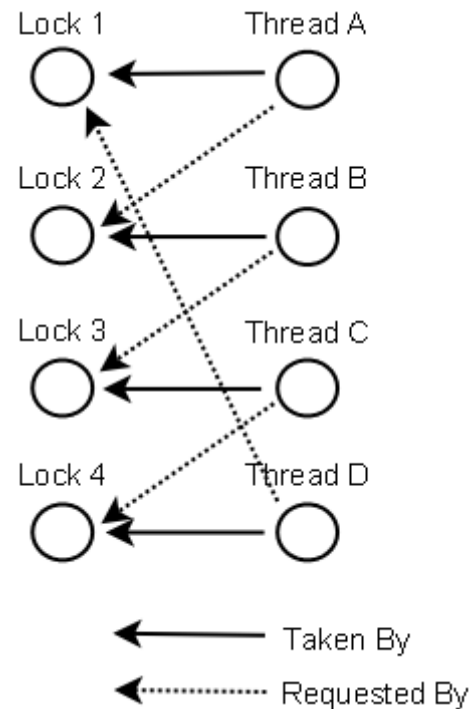
死锁检测是一个更好的死锁预防机制，它主要是针对那些不可能实现按序加锁并且锁超时也不可行的场景。

每当一个线程获得了锁，会在线程和锁相关的数据结构中（map、graph等等）将其记下。除此之外，每当有线程请求锁，也需要记录在这个数据结构中。

当一个线程请求锁失败时，这个线程可以遍历锁的关系图看看是否有死锁发生。例如，线程A请求锁7，但是锁7这个时候被线程B持有，这时线程A就可以检查一下线程B是否已经请求了线程A当前所持有的锁。如果线程B确实有这样的请求，那么就是发生了死锁（线程A拥有锁1，请求锁7；线程B拥有锁7，请求锁1）。

当然，死锁一般要比两个线程互相持有对方的锁这种情况要复杂的多。线程A等待线程B，线程B等待线程C，线程C等待线程D，线程D又在等待线程A。线程A为了检测死锁，它需要递进地检测所有被B请求的锁。从线程B所请求的锁开始，线程A找到了线程C，然后又找到了线程D，发现线程D请求的锁被线程A自己持有着。这是它就知道发生了死锁。

下面是一幅关于四个线程（A,B,C和D）之间锁占有和请求的关系图。像这样的数据结构就可以被用来检测死锁。



那么当检测出死锁时，这些线程该做些什么呢？

一个可行的做法是释放所有锁，回退，并且等待一段随机的时间后重试。这个和简单的加锁超时类似，不一样的是只有死锁已经发生了才回退，而不会是因为加锁的请求超时了。虽然有回退和等待，但是如果有大量的线程竞争同一批锁，它们还是会重复地死锁（编者注：原因同超时类似，不能从根本上减轻竞争）。

一个更好的方案是给这些线程设置优先级，让一个（或几个）线程回退，剩下的线程就像没发生死锁一样继续保持着它们需要的锁。如果赋予这些线程的优先级是固定不变的，同一批线程总是会拥有更高的优先级。为避免这个问题，可以在死锁发生的时候设置随机的优先级。

用户栈和内核栈有什么区别

内核在创建进程的时候，在创建 `task_struct` 的同时，会为进程创建相应的堆栈。每个进程会有两个栈：一个用户栈，存在于用户空间。一个内核栈，存在于内核空间。**当进程在用户空间运行时，CPU堆栈指针寄存器里面的内容是用户堆栈地址，使用用户栈；当进程在内核空间时，CPU堆栈指针寄存器里面的内容是内核栈空间地址，使用内核栈。**

当进程因为中断或者系统调用而从用户态转为内核态时，进程所使用的堆栈也要从用户栈转到内核栈。进程陷入内核态后，先把用户态堆栈的地址保存在内核栈之中，然后设置堆栈指针寄存器的内容为内核栈的地址，这样就完成了**用户栈向内核栈的转换**；当进程从内核态恢复到用户态时，把内核栈中保存的用户态的堆栈的地址恢复到堆栈指针寄存器即可。这样就实现了**内核栈和用户栈的互转**

那么，当从内核态转到用户态时，由于用户栈的地址是在陷入内核的时候保存在内核栈里面的，所以可以很容易地找到这个地址；**但是在陷入内核的时候，如何知道内核栈的地址？**关键在进程从用户态转到内核态的时候，进程的内核栈总是空的。这是因为当进程在用户态运行时，使用的是用户栈，当进程陷入到内核态时，内核栈保存进程在内核态运行的相关信息，但是，一旦进程返回到用户态后，**内核**

栈中保存的信息无效，会全部恢复，因此每次进程从用户态陷入内核时得到的内核栈都是空的，所以在进程陷入内核的时候，直接把内核栈的栈顶地址给堆栈指针寄存器就可以了。

内存泄漏

大家都知道，在堆上分配的内存，如果不再使用了，就应该及时释放，以便后面其他地方可以重用。而在 C 语言中，内存管理器不会自动回收不再使用的内存。如果忘了释放不再使用的内存，这些内存就不能被重用了，这就造成了内存泄漏。

内存泄漏几乎是很难避免的，不管是老手还是新手，都存在这个问题，甚至 Windows 与 Linux 这类系统软件也或多或少存在着内存泄漏。

也许对一般的应用软件来说，这个问题似乎不是那么突出与严重。一两处内存泄漏通常并不致于让程序崩溃，也不会带来逻辑上的错误，而且在进程退出时，系统会自动释放所有与该进程相关的内存（共享内存除外），所以内存泄漏的后果相对来说还是比较温和的。但是，量变会导致质变，一旦内存泄漏过多以致耗尽内存，后续内存分配将会失败，程序就可能因此而崩溃。

在常见情况下，内存泄漏的主要可见症状就是罪魁祸首的速度减慢。原因是体积大的进程更有可能被系统换出，让别的进程运行，而且大的进程在换进换出时花费的时间也更多。即使泄漏的内存本身并不被引用，但它仍然可能存在于页面中（内容自然是垃圾），这样就增加了进程的工作页数量，降低了性能。

最后，要避免这些内存相关的问题导致的内存越界与内存遗漏等错误，可以参考如下几点进行：

- 确保没有在访问空指针。
- 每个内存分配函数都应该有一个 free 函数与之对应，alloca 函数除外。
- 每次分配内存之后都应该及时进行初始化，可以结合 memset 函数进行初始化，calloc 函数除外。
- 每当向指针写入值时，都要确保对可用字节数和所写入的字节数进行交叉核对。
- 在对指针赋值前，一定要确保没有内存位置会变为孤立的。
- 每当释放结构化的元素（而该元素又包含指向动态分配的内存位置的指针）时，都应先遍历子内存位置 并从那里开始释放，然后再遍历回父节点。
- 始终正确处理返回动态分配的内存引用的函数返回值。

内存管理有哪几种方式

常见的内存管理方式有块式管理、页式管理、段式管理和段页式管理。最常用的是段页式管理。

1) 块式管理：把主存分为一大块一大块的，当所需的程序片断不在主存时就分配一块主存空间，把程序片断载入主存，就算所需的程序片段只有几个字节，也只能把这一块分配给它。这样会造成很大的浪费，平均浪费了50%的内存空间，但是易于管理。

2) 页式管理：用户程序的地址空间被划分成若干个固定大小的区域，这个区域被称为“页”，相应地，内存空间也被划分为若干个物理块，页和块的大小相等。可将用户程序的任一页放在内存的任一块中，从而实现了离散分配。这种方式的优点是页的大小是固定的，因此便于管理；缺点是页长与程序的逻辑大小没有任何关系。这就导致在某个时刻一个程序可能只有一部分在主存中，而另一部分则在辅存中。这不利于编程时的独立性，并给换入换出处理、存储保护和存储共享等操作造成麻烦。

3) 段式管理：段是按照程序的自然分界划分的并且长度可以动态改变的区域。使用这种方式，程序员可以把子程序、操作数和不同类型的数据和函数划分到不同的段中。这种方式将用户程序地址空间分成若干个大小不等的段，每段可以定义一组相对完整的逻辑信息。存储分配时，以段为单位，段与段在内存中可以不相邻接，也实现了离散分配。

分页对程序员而言是不可见的，而分段通常对程序员而言是可见的，因而分段为组织程序和数据提供了方便，但是对程序员的要求也比较高。

分段存储主要有如下优点：

- ①段的逻辑独立性不仅使其易于编译、管理、修改和保护，也便于多道程序共享

②段长可以根据需要动态改变，允许自由调度，以便有效利用主存空间

③方便分段共享、分段保护、动态链接、动态增长

分段存储的缺点如下：

①由于段的大小不固定，因此**存储管理比较麻烦**。

②会生成**段内碎片**，这会造成存储空间**利用率降低**。而且段式存储管理比页式存储管理方式需要更多的硬件支持。

正是由于页式管理和段式管理都有各种各样的缺点，因此，为了把这两种存储方式的优点结合起来，新引入了段页式管理。

4) 段页式管理：段页式存储组织是分段式和分页式结合的存储组织方法，这样可充分利用分段管理和分页管理的优点

①**用分段方法来分配和管理虚拟存储器**。程序的地址空间按逻辑单位分成基本独立的段，而每一段有自己的段名，再把每段分成固定大小的若干页

②**用分页方法来分配和管理内存**，即把整个主存分成与上述页大小相等的存储块，可装入作业的任何一页。程序对内存的调入或调出是按页进行的，但它又可按段实现共享和保护。

什么是虚拟内存

虚拟内存简称虚存，是计算机系统内存管理的一种技术。它是相对于物理内存而言的，可以理解为“假的”内存。它使得**应用程序认为它拥有连续可用的内存**（一个连续完整的地址空间），**允许程序员编写并运行比实际系统拥有的内存大得多的程序**，这使得许多大型软件项目能够在具有有限内存资源的系统上实现。而实际上，它通常被分割成多个物理内存碎片，还有部分暂时存储在外部磁盘存储器上，在需要进行数据交换。相比实存，虚存有以下好处：

1) **扩大了地址空间**。无论是段式虚存，还是页式虚存，或是段页式虚存，**寻址空间都比实存大**。

2) **内存保护**。每个进程运行在各自的虚拟内存地址空间，互相不能干扰对方。另外，虚存还对特定的内存地址提供写保护，可以防止代码或数据被恶意篡改。

3) **公平分配内存**。采用了虚存之后，每个进程都相当于有同样大小的虚存空间。

4) 当进程需要通信时，可采用虚存共享的方式实现不过，使用虚存也是有代价的，主要表现在以下几个方面。

1) 虚存的管理需要建立很多数据结构，这些数据结构要占用额外的内存。

2) 虚拟地址到物理地址的转换，增加了指令的执行时间。

3) 页面的换入换出需要磁盘IO，这是很耗时间的

4) 如果一页中只有一部分数据，会浪费内存。

内存碎片 内碎片 外碎片

内存碎片是由于多次进行内存分配造成的，当进行内存分配时，内存格式一般为：（用户使用段）（空白段）（用户使用段），当空白段很小的时候，可能不能提供给用户足够多的空间，如夹在中间的空白段的大小为5，而用户需要的内存大小为6，这样会产生很多的间隙，造成使用效率下降，这些很小的空隙叫碎片。

内碎片：分配给程序的存储空间没有用完，有一部分是程序不使用，但其他程序也没法用的空间。内碎片是处于区域内部或页面内部的存储块，占有这些区域或页面的进程并不使用这个存储块，而在进程占有这块存储块时，系统无法利用它，直到进程释放它，或进程结束时，系统才有可能利用这个存储块。

外碎片：空间太小，小到无法给任何程序分配（不属于任何进程）的存储空间。外部碎片是出于任何已分配区域或页面外部的空闲存储块，这些存储块的总和可以满足当前申请的长度要求，但是它们的地址不连续或其他原因，使得系统无法满足当前申请。

内碎片和外碎片是一对矛盾体，一种特定的内存分配算法，很难同时解决好内碎片和外碎片的问题，只能根据应用特点进行取舍

虚拟地址、逻辑地址、线性地址、物理地址

虚拟地址是指由程序产生的由**段选择符**和**段内偏移地址**组成的地址。这两部分组成的地址**并没有直接访问物理内存**，而是通过分段地址的变换处理后才会对应到相应的物理内存地址。

逻辑地址指由程序产生的段内偏移地址。有时直接把逻辑地址当成虚拟地址，两者并没有明确的界限。

线性地址是指**虚拟地址到物理地址变换之间的中间层**，是处理器可寻址的内存空间（称为线性地址空间）中的地址。程序代码会产生逻辑地址，或者说是段中的偏移地址，加上相应段基址就生成了一个线性地址。如果启用了分页机制，那么线性地址可以再经过变换产生物理地址。若没有采用分页机制，那么线性地址就是物理地址。

物理地址是指现在**CPU外部地址总线上的寻址物理内存的地址信号**，是地址变换的最终结果。虚拟地址到物理地址的转化方法是与体系结构相关的，一般分段与分页两种方式。以x86CPU为例，分段、分页都是支持的。内存管理单元负责从虚拟地址到物理地址的转化。**逻辑地址是段标识+段内偏移量的形式**，MMU通过查询段表，可以把逻辑地址转化为线性地址。如果CPU没有开启分页功能，那么线性地址就是物理地址；如果CPU开启了分页功能MMU还需要查询页表来将线性地址转化为物理地址：逻辑地址（段表）→线性地址（页表）→物理地址。

映射是一种多对一的关系，即不同的逻辑地址可以映射到同一个线性地址上；不同的线性地址也可以映射到同一个物理地址上。而且，同一个线性地址在发生换页以后，也可能被重新装载到另外一个物理地址上，所以这种多对一的映射关系也会随时间发生变化。

CPU对外部设备的控制方式按CPU的介入程度，从小到大依次为通道方式，DMA方式，中断方式，程序控制方式。

通道方式：通道是一种通过执行通道程序管理、控制I/O操作的控制器，它使主机与I/O操作之间达到更高的并行程度。当使用通道方式进行数据传输时，由操作系统构造通道程序和通道状态字，将通道程序保存在内存中，并将通道程序的首地址放到通道地址字中，然后执行启动I/O指令。

DMA（直接内存存取）方式。DMA方式是使用DMA控制器来管理和控制数据传输的，DMA控制器和CPU**共享系统总线**，并且都可以独立访问存储器。在使用DMA工作方式进行数据传输时，DMA控制器控制了系统总线，由DMA控制器提供存储器地址及必需的读写控制信号，实现外部设备与存储器之间的数据传输。

程序中断方式：使用程序中断方式，当I/O系统与外部设备传输数据时，CPU无须等待，也不必去查询I/O系统的状态，当I/O系统完成了数据传输后，以中断信号通知CPU。CPU保存正在执行程序的现场，转入I/O中断服务程序完成与I/O系统的数据交换，然后返回原主程序继续执行。**中断方式因为CPU无需等待而提高了效率。**

程序控制方式：程序控制方式也叫**查询控制方式**。当程序要传输数据时，CPU发出I/O指令，指令中包含了外部设备的地址信息和所要执行的操作，I/O系统接收并执行该指令，同时设置了状态寄存器；CPU定期地查询I/O系统，确定操作是否已经完成。由CPU主动查询I/O系统，完成主机与外部设备间的数据传输，方法简单，硬件开销小，但CPU因要不停执行查询操作，降低了效率。

进程的几种状态

1.进程的五状态模型：

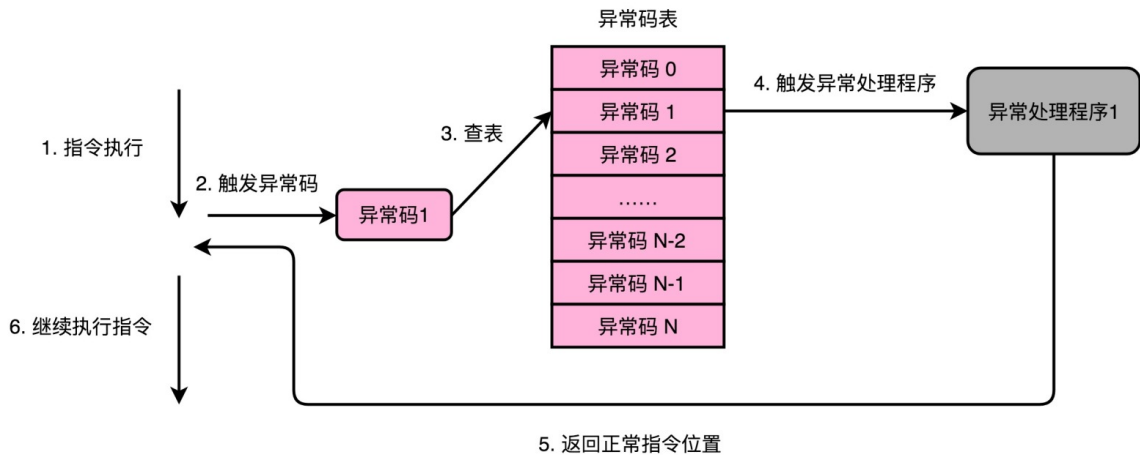
运行态：该进程正在执行。

就绪态：进程已经做好了准备，只要有机会就开始执行。

阻塞态（等待态）：进程在某些事情发生前不能执行，等待阻塞进程的事件完成。

新建态：刚刚创建的进程，操作系统还没有把它加入到可执行进程组中，通常是进程控制块已经创建但是还没有加载到内存中的进程。

退出态：操作系统从可执行进程组中释放出的进程，或由于自身或某种原因停止运行。



https://blog.csdn.net/qin_16039901

2.导致转换的事件：

空->新建：创建执行一个程序的新进程，可能的事件有：新的批处理作业、交互登录（终端用户登录到系统）、操作系统因为提供一项服务而创建、由现有的进程派生等。

新建->就绪：操作系统准备好再接纳一个进程时，把一个进程从新建态转换为就绪态。

就绪->运行：需要选择一个新进程运行时，操作系统的调度器或分配器根据某种调度算法选择一个处于就绪态的进程。

运行->退出：导致进程终止的原因有：正常完成、超过时限、系统无法满足进程需要的内存空间、进程试图访问不允许访问的内存单元（越界）、算术错误（如除以0或存储大于硬件可以接纳的数字）、父进程终止（操作系统可能会自动终止该进程所有的后代进程）、父进程请求终止后代进程等。

运行->就绪：最常见的原因是，正在运行的进程到达了“允许不中断执行”的最大时间段，该把处理器的资源释放给其他在就绪态的进程使用了；还有一中原因可能是由于具有更改优先级的就绪态进程抢占了该进程的资源，使其被中断转换到就绪态。

运行->阻塞：如果进程请求它必须等待的某些事件，例如一个无法立即得到的资源（如I/O操作），只有在获得等待的资源后才能继续进程的执行，则进入等待态（阻塞态）。

阻塞->就绪：当等待的事件发生时，处于阻塞态的进程转换到就绪态。

就绪->退出：在上图中没有标出这种转换，在某些进程中，父进程可以在任何时刻终止一个子进程，如果一个父进程终止，所有相关的子进程都被终止。

阻塞->退出：跟上一项原因类似。

异常处理

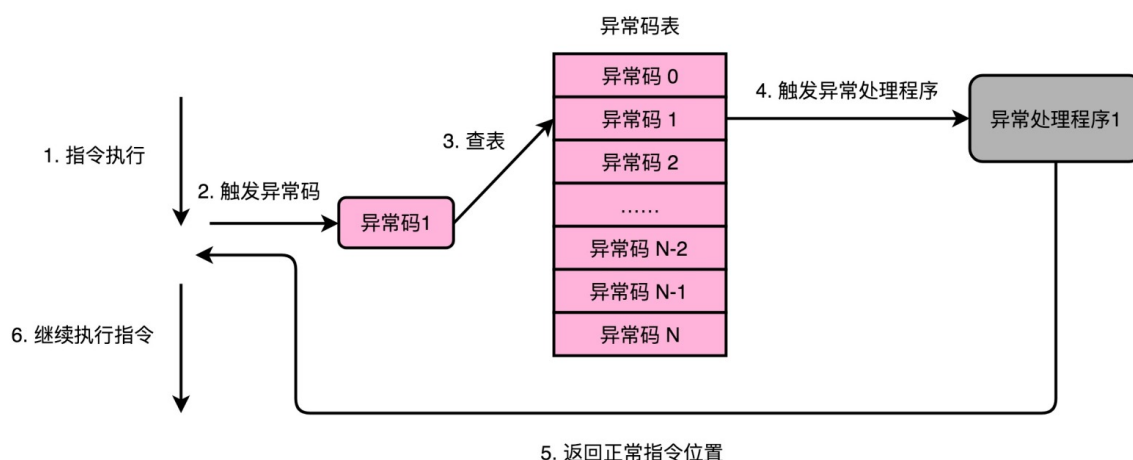
关于异常，它其实是一个硬件和软件组合到一起的处理过程。异常的前半生，也就是异常的发生和捕捉，是在硬件层面完成的。但是异常的后半生，也就是说，异常的处理，其实是由软件来完成的。

计算机会为每一种可能会发生的异常，分配一个异常代码（Exception Number）。有些教科书会把异常代码叫作中断向量（Interrupt Vector）。

异常发生的时候，通常是CPU检测到了一个特殊的信号。这些信号呢，在组成原理里面，我们一般叫作发生了一个事件（Event）。CPU在检测到事件的时候，其实也就拿到了对应的异常代码。

这些异常代码里，I/O发出的信号的异常代码，是由操作系统来分配的，也就是由软件来设定的。而像加法溢出这样的异常代码，则是由CPU预先分配好的，也就是由硬件来分配的。

拿到异常代码之后，CPU就会触发异常处理的流程。计算机在内存里，会保留一个异常表（Exception Table）。我们的CPU在拿到了异常码之后，会先把当前的程序执行的现场，保存到程序栈里面，然后根据异常码查询，找到对应的异常处理程序，最后把后续指令执行的指挥权，交给这个异常处理程序。



https://blog.csdn.net/meng_15935901

异常的分类：中断、陷阱、故障和中止

第一种异常叫中断（Interrupt）。顾名思义，自然就是程序在执行到一半的时候，被打断了。

第二种异常叫陷阱（Trap）。陷阱，其实是我们程序员“故意”主动触发的异常。就好像你在程序里面打了一个断点，这个断点就是设下的一个“陷阱”。

第三种异常叫故障（Fault）。比如，我们在程序执行的过程中，进行加法计算发生了溢出，其实就是故障类型的异常。

最后一种异常叫中止（Abort）。与其说这是一种异常类型，不如说这是故障的一种特殊情况。当CPU遇到了故障，但是恢复不过来的时候，程序就不得不中止了。

异常的处理：上下文切换

在实际的异常处理程序执行之前，CPU需要去做一次“保存现场”的操作。有了这个操作，我们才能在异常处理完成之后，重新回到之前执行的指令序列里面来。

因为异常情况往往发生在程序正常执行的预期之外，比如中断、故障发生的时候。所以，除了本来程序压栈要做的事情之外，我们还需要把CPU内当前运行程序用到的所有寄存器，都放到栈里面。最典型的就是条件码寄存器里面的内容。

像陷阱这样的异常，涉及程序指令在用户态和内核态之间的切换。对应压栈的时候，对应的数据是压到内核栈里，而不是程序栈里。

S3C2410 MMU（存储器管理单元）详述

MMU,全称Memory Manage Unit, 中文名——存储器管理单元。

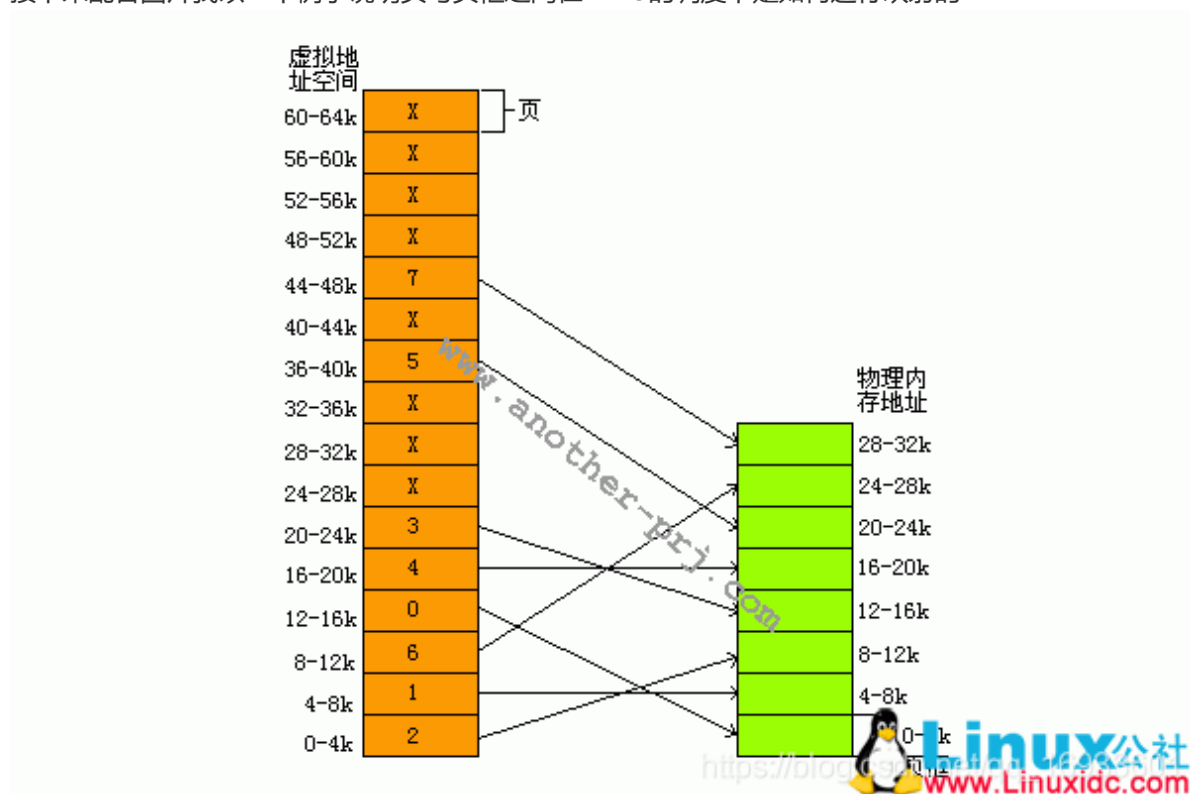
许多年以前，当人们还在使用DOS或是更古老的操作系统的时候，计算机的内存还非常小，一般都是以K为单位进行计算，相应的，当时的程序规模也不大，所以内存容量虽然小，但还是可以容纳当时的程序。但随着图形界面的兴起还用户需求的不增大，应用程序的规模也随之膨胀起来，终于一个难题出现在程序员的面前，那就是应用程序太大以至于内存容纳不下该程序，通常解决的办法是把程序分割成许多称为覆盖块（overlay）的片段。覆盖块0首先运行，结束时他将调用另一个覆盖块。虽然覆盖块的交换是由OS完成的，但是必须先由程序员把程序先进行分割，这是一个费时费力的工作，而且相当枯燥。人们必须找到更好的办法从根本上解决这个问题。不久人们找到了一个办法，这就是虚拟存储器（virtual memory）。虚拟存储器的基本思想是程序，数据，堆栈的总的大小可以超过物理存储器的大小，操作系统把当前使用的部分保留在内存中，而把其他未被使用的部分保存在磁盘上。比如对一个16MB的程序和一个内存只有4MB的机器，OS通过选择，可以决定各个时刻将哪4M的内容保留在内存中，并

在需要时在内存和磁盘间交换程序片段，这样就可以把这个16M的程序运行在一个只具有4M内存机器上了。而这个16M的程序在运行前不必由程序员进行分割。

任何时候，计算机上都存在一个程序能够产生的地址集合，我们称之为地址范围。这个范围的大小由**CPU的位数**决定，例如一个32位的CPU，它的地址范围是0~0xFFFFFFFF (4G),而对于一个64位的CPU，它的地址范围为0~0xFFFFFFFFFFFFFFFF (64T).这个范围就是我们的程序能够产生的地址范围，我们把这个地址范围称为**虚拟地址空间**，该空间中的某一个地址我们称之为虚拟地址。与虚拟地址空间和虚拟地址相对应的则是物理地址空间和物理地址，大多数时候我们的系统所具备的物理地址空间只是虚拟地址空间的一个**子集**，这里举一个最简单的例子直观地说明这两者，对于一台**内存为256MB的32bit x86主机**来说，它的**虚拟地址空间范围是0~ 0xFFFFFFFF (4G)** ,而**物理地址空间范围是0x00000000~0x0FFFFFFF (256MB)**。

在没有使用虚拟存储器的机器上，**虚拟地址被直接送到内存总线上**，使具有**相同地址的物理存储器**被读写。而在使用了虚拟存储器的情况下，虚拟地址不是被直接送到内存地址总线上，而是**送到内存管理单元——MMU**（主角终于出现了）。他由一个或一组芯片组成，一般存在与协处理器中，其功能是把虚拟地址映射为物理地址。

大多数使用虚拟存储器的系统都使用一种称为分页（paging）。**虚拟地址空间**划分成称为**页（page）**的单位,而相应的**物理地址空间**也被进行划分，单位是**页框(frame)**.页和页框的大小必须相同。接下来配合图片我以一个例子说明页与页框之间在MMU的调度下是如何进行映射的



在这个例子中我们有一台可以生成**16位地址**的机器，它的虚拟地址范围从**0x0000~0xFFFF(64K)**,而这台机器只有**32K的物理地址**，因此他可以运行64K的程序，但该程序不能一次性调入内存运行。这台机器必须有一个达到**可以存放64K程序的外部存储器**（例如磁盘或是FLASH），以保证程序片段在需要时可以被调用。在这个例子中，页的大小为4K,页框大小与页相同（这点是必须保证的，**内存和外围存储器之间的传输总是以页为单位的**），对应64K的虚拟地址和32K的物理存储器，他们分别包含了16个页和8个页框。

我们先根据上图解释一下分页后要用到的几个术语，在上面我们已经接触了页和页框，上图中绿色部分是物理空间，其中每一格表示一个物理页框。橘黄色部分是虚拟空间，每一格表示一个页，它由两部分组成，分别是**Frame Index(页框索引)**和**位p（present 存在位）**，**Frame Index**的意义很明显，它指出本页是往哪个物理页框进行映射的，**位p**的意义则是指出本页的映射是否有效，如上图，当某个页并没有被映射时（或称“映射无效”，**Frame Index**部分为X），该位为0，映射有效则该位为1。

我们执行下面这些指令（本例子的指令不针对任何特定机型，都是伪指令）

例1：

MOVE REG,0 //将0号地址的值传递进寄存器REG.

虚拟地址0将被送往MMU,MMU看到该虚地址落在页0范围内（页0范围是0到4095），从上图我们看到页0所对应（映射）的页框为2（页框2的地址范围是8192到12287），因此MMU将该虚拟地址转化为物理地址8192，并把地址8192送到地址总线上。**内存对MMU的映射一无所知，它只看到一个对地址8192的读请求并执行它。**MMU从而把0到4096的虚拟地址映射到8192到12287的物理地址。

例2：

MOVE REG,8192

被转换为

MOVE REG,24576

因为虚拟地址8192在页2中，而页2被映射到页框6（物理地址从24576到28671）

例3：

MOVE REG,20500

被转换为

MOVE REG,12308

虚拟地址20500在虚页5（虚拟地址范围是20480到24575）距开头20个字节处，虚页5映射到页框3（页框3的地址范围是12288到16383），于是被映射到物理地址 $12288+20=12308$ 。

通过适当的设置MMU，可以把16个虚页映射到8个页框中的任何一个，但是这个方法并没有有效的解决虚拟地址空间比物理地址空间大的问题。从上图我们可以看到，我们只有8个页框（物理地址），但我们有16个页（虚拟地址），所以**我们只能把16个页中的8个进行有效的映射**。我们看看例4会发生什么情况

MOV REG,32780

虚拟地址32780落在页8的范围内，从上图总我们看到页8没有被有效的进行映射（该页被打上X），这是又会发生什么？MMU注意到这个页没有被映射，于是通知CPU发生一个**缺页故障**（page fault）。这种情况下操作系统必须处理这个页故障，它必须从**8个物理页框**中找到1个当前**很少被使用的页框**并把该页框的内容写入**外围存储器**（这个动作被称为page copy），**随后把需要引用的页（例4中是页8）映射到刚才释放的页框中**（这个动作称为**修改映射关系**），然后从新执行产生故障的指令（MOV REG,32780）。假设操作系统决定释放页框1，那么它将把虚页8装入物理地址的4-8K,并做**两处修改**：首先把标记虚页1未被映射（原来虚页1是被映射到页框1的），以使以后任何对虚拟地址4K到8K的访问都引起页故障而使操作系统做出适当的动作（这个动作正是我们现在在讨论的），其次他把虚页8对应的页框号由X变为1，因此重新执行MOV REG,32780时，MMU将把32780映射为4108。

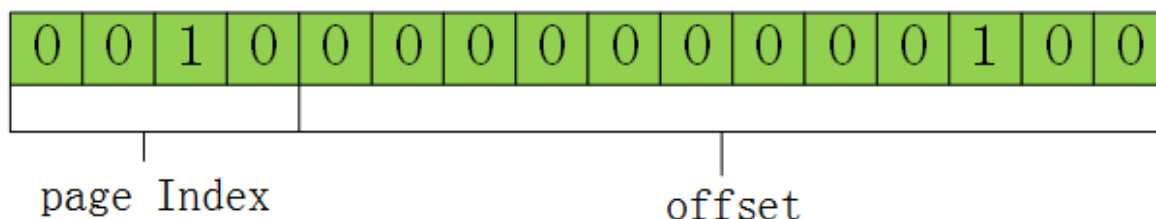
我们大致了解了MMU在我们的机器中扮演了什么角色以及它基本的工作内容是什么，下面我们将举例子说明它究竟是如何工作的（注意，本例中的MMU并无针对某种特定的机型，它是所有MMU工作的一个抽象）。

首先明确一点，MMU的主要工作只有一个，就是把虚拟地址映射到物理地址。

我们已经知道，大多数使用虚拟存储器的系统都使用一种称为**分页（paging）的技术**，就象我们刚才所举的例子，虚拟地址空间被分成**大小相同的一组页**，每个页有一个用来标示它的页号（这个页号一般是它在该组中的**索引**，这点和C/C++中的数组相似）。在上面的例子中0~4K的页号为0，4~8K的页号为1，8~12K的页号为2，以此类推。而**虚拟地址（注意：是一个确定的地址，不是一个空间）被MMU分为2个部分**，第一部分是**页号索引（page Index）**。第二部分则是**相对该页首地址的偏移量（offset）**。我们还是以刚才那个16位机器结合下图进行一个实例说明，该实例中，虚拟地址8196被送进MMU,MMU把它映射成物理地址。16位的CPU总共能产生的地址范围是0~64K,按每页4K的大小计算，

该空间必须被分成16个页。而我们的**虚拟地址第一部分所能够表达的范围也必须等于16（这样才能索引到该页组中的每一个页）**，也就是说这个部分**至少需要4个bit**。一个页的大小是4K(4096),也就是说偏移部分**必须使用12个bit来表示**($2^{12}=4096$ ，这样才能访问到一个页中的所有地址),8196的二进制码如下图所示：

Virtual Address 8192



该地址的页号索引为0010（二进制码），既索引的页为页2，第二部分为000000000100（二进制），偏移量为4。页2中的页框号为6（页2映射在页框6，见上图），我们看到页框6的物理地址是24~28K。于是MMU计算出虚拟地址8196应该被映射成物理地址24580（**页框首地址+偏移量=24576+4=24580**）。同样的，若我们对虚拟地址1026进行读取，**1026的二进制码为0000010000000010，page index=0000=0,offset=010000000010=1026**。页号为0，该页映射的页框号为2，页框2的物理地址范围是8192~12287，故MMU将虚拟地址1026映射为物理地址9218（页框首地址+偏移量=8192+1026=9218）

以上就是MMU的工作过程。

上下文有哪些？怎么理解？

为什么会有上下文这种概念

内核空间 and 用户空间是现代操作系统的两种工作模式，内核模块运行在内核空间，而用户态应用程序运行在用户空间。它们代表不同的级别，而对系统资源具有不同的访问权限。内核模块运行在最高级别（内核态），这个级下所有的操作都受系统信任，而应用程序运行在较低级别（用户态）。在这个级别，处理器控制着对硬件的直接访问以及对内存的非授权访问。内核态和用户态有自己的内存映射，即自己的地址空间。

其中处理器总处于以下状态中的一种：

内核态，运行于进程上下文，内核代表进程运行于内核空间；

内核态，运行于中断上下文，内核代表硬件运行于内核空间；

用户态，运行于用户空间。

系统的两种不同运行状态，才有了上下文的概念。用户空间的应用程序，如果想请求系统服务，比如操作某个物理设备，映射设备的地址到用户空间，必须通过**系统调用**来实现。（系统调用是操作系统提供给用户空间的**接口函数**）。

通过系统调用，用户空间的应用程序就会进入内核空间，由内核代表该进程运行于内核空间，这就涉及到上下文的切换，用户空间和内核空间具有不同的地址映射，通用或专用的寄存器组，而**用户空间的进程要传递很多变量、参数给内核，内核也要保存用户进程的一些寄存器、变量等，以便系统调用结束后回到用户空间继续执行，**

进程上下文

所谓的进程上下文，就是一个进程在执行的时候，CPU的所有寄存器中的值、进程的状态以及堆栈上的内容，当内核需要切换到另一个进程时，它需要**保存当前进程的所有状态**，即保存当前进程的进程上下文，以便再次执行该进程时，能够**恢复切换时的状态，继续执行**。

当发生进程调度时，进行**进程切换**就是上下文切换(context switch)。操作系统必须对上面提到的**全部信息进行切换**，新调度的进程才能运行。而**系统调用**进行的是**模式切换**(mode switch)。模式切换与进

程切换比较起来，容易很多，而且节省时间，因为模式切换最主要的任务只是切换进程寄存器上下文的切换。

进程上下文主要是**异常处理程序和内核线程**。内核之所以进入进程上下文是因为进程自身的一些**工作需要在内核中做**。例如，系统调用是为当前进程服务的，异常通常是处理进程导致的错误状态等。所以在进程上下文中引用current是有意义的。

中断上下文

硬件通过触发信号，向CPU发送**中断信号**，导致**内核调用中断处理程序**，进入内核空间。这个过程中，硬件的一些**变量和参数**也要传递给内核，**内核通过这些参数进行中断处理**。

所以，“中断上下文”就可以理解为**硬件传递过来的这些参数和内核需要保存的一些环境**，主要是**被中断的进程的环境**。

内核进入中断上下文是**因为中断信号而导致的 中断处理或软中断**。而中断信号的发生是随机的，中断处理程序及软中断并不能事先预测发生中断时当前运行的是哪个进程，所以在中断上下文中引用current是可以的，但没有意义。

进程上下文 VS 中断上下文

内核可以处于两种上下文：进程上下文和中断上下文。

在系统调用之后，用户应用程序进入内核空间，此后内核空间针对用户空间相应进程的代表就运行于进程上下文。

异步发生的中断会引发中断处理程序被调用，中断处理程序就运行于中断上下文。

中断上下文和进程上下文不可能同时发生。

运行于进程上下文的内核代码是可抢占的，但中断上下文则会一直运行至结束，不会被抢占。因此，内核会限制中断上下文的工作，不允许其执行如下操作：

a -- 进入睡眠状态或主动放弃CPU

由于中断上下文不属于任何进程，它与current没有任何关系（尽管此时current指向被中断的进程），所以中断上下文一旦睡眠或者放弃CPU，将无法被唤醒。所以也叫原子上下文（atomic context）。

b -- 占用互斥体

为了保护中断句柄临界区资源，不能使用mutexes。如果获得不到信号量，代码就会睡眠，会产生和上面相同的情况，如果必须使用锁，则使用spinlock。

c -- 执行耗时的任务

中断处理应该尽可能快，因为内核要响应大量服务和请求，中断上下文占用CPU时间太长会严重影响系统功能。在中断处理例程中执行耗时任务时，应该交由中断处理例程底半部来处理。

d -- 访问用户空间虚拟内存

因为中断上下文是和特定进程无关的，它是内核代表硬件运行在内核空间，所以在中断上下文无法访问用户空间的虚拟地址

e -- 中断处理例程不应该设置成reentrant（可被并行或递归调用的例程）

因为中断发生时，preempt和irq都被disable，直到中断返回。所以中断上下文和进程上下文不一样，中断处理例程的不同实例，是不允许在SMP上并发运行的。

f -- 中断处理例程可以被更高级别的IRQ中断

如果想禁止这种中断，可以将中断处理例程定义成快速处理例程，相当于告诉CPU，该例程运行时，禁止本地CPU上所有中断请求。这直接导致的结果是，由于其他中断被延迟响应，系统性能下降。

